

Custom Compiler™ and Fusion Compiler™ Co-design Assistant User Guide

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

New in This Release	10
Related Products, Publications, and Trademarks	10
Conventions	11
Customer Support	11
Statement on Inclusivity and Diversity	12

1. About Analog and Digital Co-designing	13
Analog and Digital Co-designing Using the Custom Compiler Tool	13
Software User Interface	15
Library Manager	15
Schematic Editor and Layout Editor	18
Edit Liberty Data Dialog Box	20
Main Tab	21
Extracted Data Tab	23

2. Getting Started	24
Workflows for Custom Compiler and Fusion Compiler Co-designing	24
Workflow for RTL-to-GDSII Co-designing	24
Workflow for Preroute Editing	25
Workflow for Developing and Integrating Digital Blocks	25
Starting Custom Compiler Sessions and Displaying the Library Manager	28
Enabling the Co-design Assistant	28
Custom Compiler Products and Tools	29

3. Preparing Design Libraries	30
Creating Fusion Compiler Libraries	30
Adding Fusion Compiler Libraries	34
Before You Start	34
Procedure	34
Viewing and Editing Library Attributes	38

Viewing Associated Reference Libraries	38
Editing Reference Library Associations	39
Deleting Fusion Compiler Libraries	40
Monitoring the Import and Export Progress	42

4. Running RTL-to-GDSII Co-design Flows	44
Co-designing in Unified RTL-to-GDSII Flows	44
Fusion Compiler Frontend Flows	44
Frontend Flow Tcl Script	45
Workflow for Co-designing in Unified RTL-to-GDSII Flows	46
Enabling the Co-design Frontend Flow	46
Setting Frontend Flow Options	49
Main Tab	50
Frontend Flow Tab	51
Setting Place-and-Route Options	54
Starting the Flow Process	56

5. Preroute Editing (Roundtrip Flow)	57
Workflow for Preroute Editing	58
Reviewing Designs in the Fusion Compiler Tool	59
Viewing Placement and Routing Blockages	60
Viewing Nondefault Rules and Custom Routing Constraints	62
Setting Up Co-design Filters	64
Opening Fusion Compiler Designs in the Custom Compiler Environment	66
Opening Filtered Designs	66
Opening an Entire Design	70
Checking That the Design is in the Safe-Editing Mode	71
Reviewing Designs Within the Custom Compiler Environment	72
Viewing Design Objects	73
Viewing Constraints	73
Setting and Editing Constraints	74
Displaying the Layout Editing Toolbox	74
Adjusting Cell Placements	76
Creating and Editing Blockages	78

Contents

Editing Route Guides	79
Creating and Editing Routing Corridors	81
Adding and Editing Routes	83
Before You Start	83
How Fusion Compiler Attributes Are Mapped	83
How Must-Join Pins Are Treated	84
Wire Track Considerations	84
Creating Routes	84
Editing Routes	86
Adding Shielding	86
Setting Shielding Constraints	86
Adding Shielding and Checking Coverage	87
Mapping Shield Constraints	88
Editing Other Aspects of the Design	89
Viewing and Editing Object Properties	90
Setting Terminal Currents	91
Reporting and Fixing Electrical Violations	92
Saving Modified Designs	93
Reviewing Custom Compiler Edits in the Fusion Compiler Tool	93
Starting the Fusion Compiler Tool	94
Retrieving New Object Attributes	94
Reviewing Newly Added Routes	95
Reviewing Newly Added or Modified Constraints	96
Reviewing Newly Added Shielding	97
Reviewing Fusion Compiler Edits Within the Custom Compiler Environment	98
Reviewing the Digital Edits	98
Importing DRC Data From the Fusion Compiler Tool	99
Importing Error Data From the Fusion Compiler Tool	101
Importing Error Data Using the Import Errors Command	102
Importing Error Data Using Tcl Commands	104
 6. Preroute Editing (Flow From Within the Fusion Compiler Tool)	 105
Workflow for Preroute Editing From Within the Fusion Compiler Tool	105
Enabling the Co-design Flow in the Fusion Compiler Tool	106
Opening Fusion Compiler Designs in the Custom Compiler Environment	107

7. Exporting Preroute Floorplans to Tcl Files	109
About the Floorplan Export Process	109
Workflow for Exporting Preroute Floorplans to Tcl Files	110
Guidelines for Exporting Preroute Floorplans	111
Turning on Change-Tracking Features	112
About Track Changes	113
About Highlight Changes	117
Turning On Track Changes and Highlight Changes	118
Exporting Preroute Floorplans	118
Integrating All Modifications at the Top Level	121
Export Floor Plan Dialog Box Reference	122
Output File	123
Objects	123
Nets	123
Options	123

8. Creating Black Boxes for Analog Blocks	125
About Analog Blocks and Digital Timing Shells	125
Analog Cell Timing Analysis	125
Black Box Representations of Analog Blocks	127
Workflow for Creating Black Boxes	128
Preparing for Digital Timing Shells	128
Preparing the Design Schematic	128
Creating Connection Definitions for the Timing Shell	129
Defining the Power Supplies	129
Setting Up Block Abstraction Creation	130
Creating and Editing Black Boxes	130
Running Place and Route	135

9. Developing and Integrating Digital Blocks	137
About the Place-and-Route Process	138
Place-and-Route Input	138
Place-and-Route Tcl Script	139
Workflow for Developing and Integrating Digital Blocks	142

Contents

Preparing the Technology Data	143
Setting Up the Technology Libraries	144
Mapping Technology Layer Names	144
Mapping Libraries, Cells, and Views	146
Adjusting the Design Scale	148
Mapping Recognition Layers	149
Mapping Net Bundles and Non-default Rules	149
Reviewing the Mixed-Signal Design	150
Developing Digital Blocks from Verilog Files	150
Before You Start	150
Running Place and Route Using a Verilog File	151
Developing Digital Blocks from Schematics	152
Before You Start Developing Digital Blocks From Schematics	153
Running Place and Route Using Schematics	153
Viewing the Output Log	157
Viewing the Verilog Netlist File	158
Developing Digital Blocks from Layouts	158
Workflow for Developing Digital Blocks From Layouts	158
Before You Start Developing Digital Blocks From Layouts	159
Creating the Fusion Compiler Design	159
Adding and Editing Floorplan Components	162
Copying Fusion Compiler Cells	164
Running Place-and-Route Using Layouts	164
Integrating the Digital Block at the Top Level	167
Placing Digital Block Instances	167
Remastering the Instances	168
10. Creating Frame Views	170
About Frame Views and the Create Frame Tcl Script	170
Before You Start	171
Creating Frame Views from the Custom Compiler Environment	173
11. Creating Schematics from Verilog Files	182
About Schematic Creation From Verilog Files	182
Before You Start	182

Creating Schematics from Fusion Compiler Verilog Files	183
12. Exporting Custom Compiler Designs to the Fusion Compiler Tool	185
Validating the Custom Compiler Designs	185
Exporting Custom Compiler Designs	186
Specifying Main Tab Options	192
Specifying Place & Route Tab Options	194
13. Implementing ECOs	196
About ECO Editing in Co-design Flows	196
Creating and Deleting Nets	197
Creating Nets	197
Deleting Nets	199
Creating and Deleting Instances	199
Creating Instances	200
Deleting Instances	201
Creating and Deleting Physical-Only Cells	201
Connecting and Disconnecting Nets	201
Connecting Nets	202
Disconnecting Nets	202
14. Preparing for Physical Verification (IC Validator)	204
15. Running Formality Verification	205
About the Formality Verification Flow	205
Formal Verification and the Formality Verification Tool	205
Using the Formality Verification Tool in the Co-design Process	206
Workflow for Formality Verification	206
Enabling the Formality Verification Flow	206
Setting Up Formality Verification Flow Options	209
Auto Setup	211
Link Library	211
Reference Tab	211
Implementation Tab	212

Run	213
Formality Verification Script	213
Starting the Formality Verification Flow Process	213

A. Technology Mappings	215
Viewing Mapped Constraints	215
Mapping Fusion Compiler Constraints	215
Constraints	216
Design Constraints	216
Mapping Custom Compiler Constraints	218

B. Character and Object Mappings	219
Character Mappings	219
Object Mappings	220
Object Properties	220
Shapes	221
Vias	221
Standard Via	221
Custom Via	222
Instances	222
Bounds	222
Logical Components	223
Boundary	223
Blockage	223
Placement Blockage	224
Pin Blockage (or LayerRangeBlockage)	224
Routing Blockage	225
Routing	225
Routing Corridor	226
Anchor	226
Route Guide	226
Antenna Properties	226
Voltage Area	227
Custom Compiler Objects	228

About This User Guide

This user guide describes how to use the Custom Compiler™ co-design assistants for analog and digital co-designing. The co-designing process involves iterative steps between the Fusion Compiler™ tool and the Custom Compiler environment.

This user guide assumes that you are familiar with the Custom Compiler and Fusion Compiler tools.

This preface includes the following sections:

- [New in This Release](#)
- [Related Products, Publications, and Trademarks](#)
- [Conventions](#)
- [Customer Support](#)
- [Statement on Inclusivity and Diversity](#)

New in This Release

Information about new features, enhancements, and changes, known limitations, and resolved Synopsys Technical Action Requests (STARs) is available in the Custom Compiler Release Notes on the SolvNetPlus site.

Related Products, Publications, and Trademarks

For additional information about the Custom Compiler tool, see the documentation on the Synopsys SolvNetPlus support site at the following address:

<https://solvnetplus.synopsys.com>

You might also want to see the documentation for the following related Synopsys products:

Synopsys® Custom Compiler™

Synopsys Fusion Compiler™

Synopsys Hercules™

Synopsys IC Validator™

Synopsys StarRC™

Conventions

The following conventions are used in Synopsys documentation.

Convention	Description
Courier	Indicates syntax, such as <code>write_file</code> .
<i>Courier italic</i>	Indicates a user-defined value in syntax, such as <code>write_file design_list</code>
Courier bold	Indicates user input—text you type verbatim—in examples, such as <code>prompt> write_file top</code>
Purple	• Within an example, indicates information of special interest. • Within a command-syntax section, indicates a default, such as <code>include_enclosing = true false</code>
[]	Denotes optional arguments in syntax, such as <code>write_file [-format fmt]</code>
...	Indicates that arguments can be repeated as many times as needed, such as <code>pin1 pin2 ... pinN</code> .
	Indicates a choice among alternatives, such as <code>low medium high</code>
\	Indicates a continuation of a command line.
/	Indicates levels of directory structure.
Bold	Indicates a graphical user interface (GUI) element that has an action associated with it.
Edit > Copy	Indicates a path to a menu command, such as opening the Edit menu and choosing Copy .
Ctrl+C	Indicates a keyboard combination, such as holding down the Ctrl key and pressing C.

Customer Support

Customer support is available through SolvNetPlus.

Accessing SolvNetPlus

The SolvNetPlus site includes a knowledge base of technical articles and answers to frequently asked questions about Synopsys tools. The SolvNetPlus site also gives you access to a wide range of Synopsys online services including software downloads, documentation, and technical support.

To access the SolvNetPlus site, go to the following address:

<https://solvnetplus.synopsys.com>

If prompted, enter your user name and password. If you do not have a Synopsys user name and password, follow the instructions to sign up for an account.

If you need help using the SolvNetPlus site, click REGISTRATION HELP in the top-right menu bar.

Contacting Customer Support

To contact Customer Support, go to <https://solvnetplus.synopsys.com>.

Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

About Analog and Digital Co-designing

This chapter provides an overview of the digital and analog co-designing flow using the Custom Compiler co-design assistants. This chapter also provides an overview of the menu commands that you would use for co-designing within the Custom Compiler environment.

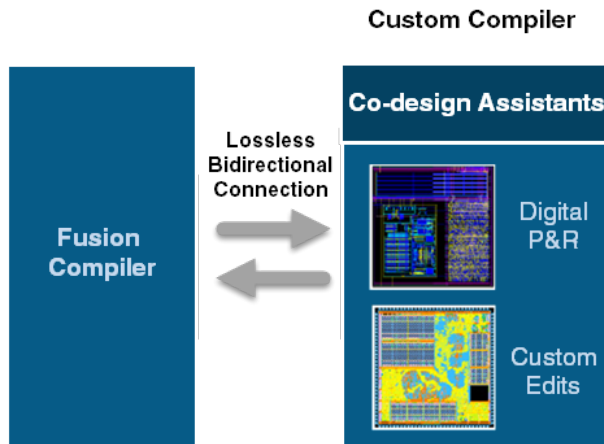
This chapter describes the following topics:

- [Analog and Digital Co-designing Using the Custom Compiler Tool](#)
- [Software User Interface](#)

Analog and Digital Co-designing Using the Custom Compiler Tool

You can use the Custom Compiler co-design assistants for digital and analog co-designing. The co-design features and flows facilitate seamless design roundtrips between the Custom Compiler and Fusion Compiler tools for the following:

- Adding digital blocks in mixed-signal custom designs
- Adding analog preroutes in digital designs
- Custom editing of digital designs



Within the Custom Compiler environment, you can:

- Design analog nets in a digital context in the Custom Compiler environment, creating preroutes before place and route.
- Create abstract (frame) views of the design in preparation for Fusion Compiler place and route processes.
- Run place-and-route using the Fusion Compiler tool.
- Open the Fusion Compiler design at any time during floorplanning, placement, or power-and-ground route creation and clock-tree synthesis.
- Perform Fusion Compiler operations directly.
- Custom edit the digital design.
- View Fusion Compiler DRC errors in the Custom Compiler environment.
- Report and fix DRC and electrical violations.
- Perform physical verification.

For instructions on how to create designs in the Fusion Compiler tool, see the available on SolvNetPlus.

Note:

You must be logged in to SolvNetPlus for the link to connect directly to the [Fusion Compiler user documentation](#). If you are prompted to log in to SolvNetPlus, log in, then click the [Fusion Compiler user documentation](#) link again to reach the help topics.

For information about the co-design features available in the Custom Compiler environment, see [Software User Interface](#).

Software User Interface

This topic describes the following:

- [Library Manager](#)
- [Schematic Editor and Layout Editor](#)
- [Edit Liberty Data Dialog Box](#)

For information on how to use the commands in co-design workflows, see [Getting Started](#).

Note:

To display the co-design menu and commands, you must first [enable the co-design assistant](#).

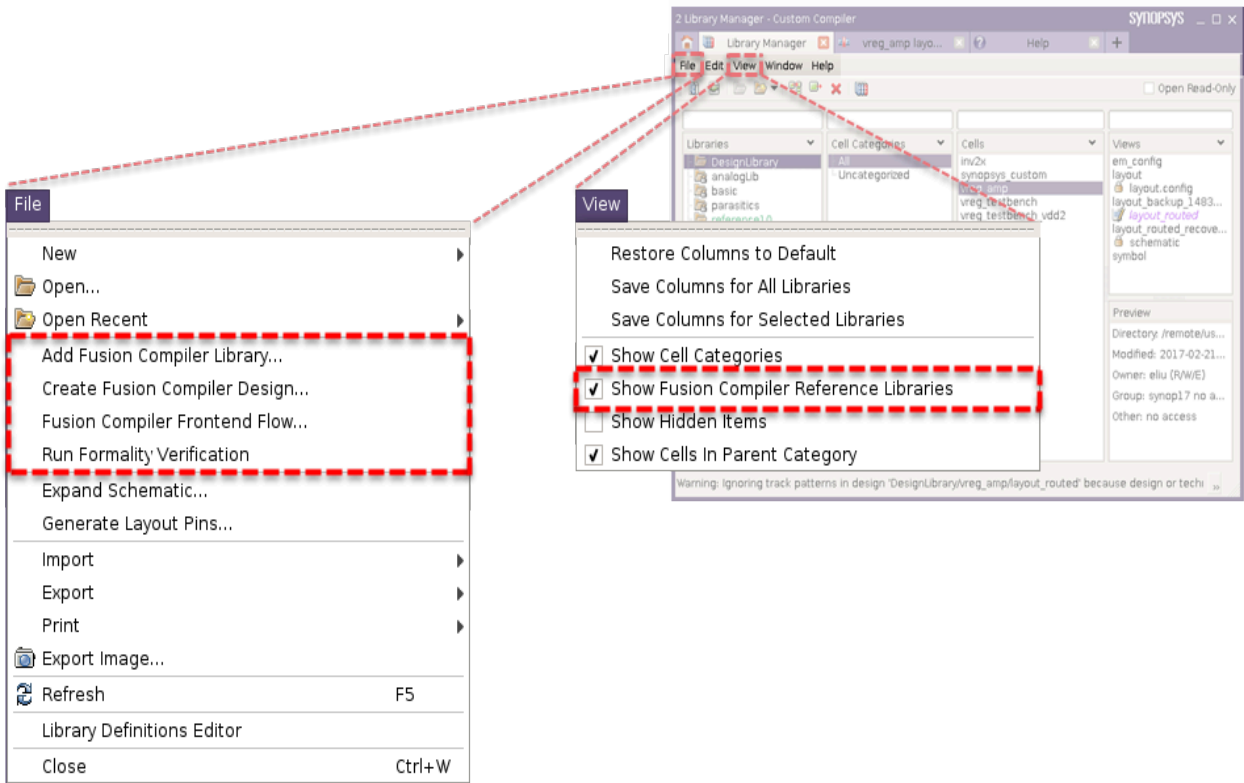
Library Manager

Before designing analog blocks or implementing digital blocks, you must use the Custom Compiler Library Manager to create or add the Fusion Compiler libraries in the Custom Compiler environment. The criteria for whether you create or add the Fusion Compiler libraries are described in [Preparing Design Libraries](#).

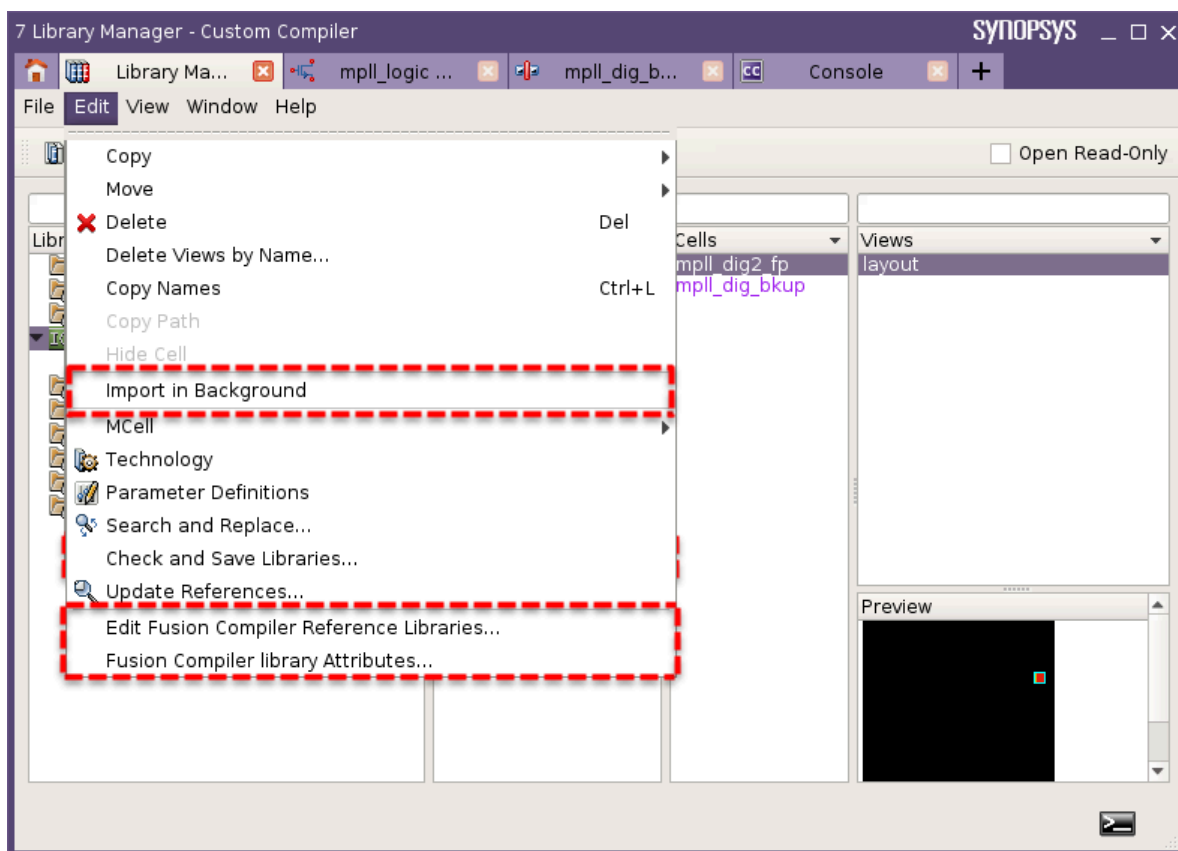
After you create or add the Fusion Compiler libraries in the Custom Compiler environment, you can use the Library Manager to view information about the designs. You can also open and manage libraries, cells, and views for the design process. For detailed information, see the *Custom Compiler Environment User Guide*.

Finally, you can set up and run the RTL-to-GDSII flow from the Library Manager. See [Running RTL-to-GDSII Co-design Flows](#).

The following Library Manager commands are available for co-designing processes:



Command	Description
File > Add Fusion Compiler Library	Opens existing Fusion Compiler libraries in the Custom Compiler environment. Tcl command: <code>dm::addFCLibrary</code>
File > Create Fusion Compiler Design	Creates new Fusion Compiler designs in the Custom Compiler environment.
File > Fusion Compiler Frontend Flow	Lets you set up and run the Fusion Compiler unified RTL-to-GDSII flows in the Custom Compiler environment. See Running RTL-to-GDSII Co-design Flows .
File > Run Formality Verification	Checks for modifications introduced in the design during development.
View > Show Fusion Compiler Reference Libraries	Displays the list of libraries in a tree structure. Using the tree structure, you can expand Fusion Compiler libraries to view the associated reference libraries. Tcl preference: <code>dmShowDigitalReferenceLibraries</code>



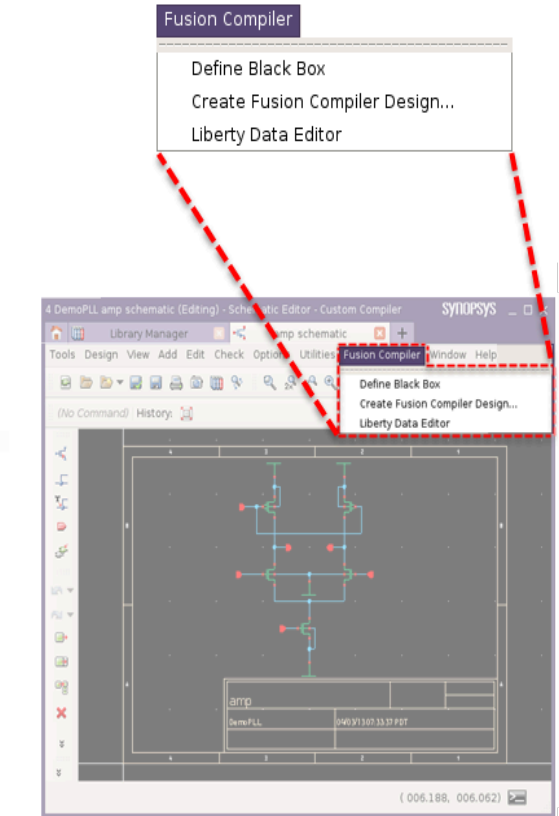
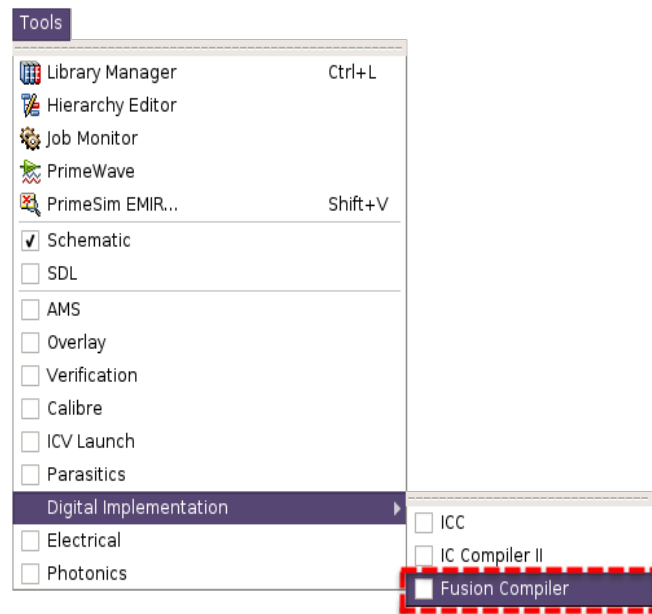
Command	Description
Edit > Import in Background	Imports the digital design data in the background when you add the Fusion Compiler design. If the digital design is greater than 1.2 GB, the digital design data takes 5 or more minutes to import into the Custom Compiler environment. Consider selecting this option to run the import process in the background. The option is grayed out (unavailable) if the digital design is already imported. Tcl preference: dmImportCellViewsInBackground
Edit > Edit Fusion Compiler Reference Libraries	Lets you edit the associated reference libraries. Tcl command: dm::showAddICC2Library
Edit > Fusion Compiler Library Attributes	Lets you view and edit the library attributes.

Other Library Manager commands are also available, such as Copy, Move, and Update References. You use these commands for general library operations.

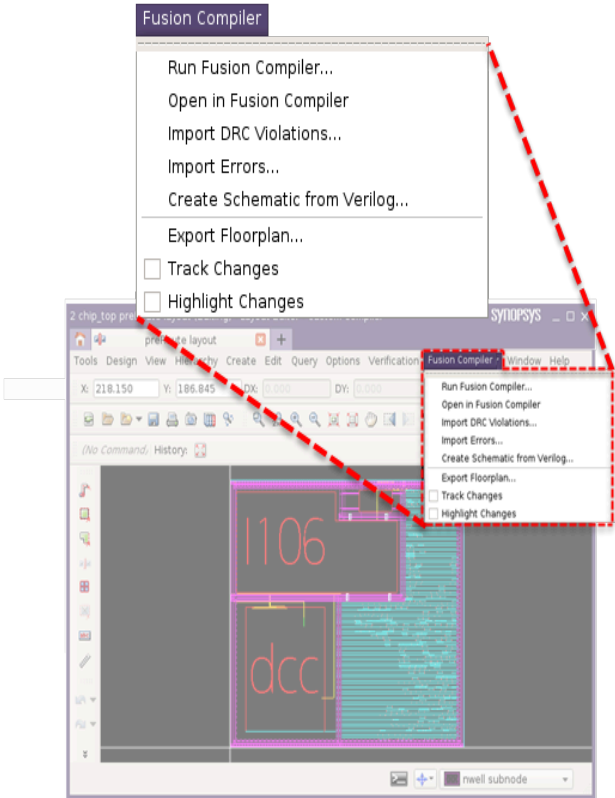
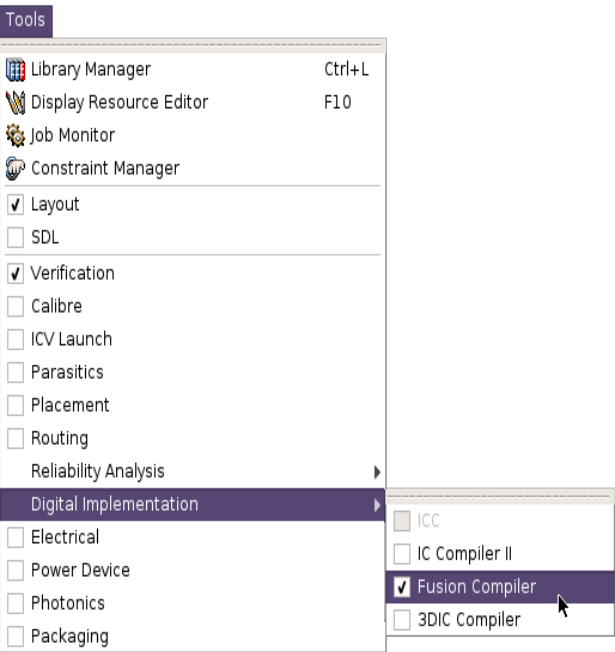
Schematic Editor and Layout Editor

You use the Custom Compiler Schematic Editor and Layout Editor when adding or editing components in mixed-signal designs. The following commands are available in the Schematic Editor and the Layout Editor:

Schematic Editor



Layout Editor



Command	Description
Tools > Digital Implementation > Fusion Compiler	Displays the Fusion Compiler menu on the Schematic Editor and the Layout Editor menu bars.
Fusion Compiler > Define Black Box	(Schematic Editor only) Displays the Define Black Box or the Edit Black Box dialog box that allows you to create digital timing shells for analog boxes. See Creating Black Boxes for Analog Blocks .
Fusion Compiler > Create Fusion Compiler Design	(Schematic Editor only) Displays the Create Fusion Compiler Design dialog box that allows you to specify logic sources, power and ground nets, and the output Fusion Compiler cells. For information, see Developing Digital Blocks from Schematics .
Fusion Compiler > Liberty Editor	(Schematic Editor only) Displays the Edit Liberty Data dialog box that allows you to view and define settings and values for a Liberty library file. See Edit Liberty Data Dialog Box .

Command	Description
Fusion Compiler > Run Fusion Compiler	(Layout Editor only) Displays the Run Fusion Compiler dialog box that allows you to specify the Tcl scripts to run.
Fusion Compiler > Open in Fusion Compiler	(Layout Editor only) Opens the current design in the Fusion Compiler tool without leaving the Custom Compiler environment. During this process, the design is changed to the read-only mode in the Custom Compiler environment. See Starting the Fusion Compiler Tool .
Fusion Compiler > Import DRC Violations	(Layout Editor only) Displays the Import DRC Violations from Fusion Compiler dialog box that allows you to import Fusion Compiler DRC data as OpenAccess DRC markers. See Importing DRC Data From the Fusion Compiler Tool .
Fusion Compiler > Import Errors	(Layout Editor only) Displays the Import Errors from Fusion Compiler dialog box that allows you to import additional Fusion Compiler error data. See Importing Error Data From the Fusion Compiler Tool .
Create Schematic from Verilog	(Layout Editor only) Displays the Import Fusion Compiler Verilog into OA dialog box, which lets you create schematics from Fusion Compiler Verilog files for layout-vs-schematic (LVS) comparisons in the Custom Compiler environment. See Creating Schematics from Verilog Files .
Fusion Compiler > Create Frame (not shown)	(Layout Editor in Custom Compiler designs only) Generates Fusion Compiler frame views from the current design.
Export Floorplan	See Exporting Preroute Floorplans to Tcl Files .
Track Changes	See Turning on Change-Tracking Features .
Highlight Changes	See About Highlight Changes .

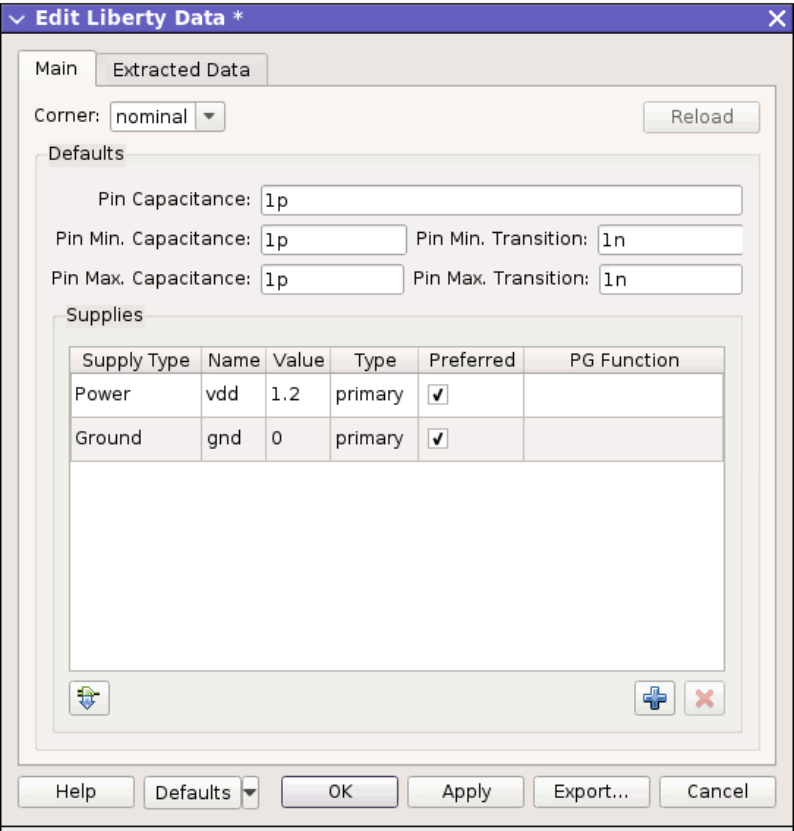
See also Schematic Editor Description and Layout Editor Description.

Edit Liberty Data Dialog Box

The **Edit Liberty Data** dialog box consists of the following tabs:

- [Main Tab](#)
- [Extracted Data Tab](#)

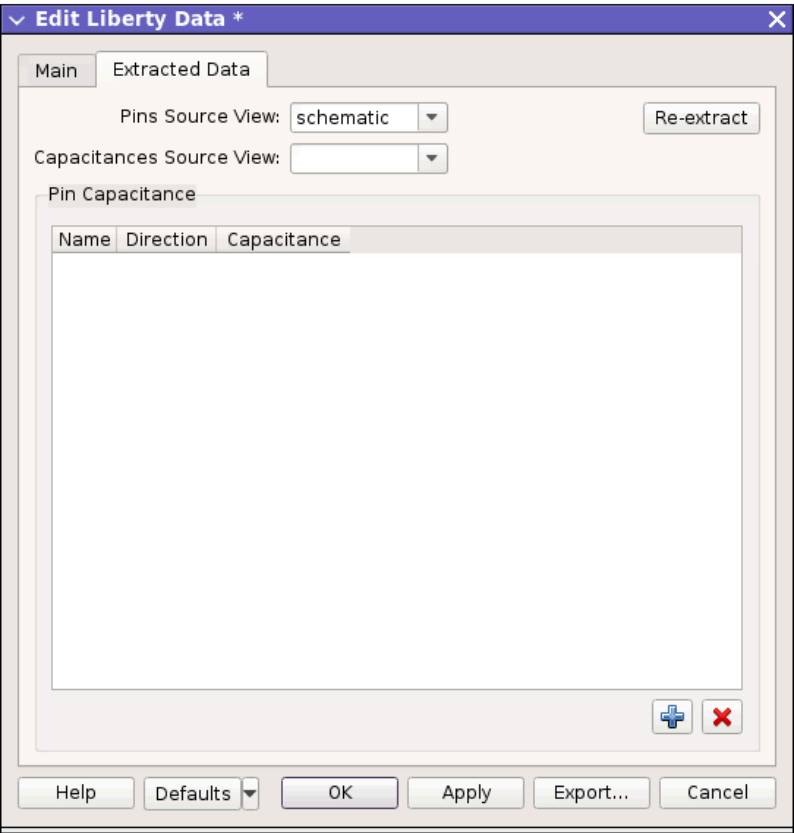
Main Tab



Option	Description
Corner	Characterization corner. Default: nominal
Reload	Reloads the corner data to revert changes. The button is only enabled when there is stored corner data in the design.
Defaults	
Pin Capacitance	Capacitance of the pin.
Pin Min. Capacitance	Minimum capacitance of the pin.
Pin Min. Transition	Minimum transition of the output pin.
Pin Max Capacitance	Maximum capacitance of the pin.
Pin Max Transition	Maximum transition of the output pin.
Supplies	

Option	Description
Supply Type	Type of power supply: power and ground.
Name	Name of the net.
Value	Power voltage value.
Type	Type of power supply: primary or backup.
Preferred	Indicator that the supply is preferred.
PG Function	Description of how the power and ground is used.
	Opens the Copy Supply Pins dialog box and lets you specify the cellview from which to copy the pins.
	Adds a new power supply to the Supplies table.
	Deletes the selected power supply from the Supplies table.

Extracted Data Tab



Option	Description
Pins Source View	View of the pin source.
Capacitance Source View	View of the capacitance source.
Re-extract	Reruns extraction.
Pin Capacitance	
Name	Net name of the pin.
Direction	Direction of signal.
Capacitance	Capacitance value.

2

Getting Started

This chapter presents the two main workflows you can use when co-designing.

This chapter describes the following topics:

- [Workflows for Custom Compiler and Fusion Compiler Co-designing](#)
- [Starting Custom Compiler Sessions and Displaying the Library Manager](#)
- [Enabling the Co-design Assistant](#)
- [Custom Compiler Products and Tools](#)

Workflows for Custom Compiler and Fusion Compiler Co-designing

The Custom Compiler and Fusion Compiler Co-design assistant supports different workflows to accommodate the co-design process.

This topic describes the following:

- [Workflow for RTL-to-GDSII Co-designing](#)
- [Workflow for Preroute Editing](#)
- [Workflow for Developing and Integrating Digital Blocks](#)

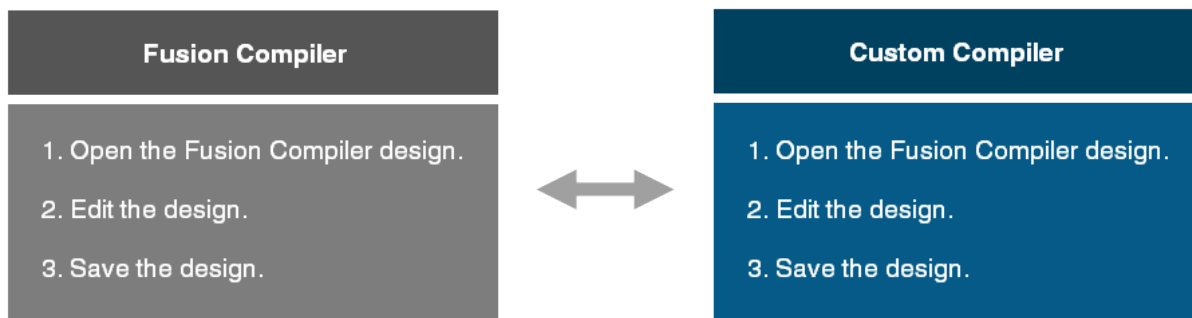
Workflow for RTL-to-GDSII Co-designing

The Fusion Compiler RTL-to-GDSII flow is a design solution that merges several frontend steps to increase productivity. With the RTL-to-GDSII interface, you have the option of running either the unified RTL-to-GDSII flow (with merged steps) or the non-unified traditional flow.

For details, see [Workflow for Co-designing in Unified RTL-to-GDSII Flows](#).

Workflow for Preroute Editing

One method of co-designing is to edit an Fusion Compiler design through roundtrips between the Fusion Compiler and Custom Compiler environment, as the following diagram shows.



Use this method if you have a predominantly digital design and you want to:

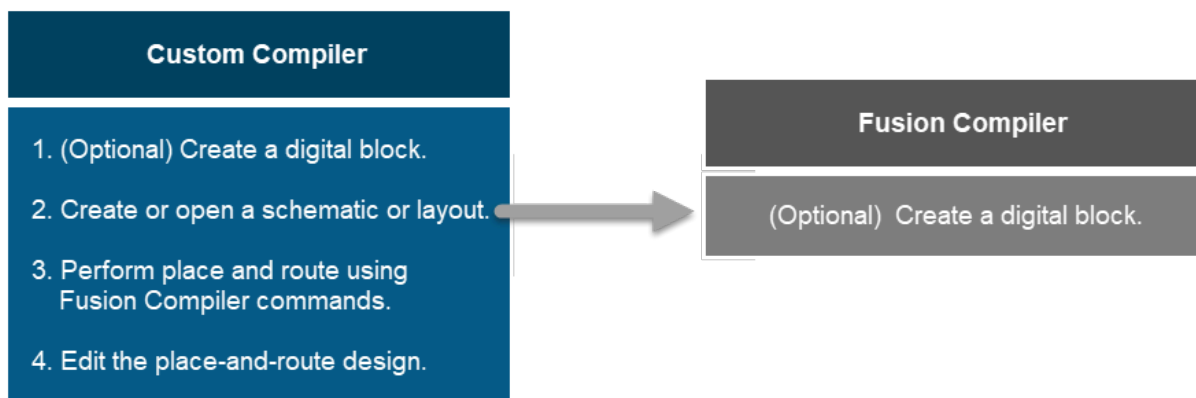
- Add analog components, such as preroutes.
- Make custom edits.

The basic workflow is as follows:

Task	Product	Description
1	Fusion Compiler	Create a digital design. See the Fusion Compiler user documentation available on SolvNet.
2	Custom Compiler	Edit the design. For example, you can add preroutes. See Preroute Editing (Roundtrip Flow) .
3	Fusion Compiler	Continue to edit the design. For example, you can route clock nets or run place and route. See the Fusion Compiler user documentation available on SolvNet.
4	—	Repeat steps 2 and 3 as needed. For example, you can repeat step 2 for custom editing.
5	Custom Compiler	Open and review the completed design.

Workflow for Developing and Integrating Digital Blocks

Another method of co-designing is to stay completely or mostly within the Custom Compiler environment throughout the co-design process, as the following diagram shows.



Use this method if you have an existing Custom Compiler mixed-signal design and you want to do one or both of the following:

- Develop a small digital block.
- Implement the digital block in the design. This task includes placing and routing the block, and then integrating it in the mixed-signal design.

You have the option of using a Verilog file, design schematic, or design layout as input for the place-and-route process.

The basic workflow for the digital place and route is as follows:

Task	Product	Description
1	Fusion Compiler	(Optional) Develop a digital block. See the Fusion Compiler user documentation available on SolvNet.
2	Custom Compiler	Prepare the design for co-designing. See Preparing the Technology Data .
3	Custom Compiler	(If the digital block does not exist) Run place and route using Fusion Compiler commands to create the digital block. The place-and-route input can be from one of the following: • Verilog file • Schematic • Layout The schematic-based place-and-route process lets you determine the size of the block to be integrated. You can use this information for floorplanning. See Developing and Integrating Digital Blocks .

Task	Product	Description
4	Custom Compiler	(Optional) In the Layout Editor, add floorplanning components, such as the boundary within which the block will be placed, power and ground routes for prerouting instances, and pin placement to specify pin locations. See About the Layout Editor and About the Layout Editor Connectivity Features.
5	Custom Compiler	Run place and route using the floorplanned layout you created in step 4. See Developing and Integrating Digital Blocks .
6	Custom Compiler	Edit and clean up the design. See About the Layout Editor and About the Layout Editor Connectivity Features.

Starting Custom Compiler Sessions and Displaying the Library Manager

To route the packaging design in the Custom Compiler environment, you must start a Custom Compiler session, and display the Library Manager. The Library Manager allows you to view and select the design you want to open.

1. Start a Custom Compiler session. See [Starting Sessions](#).
2. Display the Library Manager. See [Displaying the Library Manager](#).

Enabling the Co-design Assistant

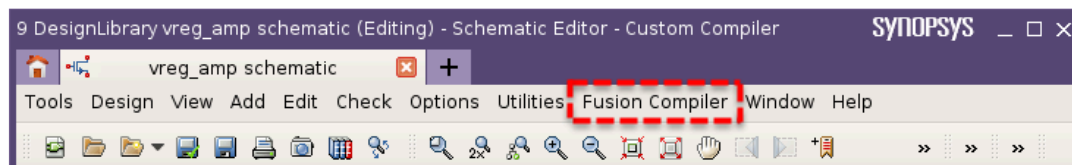
After you open a design in the Custom Compiler environment, enable the co-design assistant as follows:

1. In the Custom Compiler Console, set the following preference:

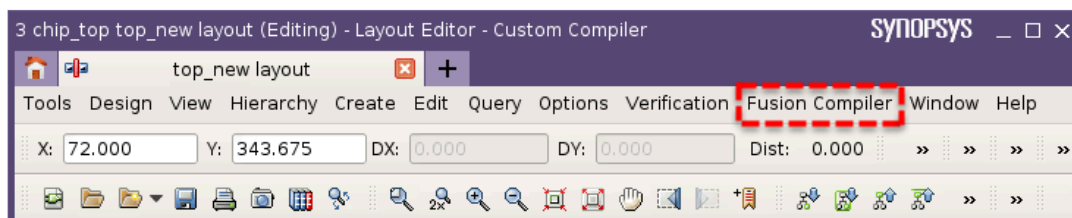
```
db::setPrefValue xtDigitalImplementationTool -value fc
```
2. In the Schematic Editor or the Layout Editor, choose **Tools > Digital Implementation > Fusion Compiler**.

The **Fusion Compiler** menu appears on the design editor menu bar.

Schematic Editor



Layout Editor



See [Software User Interface](#) for descriptions of the co-design menu commands and dialog boxes.

Custom Compiler Products and Tools

The Custom Compiler environment offers a variety of products and tools you can use when creating and editing mixed-signal designs. The following table lists some of the typical tasks and the Custom Compiler products and features you might use when co-designing. For the comprehensive list of available products and tools, see Design Phases and Products to Use in the *Custom Compiler Environment User Guide*.

Task	Custom Compiler Product or Tool
View nets, instances, terminals	Design Navigator Assistant
Set design-specific constraints	Constraint Editor Assistant
View and adjust design component properties, such as route guides	Property Editor Assistant
Add routes	Custom Router (automatically or interactively) Layout Editor (manually)
Add shielding	Auto Route Shield Tab and Adding Post-Route Shielding
Check and fix design errors	DRD tool Via Checker Metal Density Checker Color Decomposition Automation
Run electrical reports and checks	Electrical Reporter, Shield Coverage Reporter
Physical verification	Synopsys IC Validator Synopsys Hercules Mentor Graphics Calibre and Calibre Interactive StarRC SPRES Viewer Reliability Analysis feature

3

Preparing Design Libraries

Before you start the co-design flow, make sure you are using the relevant foundry constraints. You also need to prepare the design libraries in the Custom Compiler environment.

When working with digital designs in the Custom Compiler environment, you need to ensure that digital rules are applied. Use the `leTechFoundryConstraintGroups Tcl` preference to specify constraint groups that contain digital-specific rules. See [Enabling Alternate Foundry Constraints](#).

So that the modifications you make are stored in the Fusion Compiler libraries, the Fusion Compiler libraries must be created or added in the Custom Compiler environment. The criteria for whether you create or add the Fusion Compiler libraries are as follows:

If the design library...	See...	Example
Does not exist in the Fusion Compiler environment	Creating Fusion Compiler Libraries	Your design originated in the Custom Compiler environment, and you want to transfer it to the Fusion Compiler tool for digital place and route.
Exists in the Fusion Compiler environment	• Adding Fusion Compiler Libraries • Viewing and Editing Library Attributes • Viewing Associated Reference Libraries • Editing Reference Library Associations	Your design originated in the Fusion Compiler tool and you want to develop it further in the Custom Compiler environment.

Creating Fusion Compiler Libraries

So that the place-and-route results are stored in the Fusion Compiler libraries, you must add the Fusion Compiler libraries in the Custom Compiler environment. This topic describes how to use the Custom Compiler [Library Manager](#) to create the Fusion Compiler libraries.

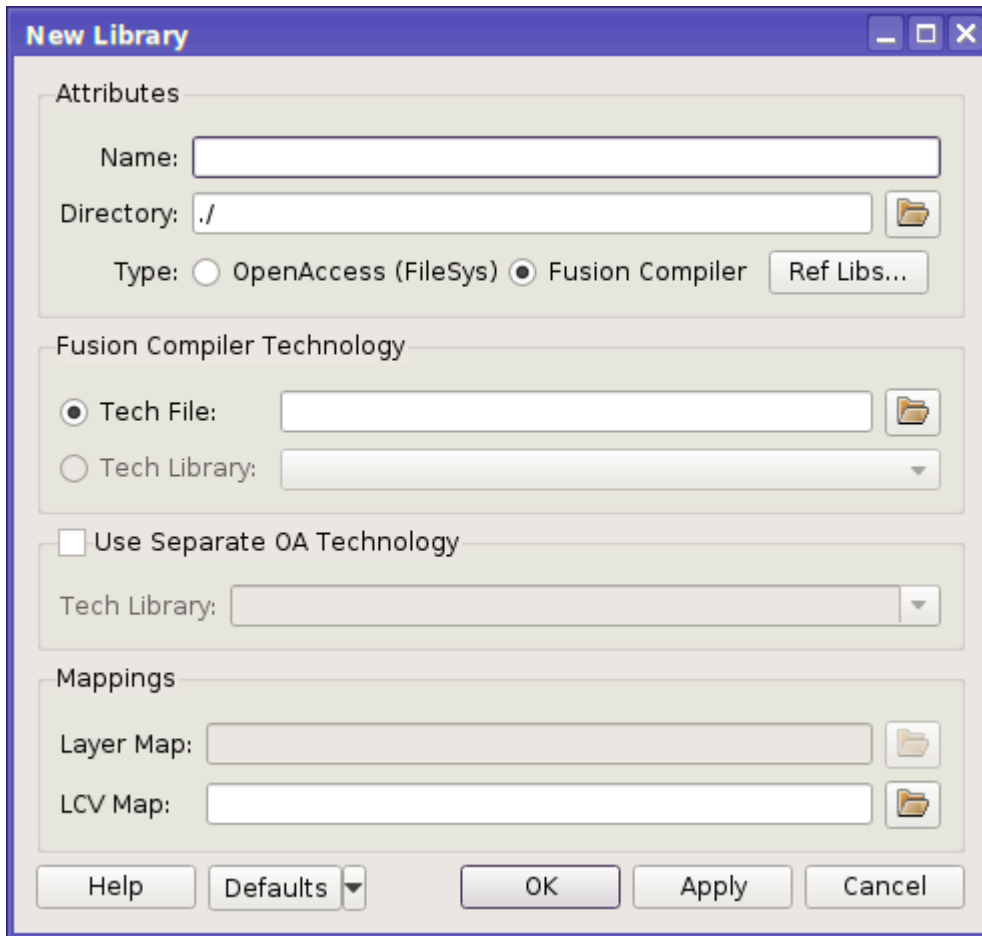
Note:

To ensure that the design uses the same scale in the Fusion Compiler and the Custom Compiler environments, set the `dmICC2ScaleFactor` Tcl preference before you create the Fusion Compiler library. Only one scale factor may be set for all Fusion Compiler libraries in a session.

When you create an Fusion Compiler library in the Custom Compiler environment, the Library Manager creates a `display.tcl` file for the layer-purpose pairs defined in the Fusion Compiler library. If a `display.tcl` file already exists, the Library Manager checks the `dmICC2TechDisplayHandling` Tcl preference setting to determine what to do: merge the data, overwrite the existing data, or keep the existing and ignore the new data. By default, the Library Manager merges the data. If you want to overwrite or keep the existing data, set the preference before you create the library.

To create Fusion Compiler libraries in the Custom Compiler environment:

1. Start a Custom Compiler session and display the Library Manager. See [Starting Custom Compiler Sessions and Displaying the Library Manager](#) and Displaying the Library Manager.
2. In the Library Manager, choose **File > New > Library**. The **New Library** dialog box opens.

The image shows a 'New Library' dialog box with a purple title bar. It contains several sections: 'Attributes' with fields for 'Name', 'Directory' (set to './'), and 'Type' (radio buttons for 'OpenAccess (FileSys)' and 'Fusion Compiler', with a 'Ref Libs...' button); 'Fusion Compiler Technology' with radio buttons for 'Tech File' and 'Tech Library' (a dropdown), and a checkbox for 'Use Separate OA Technology' with its own 'Tech Library' dropdown; and 'Mappings' with fields for 'Layer Map' and 'LCV Map'. At the bottom are 'Help', 'Defaults' (with a dropdown arrow), 'OK', 'Apply', and 'Cancel' buttons.

3. Specify the **Name** and **Directory** of the new library.
4. Select the library **Type: Fusion Compiler**, click **Ref Libs** to open the **Specify Reference Libraries** dialog box, select the reference libraries, and click **OK** to return to the New Library dialog box.

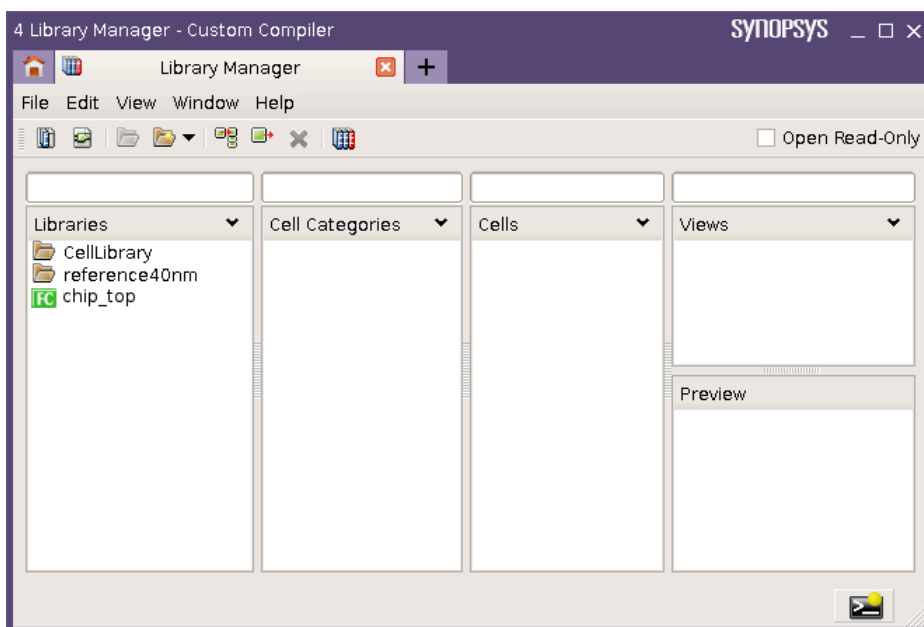
If you want to select reference libraries from other directories that are not listed in the current session, click **Add Library** in the **Specify Reference Libraries** dialog box.

5. Specify the **Fusion Compiler Tech File** and **Tech Library** you want to use.

6. Select **Use Separate OA Technology** if you want to use the selected OA technology library, and specify the following:

Settings	Description
Layer Map	(Optional) The layer map file to use to map OA layers to Fusion Compiler layers. Specify the layer map if the technology file is not available. Tcl preference: dmICC2OALayerMap
LCV Map	(Optional) The library, cell, view map file to use to map OA library, cell, view to Fusion Compiler library, cell, and view. Tcl preference: dmICC2OALibCellViewMap

7. When you are finished, click **OK**. The newly created library name appears in the Library Manager. The green **FC** icon in front of the library name indicates that it is an Fusion Compiler library.



See Also

- [Adding Fusion Compiler Libraries](#)
- [Deleting Fusion Compiler Libraries](#)

Adding Fusion Compiler Libraries

So that the place-and-route results are stored in the Fusion Compiler libraries, you must add the Fusion Compiler libraries in the Custom Compiler environment. This topic describes how to use the Custom Compiler [Library Manager](#) to add the Fusion Compiler libraries.

Before You Start

Before you start to add Fusion Compiler libraries in the Custom Compiler environment, make sure you review the prerequisites and other considerations for optimal results.

Important:

The Fusion Compiler library you are adding must already exist in the Fusion Compiler tool.

Add Fusion Compiler Library uses the mapping file that you specified with the `dmICC2OALayerMap` Tcl preference. See [Preparing the Technology Data](#).

When you add Fusion Compiler libraries in the Custom Compiler environment, the Library Manager creates a `display.tcl` file for the layer-purpose pairs defined in the Fusion Compiler library. If a `display.tcl` file already exists, the Library Manager checks the `dmICC2TechDisplayHandling` Tcl preference setting to determine what to do: merge the data, overwrite the existing data, or keep the existing and ignore the new data. By default, the Library Manager merges the data. If you want to overwrite or keep the existing data, set the preference before you create the library.

When adding digital libraries in the Custom Compiler environment, if the digital reference library path uses symbolic links, then the reference library mapping would use the symbolic links for reference libraries and instance binding. For example:

- `mp11_block.ndm` points to `/remote/my_directory/mkndm/mp11_block/rel_mp11.nlib`.
- `mp11_merge.ndm` points to `/remote/my_directory//mkndm/mp11_merge/rel_mp11.nlib`.

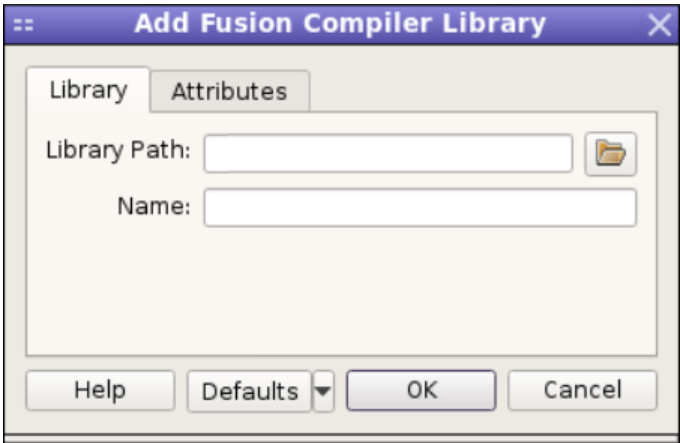
Because the `top.ndm` digital library uses `mp11_block.ndm` and `mp11_merge.ndm` as reference libraries, adding `top.ndm` in the Custom Compiler environment results in references to the directories in `mp11_block.ndm` and `mp11_merge.ndm`.

Procedure

 [Show me](#) an example of adding Fusion Compiler libraries. Although the video shows working with an IC Compiler II design, the example is also applicable for Fusion Compiler designs.

To add Fusion Compiler libraries to the Custom Compiler environment:

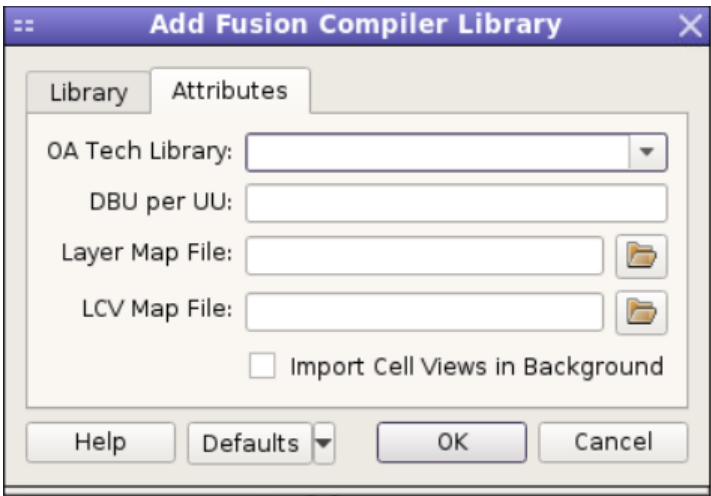
1. Start a Custom Compiler session and display the Library Manager. See [Starting Custom Compiler Sessions and Displaying the Library Manager](#).
2. In the Library Manager, choose **File > Add Fusion Compiler Library**. The **Add Fusion Compiler Library** dialog box opens.



3. Provide the following information:

Setting	Description
Library Path	The directory or file that contains the Fusion Compiler library.
Name	The new name of the Fusion Compiler library in the Custom Compiler environment.

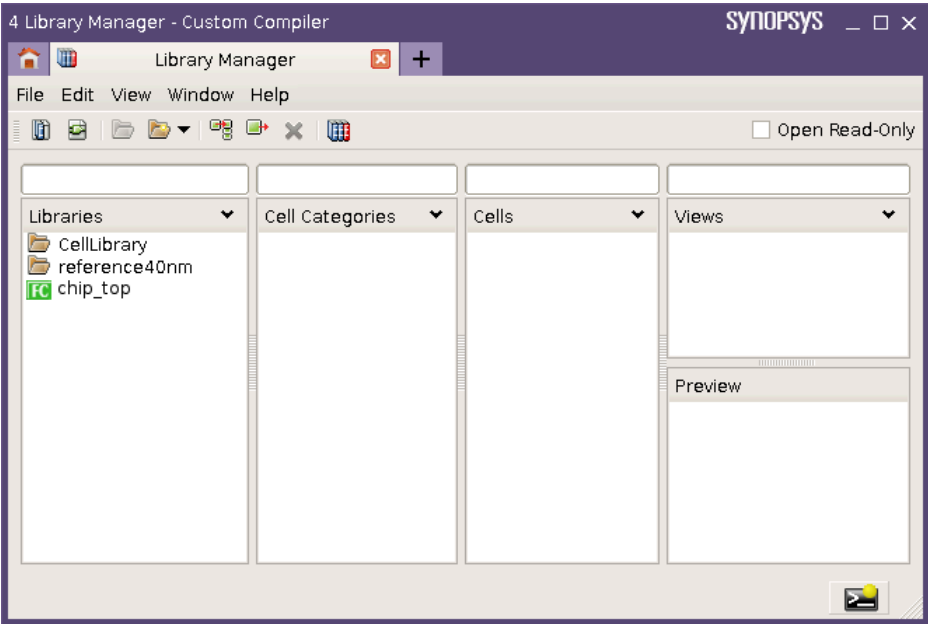
4. Click the **Attributes** tab, and provide the following information:



Setting	Description
OA Tech Library	OA (Custom Compiler) technology library. Mixed-signal designs use OA technology libraries. To ensure seamless co-designing, specify the OA technology library to use when in the Custom Compiler environment instead of the Fusion Compiler library. In cases where you have large numbers of vias to map, you can set up an incremental technology library. See Setting Up the Technology Libraries.
DBUPerUU	Design scale factor. Mixed-signal designs use technology data that do not have the same DBUPerUU as in the Fusion Compiler tool. Therefore, you must adjust the Fusion Compiler design units to ensure smooth top-level integration within the Custom Compiler environment. Synopsys recommends that you use the same DBUPerUU for the PDK and Fusion Compiler technology libraries. To scale the design units, set DBUPerUU to the target PDK resolution. See Adjusting the Design Scale for examples. For information about setting scale factors for designs in the Fusion Compiler tool, see the Fusion Compiler create_lib command.
Layer Map File	Layer mapping file. Mixed-signal designs use OA technology, but the OA (Custom Compiler) technology layer names are not equivalent to the Fusion Compiler technology layer names. Therefore, you must map Fusion Compiler layer names to OA layer names so that you can edit and view digital blocks in the context of mixed-signal designs. You must first prepare the layer mapping file. See Mapping Technology Layer Names. Then, set this option to reference the file.
LCV Map File	Library and cell name mapping file. When working with digital components in the Custom Compiler environment, the digital library and cell names must be mapped to Custom Compiler (OA) library and cell names to ensure co-design compatibility and seamless top-level integration. You must first prepare the library and cell name mapping file. See Mapping Libraries, Cells, and Views. Then, set this option to reference the file.

Setting	Description
Import Cell Views in Background	Imports all cellviews in the background when you add the Fusion Compiler design. If the digital design is greater than 1.2 GB, the co-design assistant displays a message and suggests that you use this command to import the digital design data in the background. If the import operation is in progress, this command changes to Stop Background Import so that you can halt the import process if needed. The command is unavailable if the digital design is already imported. The equivalent command, Import in Background, is available in the shortcut menu when you right-click a Library, Cell, or View name in the Library Manager. To monitor the progress of background import jobs, see Monitoring the Import and Export Progress. Tcl preference: dmImportCellViewsInBackground

5. When you are finished, click **OK**. The new library name appears in the Library Manager. The **FC** icon in front of the library name indicates that it is a Fusion Compiler library.



See Also

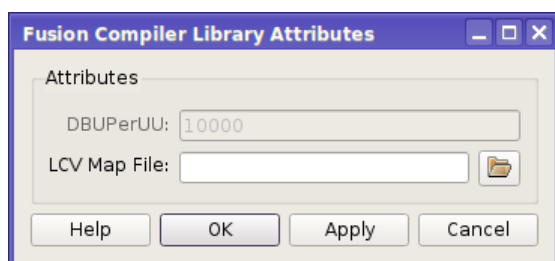
- [Creating Fusion Compiler Libraries](#)
- [Deleting Fusion Compiler Libraries](#)

Viewing and Editing Library Attributes

After you add an Fusion Compiler library, you can view the library attributes and specify a different library and cell mapping file.

To view and edit Fusion Compiler library attributes in the Custom Compiler environment:

1. In the Library Manager, right-click the Fusion Compiler library name and choose **Fusion Compiler Library Attributes**. The **Fusion Compiler Library Attributes** dialog box opens.

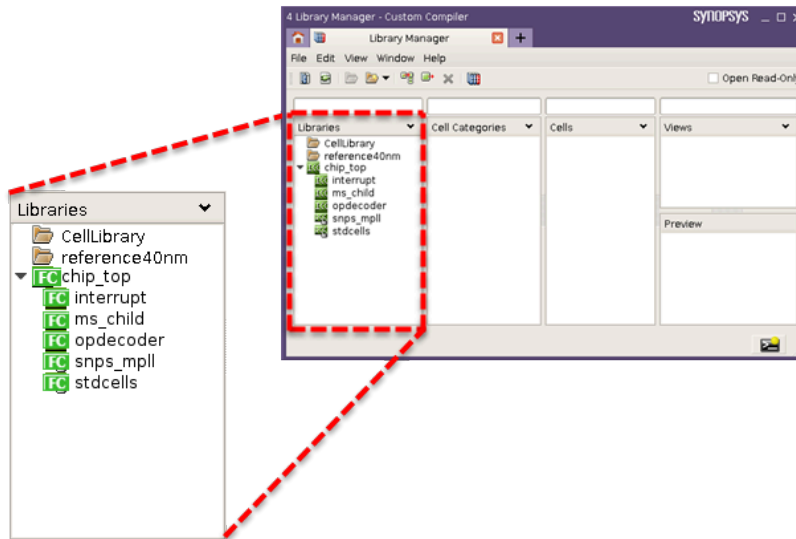


2. Review the **DBUPerUU** information and the **LCV Map File**. The DBRPerUU information cannot be changed.
3. (Optional) Specify a different **LCV Map File**. See [Mapping Libraries, Cells, and Views](#).
4. When you are finished, click **OK** to apply the changes and close the dialog box.

Viewing Associated Reference Libraries

For the Fusion Compiler library you just created or added in the Custom Compiler environment, you can view the associated reference libraries. To do this, in the Library Manager, choose **View > Show Fusion Compiler Reference Libraries**.

The list of Fusion Compiler libraries change to a tree structure. To view the referenced libraries, click the Fusion Compiler library name.



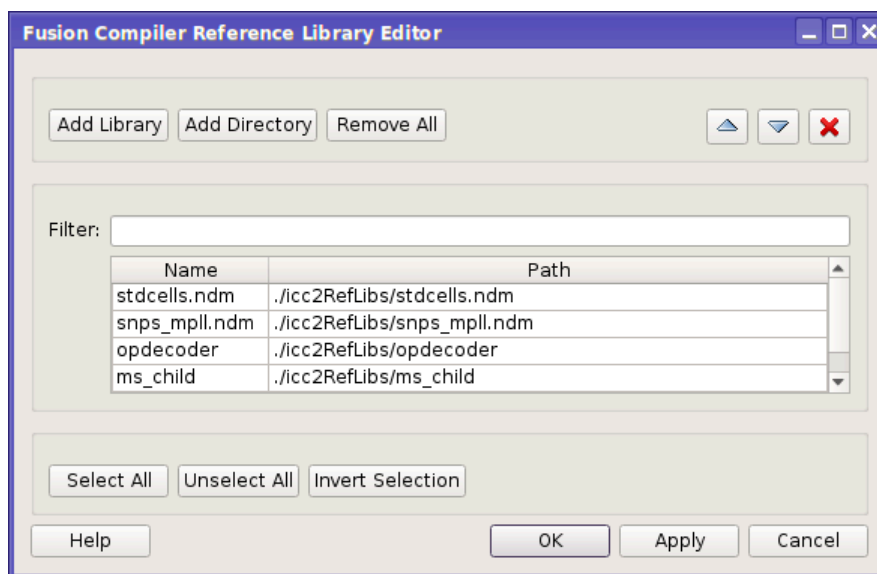
Note the following:

- If two Fusion Compiler libraries are associated with the same referenced library (same path, same name), only one referenced library is added.
- If a name conflict occurs (different paths, same name), the newly added reference library name will have a suffix ($_n$) in its name, where n is an integer greater than 0.
- All libraries that are associated with the same referenced library must use the same technology database, layer mapping, and via mapping.

Editing Reference Library Associations

To edit the Fusion Compiler reference library associations:

1. In the Library Manager, select the Fusion Compiler library.
2. Choose **Edit > Edit Fusion Compiler Reference Libraries**. The **Fusion Compiler Reference Library Editor** dialog box opens.



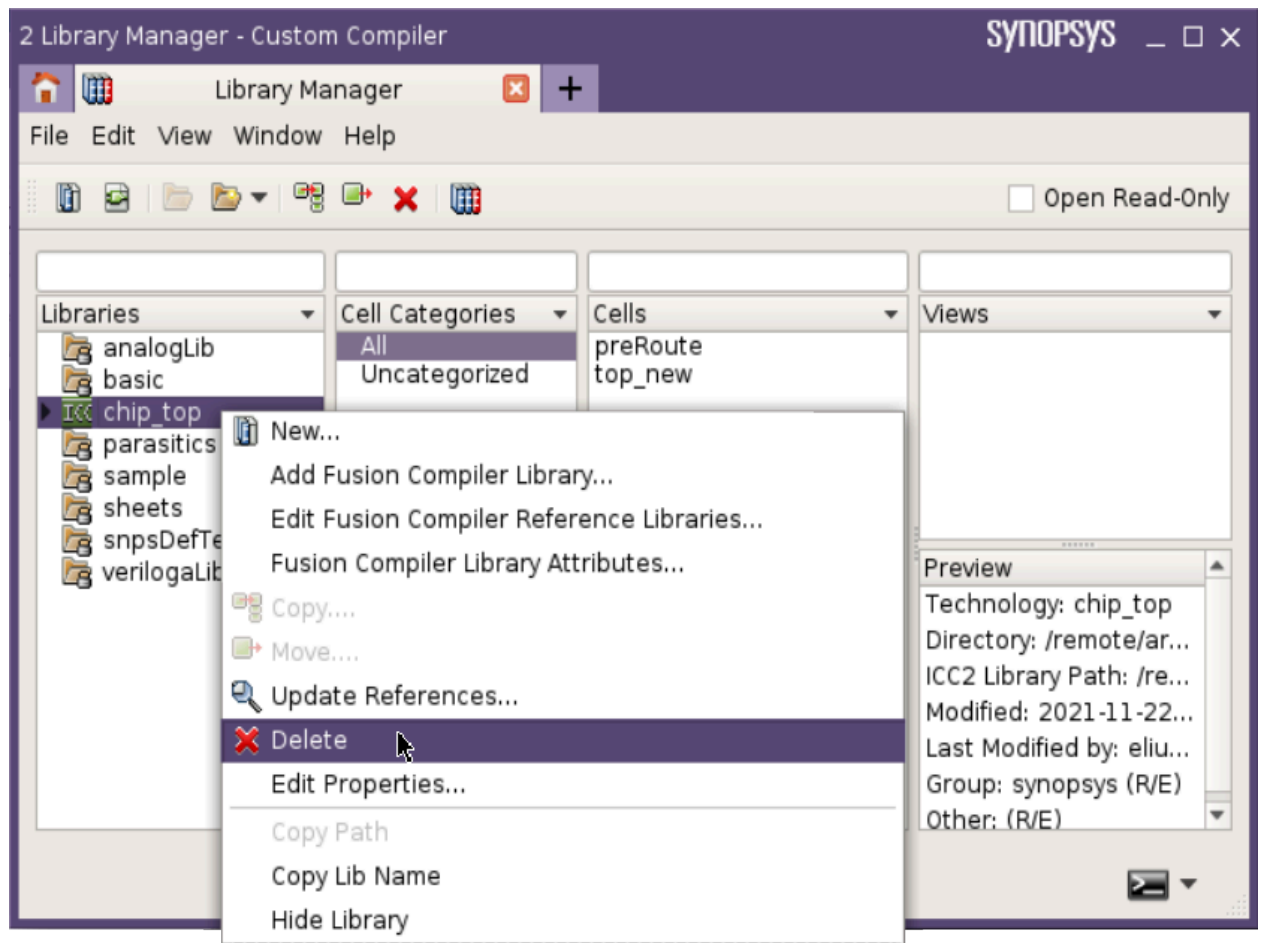
3. To add one or more reference libraries:
 - Click **Add Library** to select and add a single reference library.
 - Click **Add Directory** to add all reference libraries in the directory you select.
4. To change the position of a reference library in the list, select the reference library and click  or .
5. To select one or more reference libraries, click one of the following:
 - **Select All:** Selects all reference libraries in the list.
 - **Unselect All:** Cancels the selection of reference libraries.
 - **Invert Selection:** Toggles the selection so that the libraries that were selected are no longer selected, and libraries that were not selected are now selected.
6. To remove a reference library from the list, select the reference library and click .
7. To remove all reference libraries from the list, click **Remove All**.
8. When you are finished, click **OK**.

Deleting Fusion Compiler Libraries

You can delete Fusion Compiler libraries in the Custom Compiler environment. This topic describes how to use the Custom Compiler [Library Manager](#) to delete Fusion Compiler libraries.

To delete Fusion Compiler libraries in the Custom Compiler environment:

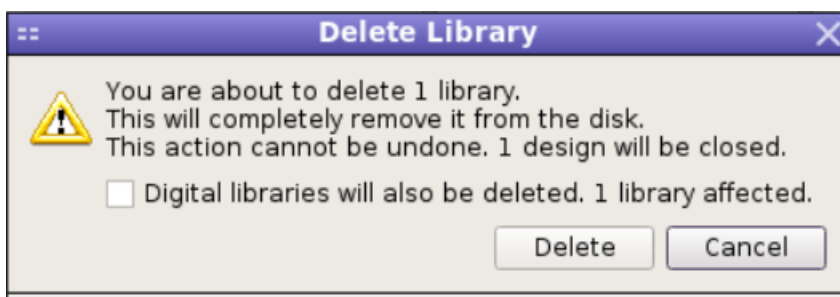
1. Display the Library Manager.
2. In the Library Manager, right-click the Fusion Compiler library you want to delete, and choose **Delete**.



3. In the **Delete Library** dialog box, select **Digital libraries will also be deleted** if you want to delete the specified Fusion Compiler library in addition to deleting its cache directory, which contains the lib.defs entry.

By default, the check box is not selected.

Tcl preference: `dmRemoveDigitalLibs`



4. Click **Delete**.

See Also

- [Creating Fusion Compiler Libraries](#)
- [Adding Fusion Compiler Libraries](#)

Monitoring the Import and Export Progress

Because Fusion Compiler designs can be large, the import and export processes can take some time. You can use the Job Monitor to track the progress of the processes. To display the Job Monitor, see the *Custom Compiler Environment User Guide*.

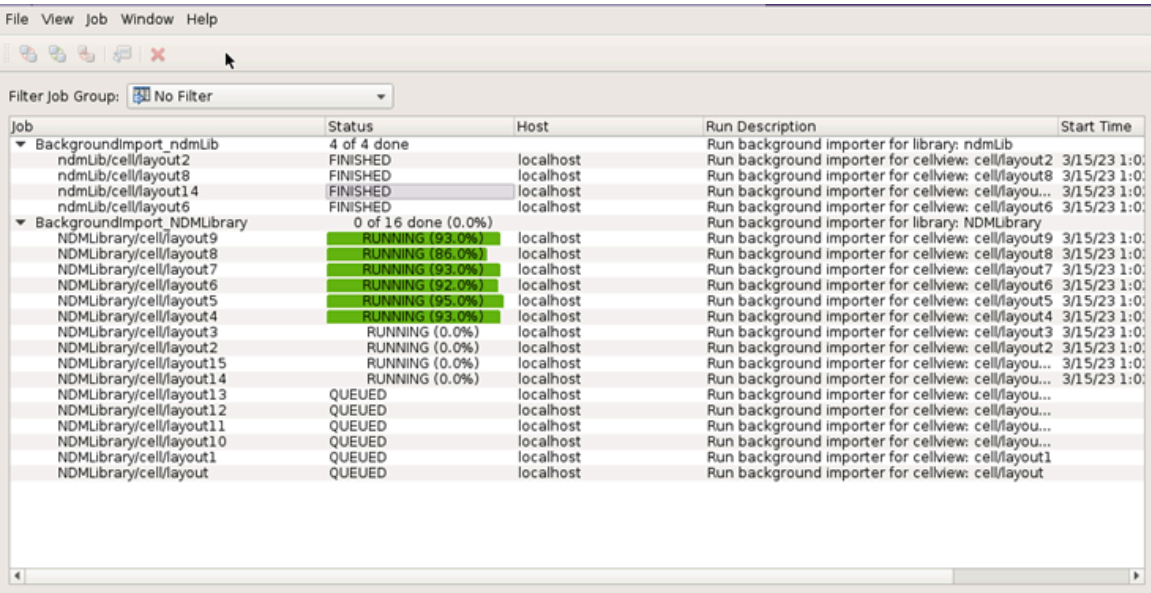
Running number of background import jobs depends on the following factors:

- Maximum number of background import jobs allowed to run.
- Sum of size of NDM design files.

Use the following preferences to set the limit for running background import job processes:

- `dmAllowedCountOfBackgroundImporters`: Limits the number of background import jobs to run. For example, if you set the number to 10, then the maximum number of background import jobs running can be less than or equal to 10.
- `dmAllowedSizeOfFilesGB`: Limits the sum of file sizes (GB) of NDM designs for which the background import jobs are running. If the sum of sizes is greater than the preference value, the next run of the background import job waits until one of the running job is finished.

During the import and export process, the Job Monitor displays the following information:



Status Information	Description
Job name	The name of the job that is currently running.
Status text	The text in the Status column indicates the design being imported or exported and the object that is currently being processed.
Green progress bar	Overlaying the status text is a green progress bar that indicates the overall progress of the job.
Blue line beneath the green progress bar	At the bottom of the green progress bar is a thin blue line that indicates the progress of the current phase of the translation.
Tooltip	If you rest the cursor on the progress bar, a tooltip appears and displays the percent completion for the overall import or export operation and the percent completion for the current object being processed.

At the end of the operation, the Status column lets you know if the job is finished or it has failed. For information about the common status messages, see the *Custom Compiler Environment User Guide*.

4

Running RTL-to-GDSII Co-design Flows

This chapter explains how to run Fusion Compiler unified RTL-to-GDSII flows in the Custom Compiler environment.

This chapter describes the following:

- [Co-designing in Unified RTL-to-GDSII Flows](#)
- [Workflow for Co-designing in Unified RTL-to-GDSII Flows](#)
- [Enabling the Co-design Frontend Flow](#)
- [Setting Frontend Flow Options](#)
- [Setting Place-and-Route Options](#)
- [Starting the Flow Process](#)

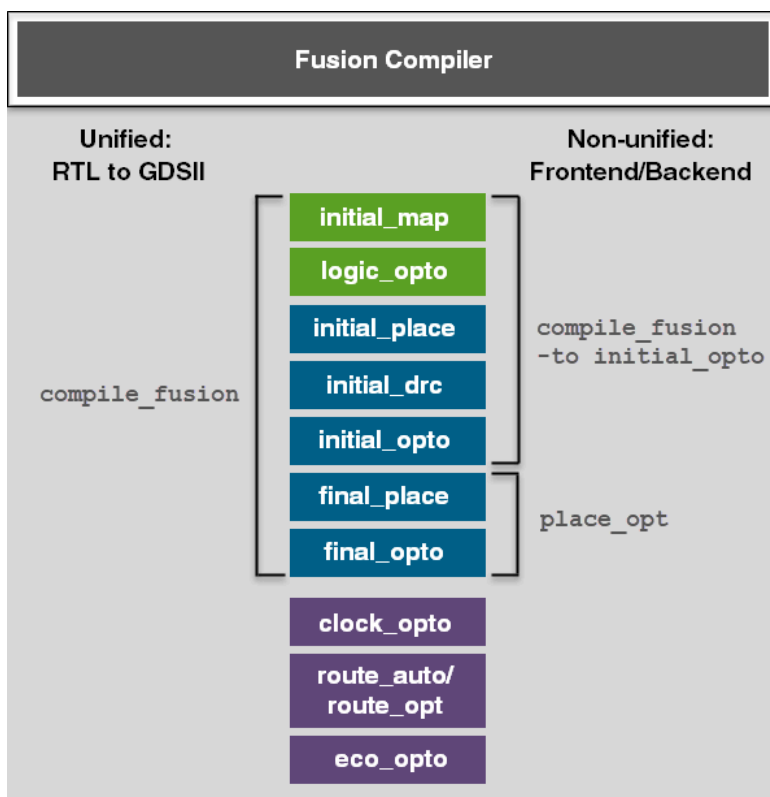
Co-designing in Unified RTL-to-GDSII Flows

This topic describes the following:

- [Fusion Compiler Frontend Flows](#)
- [Frontend Flow Tcl Script](#)

Fusion Compiler Frontend Flows

The Fusion Compiler RTL-to-GDSII flow is a design solution that merges several frontend steps to increase productivity. To accommodate different designer preferences, the Custom Compiler co-design assistant provides a user interface that lets you run either the unified RTL-to-GDSII flow or the non-unified traditional flow.



Flow	Description
Unified	The RTL-to-GDSII flow that merges the steps from <code>initial_map</code> to <code>final_opto</code> .
Non-unified	The traditional digital implementation, where the hand-off between a frontend (synthesis) and a backend (place-and-route) engineer is retained.

Frontend Flow Tcl Script

The Fusion Compiler frontend flow is driven by the following Tcl script that is supplied with the Custom Compiler environment:

Script Information	Description
Tcl script name	<code>fusion_compiler_front_end_flow.tcl</code>
Location	<code>\$\$SYNOPSYS_CUSTOM_INSTALL/ndm_oa/digital_scripts</code>
Purpose	Runs Fusion Compiler frontend (RTL-to-GDS) flows.

Script Information	Description
Content	- Graphical user interface controls to run unified frontend flow steps - Fusion Compiler Tcl commands that run the frontend flows

Workflow for Co-designing in Unified RTL-to-GDSII Flows

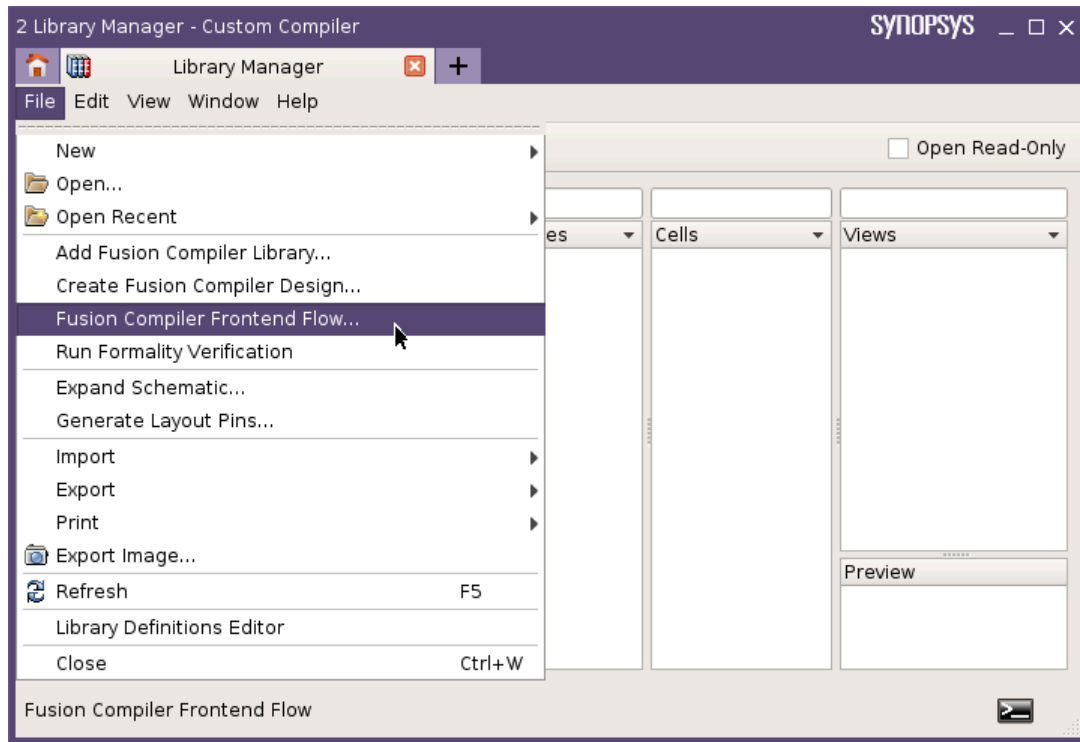
The following table presents the high-level steps for co-designing in the unified RTL-to-GDS II flow (or Fusion Compiler frontend flow).

Task	See...
1 Turn on and display the frontend flow options.	Enabling the Co-design Frontend Flow
2 Set the frontend flow options.	Setting Frontend Flow Options
3 Set the place-and-route options.	Setting Place-and-Route Options
4 Start the frontend flow operation.	Starting the Flow Process

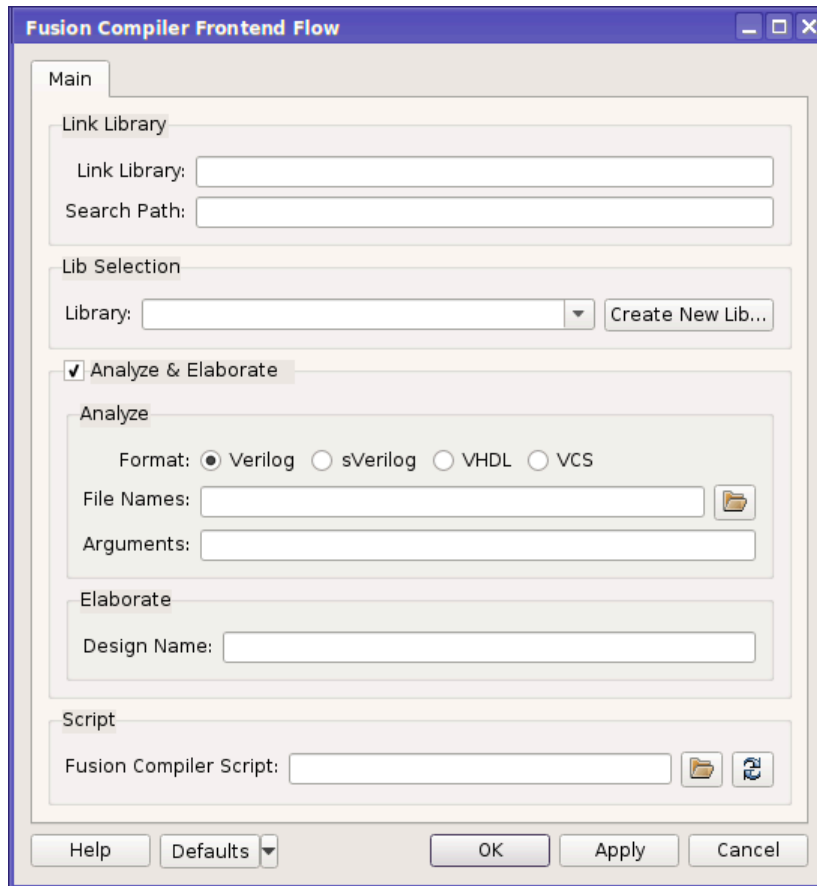
Enabling the Co-design Frontend Flow

To enable co-designing in the Fusion Compiler frontend flow:

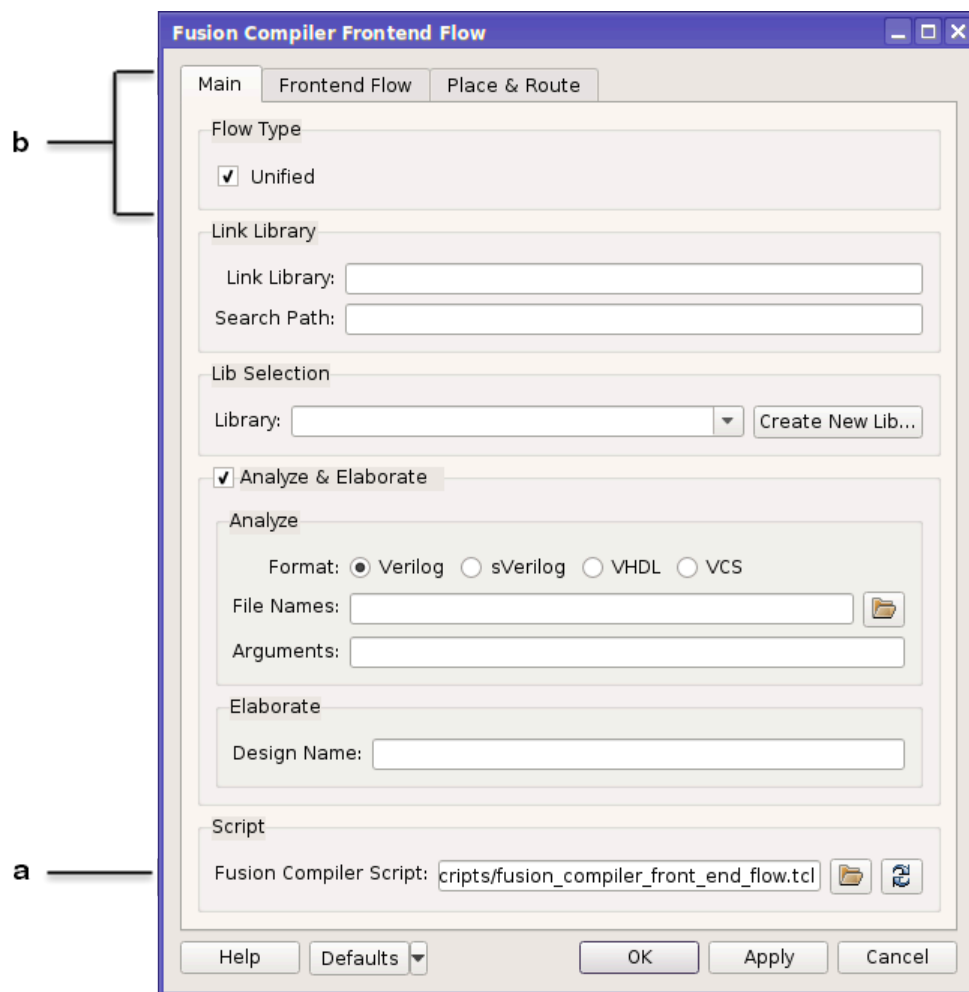
- Set the `xtDigitalImplementationTool` Tcl preference to `fc`.
- In the Library Manager, choose **File > Fusion Compiler Frontend Flow**.



The **Fusion Compiler Frontend Flow** dialog box opens.



3. In the **Main** tab:
 - a. Locate the **Script** section at the bottom of the dialog box, and specify the `fusion_compiler_front_end_flow.tcl` file. See [Frontend Flow Tcl Script](#).
 - b. Observe that the **Flow Type** section appears at the top of the **Main** tab. In addition, the **Frontend Flow** and the **Place and Route** tabs appear.



4. In the **Flow Type** section, select **Unified**, if it is not already selected. The option turns on the Fusion Compiler frontend flow feature. For additional information, see [Setting Frontend Flow Options](#).

Setting Frontend Flow Options

In the **Fusion Compiler Frontend Flow** dialog box, set the options in the following tabs:

- [Main Tab](#)
- [Frontend Flow Tab](#)

For the place-and-route options, see [Setting Place-and-Route Options](#).

Main Tab

The **Main** tab contains the following options:

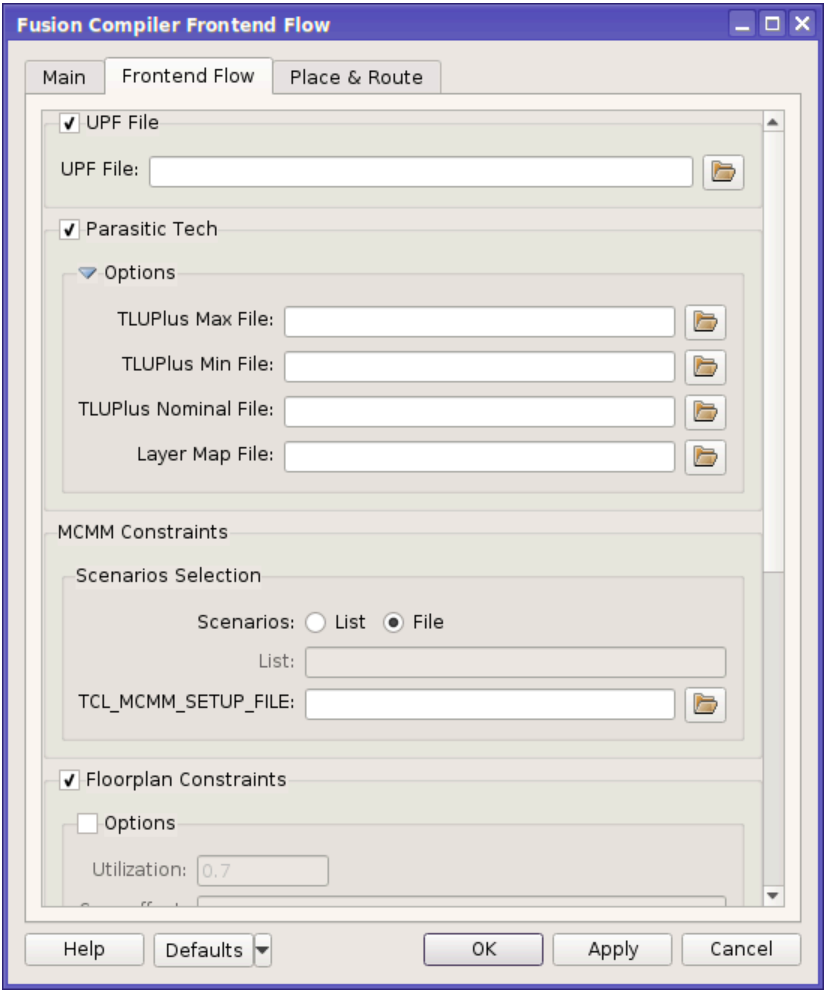
Option	Description
Flow Type	Runs the unified RTL-to-GDS flow. By default, the co-design assistant will run all the steps in the unified RTL-to-GDSII flow. To run specific steps only, see Setting Frontend Flow Options. If you prefer to run the non-unified flow, clear the Unified check box. To run specific steps in the non-unified flow, see Setting Frontend Flow Options. This option displays only if you specify the <code>fusion_compiler_front_end_flow.tcl</code> script. See Enabling the Co-design Frontend Flow.

Option	Description
Link Library	List of design files and libraries used for linking the Fusion Compiler design data and the Custom Compiler environment. References are resolved in the order that the files are specified. If you do not supply the directory names with the files, the tool uses the directory name you specify in Search Path. The Link Library settings are not stored with the design and must be set and applied with each Fusion Compiler session.
Lib Selection	Fusion Compiler Library. Choose an existing library name, or click Create New Lib to create a new library.
Analyze and Elaborate	Links the specified top module to the sub-modules and CLIB cells, and creates a block for the top module in the design library. If name conflicts occur, the existing design is overwritten. Clear the check box if you want to
Analyze	Analyzes the Verilog, SystemVerilog, or VHDL source files, and saves the resulting intermediate templates in the <code>HDL_LIBRARIES/WORK</code> folder.
Elaborate	Builds generic logic modules for the analyzed module, entity, or configuration template, and creates a block. The block is incomplete at this stage. Specify the Design Name of the top module in the RTL.
Script	Specifies the Tcl script to run. To run the frontend flow, specify <code>fusion_compiler_front_end_flow.tcl</code>. See Frontend Flow Tcl Script and Enabling the Co-design Frontend Flow. Specifying <code>fusion_compiler_front_end_flow.tcl</code> displays the Flow Type option at the top of the tab. If you do not specify <code>fusion_compiler_front_end_flow.tcl</code>, the co-design assistant will open the library in the Fusion Compiler tool, run the Analyze and Elaborate operations, and then open the block in the Custom Compiler environment.

Frontend Flow Tab

The **Frontend Flow** tab contains options that correspond to Fusion Compiler commands and options. This section provides an overview of the options. For the detailed

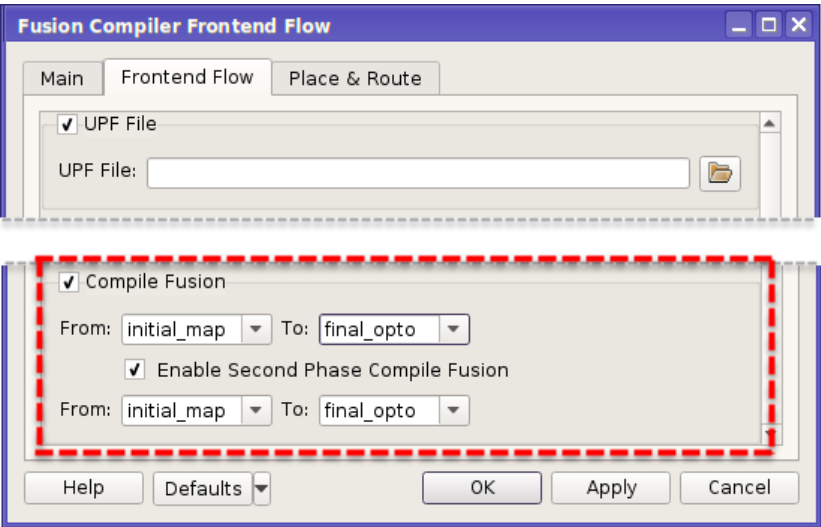
descriptions of the corresponding Fusion Compiler commands and options, see the [Fusion Compiler user documentation](#).



Option	Description
UPF File	Loads the specified unified power format (UPF) file. The UPF file defines the entire power intent and structure of a multi-voltage design.
Parasitic Tech	Loads the TLUPlus and layer map files for RC parasitic extraction, delay calculation, net estimations, and other applications. If the TLUPlus data was not loaded into the technology library, use this option to load the parasitic extraction data into the design library.

Option	Description
MCMM Constraints	Specifies the corners, or operating conditions and parasitic parameters. Use one of the following methods: • List: The list of corners that specify the design constraints. Type the list of corners in the box. Use a character space to between corner names. • File: The Tcl file containing Fusion Compiler commands that creates corners for timing analysis.
Floorplan Constraints	Generates the required floorplan. Use one of the following methods: • Options: Runs <code>compile.auto_floorplan.enable</code> and generates the floorplan automatically. You can manually adjust the floorplan. • Tcl File: Loads the Tcl file that specifies the floorplan generated by the Fusion Compiler design planning stage.
Block Pin Constraints	Sets pin placement constraints on the specified block layers. The constraints are honored during pin placement and are saved with the design.
Parasitic Parameters	Specifies the TLUPlus model for each corner after the design is loaded.
Compile Fusion	Specifies the flow steps to run. • Unified flow: When all steps are completed, the design can proceed to the <code>clock_opt</code> step. • Non-unified flow: When all steps are completed, the design can proceed to the <code>place_opt</code> step. See Fusion Compiler Frontend Flows. Note: Do not skip or repeat a step.

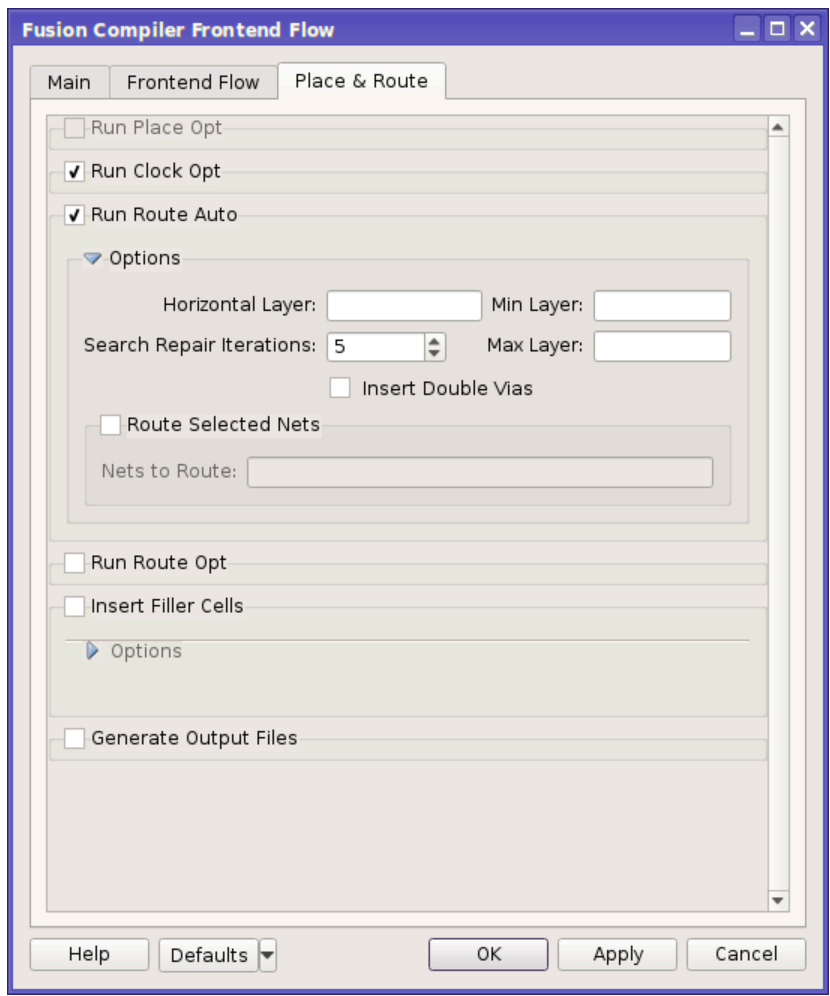
In the **Compile Fusion** section of the tab, you can specify the flow steps to run:



Flow Type	Steps	Placement Optimization
Unified (first and second phase)	From: initial_map To: - initial_map - logic_opto - initial_place - initial_drc - initial_opto - final_place - final_opto	Disabled See Setting Place-and-Route Options .
Non-unified (first and second phase)	From: initial_map To: - initial_map - logic_opto - initial_place - initial_drc - initial_opto	Enabled See Setting Place-and-Route Options .

Setting Place-and-Route Options

The **Place & Route** tab contains options that correspond to Fusion Compiler place-and-route commands and options. This section provides an overview of the options. For the detailed descriptions of the corresponding Fusion Compiler commands and options, see the [Fusion Compiler user documentation](#). For the frontend flow options, see [Setting Frontend Flow Options](#).



Option	Description
Run Place Opt	Places and optimizes the current design. The option is only available for non-unified flows.
Run Clock Opt	Runs clock synthesis and optimization commands.
Run Route Auto	Runs global routing, track assignment, and detail routing in one step.
Run Route Opt	Optimizes the current routed design for timing, electrical DRC violations, area, and power. It also performs legalization and ECO routing on the optimized design.
Insert Filler Cells	Creates standard cell metal and non-metal fillers.

Option	Description
Generate Output Files	Generates ASCII data (Verilog, UPF, DEF, scripts or SDC, parasitics, and GDS).

Starting the Flow Process

To start the flow process:

- In the **Fusion Compiler Frontend Flow** dialog box, click **OK**.

The specified flow operations start. You can open the Job Monitor to check the progress.

5

Preroute Editing (Roundtrip Flow)

During preroute editing, a design takes roundtrips between the Fusion Compiler tool and the Custom Compiler environment. This chapter describes how to edit Fusion Compiler designs within the Custom Compiler environment. Instructions are also provided for how to review the designs in the two environments.

This chapter describes the following:

- [Workflow for Preroute Editing](#)
- [Reviewing Designs in the Fusion Compiler Tool](#)
- [Opening Fusion Compiler Designs in the Custom Compiler Environment](#)
- [Checking That the Design is in the Safe-Editing Mode](#)
- [Reviewing Designs Within the Custom Compiler Environment](#)
- [Setting and Editing Constraints](#)
- [Displaying the Layout Editing Toolbox](#)
- [Adjusting Cell Placements](#)
- [Creating and Editing Blockages](#)
- [Editing Route Guides](#)
- [Creating and Editing Routing Corridors](#)
- [Adding and Editing Routes](#)
- [Adding Shielding](#)
- [Editing Other Aspects of the Design](#)
- [Viewing and Editing Object Properties](#)
- [Reporting and Fixing Electrical Violations](#)
- [Saving Modified Designs](#)

- [Reviewing Custom Compiler Edits in the Fusion Compiler Tool](#)
- [Reviewing Fusion Compiler Edits Within the Custom Compiler Environment](#)

After adding the preroutes, if you want to run place-and-route on the design from within the Custom Compiler environment, see [Developing and Integrating Digital Blocks](#).

Workflow for Preroute Editing

This topic shows the co-design roundtrip workflow when running the Custom Compiler environment independently of the Fusion Compiler tool. If you prefer to do the preroute editing in the Custom Compiler environment while in the Fusion Compiler tool, see [Preroute Editing \(Flow From Within the Fusion Compiler Tool\)](#).

Note:

No technology or PDK setup is required. In addition, roundtrip editing preserves all relevant data in both environments.

You can only edit Fusion Compiler design libraries or cell libraries that are also design libraries.

 [Show me](#) an example of preroute editing.

The following table shows the basic workflow for preroute editing.

Task	See...
1 Review the design in the Fusion Compiler tool.	Reviewing Designs in the Fusion Compiler Tool
2 Add the Fusion Compiler library to the Custom Compiler environment.	Adding Fusion Compiler Libraries
3 Open the Fusion Compiler design within the Custom Compiler environment.	Opening Fusion Compiler Designs in the Custom Compiler Environment
4 Review the Fusion Compiler design objects and constraints using the Custom Compiler Layout Editor.	Reviewing Designs Within the Custom Compiler Environment
5 Custom edit the Fusion Compiler design using the Custom Compiler tools. For example, you can create analog preroutes and fine-tune the design.	One or more of the following: • Setting and Editing Constraints • Adjusting Cell Placements • Creating and Editing Blockages • Editing Route Guides • Creating and Editing Routing Corridors • Adding and Editing Routes • Editing Other Aspects of the Design • Saving Modified Designs

Task	See...	
6	Review the Custom Compiler edits in the Fusion Compiler tool to make sure the updates appear correctly in the Fusion Compiler tool. After you confirm the changes, you can continue to edit the design in the Fusion Compiler tool. For example, you might want to route clock nets or run place-and-route operations.	Reviewing Custom Compiler Edits in the Fusion Compiler Tool
7	Review the Fusion Compiler edits in the Custom Compiler Layout Editor to make sure the updates appear correctly within the Custom Compiler environment. After you confirm the changes, you can proceed to finalize the design.	Reviewing Fusion Compiler Edits Within the Custom Compiler Environment

Reviewing Designs in the Fusion Compiler Tool

During the co-design process, foundry rules are applicable when using both Fusion Compiler and Custom Compiler products. When you open Fusion Compiler designs within the Custom Compiler environment, the following are automatically mapped to their equivalents:

- Placement blockages
- Routing blockages
- Nondefault rules
- Custom routing constraints

For mapping details, see [Character and Object Mappings](#) and [Technology Mappings](#).

Before you start editing designs using the Custom Compiler environment, you should review the design using the Fusion Compiler tool and note how the blockages and constraints are set up. This provides a baseline you can use to check the mappings on the Custom Compiler environment.

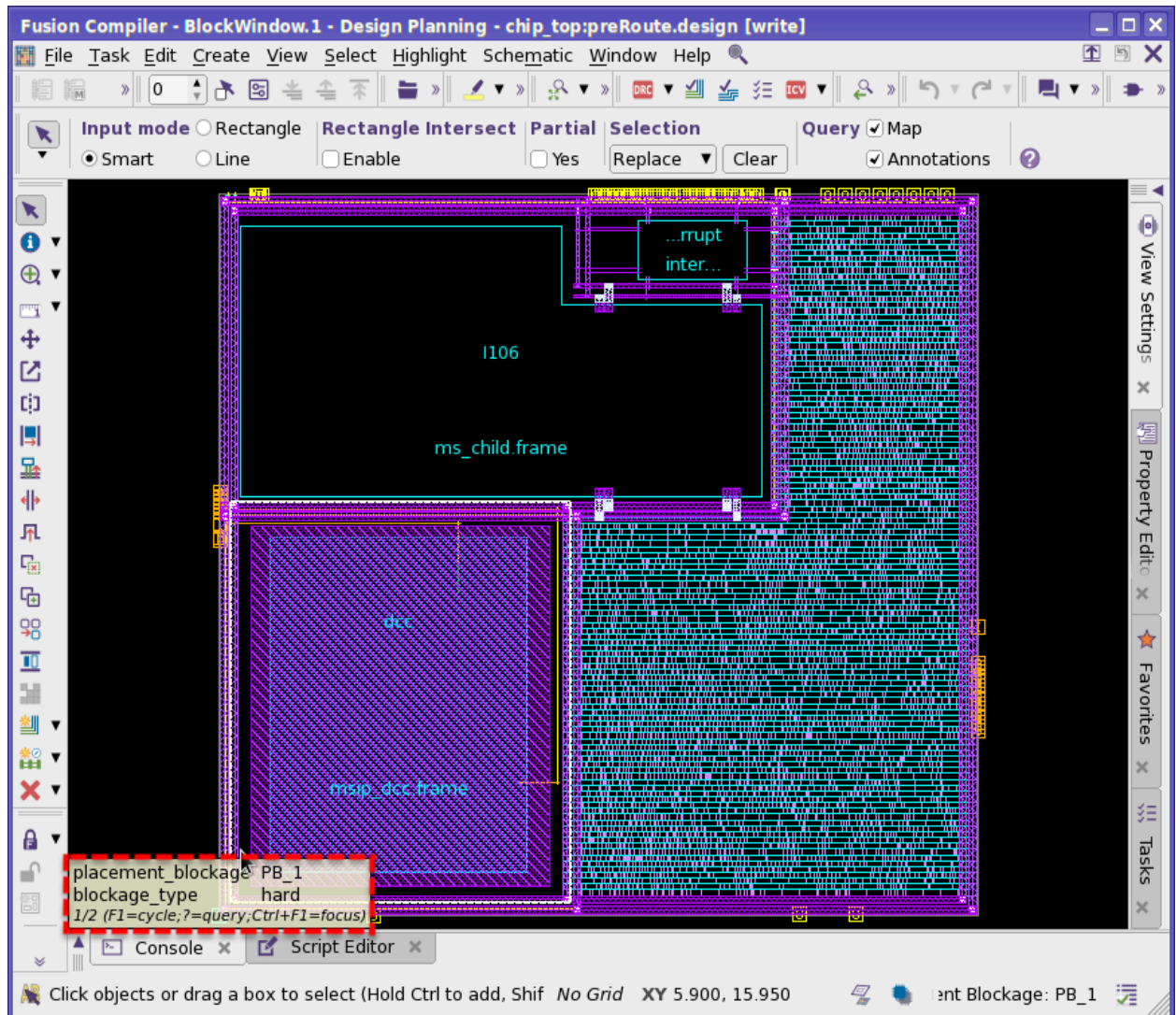
This topic describes the following:

- [Viewing Placement and Routing Blockages](#)
- [Viewing Nondefault Rules and Custom Routing Constraints](#)
- [Setting Up Co-design Filters](#)

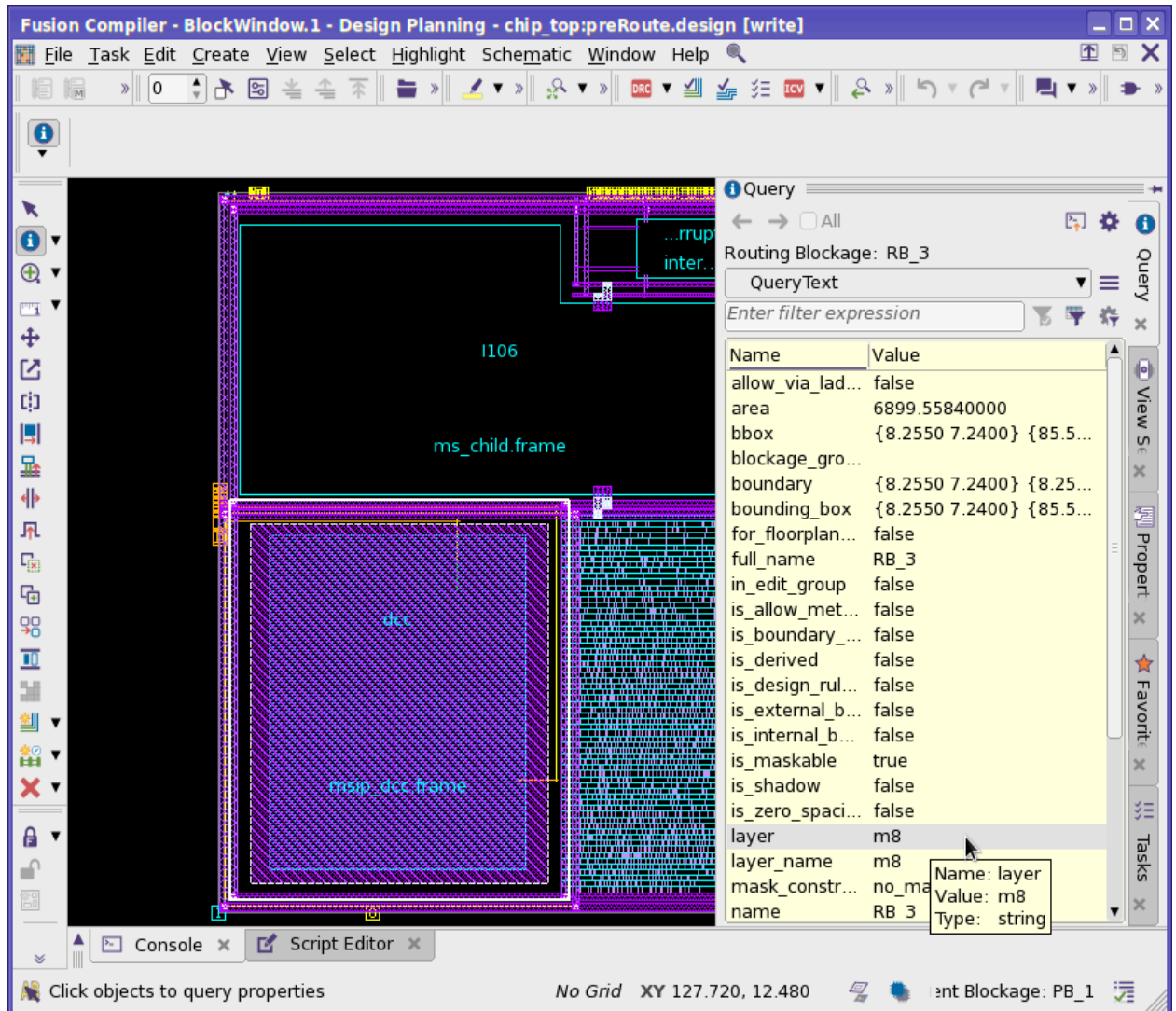
Viewing Placement and Routing Blockages

To view the placement and routing blockages:

1. Open the design in the Fusion Compiler tool.
2. Activate the placement blockage. Information about the placement blockage appears in the lower-left corner of the window. Note the information.



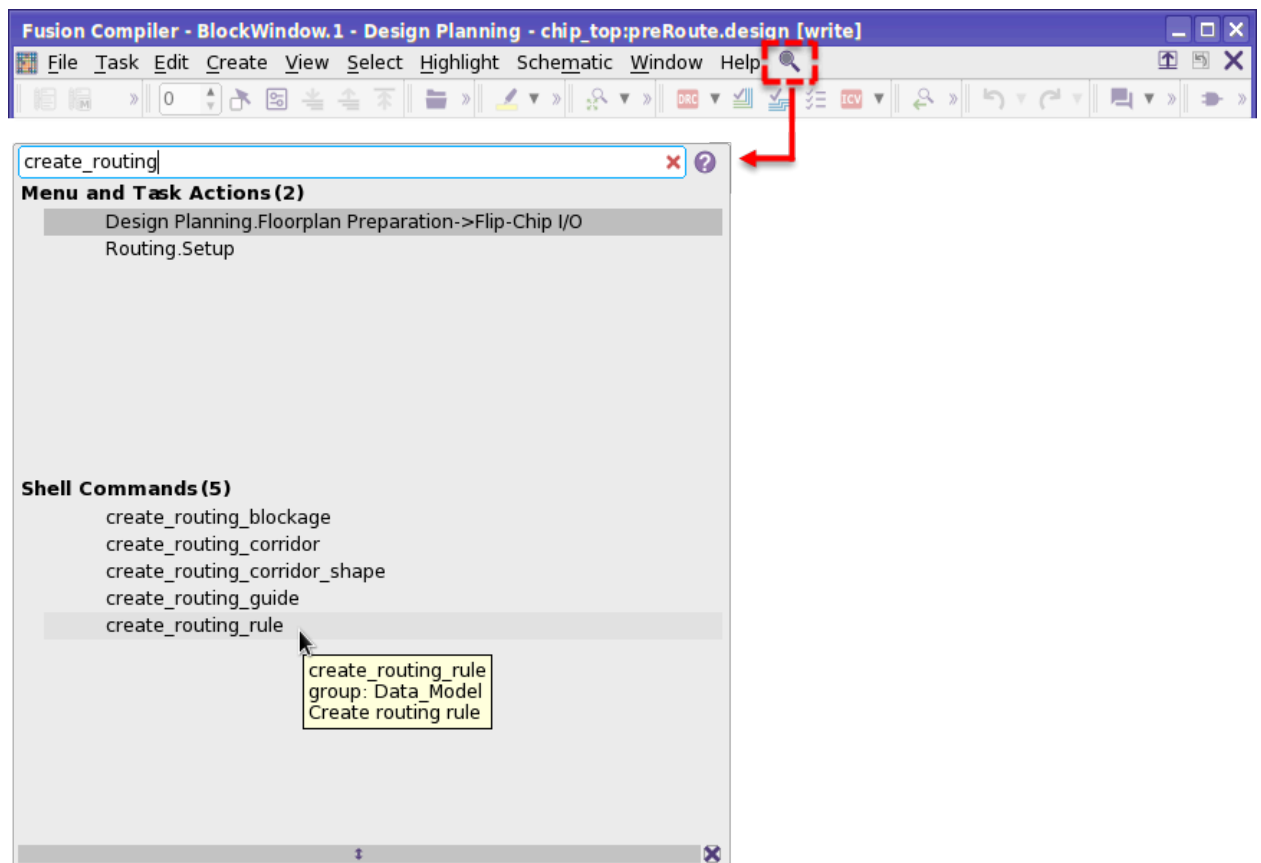
3. Choose **View > Mouse Tools > Query Tool**, and then select the routing blockage. Information about the selected routing blockage appears in the **Query** tab. Locate the **layer** attribute and note the layer information.



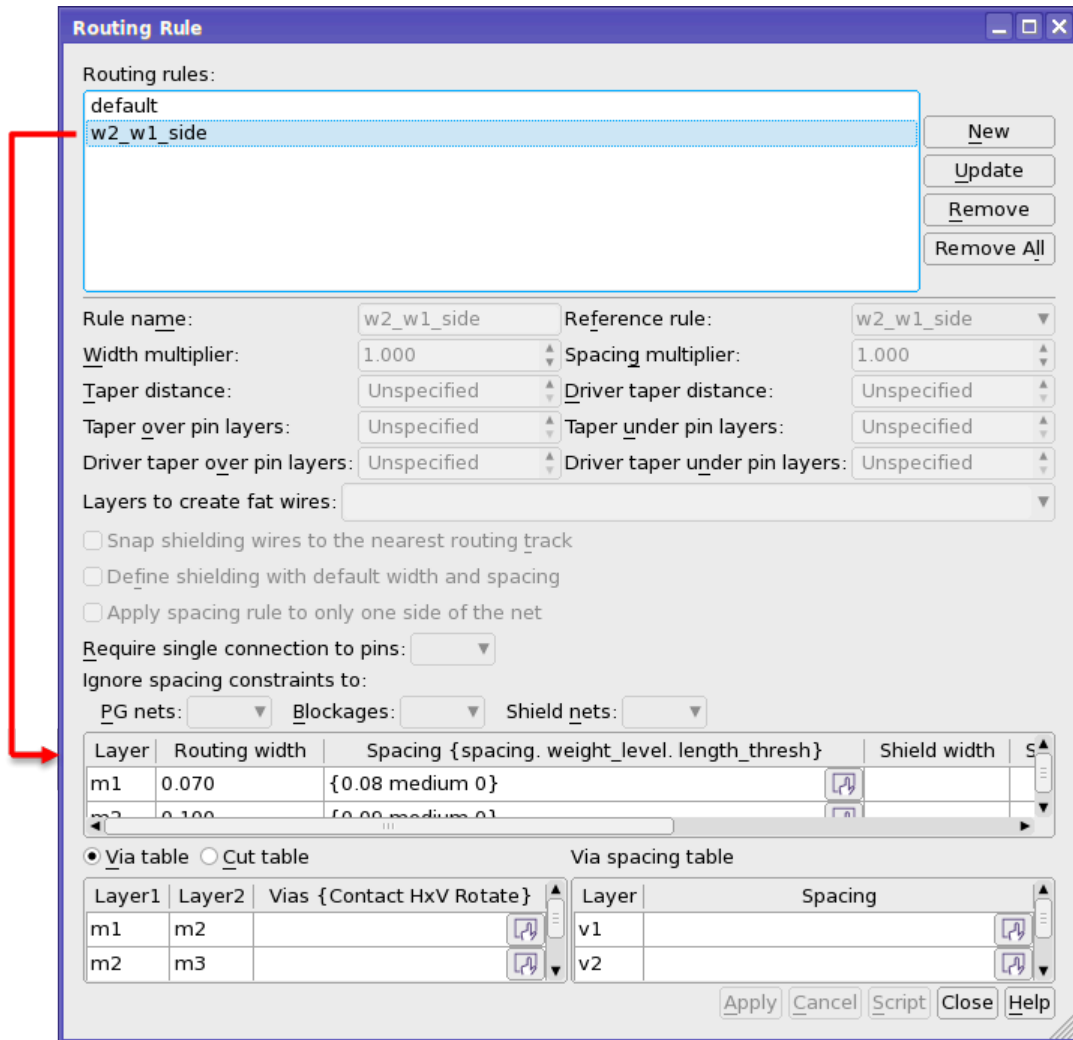
Viewing Nondefault Rules and Custom Routing Constraints

To view the nondefault rules and custom routing constraints:

1. In the Fusion Compiler window, display the heads-up display (HUD). You can use the keyboard shortcut `h`, or click  (magnifying glass) on the menu bar.
2. In the window that appears, type `create_routing`. A list of shell commands appears in the window.
3. Choose **create_routing_rule**.



4. In the **Routing Rule** dialog box that opens, choose the name of the nondefault rule. Note the layers that are associated with this rule and the routing width of each layer.



5. In the Fusion Compiler console window, type the following command to list the nets associated with the rule:

```
> get_nets -filter "routing_rule == <ruleName>"
```

<ruleName> is the name of the rule you are querying.

6. In the Fusion Compiler console window, type the following command to list the constraint groups used in the design:

```
> report_constraint_groups
```

The constraint group report appears in the window and shows the following columns:

- **Name:** The name of the custom routing constraint
- **Type:** The type of constraint
- **Objects:** The nets to which the constraint applies

7. Review the constraint group report and note the custom routing constraints and the associated nets.

8. Close the design in the Fusion Compiler tool.

Setting Up Co-design Filters

Loading and editing large Fusion Compiler designs on the Custom Compiler environment can be cumbersome and impact tool performance. Instead, you can define a region of interest and load only that region on the Custom Compiler environment.

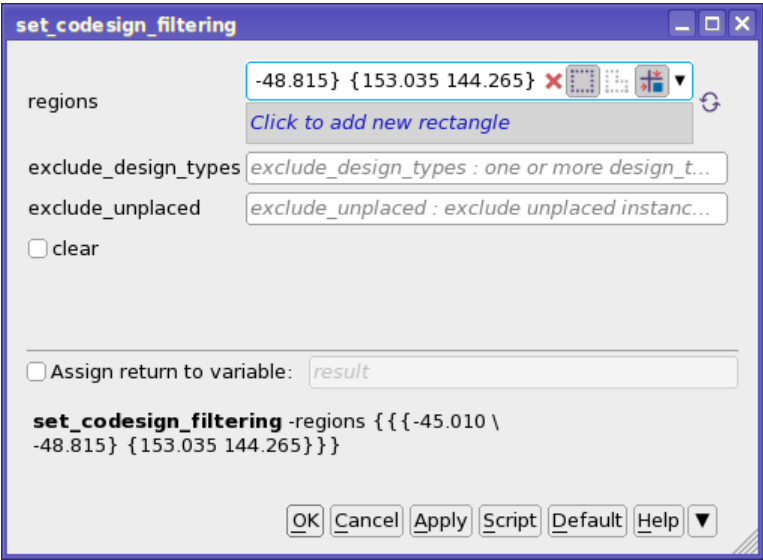
You define the region of interest with a set of filtering criteria. You can set up the filters while in the Fusion Compiler tool or when you open the design on the Custom Compiler environment.

This section describes how to set up filters in the Fusion Compiler tool. To set up filters when opening the design on the Custom Compiler environment, see [Opening Fusion Compiler Designs in the Custom Compiler Environment](#).

 [Show me](#) an example of setting up filters for region-based editing.

To set up filters in the Fusion Compiler tool:

1. With the design open, choose **CoDesign > Setup Filtering**. The **set_codesign_filtering** dialog box opens.



2. Set the following options:

Option	Description
regions	Specifies the region of interest. Provide the coordinates (x, y) that define the vertices of the rectangular  or polygonal  region.
exclude_design_types	Filters the type of design.
exclude_unplaced	Excludes the unplaced instances.
clear	Removes the current setting values in the dialog box.
Assign return to variable	Assigns the return of the command (the filtering result, such as the object ID) to a Tcl variable. You must provide the name of the variable in the box.

3. Click **OK** to save the filters with the design. These filters are applied when you open the design on the Custom Compiler environment.

Opening Fusion Compiler Designs in the Custom Compiler Environment

After reviewing the design in the Fusion Compiler tool, you can open the design in the Custom Compiler environment and started editing.

Note:

The source digital design is locked in the Fusion Compiler tool to protect the data while you work with the design within the Custom Compiler environment.

This topic describes the following:

- [Opening Filtered Designs](#)
- [Opening an Entire Design](#)

Before opening the designs, make sure you have added or created the library in the Custom Compiler environment. See [Preparing Design Libraries](#).

Opening Filtered Designs

Loading and editing large Fusion Compiler designs in the Custom Compiler environment can be cumbersome and impact tool performance. Instead, you can define a region of interest and load only that region in the Custom Compiler environment.

You define the region of interest with a set of filtering criteria. You can set up the filters while in the Fusion Compiler tool or when you open the design in the Custom Compiler environment.

This section describes how to set up and apply filters while opening the design in the Custom Compiler environment. To set up filters in the Fusion Compiler tool, see [Setting Up Co-design Filters](#).

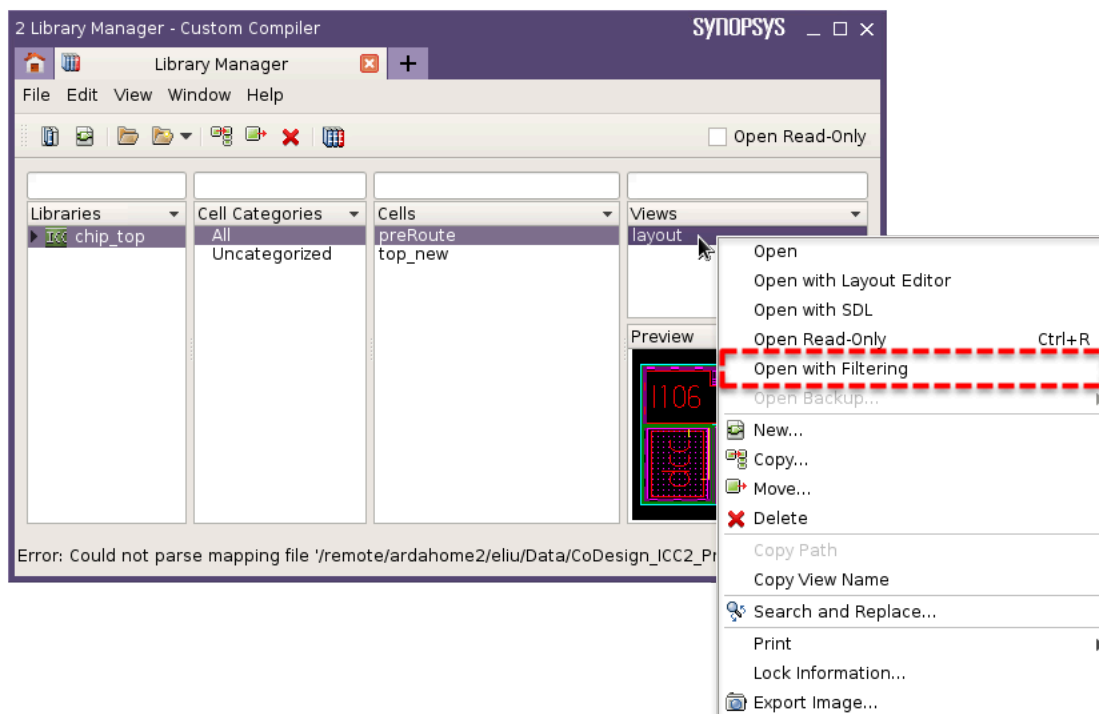
 [Show me](#) an example of setting up filters for region-based editing.

To open a filtered design:

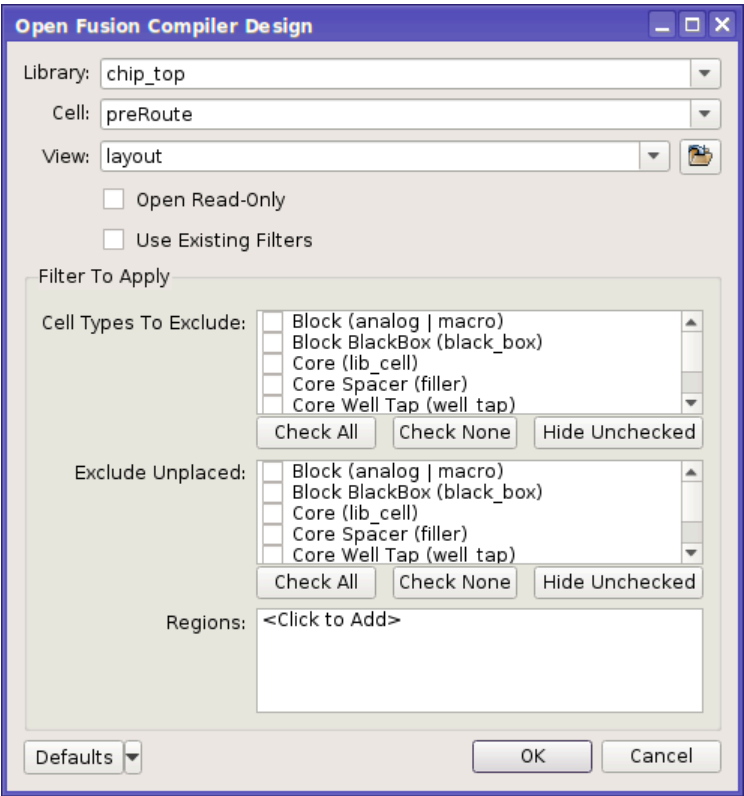
1. In the Custom Compiler [Library Manager](#), choose the library name in the **Libraries** pane. The associated Cell Categories and Cells appear.
2. In the **Cells** pane, choose the cell. The **Views** pane lists the available design views.
3. Right-click the **layout** view and choose **Open with Filtering**.

Chapter 5: Preroute Editing (Roundtrip Flow)

Opening Fusion Compiler Designs in the Custom Compiler Environment



4. In the **Open Fusion Compiler Design** dialog box that opens, specify the design **Library**, **Cell**, and **View** names.



- 5. Select **Open Read-Only** if you want to open the design in the read-only mode.
- 6. To use the filters you set up in the Fusion Compiler tool, select **Use Existing Filters**. See [Setting Up Co-design Filters](#).
- 7. To specify new filters and define the region of interest, set the following in the **Filter to Apply** section:

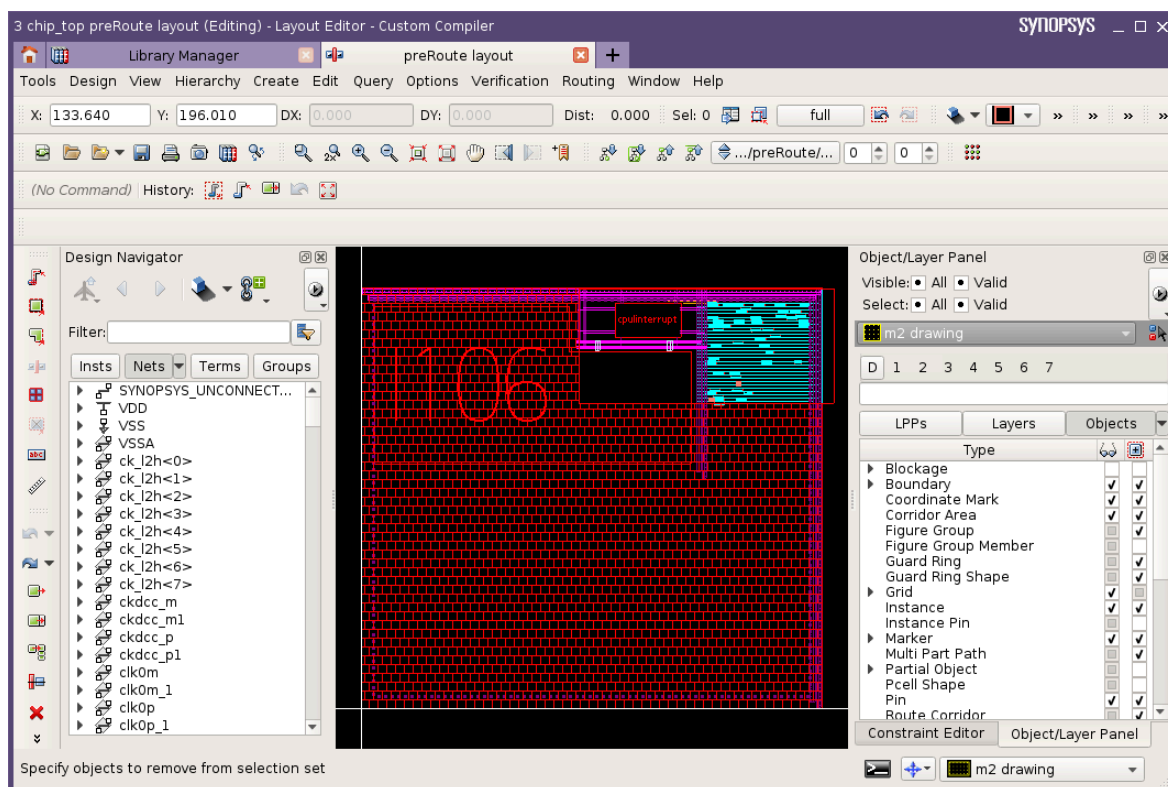
Option	Description
Cell Types to Exclude	Filters the type of cell.
Excludes Unplaced	Excludes the selected unplaced design components.
Regions	Specifies the coordinates (x, y) that define the vertices of the rectangular region.

- 8. Click **OK**.

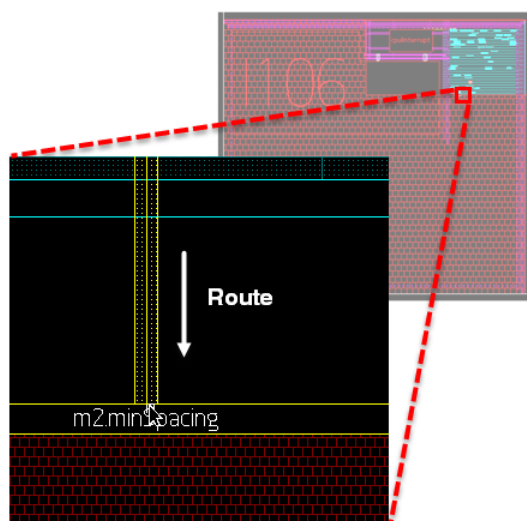
The Layout Editor opens and displays the filtered Fusion Compiler design within the viewport of the canvas.

Chapter 5: Preroute Editing (Roundtrip Flow)

Opening Fusion Compiler Designs in the Custom Compiler Environment



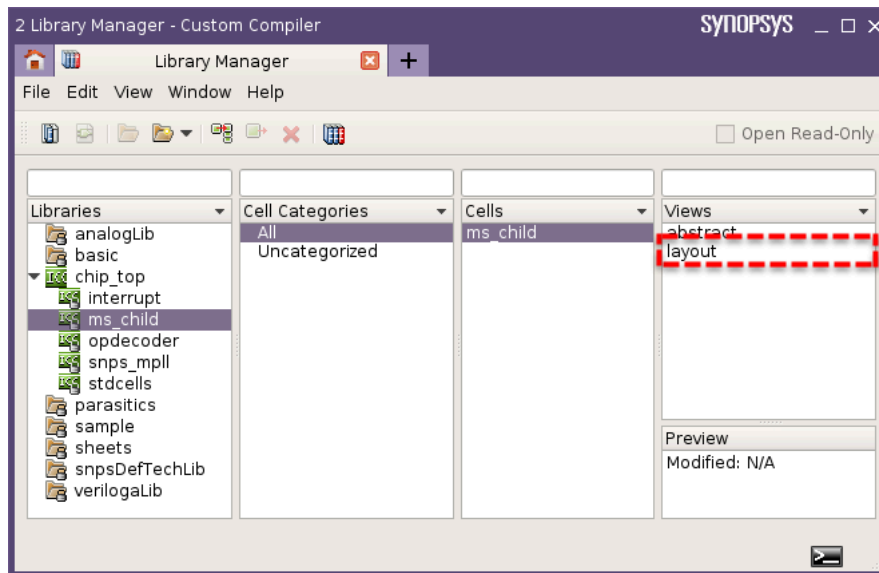
A mesh overlay lets you know that the regions beneath are not loaded, and you will not be permitted to edit the mesh-covered regions. The mesh overlay is treated as a blockage, and remains visible even if you disable the visibility of **Blockage** in the Object/Layer Panel.



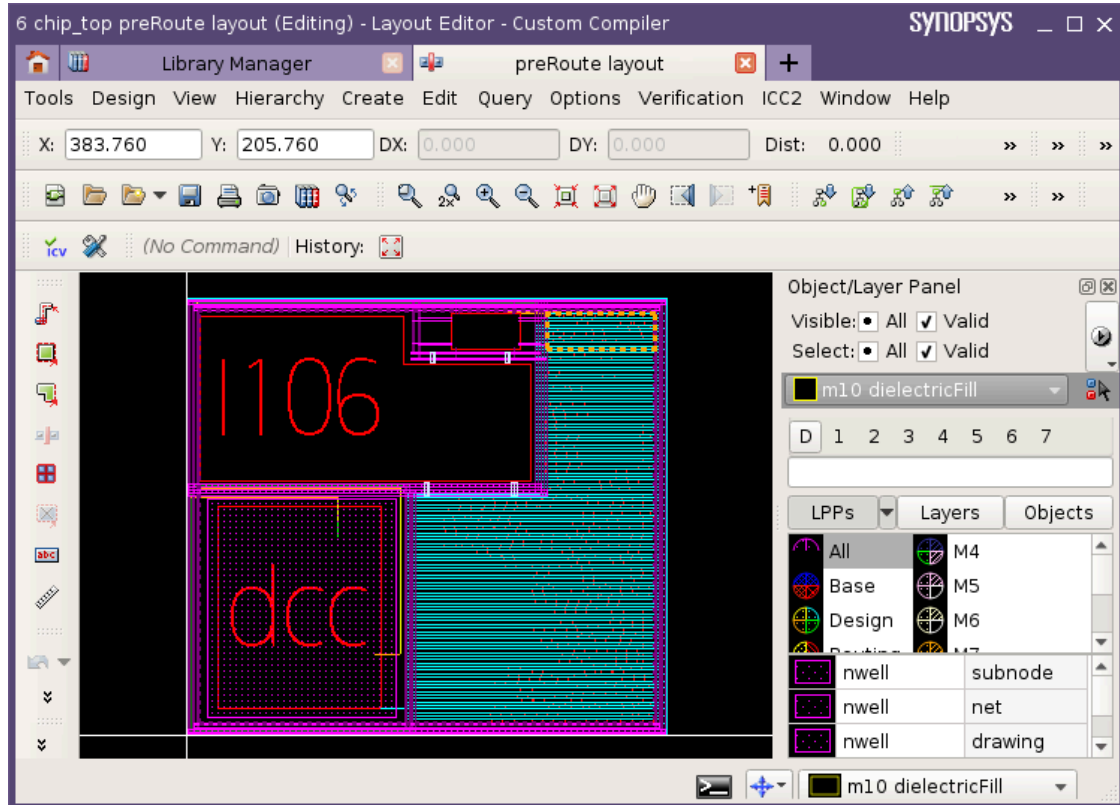
Opening an Entire Design

To open an entire design:

1. In the Custom Compiler [Library Manager](#), choose the library name in the **Libraries** pane. The associated Cell Categories and Cells appear.
2. In the **Cells** pane, select the cell. The **Views** pane lists the available design views.
3. Double-click the **layout** view.



The Layout Editor opens and displays the Fusion Compiler design within the viewport of the canvas. Notice that the names of the library, cell, and view appear at the top of the tab.



Checking That the Design is in the Safe-Editing Mode

The Custom Compiler safe-editing mode prevents you from modifying the logical elements of the design that would affect the Fusion Compiler view consistency check. It ensures that you can safely continue to use Fusion Compiler for the design implementation after editing it in the Custom Compiler environment.

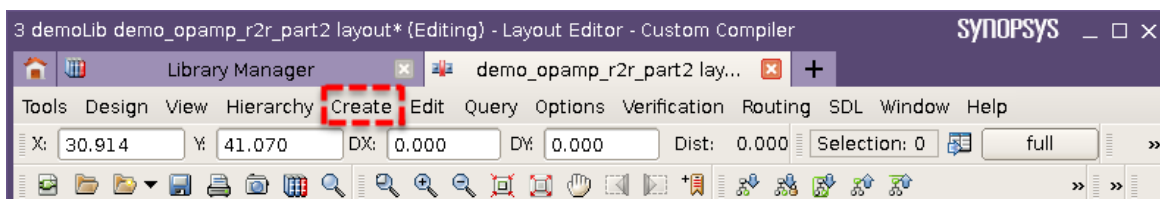
Note:

The original Fusion Compiler design remains locked while you edit the design in the Custom Compiler environment.

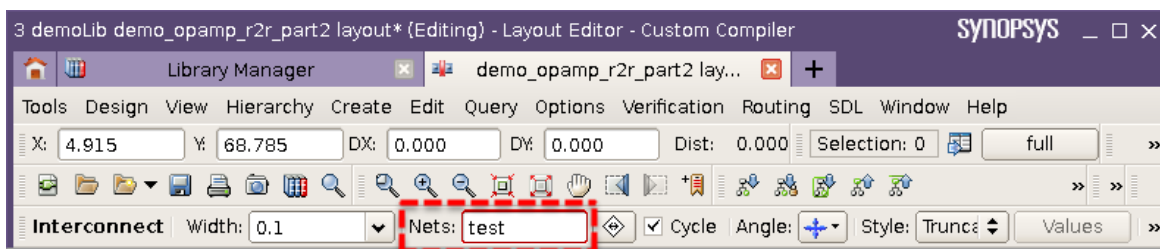
When working with Fusion Compiler designs, the Custom Compiler safe-editing mode is on by default.

To check that the safe-editing mode is turned on:

1. Display the Console so that you can see the messages that display during this procedure.
2. Click the **Create** menu, and then check to see whether the **Instance** and **Pin** commands are unavailable.



3. Choose **Create > Interconnect**. On the **Create Interconnect** toolbar that appears, type any name in the **Nets** box. (For example, type `test`.) In the **Console**, a warning message should appear and explain that the command is in the read-only mode.



4. Display the Property Editor, and then select an instance or shape on the canvas. In the Property Editor, the **Attributes** for the instance or shape should be unavailable.
5. Press Ctrl+D to clear the instance or shape selections.

Reviewing Designs Within the Custom Compiler Environment

After you open the Fusion Compiler design in the Custom Compiler Layout Editor and make sure that you are in the safe-editing mode, you should check that the constraints and blockages are mapped correctly.

This topic describes the following:

- [Viewing Design Objects](#)
- [Viewing Constraints](#)

Viewing Design Objects

To view the design objects, such as blockages:

1. In the design that you opened, make sure rows are imported.
2. Make sure placement blockages are imported.
3. To check the routing blockages:
 - a. On the canvas, select the routing blockages.
 - b. In the Property Editor, under the **Attributes** section, look at the **Layer** attribute. Make sure the blockage layers match the layers that you reviewed in the Fusion Compiler tool.
 - c. Press Ctrl+D to clear the routing blockage selections.

Viewing Constraints

To view the constraints:

1. Display the Constraint Editor and the Design Navigator.
2. To check the layer constraints:
 - a. In the Constraint Editor, click **Routing**, and then select the **Layers** constraint for the constraint group of interest.
 - b. In the parameters area, check the width of the routing layers and make sure they match the values that you noted when reviewing the design in the Fusion Compiler tool. See [Viewing Nondefault Rules and Custom Routing Constraints](#).
 - c. In the Design Navigator, click **Nets**  and choose **Net by Group**. Expand the net group of interest. Check that the nets of interest belong in the group as shown in the constraint group report you generated in the Fusion Compiler tool. See [Viewing Nondefault Rules and Custom Routing Constraints](#).
3. In the Constraint Editor, review the list of routing constraints and make sure they match the list in the constraint group report you generated in the Fusion Compiler tool. See [Viewing Nondefault Rules and Custom Routing Constraints](#).

After confirming that the mappings are correct, you can proceed to edit the design in the Custom Compiler environment. For example, you can set or edit constraints, and then add preroutes.

The remaining topics provide high-level guidance for common editing tasks. Some details are provided if the information is specific to co-designing.

Setting and Editing Constraints

Before you start to edit the designs, set or modify the constraints so that the modifications you make are compliant.

Use the following Custom Compiler features to review, set, and modify constraints:

- Constraint Manager Tool: Lets you view and manage global or process-level constraints.
- Constraint Editor Assistant: Lets you view, create, and modify nondefault rule (or design-specific) constraints.

You use the Constraint Editor to set and modify placement and routing constraints. See [Placement and Setting Routing Constraints](#).

If you create placement constraints, you must export and store the constraint data with the design using the `cm::exportIntents` Tcl command. When you re-open the design in the Custom Compiler environment, you must use the `cm::importIntents` Tcl command to restore the placement constraints.

Before adding preroutes, you should review, set, or modify the Routing constraints using the Constraint Editor. See [Adding and Editing Routes](#).

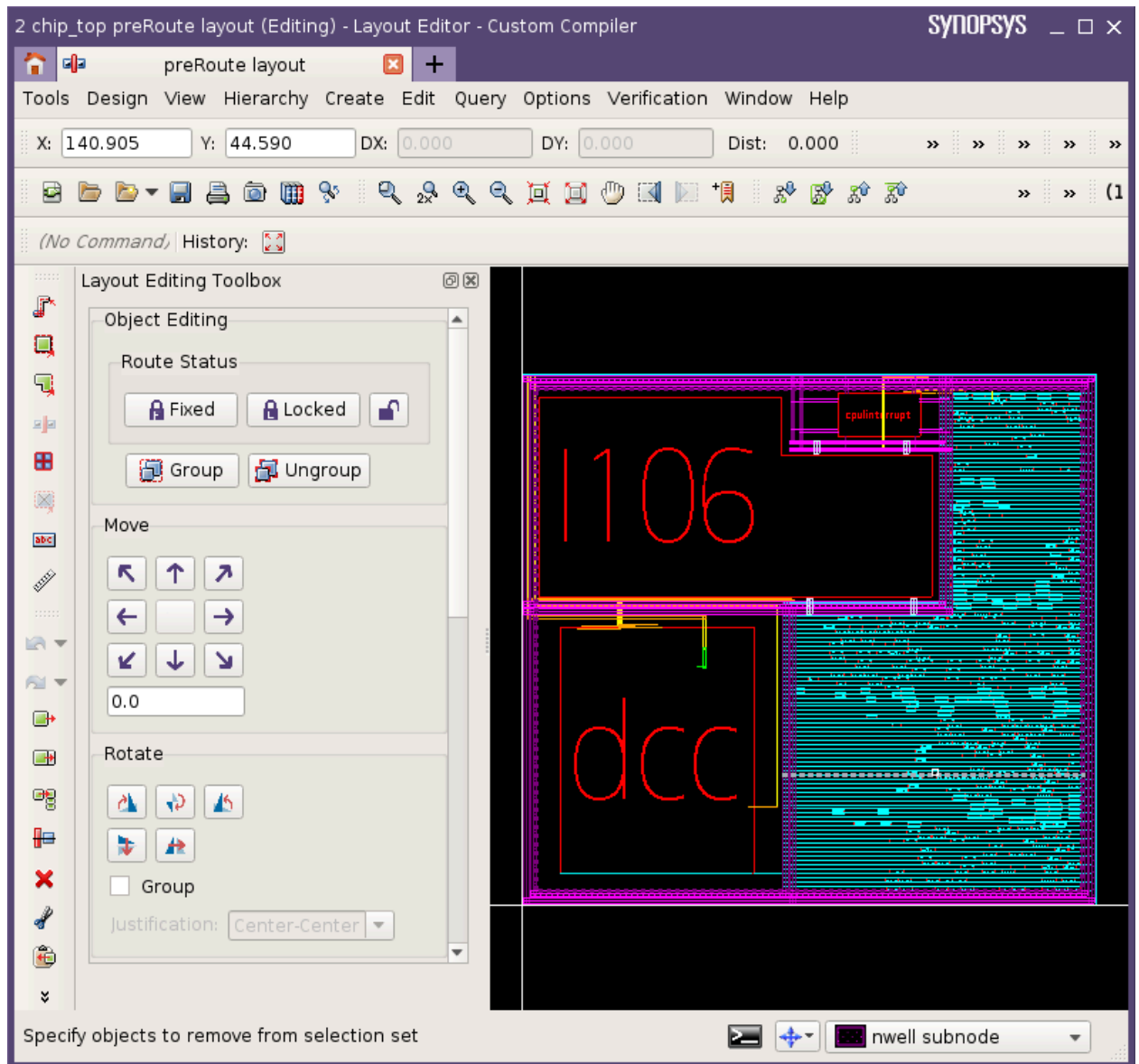
Displaying the Layout Editing Toolbox

The Layout Editing Toolbox provides quick access to frequently used features during the co-design process. Displaying the toolbox eliminates the need to hunt for the features in the various command menus.

To display the Layout Editing Toolbox:

- In the Layout Editor, choose **Window > Assistants > Layout Editing Toolbox**.

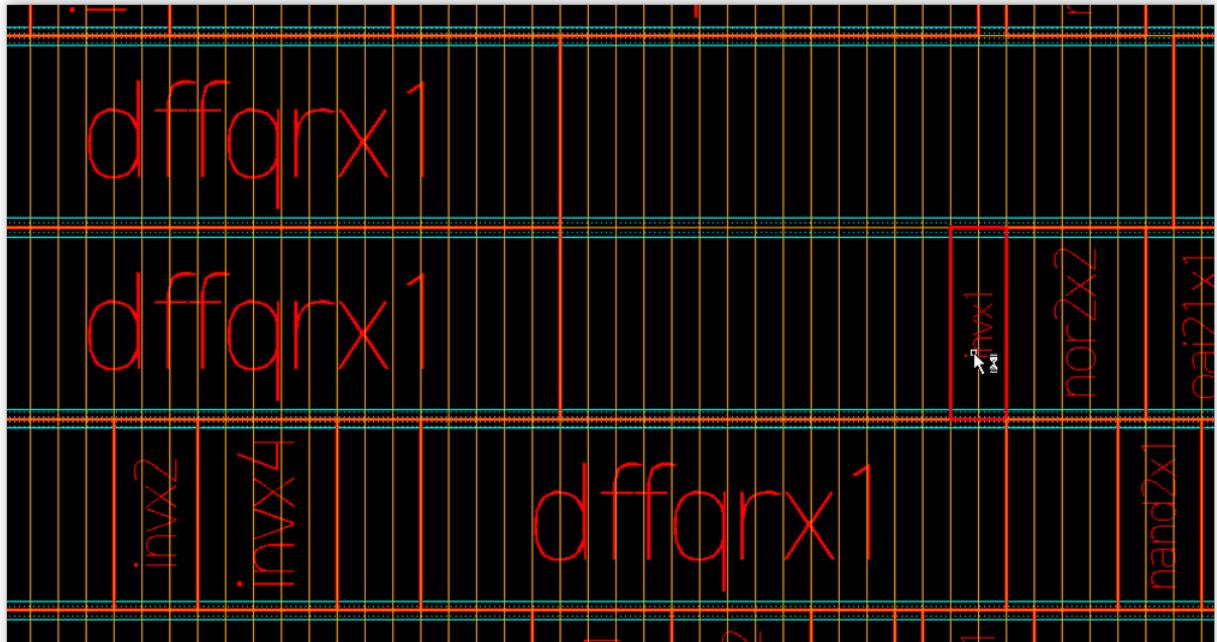
The Layout Editing Toolbox appears on the left side of the Layout Editor tab. For information about the toolbox, see Using the Layout Editing Toolbox Assistant.



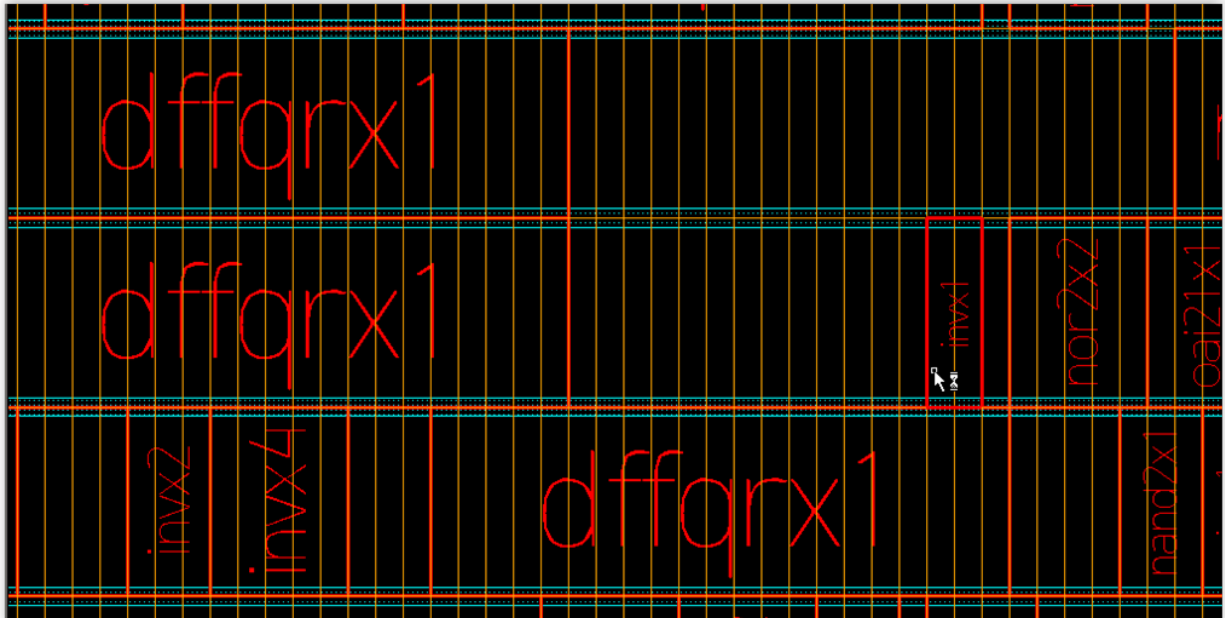
Adjusting Cell Placements

You can move standard cells in an Fusion Compiler design and the tool snaps the standard cells to the rows in the design, at proper multiple locations of the unit tile. The following figures illustrate the move and snap to row and site locations.

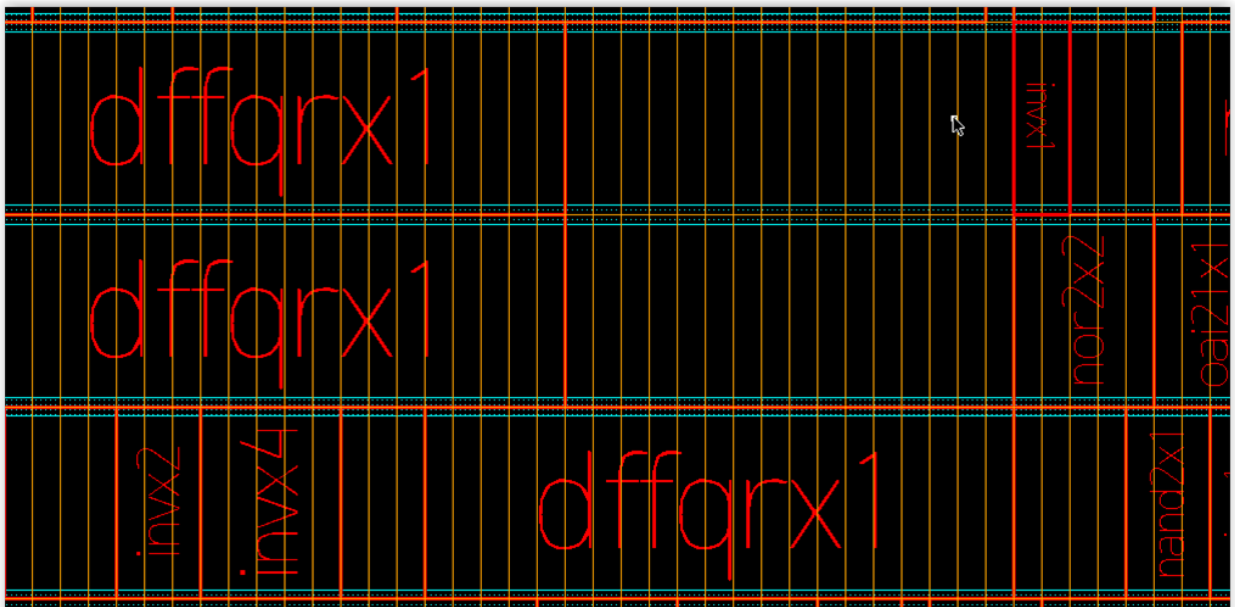
Selecting the instance to move it:



Snapping the instance to row sites during the Move command:



When moving the instance to the next row in a double back row configuration, the instance orientation adjusts automatically:



In the figures, rows are separated by the blue m1 layer power and ground straps. Site locations are identified by the orange lines.

Cells match the required orientation of the row, as shown in the third figure, where the cell has been flipped (as seen by the location of the beveled corner). Make sure that cells do not overlap when you move them, because the tool allows cells to be moved over other ones, resulting in overlaps, even though cell overlaps are not legal in the Fusion Compiler tool. After you move cells to the target location, lock their placement by changing the placement status in the Property Editor assistant. When the cell placement status is locked, neither Custom Compiler or Fusion Compiler tools allow moving the cell either manually or automatically. Macro cells are not bound to respect the row site locations.

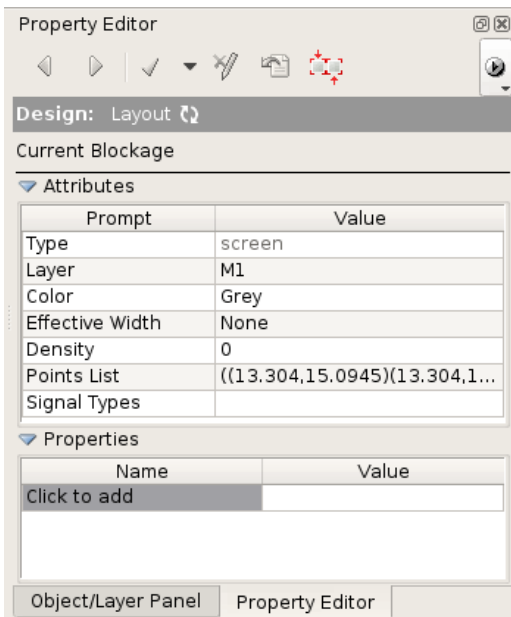
If metal pin shapes are present in the cells or instances, and routing tracks are present in the design, you have the option of snapping the metal pin shapes to the routing tracks. See Snapping Instances.

Creating and Editing Blockages

Before you add components to the design, you can create or edit placement or routing blockages. See Creating Blockages and Editing Blockages.

To view or edit a blockage you created in the Fusion Compiler tool:

1. Display the Property Editor.
2. On the canvas, select the blockage. The blockage attributes appear in the Property Editor. For the description of the attributes, see Editing Blockages and [Object Mappings](#).

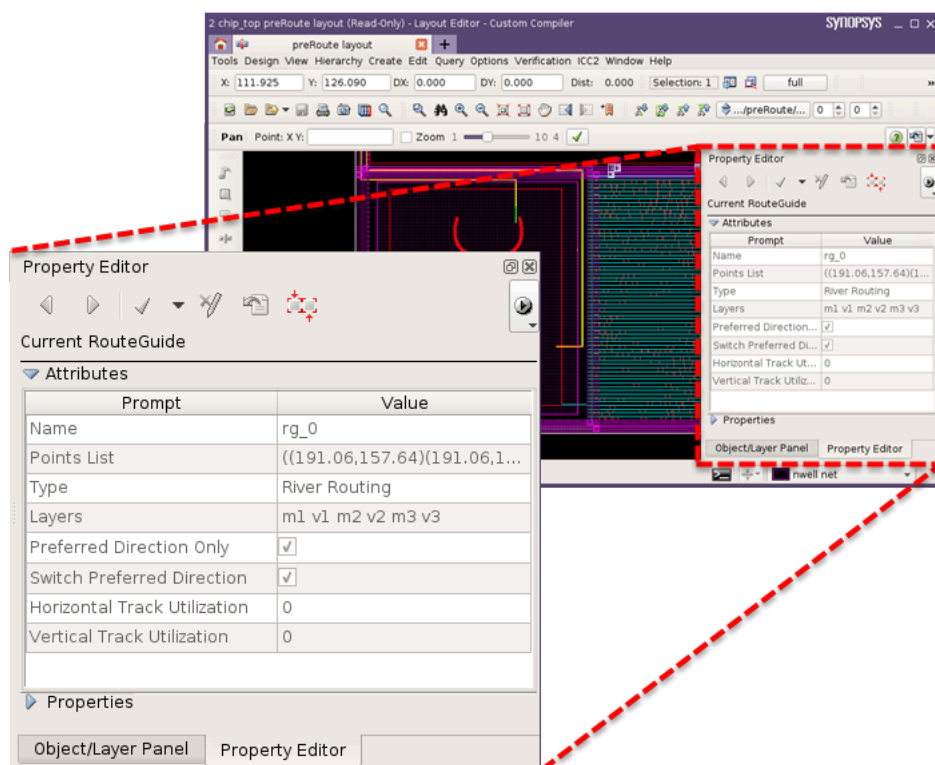


Editing Route Guides

You can edit route guides to further refine the design before adding routes. Route guides are rectangular shapes created in Fusion Compiler designs to provide routing directives. For example, you can use route guides to define regions where routes are not permitted. Route guides appear as area boundaries in the Custom Compiler Layout Editor. For more information about route guides, see the [Fusion Compiler user documentation](#) available on SolvNet. For information about how the route guide objects are mapped between Fusion Compiler tool and the Custom Compiler environment, see [Object Mappings](#).

To view or edit route guides in the Layout Editor:

1. Open the Fusion Compiler design in the Layout Editor.
2. Display the Property Editor.
3. On the canvas, locate and select the route guide you want to modify. The attributes of the selected route guide appear in the Property Editor. By default, the **Attributes** section is expanded.



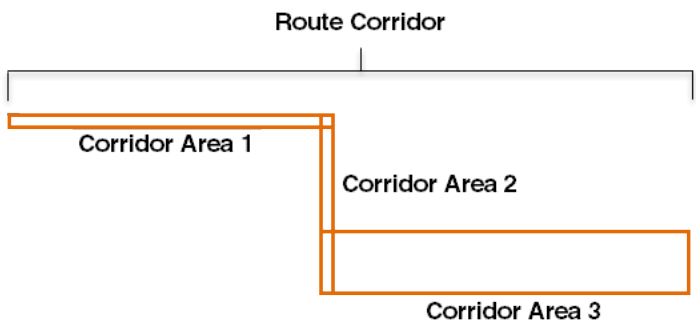
4. Modify the desired route guide **Attributes**.

Attribute	Description
Name	Identifies the selected route guide. By default, the name assigned in the Fusion Compiler tool is displayed. To change the name, double-click the box, and then type a unique name.
Points List	Specifies the location of the route guide. To change the location, click  . In the Edit Points dialog box, update the coordinates of route guide vertices.
Type	Specifies the type of route guide. See the Fusion Compiler user documentation for the list and description of the route guides.
Layers	Lists the layers on which the route guide is placed.
Preferred Direction Only	Routes all nets in the preferred direction within the routing guide boundary. This attribute is equivalent to the preferred_direction_only property in the Fusion Compiler tool. For more information, see the Fusion Compiler user documentation . The Custom Router will not use this attribute during routing. However, you can change the value and the updated value will be available when you open the design in the Fusion Compiler tool.
Switch Preferred Layers	Specifies the layers where the preferred routing direction can be switched. This attribute is equivalent to the switch_preferred_direction property in the Fusion Compiler tool. For more information, see the Fusion Compiler user documentation . The Custom Router will not use this attribute during routing. However, you can change the value and the updated value will be available when you open the design in the Fusion Compiler tool.
Horizontal Track Utilization	Controls the routing density for horizontal tracks within the routing guide boundary.
Vertical Track Utilization	Controls the routing density for vertical tracks within the routing guide boundary.

5. When you are finished, click **Apply Changes** () to save the modifications.

Creating and Editing Routing Corridors

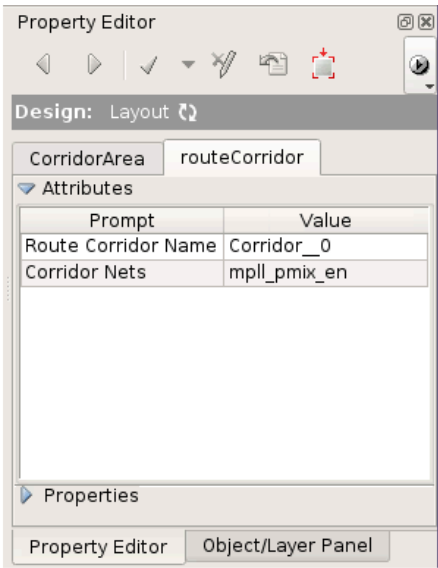
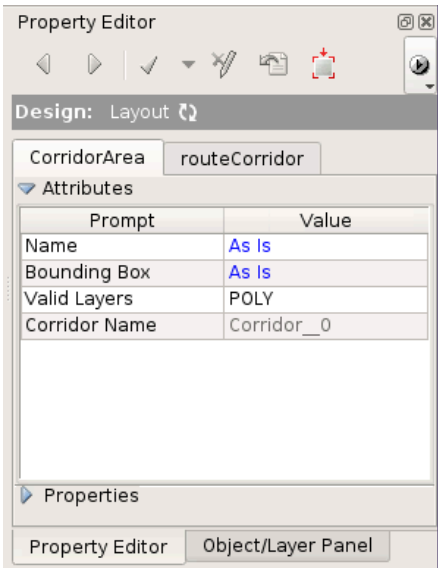
Routing corridors are passageways that restrict where routes can be placed. Corridors can be polygonal or path-shaped. A corridor can consist of multiple corridor regions in the Fusion Compiler tool or corridor areas in the Custom Compiler environment. In the following example, the routing corridor consists of three corridor areas.



You can view, create, and edit routing corridors in the Custom Compiler environment before adding routes. See [Creating Route Corridors](#).

To view and edit Fusion Compiler routing corridors in the Custom Compiler environment:

1. Display the Property Editor.
2. On the canvas, locate and select the routing corridor or corridor area you want to modify. The **Attributes** of the selected routing corridor or corridor area appear in the Property Editor.



3. View or modify the **Corridor Area** attributes. The attributes vary, depending on if you are viewing an Fusion Compiler corridor or a Custom Compiler corridor. The variation also depend on whether you are viewing a corridor or corridor region or area.

Attribute	Description
Name	Name of the selected routing corridor area. By default, the name assigned in the Fusion Compiler tool is displayed. To change the name, double-click the box, and then type a unique name.
Bounding Box	Boundary coordinates of the rectangular routing corridor: { (lowerLeftX, lowerLeftY) (upperLeftX, upperLeftY)}. To change the vertices, click  and update the coordinates in the dialog box.
Points List	List of coordinates that specify the vertices of the polygonal or path routing corridor. To change the vertices, click  and update the coordinates in the dialog box.
Width	Width of the path-shaped routing corridor.
Begin Style	End style where the path shape begins.
End Style	End style where the path shape stops.
Valid Layers	Mask number of the minimum and maximum layer.
Corridor Name	Name of the Route Corridor that contains this specific corridor area. By default, the name assigned in the Fusion Compiler tool is displayed. To change the name, double-click the box, and then type a unique name.

4. View or modify the **Route Corridor** attributes:

Attribute	Description
Route Corridor Name	Name of the routing corridor that contains one or more corridor areas. By default, the name assigned in the Fusion Compiler tool is displayed. To change the name, double-click the box, and then type a unique name.
Corridor Nets	List of nets assigned to the corridor object.

5. When you are finished, click **Apply Changes** () to save the modifications.

Adding and Editing Routes

You can preroute sensitive nets using a set of custom routing options and continue with the Fusion Compiler tool to complete the physical implementation.

This topic describes the following:

- [Before You Start](#)
- [Creating Routes](#)
- [Editing Routes](#)

To add shielding to the routes, see [Adding Shielding](#).

Before You Start

Before you start to add and edit routes, review how Fusion Compiler attributes are mapped in the Custom Compiler environment. You should also understand the use of routing tracks in the design.

This topic describes the following:

- [How Fusion Compiler Attributes Are Mapped](#)
- [How Must-Join Pins Are Treated](#)
- [Wire Track Considerations](#)

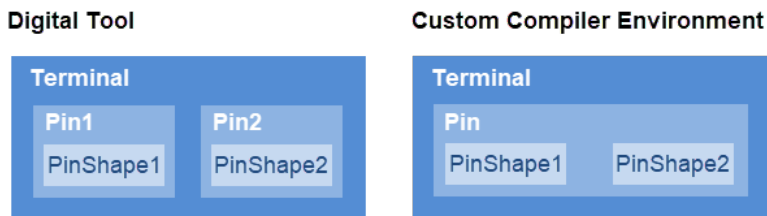
How Fusion Compiler Attributes Are Mapped

As you prepare to add or edit routes in mixed-signal designs, review how Fusion Compiler attributes are mapped in the Custom Compiler environment.

Fusion Compiler Attribute	Custom Compiler Property
<code>must_join_group</code>	<code>ICC_Mustjoin</code> The <code>must_join_group</code> attribute set on pins in the Fusion Compiler reference blocks are mapped to the <code>ICC_Mustjoin</code> property in the Custom Compiler environment. The Custom Router honors this property and connects all the pins that have this property.
<code>is_pattern_must_join</code>	<code>patternMustJoin</code> An attribute of the <code>via_ladder_attributes</code> application option, <code>is_pattern_must_join</code> specifies that the type of via ladder is Pattern Must Join. The attribute is mapped to the Custom Compiler <code>patternMustJoin</code> property.

How Must-Join Pins Are Treated

Digital terminals containing multiple pins are treated as a single pin containing individual pin shapes in the Custom Compiler environment.



If digital terminals contain pins that have the `must_join_group` attribute, a `LEF58_MUSTJOINALLPORTS` property is created for the terminal, and each pin has the `ICC_Mustjoin` property.



Wire Track Considerations

Fusion Compiler designs have wire tracks created automatically as part of the floorplan setup. The wire tracks are used by the Fusion Compiler Zroute to route the design efficiently and to help ensure the layout is free of violations. When manually editing Fusion Compiler designs, you should keep the routes on track as much as possible. You can use the available track and grid adherence options to adjust the strictness of track and grip snapping. These options are available in the Auto Route General Tab and the Interactive Route Layer Tab.

You can use the Custom Compiler Track Pattern and Track Region assistants to display or hide wire tracks. The assistants also provide options for snapping route to tracks during **Create > Interconnect** operations. For details, see Track Region Assistant and Track Pattern Assistant.

Creating Routes

You can edit the digital design in the Custom Compiler environment. This topic describes ways you can add routes to the digital design while in the Custom Compiler environment.

Before you create or modify routes, make sure you:

- Set new or modify existing routing constraints. See [Routing Constraint Reference](#).
- If you are adding routes in digital blocks, set the `rtCustomRouteDesignType` Tcl preference to `digitalBlock`.
- If you want to use digital vias only, set the `leUseDigitalVias` Tcl preference to `true`.

Use one of the following methods to create routes:

Method	Description
Automatically	The Auto Router lets you route multiple nets simultaneously. This method allows the router to control the route path. For instructions, see Auto Routing .
Interactively	The Interactive Router lets you route single nets interactively. This method allows you to control the route path. For instructions, see Interactive Routing .
Automatically, using existing route patterns	The Pattern Route Assistant lets you choose a route pattern from the supplied catalog and create new routes based on the pattern. For instructions, see Pattern Routing .
Manually	The Layout Editor Create > Interconnect command to add interconnects and path segments individually. This method lets you control the placement of the interconnects and path segments. You can also use the Create > Bus command to add bus trunks interactively.

The Auto Router and the Interactive Router integrate both electrical and topological rules as part of the routing process. In addition, you can use available options to add shielding during the routing process or post route. See [Adding Shielding](#).

Routes (shapes) created by the Custom Router (Auto Router, Interactive Router, or Route Pattern Assistant) have the **Custom Route** Route Type property. Routes (shapes) added by other methods within the Custom Compiler environment have the **User Entered** Route Type property. You can view the property in the Property Editor. You can confirm the property values when you review the design in the Fusion Compiler tool. See [Reviewing Custom Compiler Edits in the Fusion Compiler Tool](#).

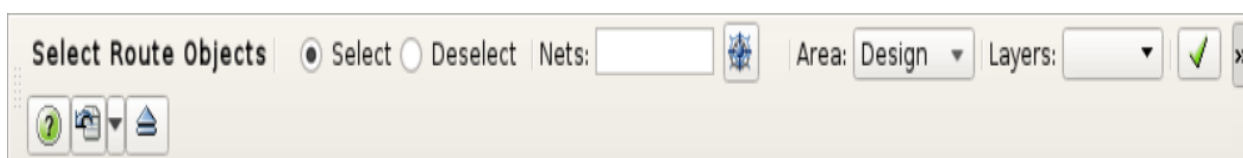
After you finish adding routes, you can save the modified design. Doing so commits all changes to the Fusion Compiler database. See [Saving Designs in the Custom Compiler Environment User Guide](#).

From this point, you can optionally open the design in Fusion Compiler, run the place-and-route operation, and then optionally return the design to the Custom Compiler environment to continue designing.

Before you can open the design for writing in Fusion Compiler, you must either close the design or change its access to read-only while in the Custom Compiler environment. To change the access to read-only, see the *Custom Compiler Environment User Guide*.

Editing Routes

To facilitate selecting and editing tasks, you can use the Layout Editor **Edit > Select > Route Objects** command. The command lets you select, deselect, or delete route shapes by net name, signal type, layer, route type, route status, and specified area. The ability to complete these frequent operations from a single toolbar instead of using different assistants simplifies the selection and deletion tasks. See *Selecting Route Objects*.



For other Layout Editor commands, see *Layout Entry Workflow*.

Adding Shielding

If you enabled the shielding options, shielding is automatically added, based on your specifications, during the auto routing and interactive routing operations. You can choose from different shielding styles. In addition, you can set up shields such that they are shared between two adjacent signal routes.

You can choose to disable shielding during routing and add shielding after the routing operation is finished. Post-route shields are wrapped around existing routes without modifying the route.

This topic describes the following:

- [Setting Shielding Constraints](#)
- [Adding Shielding and Checking Coverage](#)
- [Mapping Shield Constraints](#)

Setting Shielding Constraints

The shielding you add to routes are based on the specifications you provided in the Net Shielding constraints. You can create shielding constraints for one or more nets.

You use the following when setting constraints:

Custom Compiler Component	Task
Design Navigator	Select one or more applicable nets.
Constraint Editor	Create, edit, or modify the Net Shielding constraint.

For instructions on selecting the nets and setting the constraints, see the following topics in the Custom Router User Guide:

- Creating Routing Constraints
- Net Shielding Constraint

Adding Shielding and Checking Coverage

To add shielding during routing:

- *Auto Routing*: Use the options in the Shield Tab of the **Auto Route** dialog box.

To add shielding post-route, choose **Routing > Add Shield**, or choose **Routing > P2P Shield**. See Adding Post-Route Shielding.

You can also use the `cm::createNetShielding` Tcl command. For information, see the *Custom Compiler Tcl Reference Manual*.

Shield vias are automatically added when you add shielding. By default, the **Add Shield** command uses `ndmViaDefs`, which is the via definition created when you added the Fusion Compiler library in the Custom Compiler environment. To ensure that valid vias are used instead of `ndmViaDefs`, you must set the following Tcl preferences:

- `rtSyncShieldViasWithSignalVias = false`
- `rtShieldUseValidViasOnly = true`

`ndmViaDefs` are special via definitions created for the two consecutive metal layers and the cut layer between them when you add the Fusion Compiler library in the Custom Compiler environment. `ndmViaDefs` is used to read vias in Fusion Compiler designs when the via definitions are not in the technology database. The via definitions created by `ndmViaDefs` are named `ndmViaDef_{LowerMetalLayerName}_{UpperMetalLayerName}`.

After you add shielding, you can check the shield coverage to ensure that it is optimal. For more information, see Reporting and Checking Shield Coverage.

Shielding that is created in the Fusion Compiler tool is preserved and not modified in the Custom Compiler environment. Shielding that is created using the Custom Router are preserved and not modified in the Fusion Compiler tool.

Mapping Shield Constraints

The constraints created by the following Fusion Compiler commands are mapped to the Custom Compiler Net Shielding constraint:

Fusion Compiler Constraints	Custom Compiler Constraints
<code>create_net_shielding</code> <code>rule_name</code>	Net Shielding constraint Constraint name
<code>create_routing_rule</code> <code>constraint_group_name</code> or <code>-force name</code>	Net Shielding constraint Constraint name

Net Shielding constraints created within the Custom Compiler environment are mapped to Fusion Compiler constraints as follows:

Custom Compiler Constraints	Fusion Compiler Constraints
Net Shielding constraint for net group <code>groupName</code>	Custom Router shielding constraint Constraint group name

In the following example, the `cm::createNetShielding` Tcl command creates a Net Shielding constraint named `nets2route3`. When you view the design using the Fusion Compiler tool, you should verify that shielding constraint group is named `nets2route3`.

```
set netsList3 {test_si<0> test_si<1>}
set netColl3 [createNetColl $netsList3 [ed]]
set g3 [le::createNetGroup $netColl3 -name nets2route3]
set cs [cm::createNetShielding -nets $g3 -shieldNet VSS -enclosePin false
    -disabledLayers [list PO M5 M6 AP] -style splitCoaxial -layerGaps [list
    "M1 0.14" "M2 0.14" "M3 0.14" "M4 0.14"] -encloseVia true]
```

In the following example, the `cm::createNetShielding` Tcl command creates a Net Shielding constraint named `testgrp4`. When you view the design using the Fusion Compiler tool, you should verify that the shielding constraint group is named `testgrp4`.

```
set netsList4 {test_si<2> test_si<3>}
set netColl4 [createNetColl $netsList4 [ed]]
set cs [cm::createNetShielding testgrp4 -nets $netColl4 -shieldNet
    VSS -gap 0.5 -enclosePin false -disabledLayers [list PO M5 M6] -style
    splitCoaxial -layerGaps [list "M4 0.275"] -encloseVia true]
```

Note that `createNetColl` is a custom Tcl script that converts a list of nets to net collections.

Editing Other Aspects of the Design

You can use other Custom Compiler features to edit the Fusion Compiler design. For example, you can:

- Use **Edit > Wire > Trim Wire** to trim dangling wires that were created by the `add_buffer_on_route` command.
- Adjust pin positions using the **Edit > Distribute** command.
- Add or remove cut metal shapes using the **Edit > Generate Cut Metal** and **Edit > Delete Cut Metal** commands.
- Use editing commands to edit [voltage areas](#). Note that you cannot edit default voltage areas (:DEFAULT_VA).
- Check for missing vias using the **Verification > Via Checker** command, and then add vias using the **Create > Via** command.
- Verify the design using the IC Validator tool.

For the list of features and tools available, see [Custom Compiler Products and Tools](#).

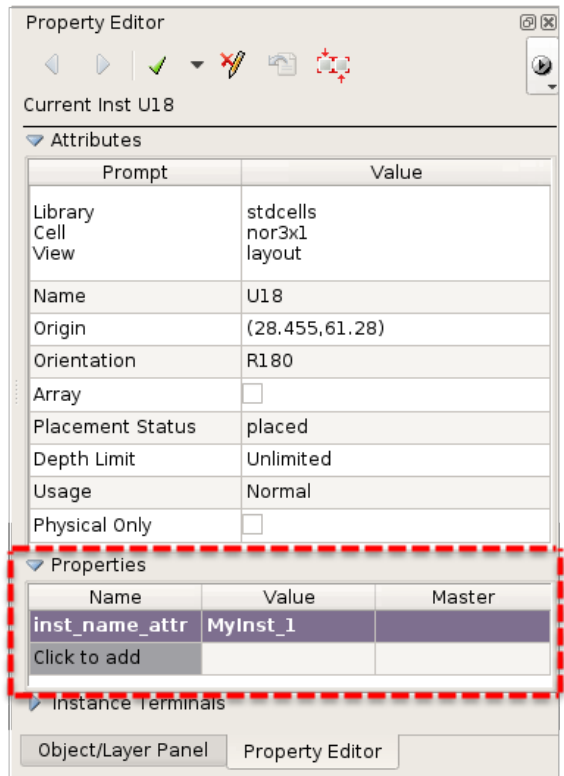
 [Show me](#) an example of cloning instances and bridging interconnects.

Viewing and Editing Object Properties

While viewing the design in the Custom Compiler environment, you can view, edit, and add object properties. Newly added object properties are saved and mapped to object attributes in the Fusion Compiler tool.

To view, edit, or add object properties:

1. Open the Fusion Compiler design in the Layout Editor.
2. Display the Property Editor.
3. On the canvas, locate and select the object of interest. The properties of the selected object appear in the **Property Editor**.
4. View, edit, or add **Properties**. Note that the following data types are permitted: **Boolean, string, integer, and floating numbers**.



To view newly added object properties in the Fusion Compiler tool, see [Retrieving New Object Attributes](#).

The cells with the `is_physical_only` attribute in IC Compiler II is mapped to the `physicalOnly` attribute of the instances in Custom Compiler, as shown in the following examples:

Figure 1 Cell with `is_physical_only` attribute in IC Compiler II

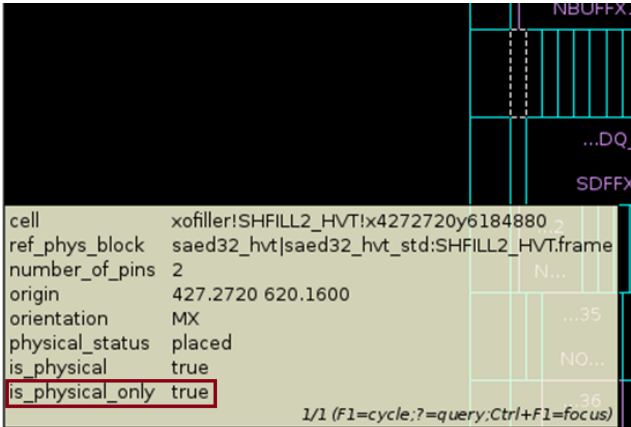


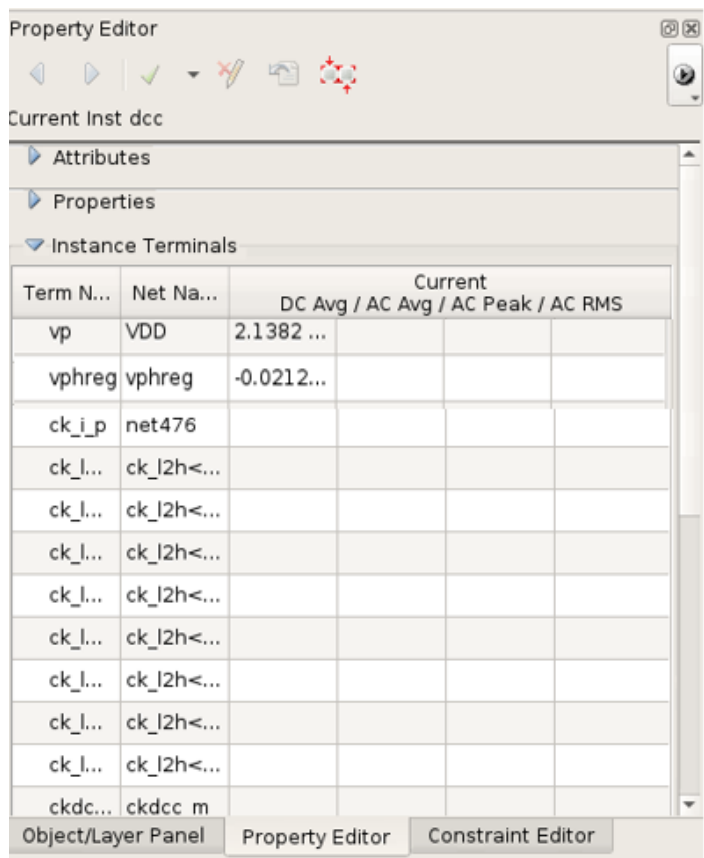
Figure 2 Cell mapped to the `physicalOnly` attribute in Custom Compiler



Setting Terminal Currents

In the **Property Editor**, locate the **Instance Terminals** section, and set the **Current** for the instance terminal. You can set any one of the following terminal currents for an instance: **DC Avg**, **AC Avg**, **AC Peak**, **AC RMS**.

For example, in the following figure, the **DC Avg** terminal current is set for **VDD** and **vphreg** instances.



The saved terminal current values are restored when the **Property Editor** dialog box is closed and launched again. The terminal currents are also saved in the Fusion Compiler™ design.

Reporting and Fixing Electrical Violations

Several Custom Compiler in-design tools allow you to report electrical violations while you are working with the design. Running in-design reports allow you to fix the violations before completing the design, thus reducing the number of costly and time-consuming post-design edits.

 [Show me](#) an example of how to check and report resistance during the co-design process.

 [Show me](#) an example of how to check and report total capacitance during the co-design process.

For details about the in-design electrical reporting features, see [Tools to Use for Electrical Checking and Reporting](#).

Saving Modified Designs

To save the modifications you made within the Custom Compiler environment:

1. In the Layout Editor, save the design. See *Saving Designs in the Custom Compiler Environment User Guide*.
2. Choose **Design > Make Read-Only** or **Design > Close** to release the lock on the design.

When the Fusion Compiler design is unlocked, the modifications you made within the Custom Compiler environment are incorporated. The preroutes you added are marked as `user_route` and the shielding you added are marked as `shield_route` in the Fusion Compiler tool. You can verify these changes in [Reviewing Custom Compiler Edits in the Fusion Compiler Tool](#).

Reviewing Custom Compiler Edits in the Fusion Compiler Tool

After editing designs using the Custom Compiler tool, you can continue editing the design using the Fusion Compiler tool. For example, you can route clock nets or run the place-and-route operation. Before doing so, you should confirm that the changes you made to the design in the Custom Compiler environment appear correctly in the Fusion Compiler tool.

This topic describes the following:

- [Starting the Fusion Compiler Tool](#)
- [Retrieving New Object Attributes](#)
- [Reviewing Newly Added Routes](#)
- [Reviewing Newly Added or Modified Constraints](#)
- [Reviewing Newly Added Shielding](#)

When you finish reviewing the designs, you can continue to edit them using the Fusion Compiler tool. For instructions, see the [Fusion Compiler user documentation](#) available on SolvNetPlus.

Note:

You must be logged in to SolvNetPlus for the link to connect directly to the [Fusion Compiler user documentation](#). If you are prompted to log in to SolvNetPlus, log in, then click the [Fusion Compiler user documentation](#) link again to reach the help topics.

Starting the Fusion Compiler Tool

You can open the current design in the Fusion Compiler tool without leaving the Custom Compiler environment. During this process, the design is changed to the read-only mode in the Custom Compiler environment.

To start open the current design in the Fusion Compiler tool:

- In the Layout Editor, choose **Fusion Compiler > Open in Fusion Compiler**.

The design becomes read-only in the Custom Compiler environment. If you have unsaved changes, a message prompts you to save the changes.

The Fusion Compiler tool opens and displays the design.

Retrieving New Object Attributes

Object properties you added while working in the Custom Compiler environment are saved with the design and mapped to object attributes. To view these attributes in the Fusion Compiler tool, you must retrieve them using Fusion Compiler commands.

To retrieve the object attributes:

1. Define the attribute using the `Fusion Compiler define_user_attribute` command.
2. Retrieve the attribute using the Fusion Compiler `get_attribute` command.

See the [Fusion Compiler user documentation](#) for information about the commands. Note that the following data types are permitted: Boolean, string, integer, and floating numbers.

See [Viewing and Editing Object Properties](#) for information about Custom Compiler object properties.

Example:

While editing the design in the Custom Compiler environment, you added an `inst_name_attr` attribute to an instance.

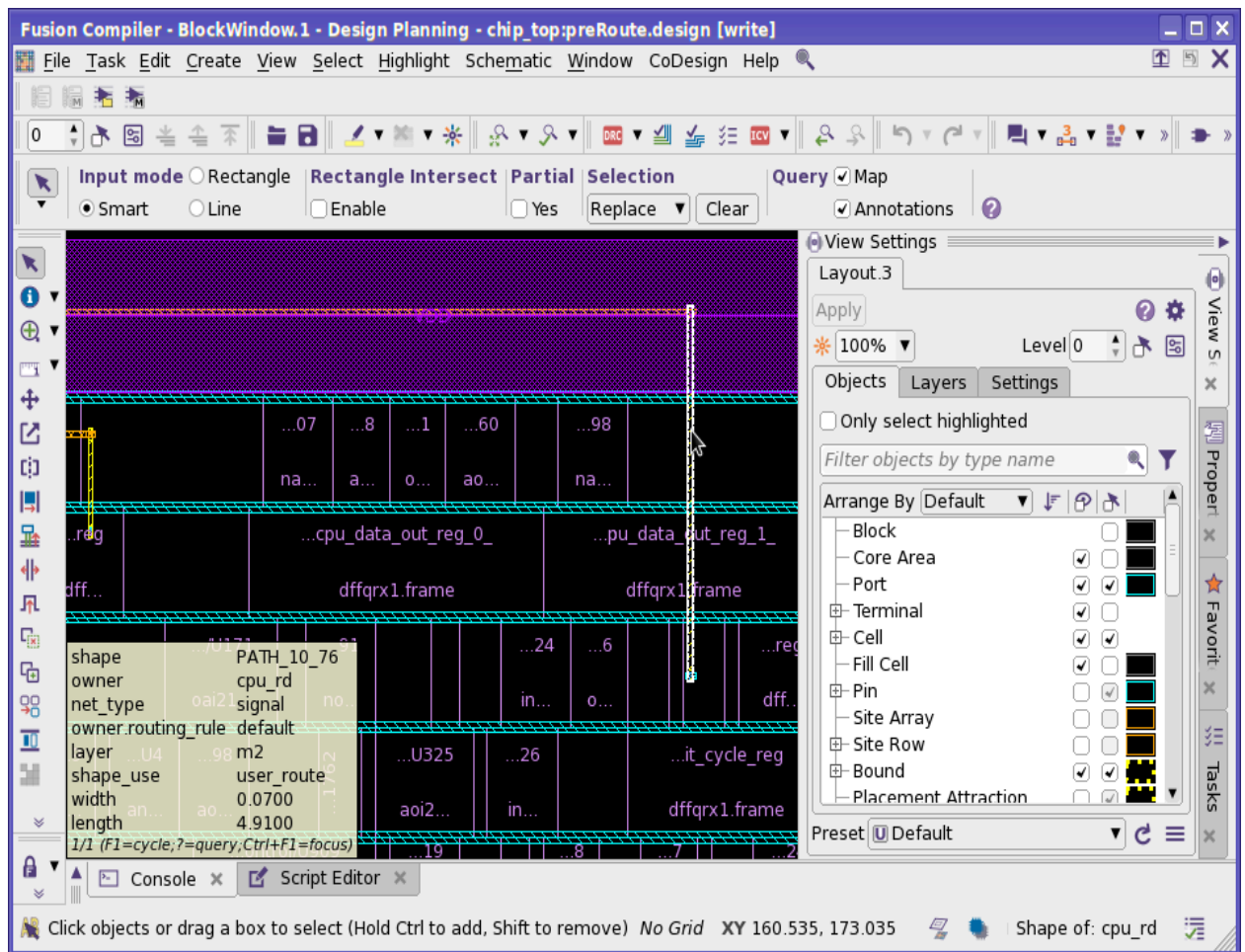
To view that attribute in the Fusion Compiler tool, define and retrieve the attribute using the Fusion Compiler commands:

```
define_user_attribute -type string -classes cell -name inst_name_attr  
get_attribute [get_cells I_PCI_TOP/mult_x_24/U273] inst_name_attr
```

Reviewing Newly Added Routes

To review the newly added routes using the Fusion Compiler tool:

1. Open the design in the Fusion Compiler tool.
2. Highlight routes that were added in the Custom Compiler environment. Information about the routes appears in the lower-left corner.
3. Check that the **shape_use** property for routing and differential-pair routing shapes is **user_route**. The **user_route** property value confirms that the routes were created in the Custom Compiler environment. The newly added routes will not be altered by the Fusion Compiler router.



4. Check that the routing_rule is assigned to the nondefault rules (or constraints) you created in the Custom Compiler environment.

Reviewing Newly Added or Modified Constraints

When reviewing the design, you should check that Custom Compiler constraints are mapped correctly in the Fusion Compiler tool. In addition, make sure that Custom Compiler net groups with nondefault width, spacing, and via constraints are mapped to Fusion Compiler routing rules. For more information, see [Technology Mappings](#).

To check the mappings:

1. Open the **Routing Rule** dialog box, and then:
 - a. Confirm that the new layer constraints you created in the Custom Compiler environment are listed in the dialog box.
 - b. For each new or modified constraint, confirm that the parameter values, such as width, match the values that were set in the Custom Compiler environment.
2. Close the **Routing Rule** dialog box.
3. In the Fusion Compiler console window, type the following command to list the constraint groups used in the design:

```
> report_constraint_groups
```

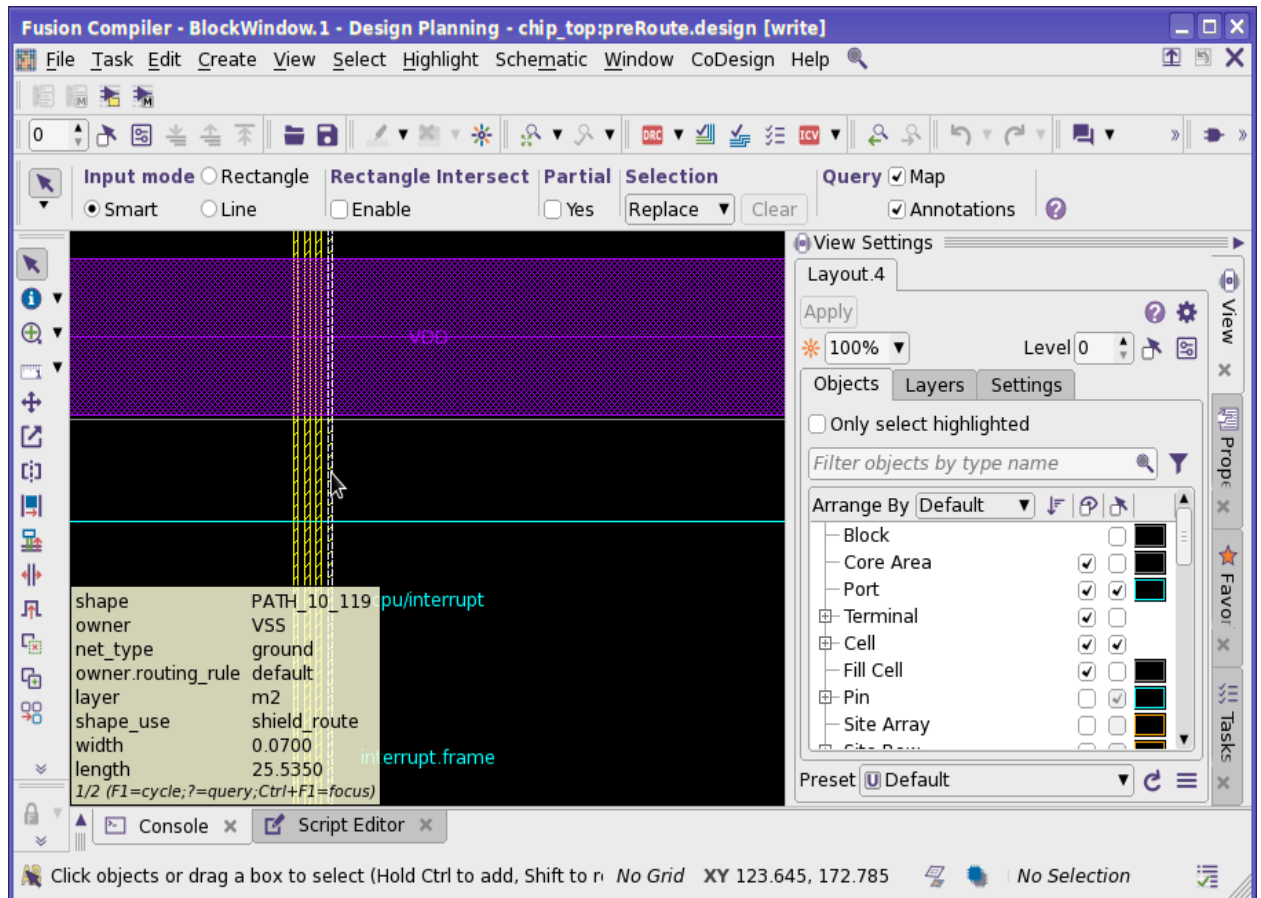
The constraint group report appears in the window and shows the following columns:

- **Name:** The name of the custom routing constraint
 - **Type:** The type of constraint
 - **Objects:** The nets to which the constraint applies
4. Review the constraint group report and make sure newly added routing and shielding constraints appear in the report.
 5. Close the design in the Fusion Compiler tool.

Reviewing Newly Added Shielding

To review the newly added shielding:

1. On the canvas, zoom in and select the newly created nets. Information about the selected objects appears in the lower-left corner.
2. Check that the `shape_use` property is `shield_route` to confirm that the newly created Custom Compiler route shielding appear in the Fusion Compiler tool.



After verifying that the newly added Custom Compiler components appear correctly in the Fusion Compiler tool, you can proceed to route clock nets or run place and route operations. For details, see the [Fusion Compiler user documentation](#) available on SolvNet.

Reviewing Fusion Compiler Edits Within the Custom Compiler Environment

You can open the design using the Custom Compiler tool to review the Fusion Compiler modifications and finalize the design.

This topic describes the following:

- [Reviewing the Digital Edits](#)
- [Importing DRC Data From the Fusion Compiler Tool](#)
- [Importing Error Data From the Fusion Compiler Tool](#)

Reviewing the Digital Edits

To review the edits:

1. Open the design layout in the Custom Compiler environment. See [Opening Fusion Compiler Designs in the Custom Compiler Environment](#).
2. Choose **Design > Make Editable** so that the design is in the edit mode.
3. Display the Design Navigator.
4. In the **Nets** tab, locate and select the new nets that were added in the Fusion Compiler tool.
5. Click  **Shadow Mode** to highlight the selected nets. Check that the newly added Fusion Compiler routes are present and placed correctly.
6. Click  **Shadow Mode** to turn off the highlighting.
7. Press Ctrl+D to clear the net selections.
8. Save the design.

For information about the Custom Compiler products and tools you can use for in-design rule checks and physical verification, see [Custom Compiler Products and Tools](#).

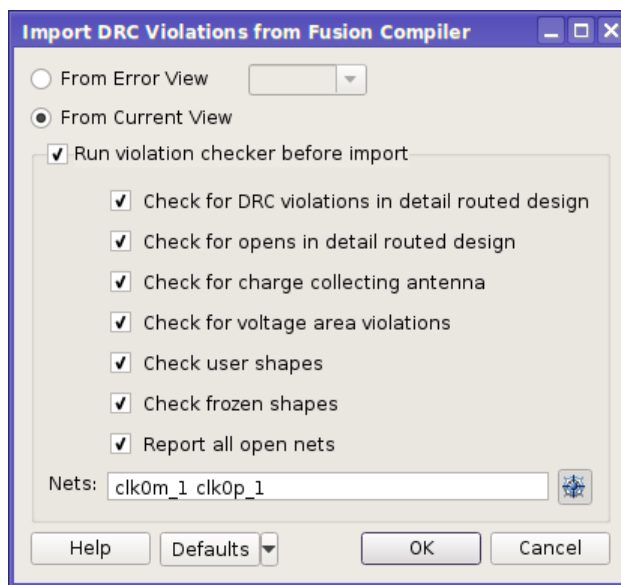
Importing DRC Data From the Fusion Compiler Tool

While reviewing the Fusion Compiler edits, you can:

- Import the Fusion Compiler DRC data and resolve layout errors without having to leave the Custom Compiler environment. You can use the **Autofix** option to locate and fix the violations. Alternatively, you can manually edit the layout to correct the errors. For information about the DRD feature or instructions on how to edit the design, see the Custom Compiler Design Checking and Physical Verification User Guide.
- Run the Fusion Compiler `check_routes` command before the import process if the Fusion Compiler DRC data does not exist, or if you want to update the existing DRC data.

To import DRC data from the Fusion Compiler tool:

1. In the Layout Editor, choose **Fusion Compiler > Import DRC Violations**. The **Import DRC Violations from Fusion Compiler** dialog box opens.



2. Select one of the following:

Import Selection	Description
From Error View	Imports the DRC data from the Fusion Compiler error view. If you select From Error View , you must also choose the error view (error file) to import.

Import Selection	Description
From Current View	Imports the DRC data from the Fusion Compiler layout view. If you select From Current View , you have the option of running the Fusion Compiler <code>check_routes</code> command. See step 3.

3. (Available only if you select **From Current View**) If you want to run the Fusion Compiler `check_routes` command before importing the DRC data, select **Run violation checker before import**, and then select one or more of the following:

DRC Checking Options	Description
Check for DRC violations in detail routed design	Checks for DRC violations of all nets. This option is equivalent to specifying the <code>check_routes -drc</code> argument.
Check for opens in detail routed design	Checks for open nets in the design. This option is equivalent to specifying the <code>check_routes -open_net</code> argument.
Check for charge collecting antenna	Checks for charge-collecting antenna rule violations on the nets. This option is equivalent to specifying the <code>check_routes -antenna</code> argument.
Check for voltage area violations	Checks for voltage-area rule violations on the nets. This option is equivalent to specifying the <code>check_routes -voltage_area</code> argument.
Check for user shapes	Checks for DRC violations that are caused by user-created shapes. This option is equivalent to specifying the <code>check_routes -check_from_user_shapes</code> argument.
Check for frozen shapes	Checks for DRC violations that are caused by shapes on frozen layers and frozen nets. This option is equivalent to specifying the <code>check_routes -check_from_frozen_shapes</code> argument.
Report all open nets	Reports all open nets found. If you do not select the option, the command reports a maximum of 200 open nets. This option is equivalent to specifying the <code>check_routes -report_all_open_nets</code> argument.
Nets	Specifies the nets to verify. During verification, the command checks the specified nets for the following: open nets (<code>-open_net</code>), antenna violations (<code>-antenna</code>), and voltage area violations (<code>-voltage_area</code>). The verify command ignores all other options. The Nets option is equivalent to specifying the <code>check_routes -nets</code> argument.

4. When you are finished, click **OK**.

The co-design assistant saves the current design and makes it read-only and starts an `xtlImportDRC_#` job for the Job Monitor so that you can view the job progress.

If you selected the option to run the Fusion Compiler `check_routes` command, the Custom Compiler II tool creates a custom command file that applies the settings you specified in the **Import DRC Violations from Fusion Compiler** dialog box and runs `check_routes` on the current design.

When the `check_routes` command is finished, the DRC data is imported into the design.

When the import process is finished, the list of DRC violations are displayed in the Error Viewer. For information about the Error Viewer, see the Custom Compiler Environment User Guide.

Note that the tool saves the following files in the `digitalbridge` directory by default:

- `check_routes.tcl`: The Tcl script for the DRC and import process
- `check_routes.log`: The log from the DRC and import process

Importing Error Data From the Fusion Compiler Tool

While reviewing the Fusion Compiler edits, you can:

- Run various Fusion Compiler checking and reporting commands.
- Import the reported layout errors and resolve them without having to leave the Custom Compiler environment.

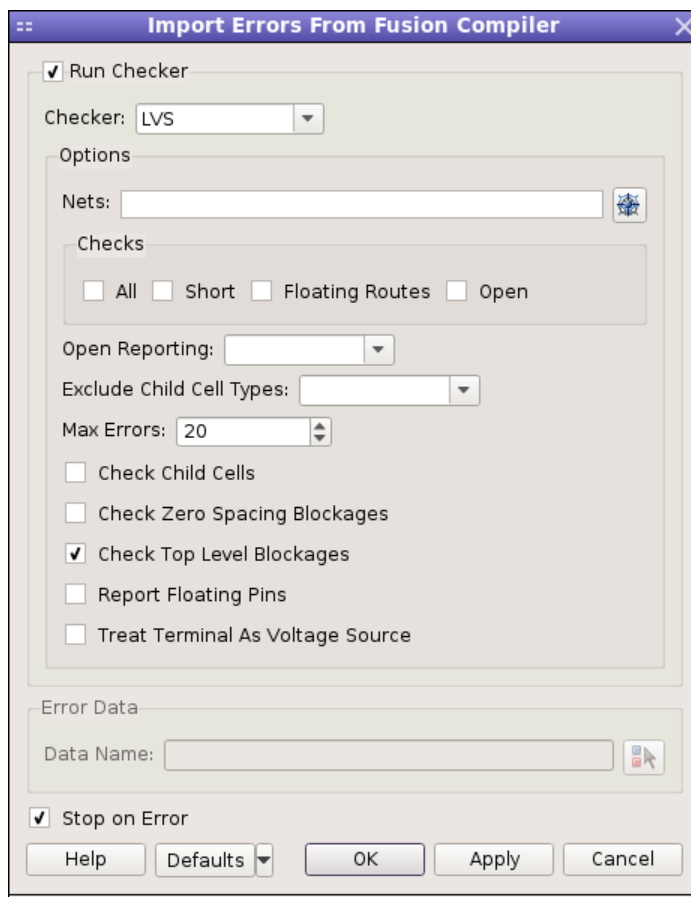
This topic describes the two methods for importing error data:

- [Importing Error Data Using the Import Errors Command](#)
- [Importing Error Data Using Tcl Commands](#)

Importing Error Data Using the Import Errors Command

To import error data from the Fusion Compiler tool:

1. In the Layout Editor, choose **Fusion Compiler > Import Errors**. The **Import Errors from Fusion Compiler** dialog box opens.



2. Select **Run Checker** to run checking and reporting commands. Clear the **Run Checker** check box if you want to import the errors only. See [Step 4](#).
3. If you selected **Run Checker**, set the following options:
 - a. Choose the error **Checker** tool to run:
 - **LVS**: Runs the `check_lvs` command to check for inconsistencies in the physical layout of the current design. When selecting **Nets**, you can select defined nets or net groups.
 - **Legality**: Runs the `check_legality` command to check the legality of the current placement.

- **PG Missing Vias:** Runs the `check_pg_missing_vias` command to check missing vias within the power and ground network.
 - **PG Connectivity:** Runs the `check_pg_connectivity` command to verify physical connectivity of the power ground network.
 - **Routability:** Runs the `check_routability` command to check for issues that can degrade the performance or quality of results of the Fusion Compiler routing commands.
- b. Select the checks and reports you want to run in the **Options** section:
- For the description of the LVS options, see the [check_lvs](#) topic.
 - For the description of the Legality options, see the [check_legality](#) topic.
 - For the description of the PG Missing Vias options, see the [check_pg_missing_vias](#) topic.
 - For the description of the PG Connectivity options, see the [check_pg_connectivity](#) topic.
 - For the description of the Routability options, see the [check_routability](#) topic.
4. If you cleared the **Run Checker** check box, specify the name of the error data file (**Data Name**) for the error data you are importing.
- If you want to overwrite an existing error data file, click  and select the file name.
5. Select **Stop on Error** to run the following commands to halt the checking when errors are encountered:
- `set sh_continue_on_error false`
 - `set sh_script_stop_severity E`
- If you do not select the option, checks will proceed even if there are design errors.
6. When you are finished, click **OK**.

The checks you selected are run in batch mode, and the errors are imported into the Custom Compiler environment as follows:

- `check_pg_missing_vias` creates *blockName_missingVia.err*.
- `check_lvs` creates error markers in *blockName_lvs.err*.
- `check_legality` creates error markers in *blockName_check_legality.err*.

- `check_pg_connectivity` creates error markers in `blockName_check_pg_connectivity.err`.
- `check_routability` creates error markers in `blockName_check_routability.err`.

Importing Error Data Using Tcl Commands

Instead of using the graphical user interface (**Fusion Compiler > Import Errors**), you can use Tcl commands to run the checks and import the error data.

To use the Tcl commands to import the error data:

► In the Console or in a Tcl script, use one or more of the following Tcl commands:

- `xt::checkLVS`
- `xt::checkLegality`
- `xt::checkPGConnectivity`
- `xt::checkPGMissingVias`
- `xt::checkRoutability`
- `xt::checkRoutes`

When running the Tcl commands, the current design opens in the digital tool, and then the digital command (for example, `check_lvs`) checks the design. When the check is finished, the error markers are automatically imported into the Custom Compiler environment.

6

Preroute Editing (Flow From Within the Fusion Compiler Tool)

During preroute editing, a design takes roundtrips between the Fusion Compiler tool and the Custom Compiler environment. This chapter describes how to edit Fusion Compiler designs within the Custom Compiler environment while remaining in the Fusion Compiler tool.

This chapter describes the following:

- [Workflow for Preroute Editing From Within the Fusion Compiler Tool](#)
- [Enabling the Co-design Flow in the Fusion Compiler Tool](#)
- [Opening Fusion Compiler Designs in the Custom Compiler Environment](#)

Workflow for Preroute Editing From Within the Fusion Compiler Tool

This topic shows the basic workflow for preroute editing in the Custom Compiler environment while in the Fusion Compiler tool. If you prefer to work in the Custom Compiler environment independently during the co-design roundtrips, see [Preroute Editing \(Roundtrip Flow\)](#).

Task	See...
1 Enable the co-design flow in the Fusion Compiler tool.	Enabling the Co-design Flow in the Fusion Compiler Tool
2 Open the design in the Fusion Compiler tool.	Reviewing Designs in the Fusion Compiler Tool See also Fusion Compiler user documentation .
3 Open the Fusion Compiler design within the Custom Compiler environment.	Opening Fusion Compiler Designs in the Custom Compiler Environment

Task	See...
4 Review the Fusion Compiler design objects and constraints using the Custom Compiler Layout Editor.	Reviewing Designs Within the Custom Compiler Environment
5 Custom edit the Fusion Compiler design using the Custom Compiler tools. For example, you can create analog preroutes and fine-tune the design.	One or more of the following: • Setting and Editing Constraints • Adjusting Cell Placements • Creating and Editing Blockages • Editing Route Guides • Creating and Editing Routing Corridors • Adding and Editing Routes • Editing Other Aspects of the Design • Saving Modified Designs
6 Review the Custom Compiler edits in the Fusion Compiler tool to make sure the updates appear correctly in the Fusion Compiler tool. After you confirm the changes, you can continue to edit the design in the Fusion Compiler tool. For example, you might want to route clock nets or run place-and-route operations.	Reviewing Custom Compiler Edits in the Fusion Compiler Tool

Enabling the Co-design Flow in the Fusion Compiler Tool

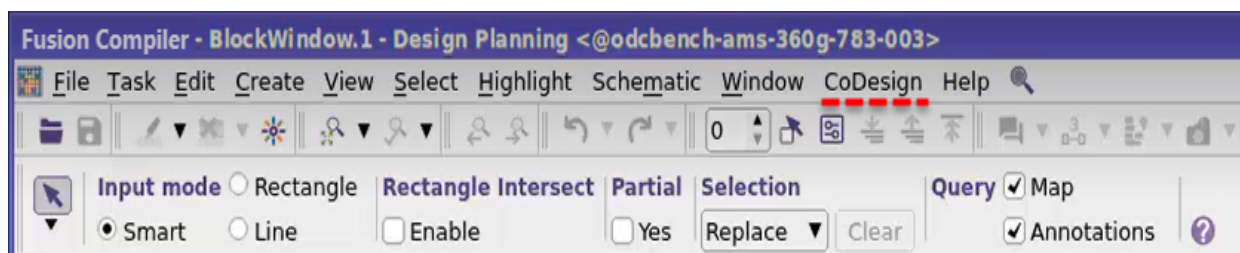
If you want to custom-edit digital designs using the Custom Compiler tool, you must enable the co-design assistant. This topic describes how to enable the co-design assistant in the Fusion Compiler tool.

To enable the co-design assistant:

1. Start the Fusion Compiler tool.
2. At the `fc_shell` prompt, type the following to source the `icc2_codesign.tcl` script.

```
fc_shell > source <installation_dir>/contrib/ndm_oa/icc2_codesign.tcl
```

The **CoDesign** menu appears on the Fusion Compiler menu bar.



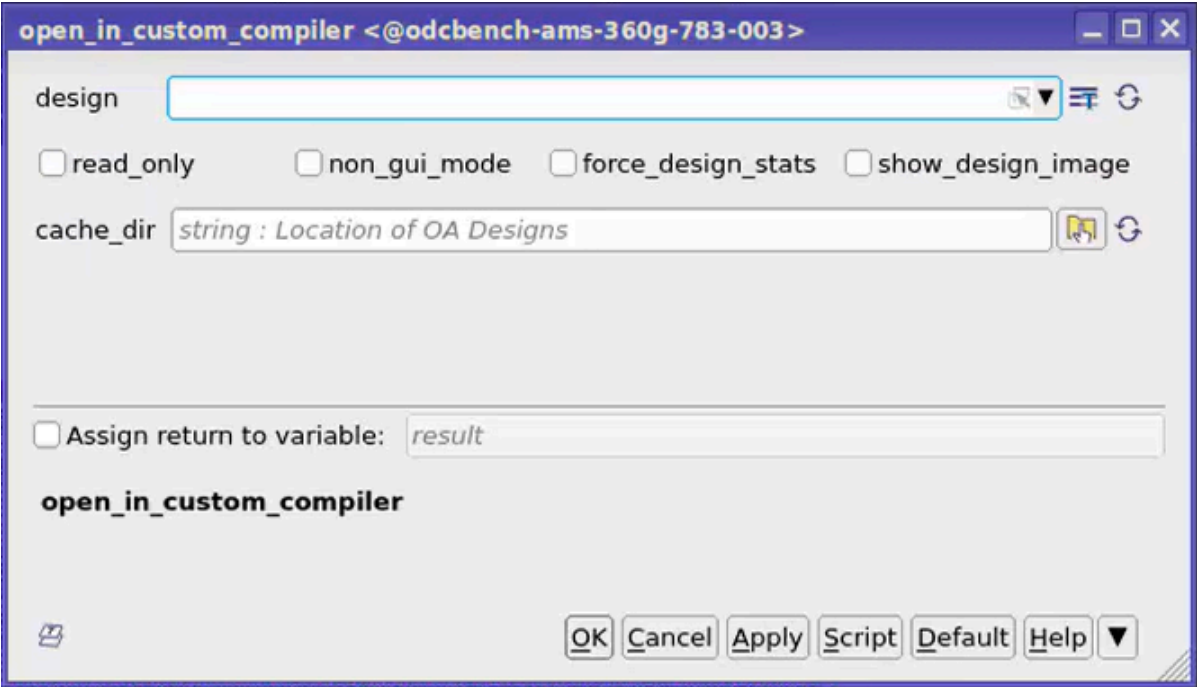
After you enable the co-design assistant, you can proceed to step 2 in the [Workflow for Preroute Editing From Within the Fusion Compiler Tool](#).

Opening Fusion Compiler Designs in the Custom Compiler Environment

After opening and [Reviewing Designs in the Fusion Compiler Tool](#) to obtain a baseline for the blockage, rule, and constraint mappings, you can open the design in the Custom Compiler environment and start editing.

To open the Fusion Compiler in the Custom Compiler environment:

1. In the Fusion Compiler tool, choose **CoDesign > Open in Custom Compiler**.
2. In the **open_in_custom_compiler** dialog box, set the following:



Option	Description
design	The digital design you want to open in the Custom Compiler environment.
read_only	Opens the design in the read-only mode.

Option	Description
non_gui_mode	Creates cached design data in the specified cache_dir directory without opening the Custom Compiler graphical user interface. Select this option if the design is large and would take a long time to open. The cached data is created automatically in the codesign directory whenever you open the design for the first time in the Custom Compiler environment. However, when the design is significantly larger, this option provides the convenience of separating the caching operation from the design-open operation so that the overall process does not take a long time.
force_design_stats	Displays design statistics (number of cells, nets, vias, and shapes) when opening the design in the Custom Compiler environment. If you applied a filter when opening the design, that information is also displayed.
show_design_image	Displays a thumbnail of the design you are opening.
cache_dir	Directory in which the cached design data is stored. The default is codesign.
Assign return to variable	Returns the specified value after the design opens in the Custom Compiler environment.

3. Click **OK**.

The design opens in the Custom Compiler environment, and the design becomes read-only in the Fusion Compiler tool.

After you open the design successfully in the Custom Compiler environment, proceed to step 4 in [Workflow for Preroute Editing From Within the Fusion Compiler Tool](#).

7

Exporting Preroute Floorplans to Tcl Files

After completing the preroute edits, you have the option of exporting the preroute floorplans to Tcl files.

This chapter describes the following:

- [About the Floorplan Export Process](#)
- [Workflow for Exporting Preroute Floorplans to Tcl Files](#)
- [Guidelines for Exporting Preroute Floorplans](#)
- [Turning on Change-Tracking Features](#)
- [Exporting Preroute Floorplans](#)
- [Integrating All Modifications at the Top Level](#)
- [Export Floor Plan Dialog Box Reference](#)

About the Floorplan Export Process

Exporting preroute floorplans to Tcl files allows multiple designers to work on a design simultaneously, thus contributing to increased efficiency and productivity in the co-design process.

In the co-design flow, multiple designers typically work on different components in mixed-signal designs.

- While analog designers work on analog components in the Custom Compiler environment, access to the design is locked so that the digital designer cannot write to the same design in the Fusion Compiler tool.
- While one analog designer is modifying some part of the design, the design is locked to prevent other analog designers from accidentally writing to the same design.

Although design-locking is standard practice in group collaborations, the policy can create bottlenecks in the design process. To work around the potential bottlenecks, multiple

designers can use **Track Changes** and **Export Floorplan** to modify the same design simultaneously.

When multiple designers are given a copy of the same design, they can turn on **Track Changes**, modify a portion of the design, and use **Export Floorplan** to export only the modifications to a Tcl file. See [Workflow for Exporting Preroute Floorplans to Tcl Files](#).

The **Export Floorplan** command exports all or parts of the layout to let each designer:

- Focus on analog nets, associated connection macros, and instances set up in the Custom Compiler environment.
- Work on cleaning up the preroute or postroute edits while the digital designer continues to work on the digital components in the Fusion Compiler tool.
- Reuse the exported floorplan components in other designs or another version of the same design after ECO.

Export Floorplan supports manually inserted filler cells. During the export process, the command creates new cells if they do not exist, or updates the cell attributes if they already exist.

See Also

- [Workflow for Exporting Preroute Floorplans to Tcl Files](#)
- [Guidelines for Exporting Preroute Floorplans](#)
- [Turning on Change-Tracking Features](#)
- [Exporting Preroute Floorplans](#)
- [Integrating All Modifications at the Top Level](#)

Workflow for Exporting Preroute Floorplans to Tcl Files

The **Track Changes** and **Export Floorplan** features allow multiple designers to modify the same design simultaneously. This topic describes the basic workflow for using the features.

Task	See...
1 Before starting, all designers should review the guidelines for exporting the preroute floorplans.	Guidelines for Exporting Preroute Floorplans

Task	See...
2 Because several designers will verify and edit the Fusion Compiler design simultaneously, they are each provided a copy of the design.	-
3 Each designer opens the design with Open By Region and loads only the assigned region.	Opening Filtered Designs
4 Turn on the Connectivity Engine.	Enabling the Connectivity Engine
5 Each designer turns on Track Changes and makes corrections to routes on specific nets.	Turning on Change-Tracking Features
6 Each designer uses the Export Floorplan command and selects either Modified Nets or Modified Routes to export only the modified nets or routes that are tracked. The exported data are stored in a Tcl file.	Exporting Preroute Floorplans
7 The top-level designer loads one or more of the exported Tcl files to merge the edits from the multiple designers.	Integrating All Modifications at the Top Level

Guidelines for Exporting Preroute Floorplans

Before starting the simultaneous modifications and exporting process, be sure to review the guidelines and considerations.

Having multiple designers modifying a design simultaneously requires awareness of the following:

- For power and ground nets or nets that span different design regions:
 - Turn on **Track Changes** and **Highlight Changes** so that you can track the modifications in the different regions in detail. See [Turning on Change-Tracking Features](#).
 - When exporting the modifications, select **Modified Routes** and export only the edits that are made locally.
- All modifications from different designers or design regions must be checked together before merging with the original design.
- Instances and pins connected to the nets you are modifying must remain at the same locations as in the original design.

If you plan to reuse the exported floorplan data in other Fusion Compiler designs, review the following guidelines:

- Instances and pins must remain at the same locations as in the original design.
- Net names in the exported floorplan must match the net names in the Fusion Compiler design.

If the net names do not match, you can rename the nets in the Fusion Compiler design.

See Also

- [About the Floorplan Export Process](#)
- [Workflow for Exporting Preroute Floorplans to Tcl Files](#)
- [Turning on Change-Tracking Features](#)
- [Exporting Preroute Floorplans](#)
- [Integrating All Modifications at the Top Level](#)

Turning on Change-Tracking Features

You can turn on **Track Changes** and **Highlight Changes** to keep track of the modifications you make in the Fusion Compiler design. Later, when you export the preroute floorplan, you can export only the objects that have been modified.

This topic describes the following:

- [About Track Changes](#)
- [About Highlight Changes](#)
- [Turning On Track Changes and Highlight Changes](#)

See Also

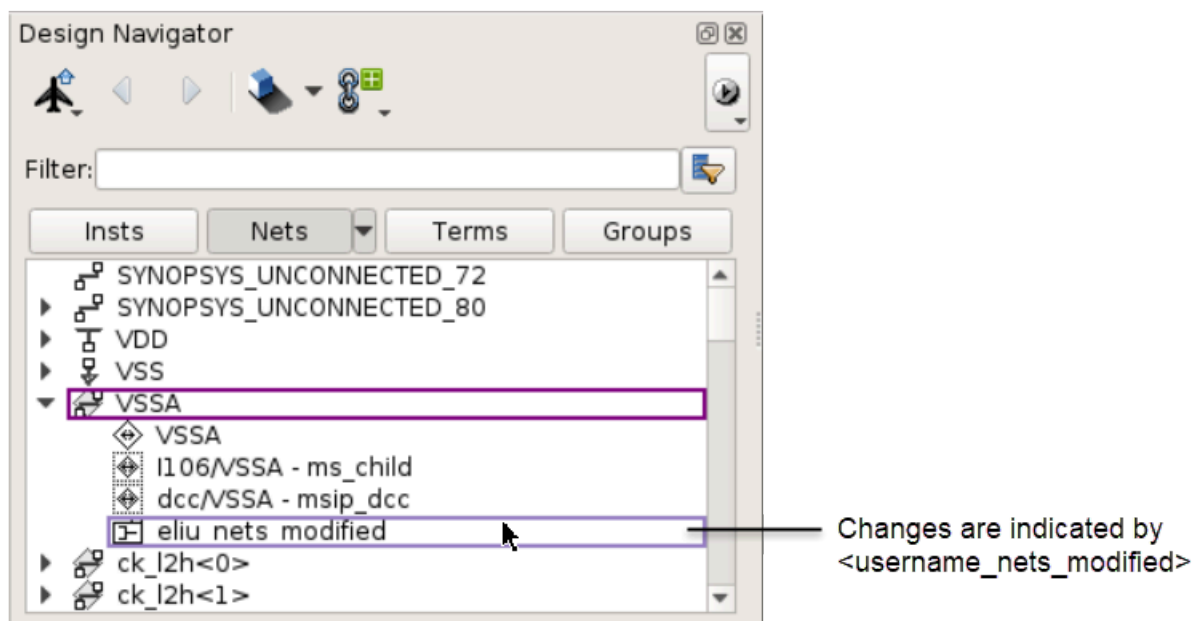
- [About the Floorplan Export Process](#)
- [Workflow for Exporting Preroute Floorplans to Tcl Files](#)
- [Guidelines for Exporting Preroute Floorplans](#)
- [Exporting Preroute Floorplans](#)
- [Integrating All Modifications at the Top Level](#)

About Track Changes

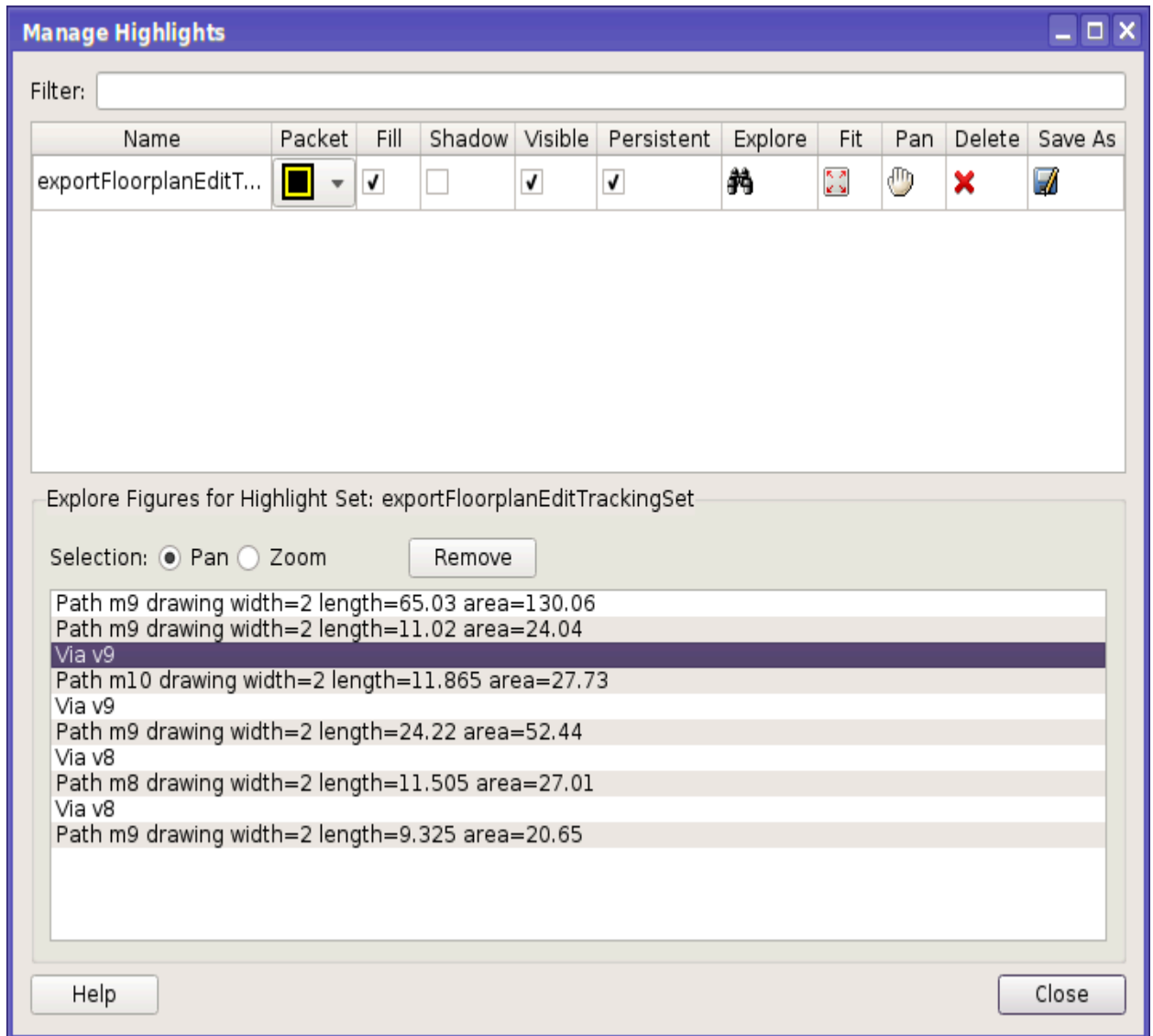
Track Changes displays indicators in the Design Navigator to let you know which net components are modified. In addition, the feature keeps track of the modifications you make so that you can export only the objects that have been modified.

When you turn on **Track Changes**, the nets you modify appear in the Design Navigator. The changes are listed under the net name, and are indicated by **<username_nets_modified>**.

Track Changes



Turning on **Track Changes** also creates a highlight set called **exportFloorplanEditTrackingSet**. Thereafter, modified objects are added to this highlight set. You can view the detailed list of objects that are modified and change the properties of this highlight set (for example, display packet and visibility) in the **Manage Highlights** dialog box. See Managing Highlight Settings.

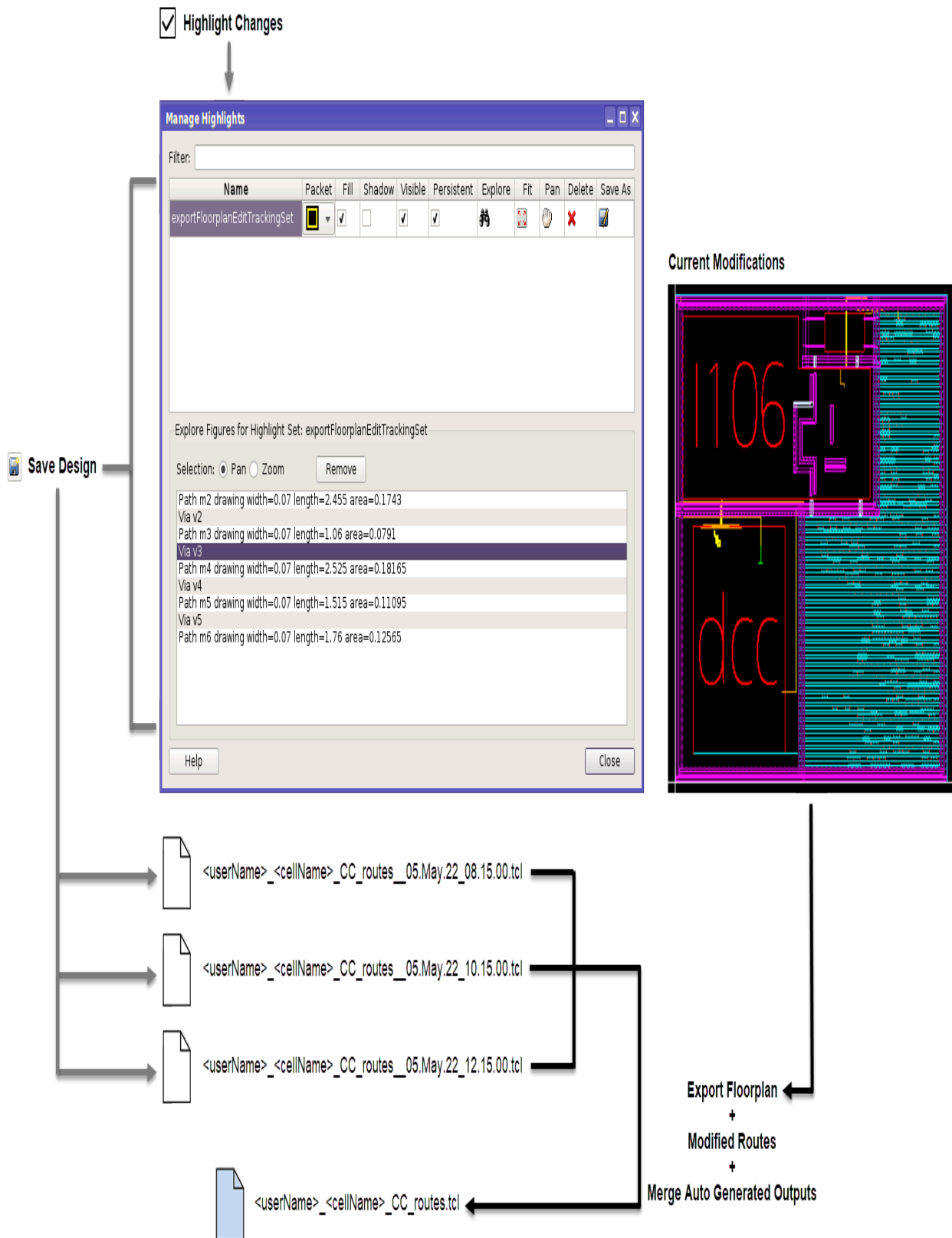


Every time you save the design:

- The tracked modifications in the **exportFloorplanEditTrackingSetp** highlight set are saved to the following file:
`.../<userName>_<cellName>_CC_routes_<datetime>.tcl`
- The tracked changes in the **exportFloorplanEditTrackingSetp** highlight set are cleared.

While in the current design session, if you export the floorplan with the **Modified Routes** and **Merge Auto Generated Outputs** options, the various modification .tcl files saved

during the design session are combined with any unsaved modifications (if any) into a single file, `.../<userName>_<cellName>_CC_routes.tcl`.

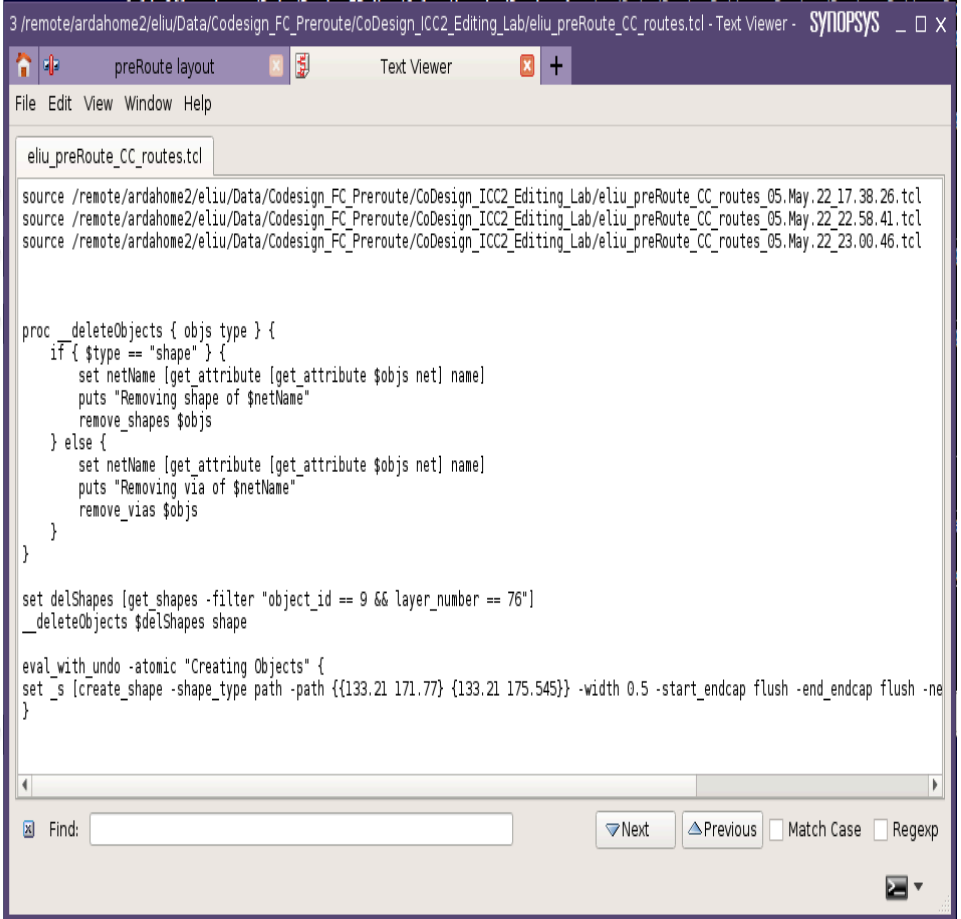


The .../<userName>_<cellName>_CC_routes.tcl  file consists of:

- Lines of code to source the individual <userName>_<cellName>_CC_routes_<datetime>.tcl files that contain previously saved modifications.
- Code that replays the non-saved modifications from the current design session.

Previously saved modifications

Non-saved modifications from
the current design session



```
3 /remote/ardahome2/eliu/Data/Codesign_FC_Preroute/CoDesign_ICC2_Editing_Lab/eliu_preRoute_CC_routes.tcl - Text Viewer - SYNOPSYS
preRoute layout
Text Viewer
File Edit View Window Help
eliu_preRoute_CC_routes.tcl

source /remote/ardahome2/eliu/Data/Codesign_FC_Preroute/CoDesign_ICC2_Editing_Lab/eliu_preRoute_CC_routes_05.May.22.17.38.26.tcl
source /remote/ardahome2/eliu/Data/Codesign_FC_Preroute/CoDesign_ICC2_Editing_Lab/eliu_preRoute_CC_routes_05.May.22.22.58.41.tcl
source /remote/ardahome2/eliu/Data/Codesign_FC_Preroute/CoDesign_ICC2_Editing_Lab/eliu_preRoute_CC_routes_05.May.22.23.00.46.tcl

proc _deleteObjects { objs type } {
    if { $type == "shape" } {
        set netName [get_attribute [get_attribute $objs net] name]
        puts "Removing shape of $netName"
        remove_shapes $objs
    } else {
        set netName [get_attribute [get_attribute $objs net] name]
        puts "Removing via of $netName"
        remove_vias $objs
    }
}

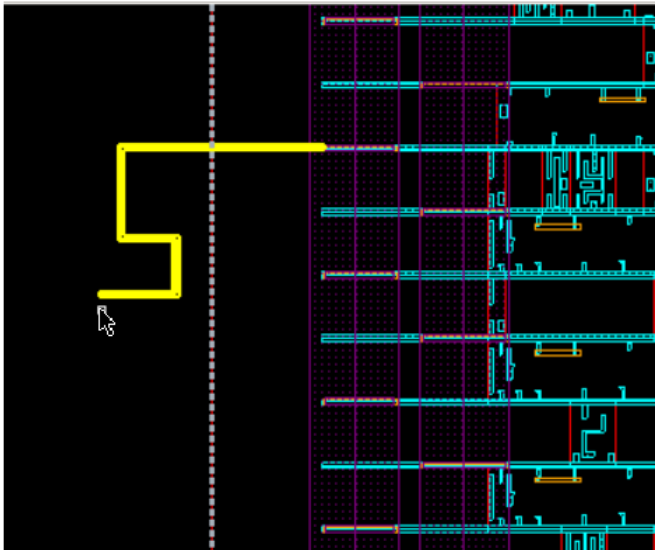
set delShapes [get_shapes -filter "object_id == 9 && layer_number == 76"]
_deleteObjects $delShapes shape

eval_with_undo -atomic "Creating Objects" {
    set _s [create_shape -shape_type path -path {{133.21 171.77} {133.21 175.545}} -width 0.5 -start_endcap flush -end_endcap flush -ne
}
```

About Highlight Changes

Highlight Changes adds highlighting color to the components you modify in the layout for tracking the changes visually. The feature can be helpful when reviewing the modifications you want to export.

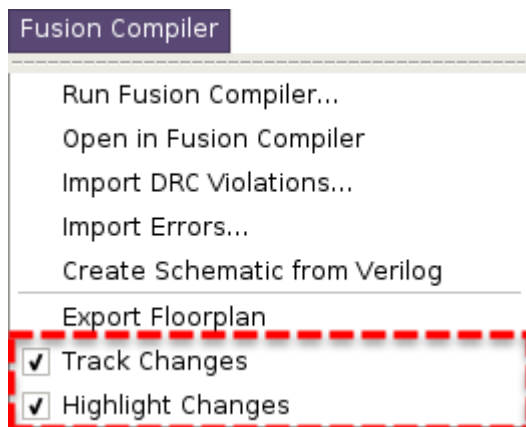
Highlight Changes



Turning On Track Changes and Highlight Changes

To turn on the change-tracking features:

- In the Layout Editor, choose **Fusion Compiler > Highlight Changes** or **Fusion Compiler > Track Changes**.



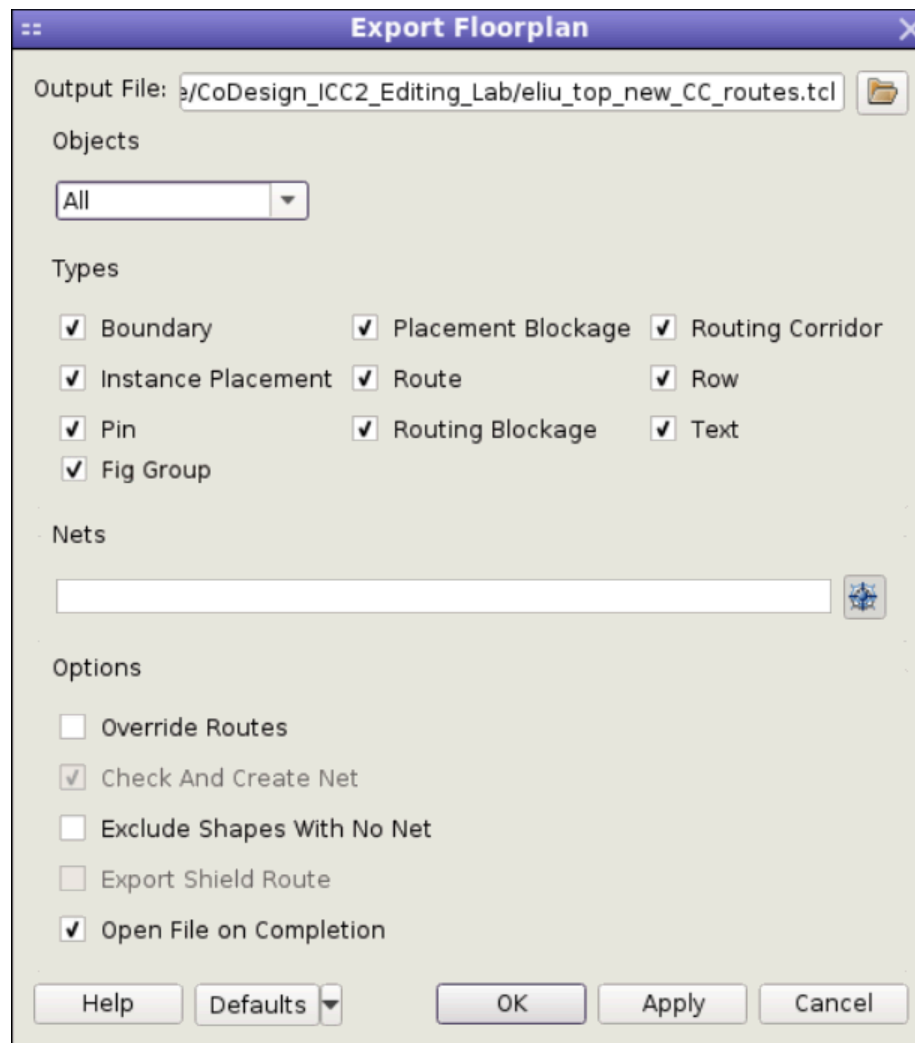
Exporting Preroute Floorplans

You can export the preroute floorplans to a Tcl file to run cleanup processes while the digital designers continue to work on the design in the Fusion Compiler tool. Exporting floorplans also lets you reuse the floorplan in other designs.

To export the preroute floorplans:

1. Make sure you have enabled the co-design assistant. See [Enabling the Co-design Assistant](#).
2. With the design opened in the Layout Editor, choose **Fusion Compiler > Export Floorplan to File**.

The **Export Floorplan** dialog box opens.



3. Specify the **Output File Name**, the name and location of the exported Tcl file containing the floorplan data.

4. Choose the floorplan **Objects** to export:

Argument	Description
All	All nets and routes in the floorplan.
Modified Nets	Only nets that have been modified in the Custom Compiler environment.
Modified Routes	Only routes that have been modified in the Custom Compiler environment.
Selected Routes	Only the routes you selected.

5. If you selected **Objects: All**, select one or more of the object **Types** to export.

6. If you selected **Objects: All**, specify the **Nets** to further filter the objects.

To specify a list of nets, add a character space between the net names.

7. Select the **Options** to be applied during the export process.

The options that are available depends on the objects you selected in step 4.

Option	Description
Override Routes	Replaces the routes in the Fusion Compiler design.
Check and Create Nets	Adds the following commands in the exported Tcl file for checking and creating nets: <pre>set _tmpNet [get_nets <netName> -quiet] if { \$_tmpNet == "" } { set _tmpNet [create_net <netName>] }</pre> For information about the <code>create_net</code> command, see the Fusion Compiler user documentation. This option is available only if you choose Objects: All or Objects: Modified Nets.
Exclude Shapes With No Net	Excludes objects if they do not have net assignments.
Export Shield Route	If routes have shielding, includes the shield objects during the export process.
Open File on Completion	Opens the Tcl file in the system default text editor when the export process is finished.

Option	Description
Merge Auto Generated Outputs	When used with the Modified Routes selection, combines all modification data from the current design session into a single file. The following are merged: ◦ Tcl files that are generated every time you save the design within the current design session. ◦ Any unsaved modifications in the current design session. See About Track Changes .

8. Click **OK**.

The selected floorplan data and the associated highlight set information (**exportFloorplanEditTrackingSet**) are exported to the Tcl file (`.../
<userName>_<cellName>_CC_routes.tcl`).

See Also

- [About the Floorplan Export Process](#)
- [Workflow for Exporting Preroute Floorplans to Tcl Files](#)
- [Guidelines for Exporting Preroute Floorplans](#)
- [Turning on Change-Tracking Features](#)
- [Integrating All Modifications at the Top Level](#)

Integrating All Modifications at the Top Level

As the top-level designer, you can collect all the exported floorplan files from the multiple designers and integrate them at the top level.

Before you start, make sure you check all modifications from the different designers or design regions together. See [Guidelines for Exporting Preroute Floorplans](#).

To integrate all the various modifications at the top-level:

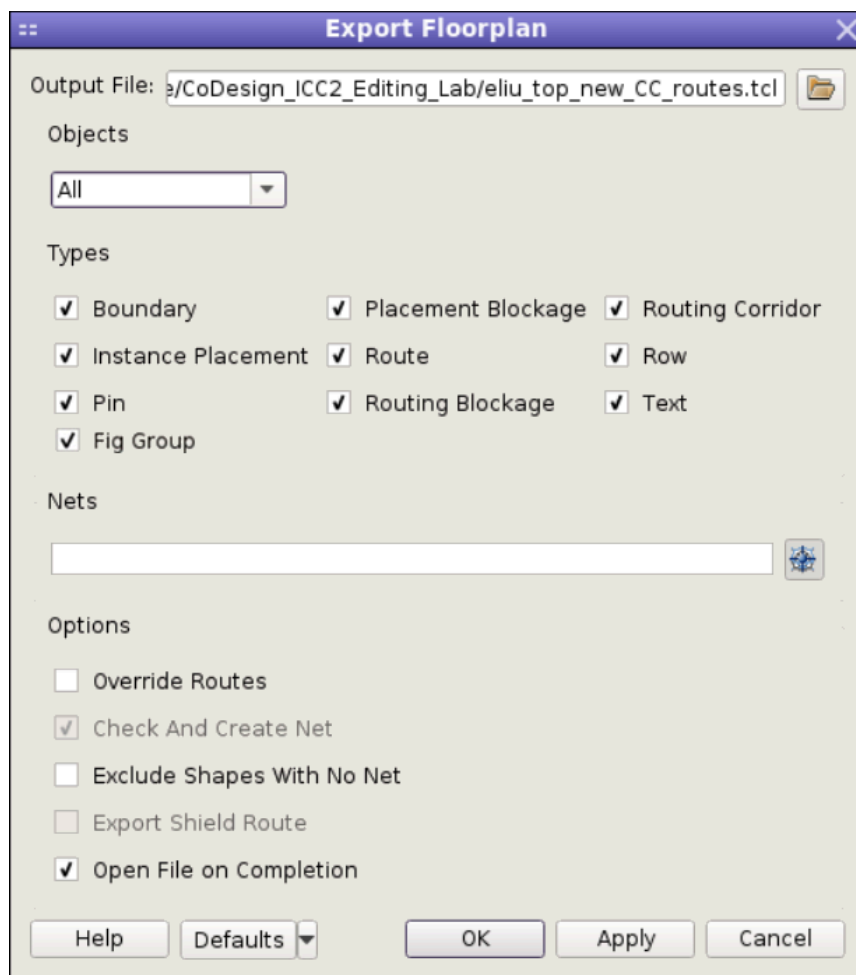
1. Collect the exported floorplan files (`<userName>_<cellName>_CC_routes.tcl`) from your designers.
2. Open the design in the Fusion Compiler tool.
3. Source the individual `<userName>_<cellName>_CC_routes.tcl` files.

See Also

- [About the Floorplan Export Process](#)
- [Workflow for Exporting Preroute Floorplans to Tcl Files](#)
- [Guidelines for Exporting Preroute Floorplans](#)
- [Turning on Change-Tracking Features](#)
- [Exporting Preroute Floorplans](#)

Export Floor Plan Dialog Box Reference

This topic presents a quick reference of the **Export Floorplan** dialog box. For instructions on exporting preroute floorplans to Tcl files, see [Exporting Preroute Floorplans](#).



Output File

Option	Description
Output File	Name and location of the exported Tcl file.

Objects

Route objects to export to the Tcl file.

Option	Description
Floorplan objects to export	• All: All nets and routes in the floorplan. • Modified Nets: Only nets that have been modified in the Custom Compiler environment. • Modified Routes: Only routes that have been modified in the Custom Compiler environment. • Selected Routes: Only the routes that you select.
Types of objects to export	• Boundary: Boundary objects • Instance Placement: Instance objects. • Pin: Pin objects. • Placement Blockage: Placement blockage objects. • Route: Route objects. • Routing Blockage: Route blockage objects. • Routing Corridor: Route corridor objects. • Row: Row objects. • Text: Text objects. • Fig Group: Figure group objects.

Nets

Option	Description
Nets	List of nets of the selected objects. Add a character space between net names.

Options

Options to be applied during the export process. The options that are available depends on the [Objects](#) you selected.

Option	Description
Override Routes	Replaces the routes in the Fusion Compiler design.
Check and Create Nets	Adds the following commands in the exported Tcl file for checking and creating nets: <pre>set _tmpNet [get_nets <netName> -quiet] if { \$_tmpNet == } { set _tmpNet [create_net <netName>] }</pre> For information about the <code>create_net</code> command, see the Fusion Compiler user documentation. This option is available only if you choose Objects: All or Objects: Modified Nets.
Exclude Shapes With No Net	Excludes objects if they do not have net assignments.
Export Shield Route	If routes have shielding, includes the shield objects during the export process.
Open File on Completion	Opens the Tcl file in the system default text editor when the export process is finished.
Merge Auto Generated Outputs	When used with the Modified Routes selection, combines all modification data from the current design session into a single file. The following are merged: • Tcl files that are generated every time you save the design within the current design session. • Any unsaved modifications in the current design session. See About Track Changes.

8

Creating Black Boxes for Analog Blocks

This chapter describes how to create black boxes for analog blocks to ensure accurate timing analysis.

This chapter describes the following:

- [About Analog Blocks and Digital Timing Shells](#)
- [Workflow for Creating Black Boxes](#)
- [Preparing for Digital Timing Shells](#)
- [Creating and Editing Black Boxes](#)
- [Running Place and Route](#)

About Analog Blocks and Digital Timing Shells

This topic describes the following:

- [Analog Cell Timing Analysis](#)
- [Black Box Representations of Analog Blocks](#)

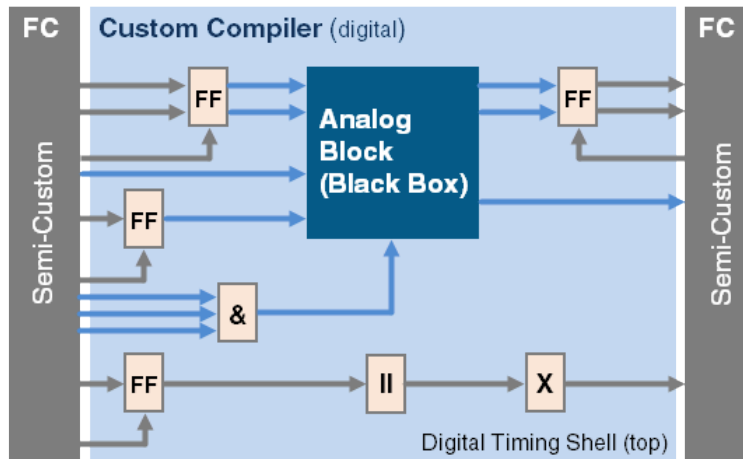
Analog Cell Timing Analysis

To verify that your design meets defined timing constraints, you can perform timing analysis. Accurate timing analysis of analog cells requires that you create timing models for every instance in the design before place and route. The traditional analysis process can be extensive and lengthy: The setup is tedious, complete layouts are required, and transistor-level simulations must be performed. In addition, the accuracy of the timing analysis depends on the limited operating conditions available, and changes require rerunning the time-consuming simulation.

To simplify timing models and improve the accuracy of the analysis, you can wrap digital timing shells around analog blocks. Digital timing shells consist of flip-flops, which enable the generation of a block abstraction in the Fusion Compiler tool. The abstract models

provide highly accurate timing representations of the analog blocks for top-level design timing analysis.

Digital Timing Shell Wrapped Around an Analog Block



With timing shells:

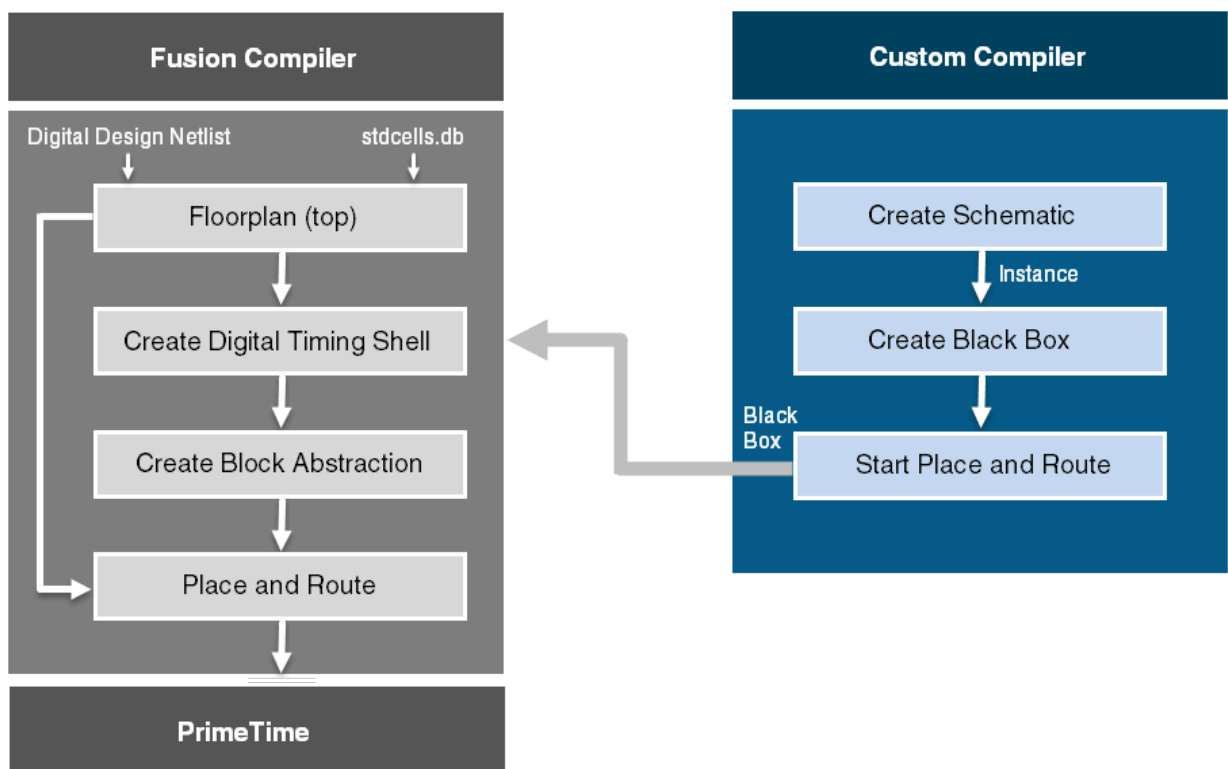
- No layout is required.
- The analog block can be a black box.
- No SPICE simulation needed.
- The process covers all operating conditions.
- Support for multi-voltage automated flow is available.
- The model view is not dependent on analog cell changes.

Black Box Representations of Analog Blocks

The following figure shows the timing shell implementation in the co-design process. Notice that you must first create black box representations of the analog blocks in the Custom Compiler environment before generating abstracts of these blocks in the Fusion Compiler tool.

The black boxes are created from instances in the design schematic and contain floorplanning, power, and ground information. After you create the black boxes, you run place and route to create the timing shell and block abstraction. See [Workflow for Creating Black Boxes](#).

Timing Shell Implementation



Workflow for Creating Black Boxes

The basic workflow for creating black boxes is as follows:

Task	See...
1 Prepare for timing shells.	Preparing for Digital Timing Shells
2 Create frame views for the black boxes.	Creating Frame Views
3 Create black boxes for the analog blocks.	Creating and Editing Black Boxes
4 Add the Fusion Compiler library.	Adding Fusion Compiler Libraries
5 Run place and route.	Running Place and Route

Preparing for Digital Timing Shells

This topic describes the requirements for creating digital timing shells:

- [Preparing the Design Schematic](#)
 - [Creating Connection Definitions for the Timing Shell](#)
 - [Defining the Power Supplies](#)
 - [Setting Up Block Abstraction Creation](#)
-

Preparing the Design Schematic

Make sure the design schematic and the instances (analog blocks) meet the following requirements:

- The design must contain standard cells.
- All power must use connection definitions. See [Creating Connection Definitions for the Timing Shell](#).
- The design must not contain primitive devices and explicit power and ground connections.
- The power supplies are defined. See [Defining the Power Supplies](#).

Creating Connection Definitions for the Timing Shell

Connection definitions enable the use of IP from different sources using multiple power supplies by mapping to power domains and connecting supply statements. For a multi-power-supply flow that uses UPF file generation, the schematic needs to have connection definitions.

After preparing your timing libraries and creating the necessary views by modifying the Fusion Compiler Place and Route command file, you can create connection definitions.

To create the connection definitions:

1. Open the design schematic.
2. Follow the instructions for Creating Connection Definitions in the *Custom Compiler Schematic Editor User Guide*.

The connection definitions are mapped to UPF power domains and supply nets as follows:

Connection Definitions	UPF Mapping	Example
All connection definitions	Supply nets/ports are created Note: ! is removed from the names	<pre>create_supply_port vdd create_supply_net vdd connect_supply_net vdd -ports vdd</pre>
stdcells connection definitions	Primary Power net/Ground net Top Power Domain is created with these nets	<pre>set_domain_supply_net Top_PD -primary_power_net vdd -primary_ground_net vss</pre>
BlackBox with connection definitions	Supply nets are created and black box ports are connected to the supply nets	<pre>create_supply_port vdda create_supply_net vdda connect_supply_net vdd -ports vdda connect_supply_net vdda -ports {BB1/vdda}</pre>

Defining the Power Supplies

To define the power supplies, in the Console, type the following:

```
db::setPrefValue dbSigTypeGroundFilter -value "*vss* *gd*"
```

```
db::setPrefValue dbSigTypePowerFilter -value "*vdd* *vp*"
```

Setting Up Block Abstraction Creation

If you want the system to create block abstractions after place and route, set the following preferences in the `TLUplus` files and store them in the constraint map folder:

Preference	Description
<code>leICCTLUPlusMapFile</code>	Sets the value of the <code>TLUPLUS_MAP_FILE</code> in the Fusion Compiler template. Specifies the name of the file that defines the layer mapping between the Fusion Compiler technology file and the Interconnect Technology Format (ITF) file.
<code>leICCTLUPlusMinFile</code>	Sets the value of the <code>TLUPLUS_MIN_FILE</code> in the Fusion Compiler template. The Fusion Compiler tool uses this TLUPlus file for minimum condition calculation of resistance and capacitance.
<code>leICCTLUPlusMaxFile</code>	Sets the value of the <code>TLUPLUS_MAX_FILE</code> in the Fusion Compiler template. The Fusion Compiler tool uses this for maximum condition calculation of resistance and capacitance.

Creating and Editing Black Boxes

Caution:

Before you start, make sure you create frame views of the analog macros (blocks). See [Creating Frame Views](#).

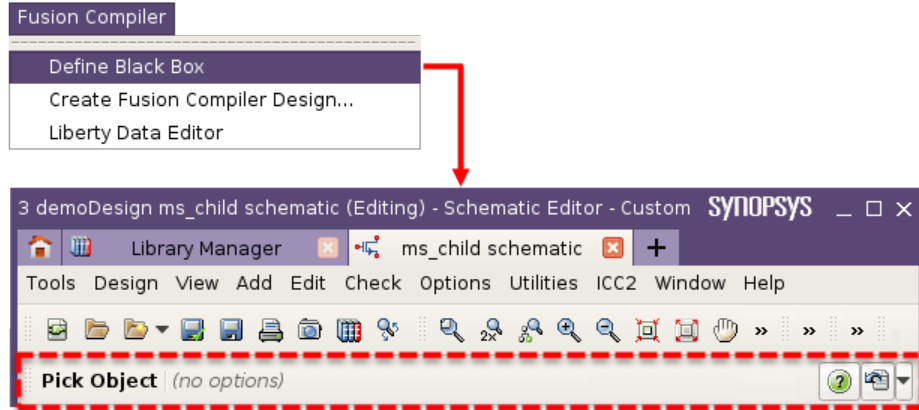
Before you run place and route to create the timing shell, you must create black boxes for analog blocks in the design schematic. See [Black Box Representations of Analog Blocks](#).

Note:

You can create only one black box per design schematic.

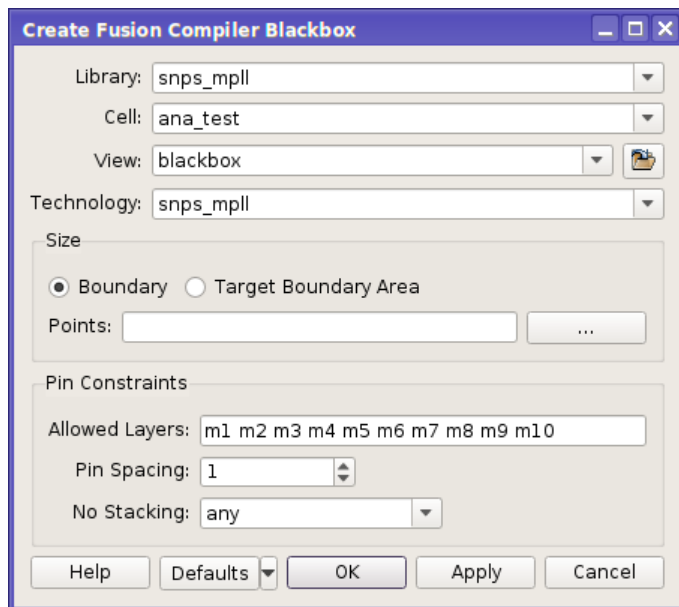
To create or edit a black box for an analog block:

1. Open the design schematic.
2. In the Schematic Editor, choose **Fusion Compiler > Define Black Box**. The **Pick Object** toolbar appears, indicating that the **Define Black Box** command is active.

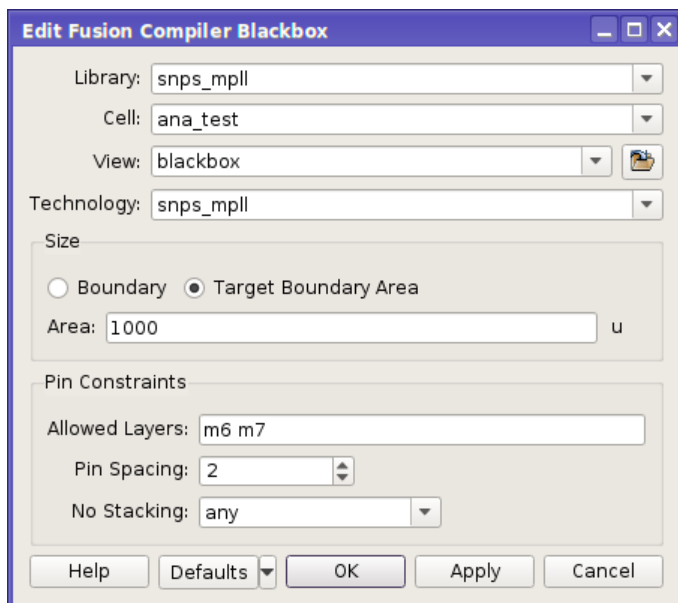


3. On the schematic canvas:

- To create a black box, select the instance on the schematic canvas. The **Create Fusion Compiler Blackbox** dialog box opens.



- To edit a black box, select the black box that was created for the instance on the schematic canvas. The **Edit Fusion Compiler Blackbox** dialog box opens.



4. Set or modify the following at the top of the **Create Fusion Compiler Blackbox** dialog box:

Setting	Description
Library, Cell, View	The library, cell, and view of the design that contains the selected instance.
Technology	The technology that contains the constraints for the design.

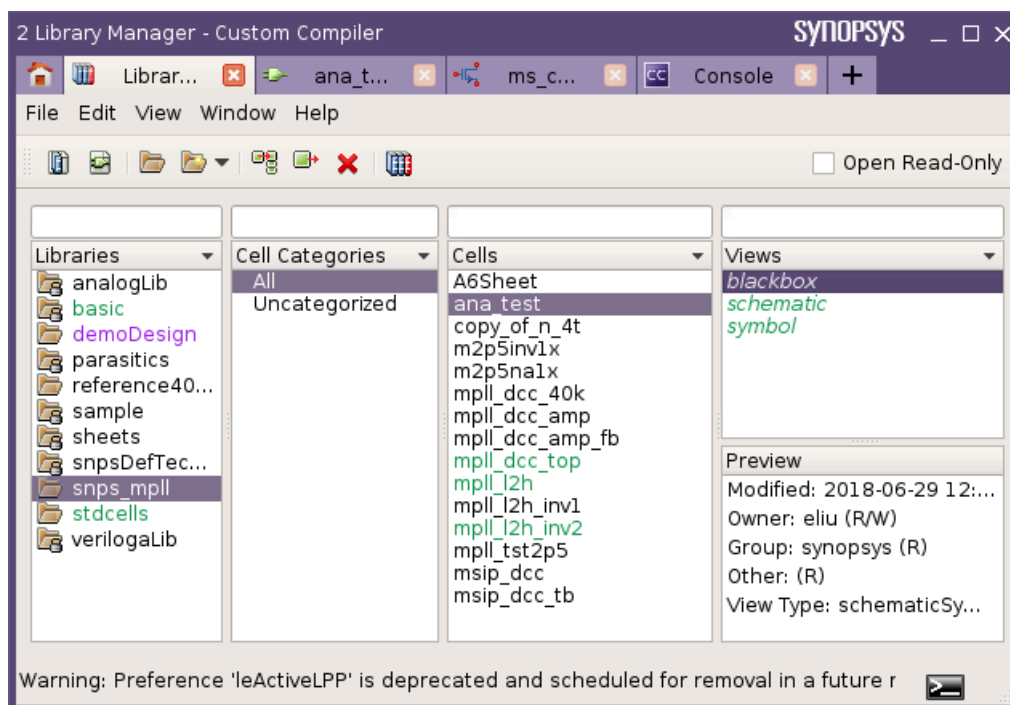
5. In the **Size** area, specify or modify the black box dimensions using one of the following methods:

Setting	Description
Boundary	The coordinates of the black box vertices. The black box can be rectangular or polygonal.
Target Boundary Area	The total area of the black box, in microns.

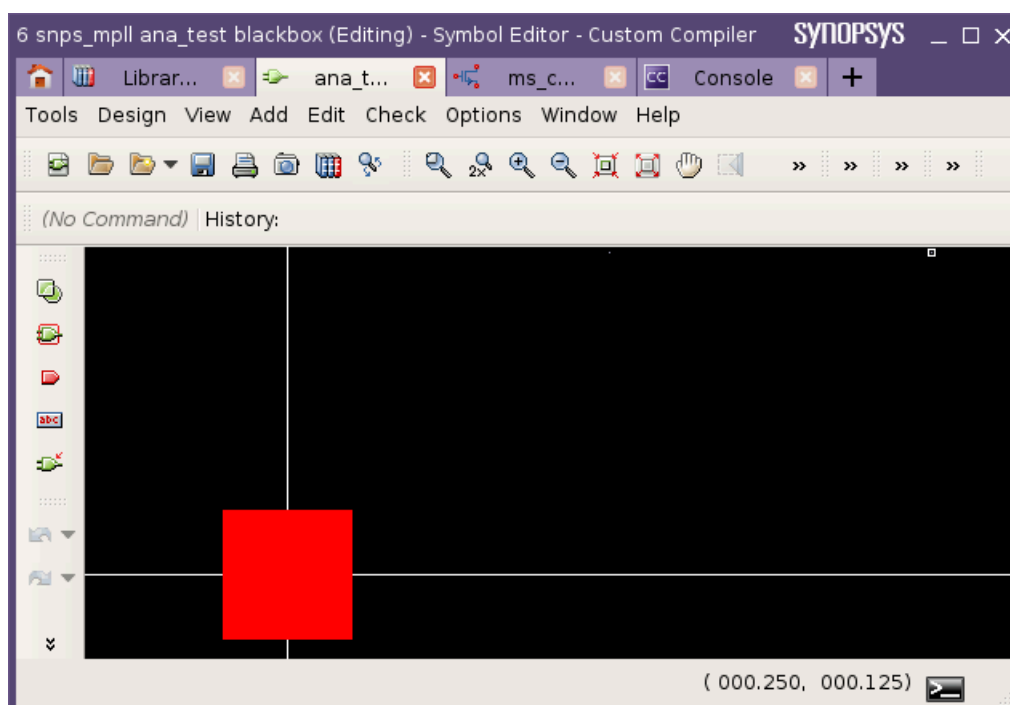
6. In the **Pin Constraints** area, specify or modify the physical constraints for the signal pins:

Setting	Description
Allowed Layers	The layers on which pins are permitted.
Pin Spacing	The distance between adjacent pins, in microns.
No Stacking	How signal pins can stack the other pins during in placement: ◦ any: Signal pins can stack with other signal pins on different layers. Signal pins can also stack with power or ground straps, power or ground pins on different layers as well. (Default) ◦ with_pg_pins_only: Signal pins cannot stack with other signal pins on different layers. But signal pins can stack with power or ground straps, power or ground pins on different layers. ◦ with_signal_pins_only: Signal pins can stack with other signal pins on different layers. But signal pins cannot stack with power or ground straps, power or ground pins on different layers. ◦ none: Signal pins cannot stack with other signal pins on different layers. Signal pins cannot stack with power or ground traps, power or ground pins on different layers neither.

7. When you are finished, click **OK**. A black box is created or the selected black box is modified.
8. To view the black box, in the Library Manager, double-click the black box cellview name.



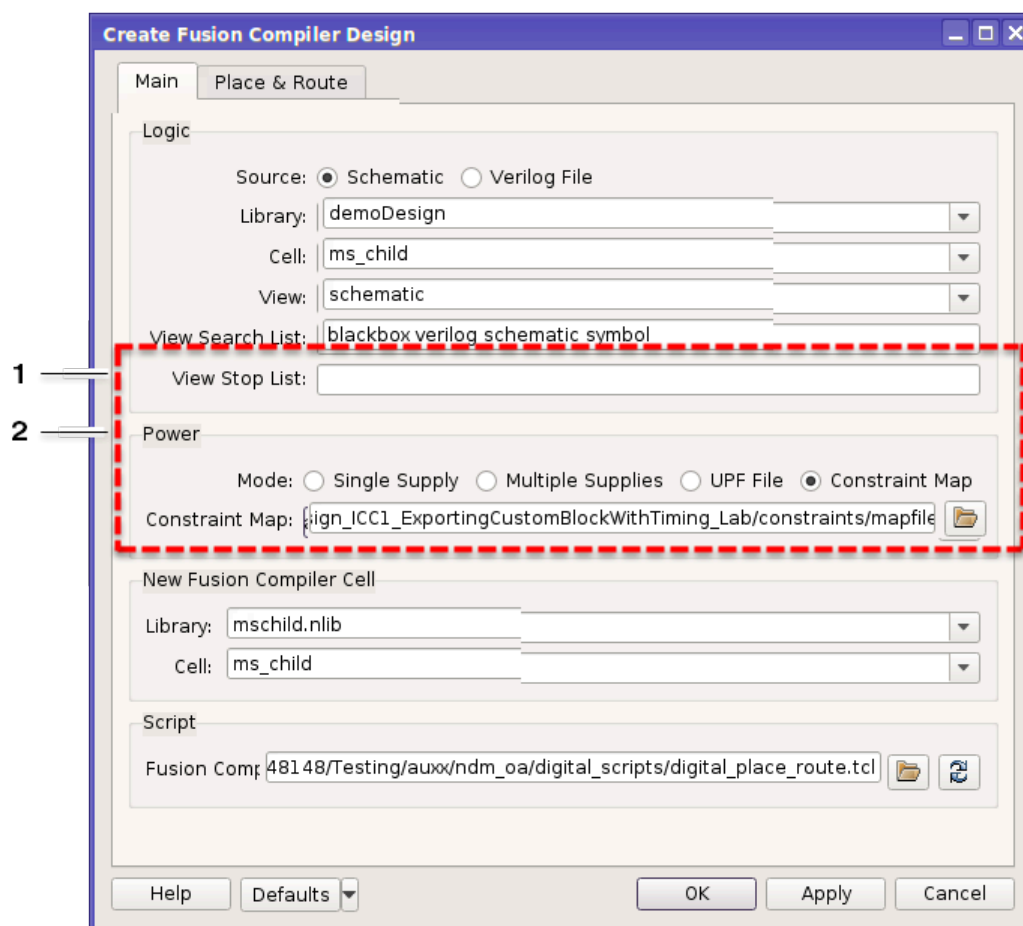
The blackbox cellview displays in the Custom Compiler window.



Running Place and Route

After you create the black box, you can run place and route using the design schematic as input. To run place and route, follow the instructions in [Running Place and Route Using Schematics](#). When setting options in the **Create Fusion Compiler Design** dialog box, set the following:

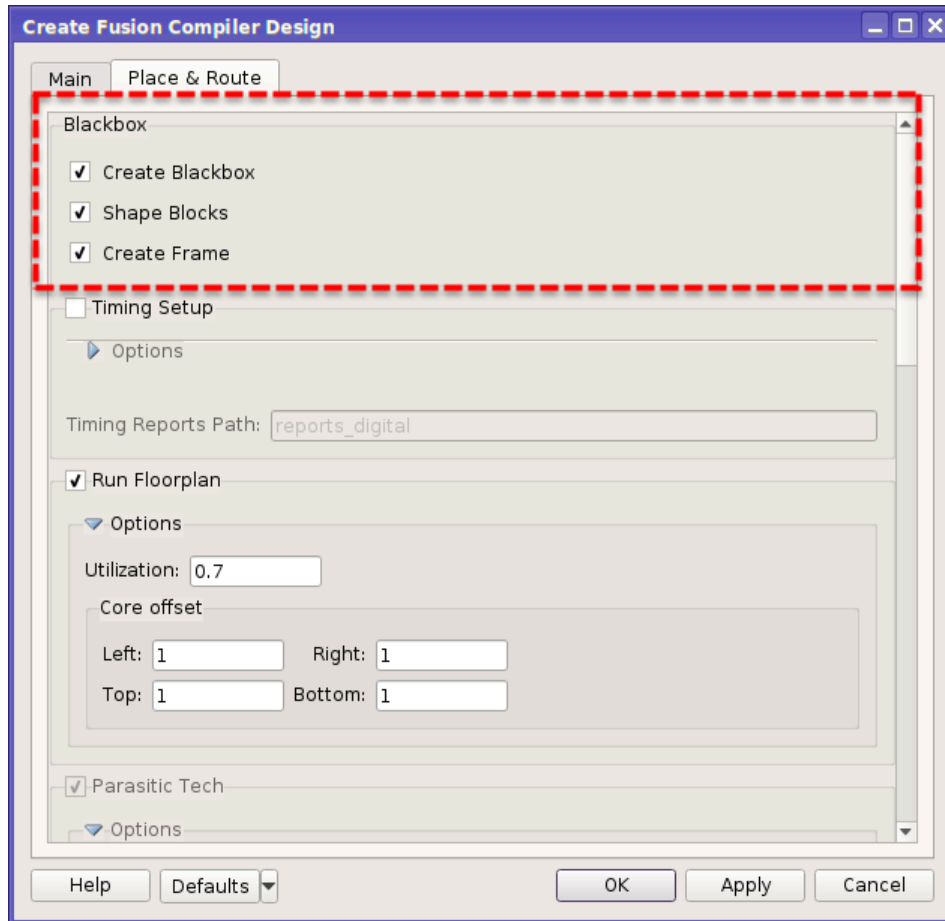
1. In the **View Stop List** box, make sure the standard cell stopping view includes only one pair of power and ground connection definitions.
2. In the **Power** area, select **Constraint Map**, and select the constraint map file.



3. In the **Place & Route** tab, set the **Blackbox** options.

The **Place & Route** tab appears only if you specify a **Script** file (`digital_place_route.tcl`) that contains the default contents or if you customized the contents to include additional graphical user interface controls. Because the `.tcl` file is customizable, the **Blackbox** options can vary. See [Place-and-Route Tcl Script](#).

The **Blackbox** options appear only if you selected **Constraint Map** in the **Power** area (step 2).



9

Developing and Integrating Digital Blocks

This chapter describes how to develop and integrate digital blocks using the Custom Compiler tool.

You can develop and integrate digital blocks in your mixed-signal designs from within the Custom Compiler environment. To develop the digital blocks, you run Fusion Compiler place-and-route commands from within the Custom Compiler environment.

This chapter describes the following:

- [About the Place-and-Route Process](#)
- [Workflow for Developing and Integrating Digital Blocks](#)
- [Preparing the Technology Data](#)
- [Reviewing the Mixed-Signal Design](#)
- [Developing Digital Blocks from Verilog Files](#)
- [Developing Digital Blocks from Schematics](#)
- [Developing Digital Blocks from Layouts](#)
- [Integrating the Digital Block at the Top Level](#)

About the Place-and-Route Process

To develop a digital block in a mixed-signal design, you can initiate the place-and-route process from within the Custom Compiler environment. The place-and-route flow supports black-box blocks that have single power-and-ground connections.

The place-and-route process runs a Tcl script that contains Fusion Compiler commands. This topic describes the following:

- [Place-and-Route Input](#)
- [Place-and-Route Tcl Script](#)

The process requires that the Fusion Compiler library be created in the Fusion Compiler tool and that you add the Fusion Compiler library in the Custom Compiler environment. See [Workflow for Developing and Integrating Digital Blocks](#).

Place-and-Route Input

You can specify one of the following as input to the place-and-route process:

Place-and-Route Input	Description
Verilog file	Produces blocks that are unaware of any floorplan components in the mixed-signal design. Use a Verilog file if you do not have a schematic source.
Schematic	Produces blocks that are unaware of any floorplan components in the mixed-signal design. Use a schematic if you do not have an existing Fusion Compiler digital block, and you want a quick estimate of the block size for floorplanning.
Fusion Compiler Layout	Serves as input at any stage of the place-and-route process: after Verilog input, as floorplanned input, as a placed design, or as a routed design. The layout can contain floorplan components, such as the block boundary, power and ground routes, and pins. The resulting block is optimized and aligns with the floorplan components in the mixed-signal design.

The place-and-route process supports black-box blocks that have single power-and-ground connections.

Place-and-Route Tcl Script

The following Tcl script is supplied with the Custom Compiler environment:

Script Information	Description
Tcl script name	digital_place_route.tcl
Location	\$SYNOPSIS_CUSTOM_INSTALL/ndm_oa/digital_scripts
Purpose	Runs place-and-route flows with schematic or layout input.
Content	- Graphical user interface controls to run placement steps (floorplan, placement, and so on) - Fusion Compiler Tcl commands that run the place-and-route flows

The script serves as templates and contain Fusion Compiler commands that perform place and route. You can modify the script to meet your design requirements. For example, if you want to create a simplified block to estimate its size for floorplanning, you can remove or disable commands and variables that create detailed components. For the descriptions of Fusion Compiler commands, see the [Fusion Compiler user documentation](#).

[Table 1](#) and [Table 2](#) list the commands and variables you can use in the supplied place-and-route script. You can refer to the tables when reviewing or modifying the contents of the script.

The variables and options in the Tcl script file are displayed in the place-and-route graphical user interface. Values you specify in the graphical user interface are propagated to the Tcl script file. See [Running Place and Route Using Schematics](#) and [Before You Start Developing Digital Blocks From Layouts](#).

Table 1 Fusion Compiler Place-and-Route Commands and Equivalent Tcl Variables

Operation: Fusion Compiler Command	Tcl Variable Controlling the Operation	Tcl Variables for Fusion Compiler Command Options	Equivalent Fusion Compiler Command Options
Floorplanning: initialize_floorplan	DIGITAL_RUN_FLOORPLAN	DIGITAL_UTILIZATION	core_utilization
		DIGITAL_CONTROL_TYPE	control_type
		DIGITAL_CORE_LEFT	core_offset
		DIGITAL_CORE_RIGHT	
		DIGITAL_CORE_TOP	
		DIGITAL_BOTTOM	

Table 1 *Fusion Compiler Place-and-Route Commands and Equivalent Tcl Variables*
(Continued)

Operation: Fusion Compiler Command	Tcl Variable Controlling the Operation	Tcl Variables for Fusion Compiler Command Options	Equivalent Fusion Compiler Command Options
		DIGITAL_FLOORPLAN_SHAPE_TYPE	shape
		DIGITAL_SIDE_LENGTH	side_length
Run Placement: create_placement, legalize_placement	DIGITAL_RUN_PLACEMENT	—	—
Create PG rings: create_pg_ring_pattern, set_pg_strategy, compile_pg*	DIGITAL_CREATE_PG_RINGS*	—	—
Add Filler Cells: create_stdcell_filler	DIGITAL_ADD_FILLER_CELL	DIGITAL_CHIP_FINISH_METAL_FILLER_LIB_CELL_LIST DIGITAL_CHIP_FINISH_METAL_FILLER_PREFIX DIGITAL_CHIP_FINISH_NON_METAL_FILLER_LIB_CELL_LIST DIGITAL_CHIP_FINISH_NON_METAL_FILLER_PREFIX	lib_cell prefix lib_cell prefix
Add tap cells: create_tap_cells	DIGITAL_ADD_TAP_CELL	DIGITAL_TAP_MASTER_CELL DIGITAL_TAP_CELL_DISTANCE DIGITAL_TAP_AVOID_LAYERS	lib_cell distance no_tap_cell_under_layers
Run Standard Cell PG Routing: create_pg_std_cell_conn_pattern set_pg_strategy compile_pg*	DIGITAL_RUN_ROUTING	—	—
Run Routing: set_ignored_layers route_auto route_detail	DIGITAL_RUN_ROUTING	DIGITAL_MIN_ROUTING_LAYER DIGITAL_MAX_ROUTING_LAYER DIGITAL_SEARCH_REPAIR_ITERATIONS	min_routing_layer max_routing_layer max_number_iterations
Run DRC/LVS: check_routes check_lvs	DIGITAL_RUN_VERIFICATION	—	—

* By default, the Custom Compiler tool derives and uses the metal1 and metal2 layers for creating PG rings and standard cell PG connections. The layer default widths are used for creating PG rings.

Table 2 *Digital Place-and-Route Commands and Equivalent Tcl Variables*

Digital Operation: Digital Command	Tcl Variable Controlling the Operation	Tcl Variables for Digital Command Options	Equivalent Digital Command Options
create_net, connect_pg_net *	CONNECT_PG_NET	DIGITAL_POWER_SUPPLY_NET DIGITAL_POWER_SUPPLY_P ORT DIGITAL_GROUND_SUPPLY_ NET DIGITAL_GROUND_SUPPLY_P ORT	-net -net
load_upf *	CONNECT_PG_NET	DIGITAL_UPF_FILE	—
create_blackbox *	DIGITAL_CREATE_BLACKBOX	TARGET_BOUNDARY_AREA BOUNDARY BLACKBOX_NAMEBOUNDARY	target_boundary_area boundary —
set_constraint_mapping_ file	DIGITAL_CREATE_BLACKBOX	CONSTRAINT_MAP_FILE	—
create_abstract	DIGITAL_CREATE_BLACKBOX	BLACKBOX_MODULE_NAME	blocks
shape_blocks	DIGITAL_SHAPE_BLOCKS	—	—
create_frame	DIGITAL_CREATE_FRAME	—	—
—	DIGITAL_RUN_TIMING_FLOW ¹	—	—
—	DIGITAL_RUN_TIMING_FLOW	DIGITAL_SCENARIOS_TYPE ² DIGITAL_SCENARIOS_LIST ³ DIGITAL_TCL_MCOMM_SETUP_F ILE ⁴ DIGITAL_SDC_FILE DIGITAL_REPORTS_DIRECT ORY ⁵	report_qor report_timing report_pvt check_design

1. Enables timing-driven flow.

2. Specifies LIST or MCOMM_SETUP_FILE.

3. Lists scenario names. You can write additional Tcl code to define constraints for operating conditions and parasitic setup.

4. Specifies constraints for operating conditions and parasitic setup in the Tcl file.

5. Specifies the directory in which generated command reports are stored.

Table 2 *Digital Place-and-Route Commands and Equivalent Tcl Variables (Continued)*

Digital Operation: Digital Command	Tcl Variable Controlling the Operation	Tcl Variables for Digital Command Options	Equivalent Digital Command Options
read_parasitic_tech	DIGITAL_PARASITIC_TECH	DIGITAL_TLUP_PLUS_MAX_FILE DIGITAL_TLUP_PLUS_MIN_FILE DIGITAL_TLUP_PLUS_NOMINAL_FILE DIGITAL_TLUP_PLUS_LAYER_MAP_FILE	-tlup -tlup -tlup -layermap
create_placement	DIGITAL_RUN_PLACEMENT	DIGITAL_CONGESTION_DRIVEN DIGITAL_TIMING_DRIVEN DIGITAL_PLACEMENT_EFFORT DIGITAL_CONGESTION_EFFORT	congestion timing_driven effort congestion_effort
place_opt	DIGITAL_RUN_PLACEMENT	DIGITAL_RUN_PLACEMENT_FROM_STAGE DIGITAL_RUN_PLACEMENT_TO_STAGE	-from -to
cts_opt	DIGITAL_RUN_CTS	DIGITAL_RUN_CTS_FROM_STAGE DIGITAL_RUN_CTS_TO_STAGE	-from -to
route_opt	DIGITAL_ROUTING_OPT	—	—

* Default uses single PG-based place-and-route flow.

Workflow for Developing and Integrating Digital Blocks

Caution:

Create the digital block using the Fusion Compiler tool before you proceed.



[Show me](#) an example of how to develop and integrate a digital block.

Task	See...
1 Prepare the technology data for place and route. Perform this step if your design originated in the Fusion Compiler tool.	Preparing the Technology Data
2 Review the top-level design.	Reviewing the Mixed-Signal Design
3 Prepare the design library for co-designing.	Preparing Design Libraries by creating or adding the Fusion Compiler library
4 Running place and route using one of the following sources: • Verilog file • Schematic • Layout	One of the following: • Developing Digital Blocks from Verilog Files • Developing Digital Blocks from Schematics • Developing Digital Blocks from Layouts
5 Integrate the digital block at the top level.	Integrating the Digital Block at the Top Level

Preparing the Technology Data

Caution:

Perform the tasks in this topic if your design originated in the Fusion Compiler tool and you want develop it further in the Custom Compiler environment. If your design originated in the Custom Compiler environment, proceed to [Reviewing the Mixed-Signal Design](#).

If your design originated in the Fusion Compiler tool, you must prepare the technology data as follows to ensure a seamless co-design process:

Task	See...
1 Specify the Custom Compiler (OA) technology library as the base technology.	Setting Up the Technology Libraries
2 Map Fusion Compiler layers to Custom Compiler (OA) layers.	Mapping Technology Layer Names
3 Map Fusion Compiler library and cell names to Custom Compiler (OA) library and cell names.	Mapping Libraries, Cells, and Views
4 Set the <code>dmICC2DBUperUU</code> and <code>dmICC2ScaleFactor</code> Tcl preferences to match the design scale used in the Custom Compiler (OA) environment.	Adjusting the Design Scale

Task	See...
5 (Optional) If the Fusion Compiler library uses recognition layers, set the <code>dmICC2RecognitionColorLayers</code> Tcl preference to map the layers.	Mapping Recognition Layers
6 Review other mapping considerations.	Mapping Net Bundles and Non-default Rules

Setting Up the Technology Libraries

Mixed-signal designs use OA technology libraries. To ensure seamless co-designing, set the `dmICC2OATechLib` Tcl preference to use the OA technology library instead of the Fusion Compiler library. For example:

```
db::setPreference dmICC2OATechLib -value Reference10
```

You also have the option of specifying the OA technology library when you create a new or add an existing Fusion Compiler library. See [Preparing Design Libraries](#). Regardless of the method you use, the value you specify in one method is automatically propagated to the other method.

In cases where you have large numbers of vias to map:

1. Create an incremental technology library where the OA technology is the base technology. See [Incremental Technology Data](#).

The incremental technology library contains all the via definitions.

2. Set the incremental library you created as the technology for analog libraries.
3. Set the `dmICC2OATechLib` Tcl preference to use the incremental technology library you created. For example:

```
db::setPreference dmICC2OATechLib -value myIncrTech
```

When you add the Fusion Compiler library, the library will have all the via definitions.

Mapping Technology Layer Names

Mixed-signal designs use OA technology, but the OA technology layer names are not equivalent to the Fusion Compiler technology layer names. Therefore, you must map Fusion Compiler layer names to OA layer names so that you can edit and view digital blocks in the context of mixed-signal designs.

To map the layer names:

1. Prepare a mapping file (.map) that maps the Fusion Compiler layer names to OA layer names. See [Mapping Libraries, Cells, and Views](#).
2. Set the dmICC2OALayerMap Tcl preference to reference it. For example:

```
db::setPrefValue dmICC2OALayerMap -value layer_digital2cc.map
```

You also have the option of specifying the layer name mapping file when you create a new or add an existing Fusion Compiler library. See [Preparing Design Libraries](#). Regardless of the method you use, the value you specify in one method is automatically propagated to the other method.

The layer mapping file must be an ASCII file. Each line in the file represents a single mapping and must use the following syntax:

```
OALayerName OAPurposeName digitalLayerNumber digitalPurposeNumber  
digitalMaskNumber digitalShapeType
```

digitalShapeTypes supports only text objects.

For example:

Mapping: Fusion Compiler to Custom Compiler	Example
layer to layer	m1 52
cut layer to cut layer	v1 1
LPP to LPP	m1 drawing 52 0 v1 boundary 2 1
LPP to LPP-mask cut metal shapes	m1 drawing 52 -2 1
LPP of any mask number with text object to LPP	m1_text drawing 10 0 * text
layer of any purpose and any mask number with text object to LPP	m2 TEXT 12 * * text

The mappings will occur in the following order:

1. All Fusion Compiler purposes are mapped to corresponding Custom Compiler purposes for the specific layer.
2. Specific Fusion Compiler layer-purpose pairs are mapped to the corresponding Custom Compiler layer-purpose pairs.

- 3. The color masks of specific Fusion Compiler layer-purpose pairs are mapped to corresponding Custom Compiler layer-purpose pairs that are usually not colorable.
- 4. Specific Fusion Compiler layers or layer-purpose pairs with text objects are mapped to corresponding Custom Compiler layers or layer-purpose pairs, respectively.

OA		Fusion Compiler			
1	Layer name		layer number		
2	Layer name	Purpose name	Layer number	Purpose number	
3	Layer name	Purpose name	layer number	purpose number	mask number
4	Layer name	Purpose name	layer number	Purpose number	text object
	Layer name				text object

Mapping Libraries, Cells, and Views

When working with digital components in the Custom Compiler environment, the digital libraries, cells, and views must be mapped to Custom Compiler (OA) libraries, cells, and views to ensure co-design compatibility and seamless top-level integration.

To map the libraries, cells, and views:

- 1. Prepare a mapping file (.map) that maps the Fusion Compiler libraries, cells, and views to OA libraries, cells, and views. See [Mapping Libraries, Cells, and Views](#).
- 2. Set the dmICC2OALibCellViewMap Tcl preference to reference the file and its directory. For example:

```
db::setPrefValue dmICC2OALibCellViewMap -value reference.map
```

The mappings will occur in the order that the names appear in the file. The library, cell, and view will all be mapped.

You also have the option of specifying the library and cell name mapping file when you add an existing Fusion Compiler library. See [Adding Fusion Compiler Libraries](#). Regardless of the method you use, the value you specify in one method is automatically propagated to the other method.

Library, cell, and view mapping file requirements:

The mapping file must be an ASCII file. Each line in the file represents a single mapping and must use the following syntax:

```
OALibName OACellName OAViewName digitalLibName digitalCellNumber
digitalViewNumber [digitalLabel]
```

For any of the fields, you can use * to represent a wildcard. If you use *, you must use it for the corresponding Fusion Compiler and OA fields. For example, if you use * for the OAViewName, you must also use it for the digitalViewName.

The wildcard lets you simplify the statements in the mapping file. In the following example, the Fusion Compiler library called sourceDigitalLib is mapped to an existing OA library named myOALib. All Fusion Compiler cells and views under ICC2lib are mapped to the OA cells and views of the same name under myOALib.

```
myOALib * * sourceDigitalLib * *
```

In the following example, sourceDigitalLib is mapped to myOALib. The two statements map the Fusion Compiler abstract (frame) views and layout views to the OA layout views with the same cell names.

```
myOALib * layout sourceDigitalLib * abstract
myOALib * layout sourceDigitalLib * layout
```

An example `reference.map` file contains the following Fusion Compiler cells, which will be mapped to the Custom Compiler (OA) library/cell/view as follows:

Custom Compiler Library/Cell/View	Fusion Compiler Cell Names
sc invx1 abstract	sc.ndm invx1 layout
saed32_hvt_oa * layout	saed32_nvt.ndm std* abstract The <code>std</code> prefix in the name is removed.
saed32_hvt_oa *_vg layout	saed32_hvt.ndm * abstract The <code>_vg</code> suffix is added to the name.
* * abstract	* * layout All layout view names are mapped to abstract.

To remaster abstract views to layout views, set the `dmICC2MapAbstractToLayout` Tcl preference to `true`. For example:

```
db::setPrefValue dmICC2MapAbstractToLayout -value true
```

Adjusting the Design Scale

Fusion Compiler designs and reference libraries must use the same scale factor. See the [Fusion Compiler user documentation](#) about the Fusion Compiler create_lib command for details.

Mixed-signal designs use technology data that do not have the same DBUPerUU as in the Fusion Compiler tool. Therefore, you must adjust the Fusion Compiler design units to ensure smooth top-level integration within the Custom Compiler environment.

Synopsys recommends that you use the same DBUPerUU for the PDK and Fusion Compiler technology libraries. To scale the design units, use one of the following methods to set the scale to the target PDK resolution:

- Set the `dmICC2DBUPerUU` Tcl preference.
- Set the DBUPerUU option when you add the Fusion Compiler library. See [Adding Fusion Compiler Libraries](#).

Regardless of the method you use, the value you specify in one method is automatically propagated to the other method.

The following table shows how to set up the scale factor under two scenarios. Although the scenarios use a scale factor of 10 000, your Fusion Compiler libraries might have different scaling factors. Be sure to check and adjust the `dmICC2DBUPerUU` Tcl preference or the DBUPerUU option accordingly.

	Fusion Compiler Ref Lib	Fusion Compiler Top Lib	Fusion Compiler Technology for Lib Creation	Tech Used in the Custom Compiler environment	Setup
1	Created in Fusion Compiler tool with scale factor 10 000	To be created in the Custom Compiler environment	Fusion Compiler technology file	Fusion Compiler tech	dmICC2ScaleFactor 10 000 (same as Fusion Compiler reference libraries) dmICC2DBUPerUU 10000
2	Created in Fusion Compiler tool with scale factor 10 000	To be created in the Custom Compiler environment	Fusion Compiler technology file	OA tech	dmICC2ScaleFactor 10 000 (same as Fusion Compiler reference libraries) dmICC2DBUPerUU 2000 (same as OA technology)

For example, suppose you have a digital block that has a height of 5 user units. If you store the block in the two environments, its height is as follows:

Environment	5 user units
Fusion Compiler	50 000 DBU
Custom Compiler	5000 DBU

In the Fusion Compiler tool, the block height is 50 000 DBU. If you place the block in the Custom Compiler environment without scaling, the block will have a height of 50 000/1000, or 50 units.

To correct the scaling, you must first determine the scaling ratio:

Components of Ratio Calculation	Value
Fusion Compiler scaling factor	10 000
PDK DBUperUU	1000
Scaling ratio = scaling factor / DBUperUU	10

Then set the preference as follows:

```
db::setPrefValue dmICC2DBperUU -value 1000
```

Mapping Recognition Layers

By default, when you add Fusion Compiler libraries in the Custom Compiler environment, Fusion Compiler layers are automatically mapped to equivalent layers in the Custom Compiler technology database. If the Fusion Compiler library has color recognition layers, you must list the recognition layers using the `dmICC2RecognitionColorLayers Tcl` preference. For information, see the *Custom Compiler Tcl Reference Manual*.

Mapping Net Bundles and Non-default Rules

Fusion Compiler net bundles and non-default rules (NDRs) are both mapped to Custom Compiler net groups. If they have the same name, the NDR mapping is skipped. To ensure proper mapping, make sure the net bundles and NDRs have different names.

Reviewing the Mixed-Signal Design

Before you start to implement the digital block, you should review the top level of the mixed-signal design to become familiar with the design components and their layout. This will facilitate the floorplanning and block implementation tasks downstream.

To review the top-level design:

1. Within the Custom Compiler environment, open the Library Manager, and then open the layout view in the SDL mode.
2. Examine the design to learn the placements of various components. For example, you might want to check the instances of interest.
3. When you are finished, close the design.

Developing Digital Blocks from Verilog Files

You can develop a digital block by running place and route using Verilog files. Because the input does not contain any floorplan information, the resulting block is created without floorplan components.

Use this method if you do not have an existing Fusion Compiler digital block, and you do not have a design schematic that you can use as the source. You can use this method to create a simplified block to estimate the block size for floorplanning.

This topic describes the following:

- [Before You Start](#)
- [Running Place and Route Using a Verilog File](#)

To create digital blocks with floorplan considerations, see [Developing Digital Blocks from Layouts](#).

Before You Start

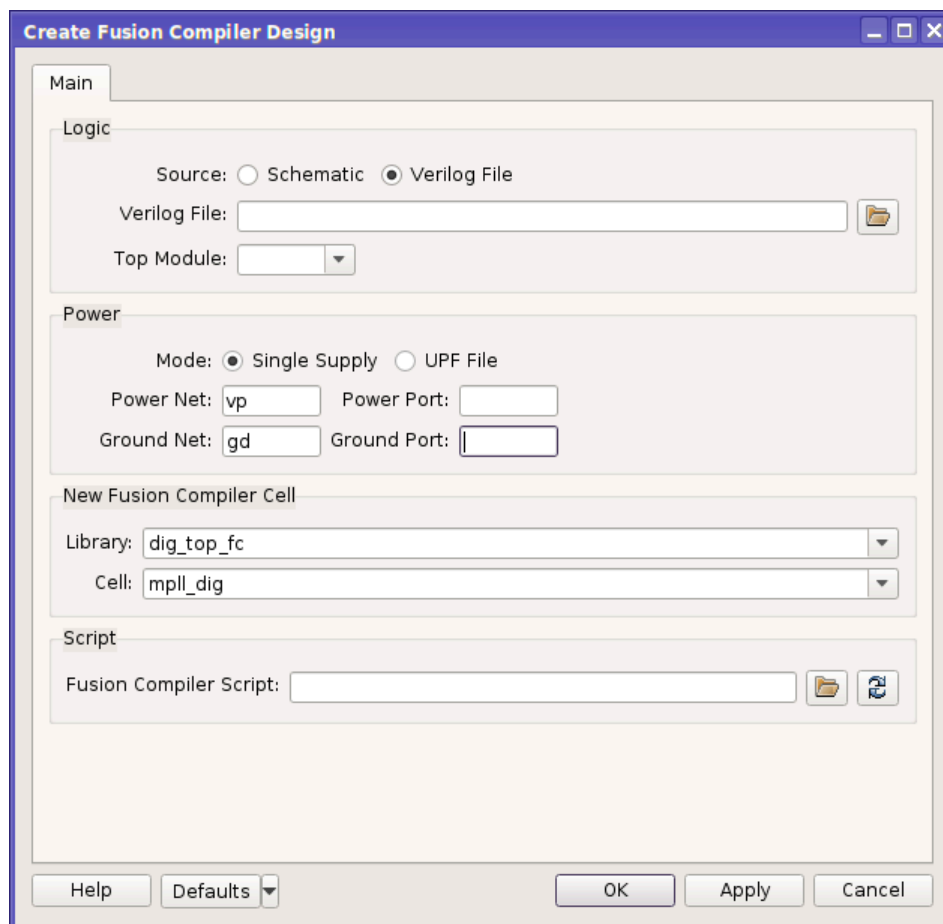
Because you will be developing a simplified block, you should review and modify the Tcl script to accommodate your design goals. For information, see [Place-and-Route Tcl Script](#).

Before running place and route, make sure you create a frame view of the block. See [Creating Frame Views](#).

Running Place and Route Using a Verilog File

To run place and route using a Verilog file:

1. From the Library Manager, choose **File > Create Fusion Compiler Design**. The **Create Fusion Compiler Design** dialog box opens.



2. In the **Logic** area:
 - a. Select **Verilog File**.
 - b. Locate and select the **Verilog File** you want to use.
 - c. In the **Top Module** box, type the name of the top-most module in the Verilog file.

3. In the **Power** area, set the following:

Setting	Description
Mode	Select Single Supply .
Power Net	Type the name of the power net (for example, VDD).
Power Port	Type the name of the power port.
Ground Net	Type the name of the ground net (for example, VSS).
Ground Port	Type the name of the ground port.

4. In the **New Fusion Compiler Cell** area, select the **Library** and **Cell** for the newly created block.
5. In the **Script** area, leave the box empty to produce an Fusion Compiler block with logical information. You can specify a script to run the flow after Verilog in.
6. When you are finished, click **OK**.

The place-and-route process starts. When the process is finished:

- The completed block appears within the viewport in the Layout Editor.
- The `ndm2oa.log` file is generated and stored in `synopsys_custom/digitalbridge/<LibName>/logFiles`.

```
synopsys_custom/digitalbridge/<LibName>/logFiles/  
<blockName>:layout:maskLayout_ndm2oa.log
```

Output	Description
<code><LibName></code>	The Fusion Compiler output library that you specified in the Create Fusion Compiler Design dialog box.
<code><LibName>/logFiles/</code>	The directory containing the <code><name>.log</code> file.

Developing Digital Blocks from Schematics

You can develop a digital block by running place and route using a schematic. Because the input does not contain any floorplan information, the resulting block is created without floorplan considerations.

Use this method if you do not have an existing Fusion Compiler digital block, and you want to create a simplified block to estimate the block size for floorplanning.

This topic describes the following:

- [Before You Start Developing Digital Blocks From Schematics](#)
- [Running Place and Route Using Schematics](#)
- [Viewing the Output Log](#)
- [Viewing the Verilog Netlist File](#)

To create digital blocks with floorplan considerations, see [Developing Digital Blocks from Layouts](#).

Before You Start Developing Digital Blocks From Schematics

Because you will be developing a simplified block, you should review and modify the Tcl script to accommodate your design goals. For information, see [Place-and-Route Tcl Script](#).

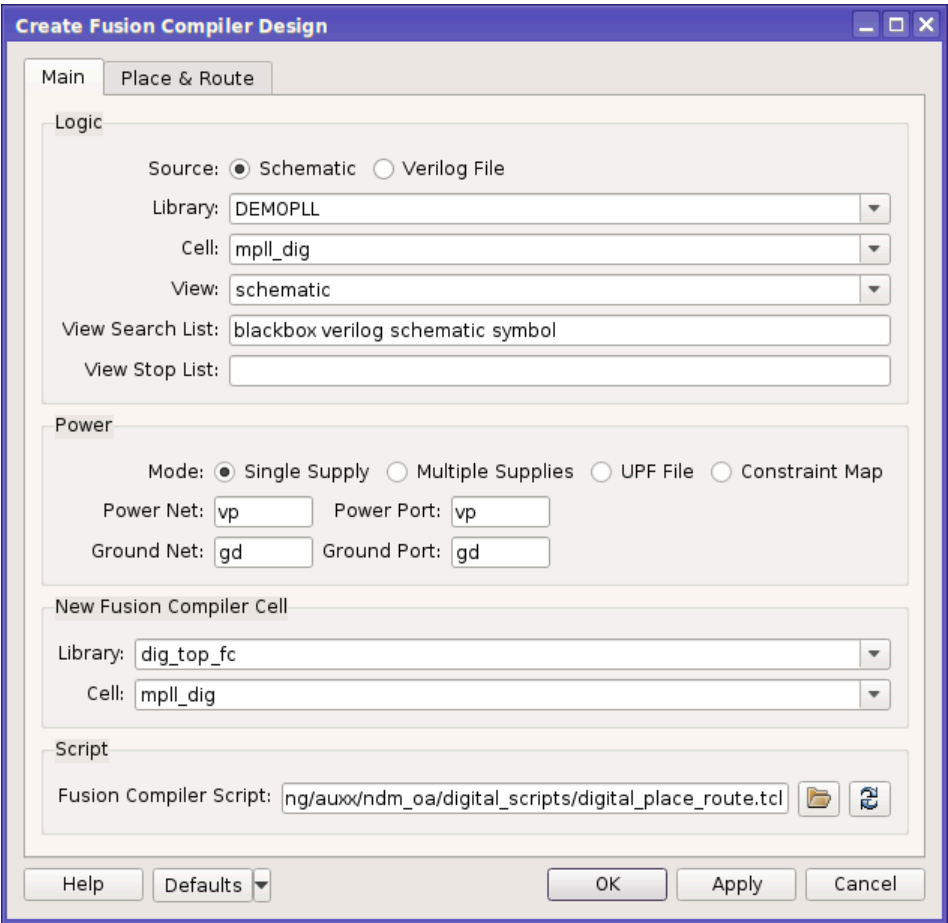
Before running place and route, make sure you create a frame view of the block. See [Creating Frame Views](#).

Running Place and Route Using Schematics

To run place and route using a design schematic of a digital block:

1. From the Library Manager, open the design schematic you want to use. The Schematic Editor opens.
2. In the Schematic Editor, choose **Tools > Digital Implementation > Fusion Compiler**. The **Fusion Compiler** menu appears on the menu bar.
3. Choose **Fusion Compiler > Create Fusion Compiler Design**. The **Create Fusion Compiler Design** dialog box opens.

Notice that in the **Logic** area, the default setting for **Source** is **Schematic**. To use the Verilog setting, see [Developing Digital Blocks from Verilog Files](#).



- 4. In the **Logic** area, choose the **Library** and **Cell** settings. In the **View** list, choose **config**.
- 5. In the **Power** area, set the following:

Setting	Description
Mode	Select one of the following: Single Supply: Specifies a single power source. You must specify the Power Net , Power Port , Ground Net , Ground Port names. Select this option if your design has a single supply net without a black box. You can also select this option if your design has a single supply net with one or more black boxes, as long as the black boxes do not have multiple power and ground terminals.

Setting	Description
	Multiple Supplies: Extracts the power network information from this design for outputting in UPF format. Running place and route once with multiple supplies generates a UPF file you can use to rerun place and route with the unified power format. ◦ Domains tab: Shows the common power and ground connections shared by the gates in the design. ◦ Black Boxes tab: Shows the power and ground details for any black boxes. ◦ Supply Nets tab: Shows all power and ground details for the design. If your design has multiple supplies, with or without one or more black boxes, you must choose this option. When your design has inherited connections to define the supplies, you must choose this option to make sure inherited connections for power and ground nets are honored. UPF File: Uses the Unified Power Format (UPF) file you specify to define power and ground settings. Constraint Map: Contains path to the constraint map file. Select this option if the schematic contains black boxes. See Creating Black Boxes for Analog Blocks.

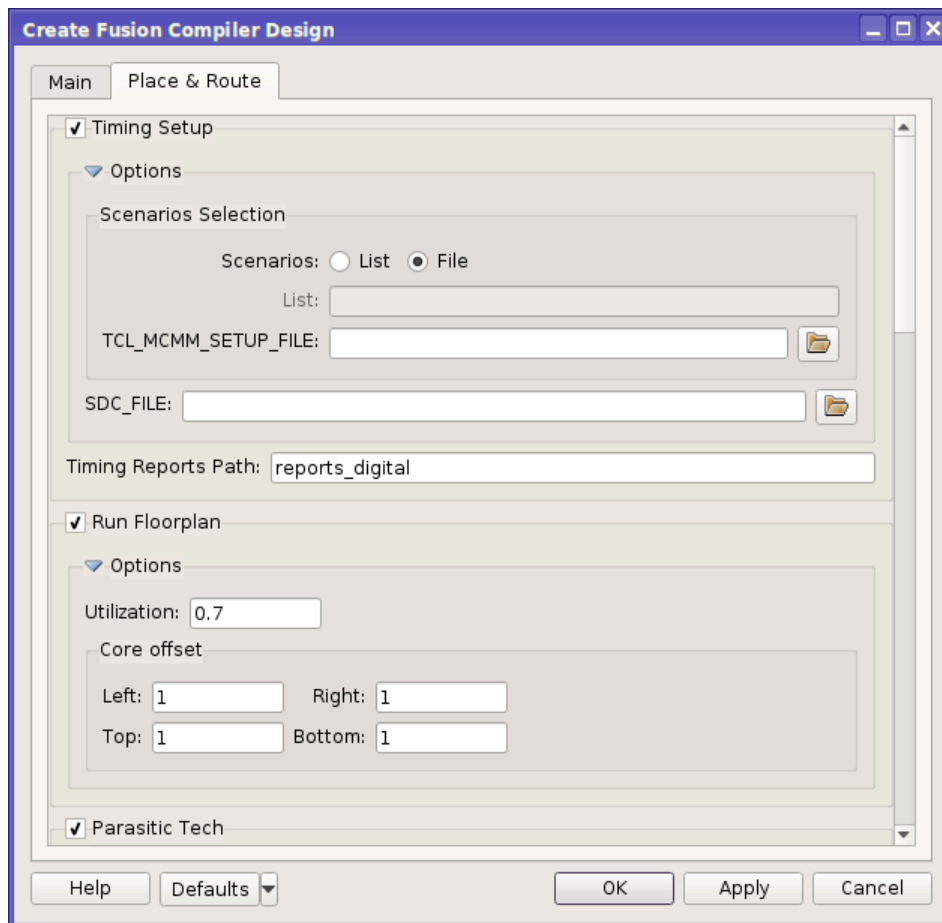
6. In the **New Fusion Compiler Cell** area, select the **Library** and **Cell** for the newly created block.
7. In the **Script** area, specify the location of `digital_place_route.tcl`. See [Place-and-Route Tcl Script](#).

If the file contains the default contents or if you customized the contents to include additional graphical user interface controls, the **Place & Route** tab appears.

If the file contains simple Fusion Compiler commands without any graphical user interface controls, no Place & Route tab will display.

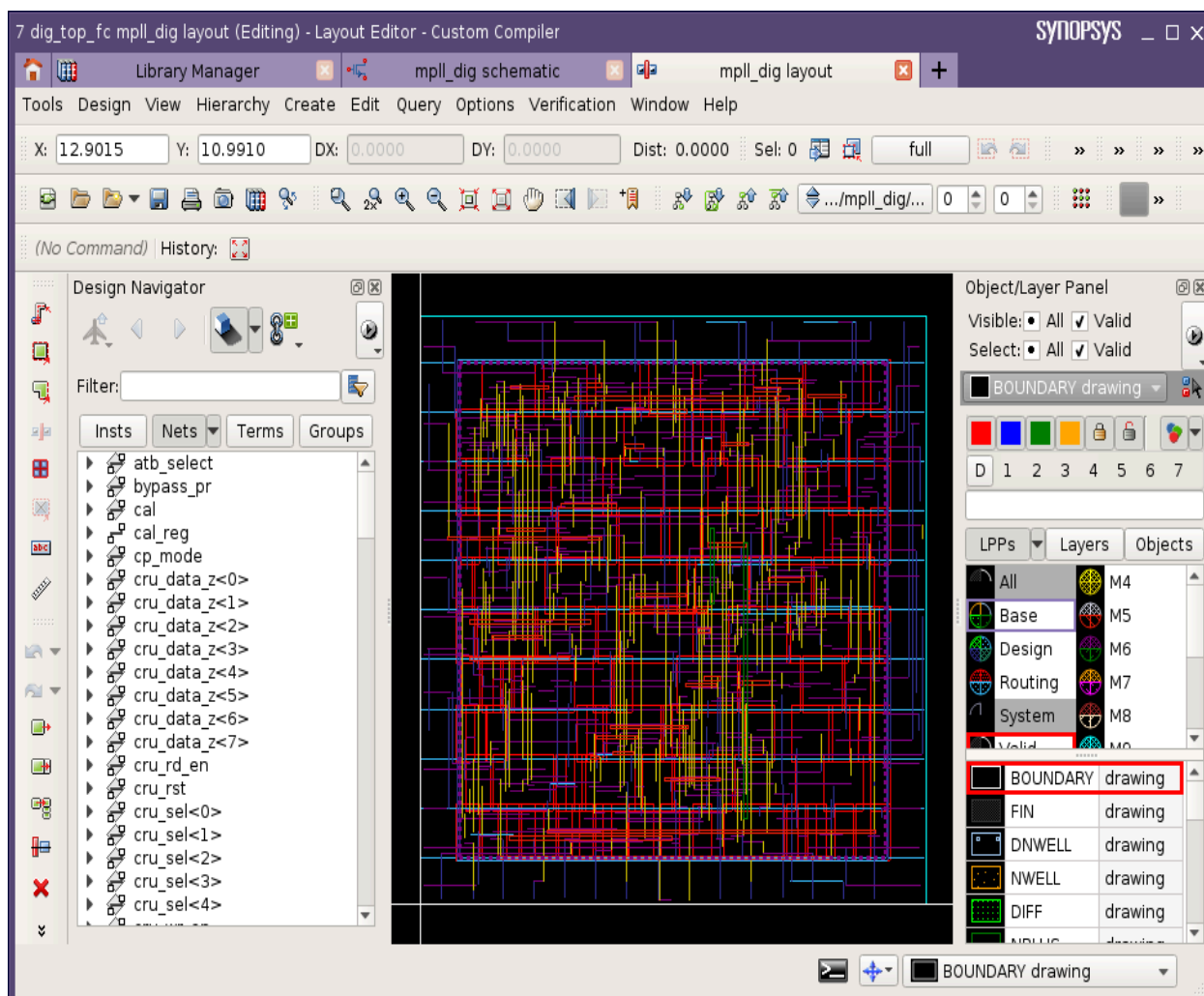
If you previously specified a Tcl script, click **Reload the specified script**  to ensure that updates made to the file since you last ran the script are synchronized with your current session.

8. (Default and customized scripts only) Click the **Place & Route** tab and set the variables and options. For the descriptions of Fusion Compiler options, see the [Fusion Compiler user documentation](#).



9. When you are finished, click **OK**.

The place-and-route process starts. When the process is finished, the completed block appears within the viewport in the Layout Editor.



If the logic source is set to a netlistable cellview, a Verilog netlist file is created.

Viewing the Output Log

To check that the process completed successfully without error:

1. In Layout Editor, choose **Tools > Job Monitor**.
2. In the Job Monitor, right-click the xtlCC2CreateDesign_n job that just finished, and then choose **View Output**.
3. In the Text Viewer that is displaying the `create_design_in_icc2.log` tab, scroll down to the DRC - SUMMARY section and check that there are no violations.

You can also view the `ndm2oa.log` file that is generated various flow and stored in `synopsys_custom/digitalbridge/<LibName>/logFiles`.

Viewing the Verilog Netlist File

To view the Verilog netlist file that is created:

1. At the UNIX prompt, navigate to the following folder:

```
.../synopsys_custom/digitalbridge/place_route
```

2. View the contents of the `<blockname>.v` file.

Developing Digital Blocks from Layouts

You can create a digital block by running place and route using a layout. Because the input can contain floorplan information, such as the block boundary, power and ground routes, and pins, the resulting block is optimized and aligns with the components in the mixed-signal design.

Use this method if you know the size of the digital block, and you want to customize the floorplan components, such as block shapes, pin positions, power and ground straps, critical instance placements, and critical routes.

This topic describes the following:

- [Workflow for Developing Digital Blocks From Layouts](#)
- [Before You Start Developing Digital Blocks From Layouts](#)
- [Creating the Fusion Compiler Design](#)
- [Adding and Editing Floorplan Components](#)
- [Copying Fusion Compiler Cells](#)
- [Running Place-and-Route Using Layouts](#)

To create simplified digital blocks without adding floorplans, see [Developing Digital Blocks from Schematics](#).

Workflow for Developing Digital Blocks From Layouts

The following table shows the workflow for developing digital blocks from layouts. The workflow can vary slightly, depending on whether you want to automatically or manually add floorplan components. If you automatically add floorplan components, you can place

the instance of the block in the mixed-signal design during the workflow. If you manually add floorplan components, you do not need to place the block instance.

Task	Description	
1	Create an Fusion Compiler design using one of the following inputs: • <i>Logic data</i>: Produces a basic floorplan without components. • <i>Floorplan components</i>: Produces a floorplan that includes components.	Creating Fusion Compiler Libraries If you are creating the design with floorplan components, make sure you edit the <code>digital_place_route.tcl</code> script Developing Digital Blocks from Layouts .
2	(Floorplans with components only) Place the digital block instance in the mixed-signal design.	Placing Digital Block Instances
3	(Floorplans without components) Add floorplan components to the Fusion Compiler digital block.	Adding and Editing Floorplan Components
4	Edit the floorplan components in the digital block and the mixed-signal design to make sure they align.	Adding and Editing Floorplan Components
5	Create a frame view of the block.	Creating Frame Views
6	Run Fusion Compiler place and route.	Running Place-and-Route Using Layouts
7	Integrate the digital block at the top level of the mixed-signal design.	Integrating the Digital Block at the Top Level
8	Review placed and routed design.	—

Before You Start Developing Digital Blocks From Layouts

If you plan to place an instance of the digital block in the mixed-signal design (see step 1 in [Workflow for Developing Digital Blocks From Layouts](#)), you should review and modify the `digital_place_route.tcl` script to add only the desired floorplan components. For example, you might want to turn off `DIGITAL_CREATE_PG_RINGS` if you plan to add them later.

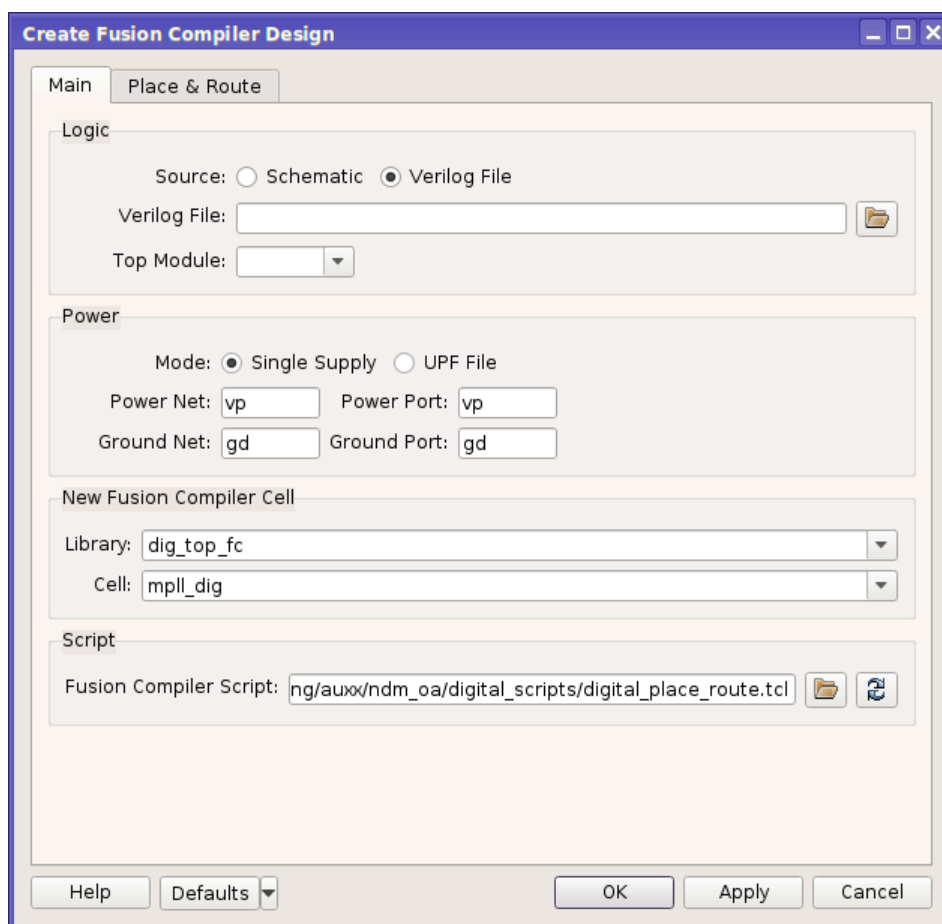
For information about the script, see [Place-and-Route Input](#).

Creating the Fusion Compiler Design

You can create an Fusion Compiler design, the digital block, while within the Custom Compiler environment.

To create the Fusion Compiler design:

1. From the Library Manager, choose **File > Create Fusion Compiler Design**. The **Create Fusion Compiler Design** dialog box opens.



2. In the **Logic** area:
 - a. Select **Verilog File**.
 - b. Locate and select the **Verilog File** you want to use.
 - c. In the **Top Module** box, type the name of the top-most module in the Verilog file.
3. In the **Power** area, set the following:

Setting	Description
Mode	Select Single Supply .
Power Net	Type the name of the power net (for example, VDD).

Setting	Description
Ground Net	Type the name of the ground net (for example, VSS).

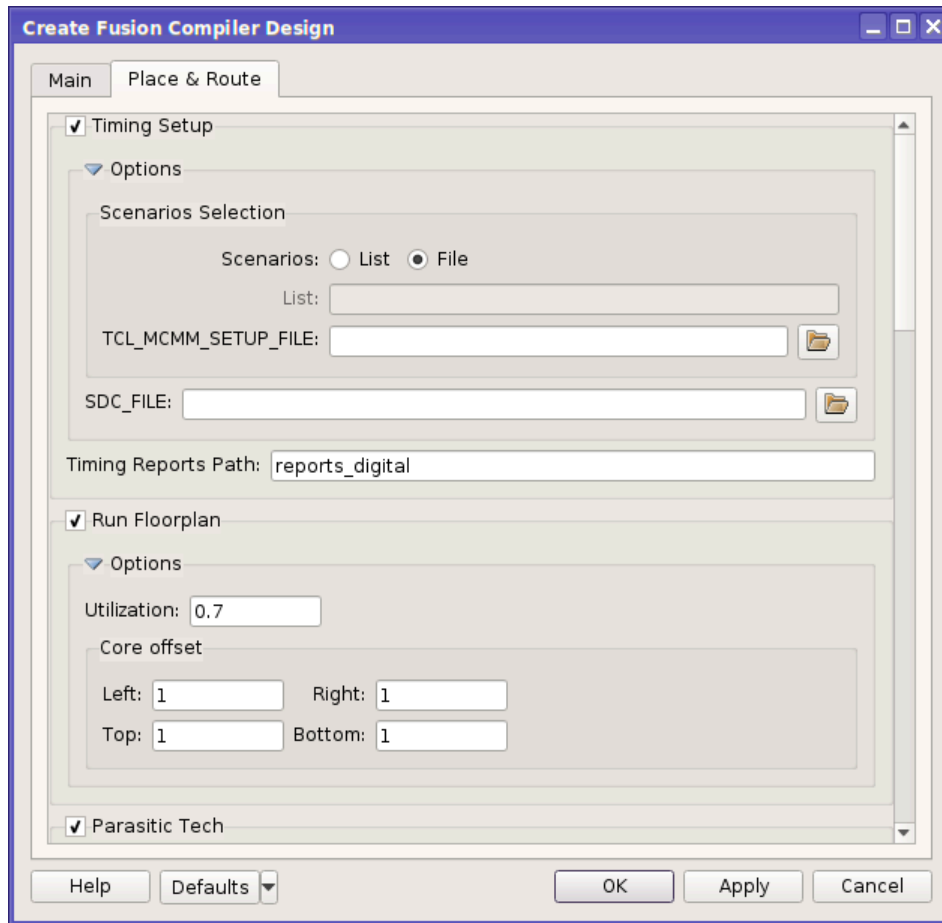
4. In the **New Fusion Compiler Cell** area, select the **Library** and **Cell** for the newly created block.
5. In the **Script** area, specify the location of `digital_place_route.tcl`. See [Place-and-Route Tcl Script](#).

If the file contains the default contents or if you customized the contents to include additional graphical user interface controls, the **Place & Route** tab appears.

If the file contains simple Fusion Compiler commands without any graphical user interface controls, no Place & Route tab will display.

If you previously specified a Tcl script, click **Reload the specified script**  to ensure that updates made to the file since you last ran the script are synchronized with your current session.

6. (Default and customized scripts only) Click the **Place & Route** tab and set the variables and options. For the descriptions of Fusion Compiler options, see the [Fusion Compiler user documentation](#).



7. When you are finished, click **OK**.

The place-and-route process starts. You can monitor the progress using the Job Monitor. When the process is finished, the completed block appears within the viewport in the Layout Editor. The output files are stored in the `synopsys_custom/digitalbridge` directory, or the directory specified by the `SYNOPSYS_CUSTOM_LOCAL` environment variable.

Adding and Editing Floorplan Components

To add or edit the floorplan components in the digital block or in the mixed-signal design:

- Make sure you are viewing the layout view of the mixed-signal design.
- To use the SDL **Pick & Place** command to add or edit floorplan components and place instances at the top (0x) level, set the `leNetListMode Tcl` preference to `off`.

For information about the Pick & Place command, see the *Custom Compiler SDL and Visually Assisted Automation Guide*.

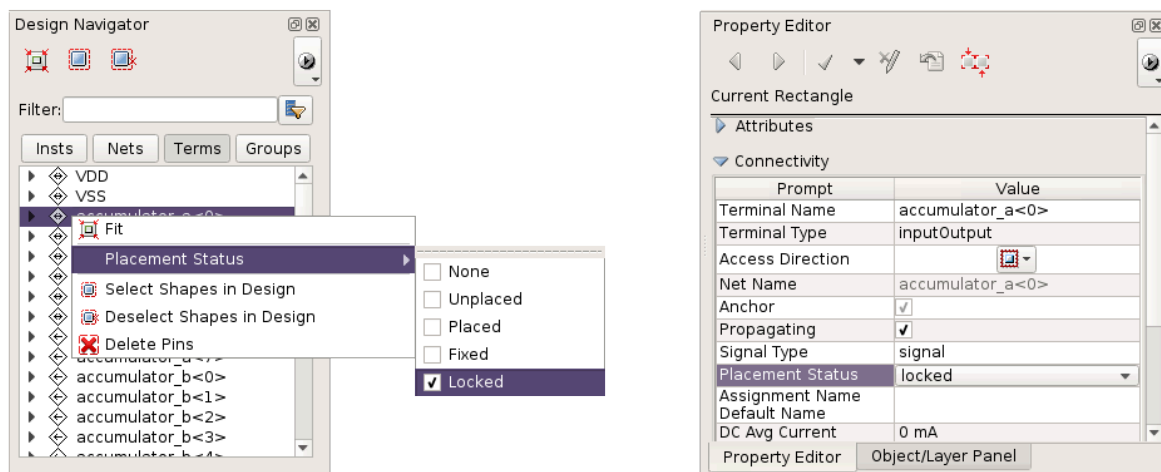
Note:

The **Replace Existing** option has no effect on the result of the **Pick & Place** command in the co-design flow.

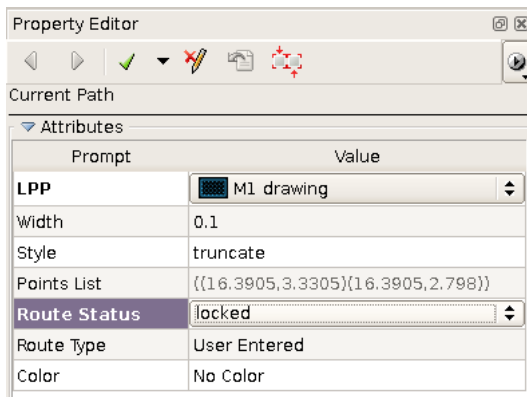
- To prevent making changes in the SDL mode, set the `leNetListMode` Tcl preference to `read`.
- Add or edit any necessary components, such as power and ground routes, pins, and so on. For instructions, see Layout Editor Description.
- To add routes and shielding, see Routing Methods, the Auto Route Shield Tab, and Adding Post-Route Shielding. See [Adding and Editing Routes](#) for additional guidelines.

When adding or editing the floorplan components, use the following guidelines:

- All shapes must have net information.
- All routes must have the correct signal type assignment.
- Power and ground routes should be of signal type **Power** and **Ground**, respectively.
- Standard cells must be bound to abstract views.
- Instances and pins should have the **Locked** placement status. You can set their status using the Design Navigator or Property Editor. Locking instance and pin placements ensures that they will not be moved during the place-and-route process.



- Routes should also have the **Locked** placement status. You can set this using the Property Editor. Locking route placements ensures that they will not be moved during the place-and-route process.



Copying Fusion Compiler Cells

You can use the **Edit > Copy Cell** command in the Library Manager to:

- Copy cells from one Fusion Compiler library to another Fusion Compiler library to create duplicate versions while in the Custom Compiler environment.
- Copy cells from the Fusion Compiler library to the Custom Compiler library.

Via objects are flattened during the copy operation to prevent OA technology modifications when introducing Fusion Compiler via definitions.

For information, see the **Edit > Copy Cell** command in the Library Manager.

Running Place-and-Route Using Layouts

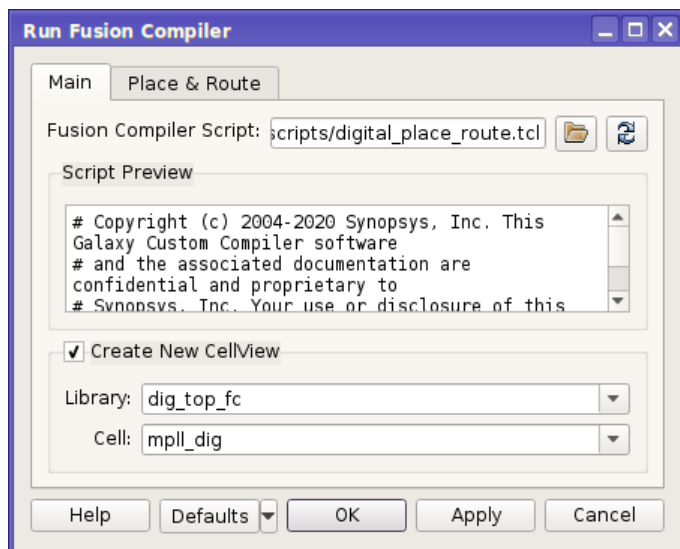
Before running place and route, make sure you create a frame view of the block. See [Creating Frame Views](#).

After you add the floorplan details, you can run place and route on the optimized digital block.

To run place and route using the layout of the block:

1. From the Library Manager, open the design layout of the block. The Layout Editor opens.
2. In the Layout Editor, choose **Tools > Digital Implementation > Fusion Compiler**. The **Fusion Compiler** menu appears on the menu bar.

3. Choose **Fusion Compiler > Run Fusion Compiler**. The **Run Fusion Compiler** dialog box opens.

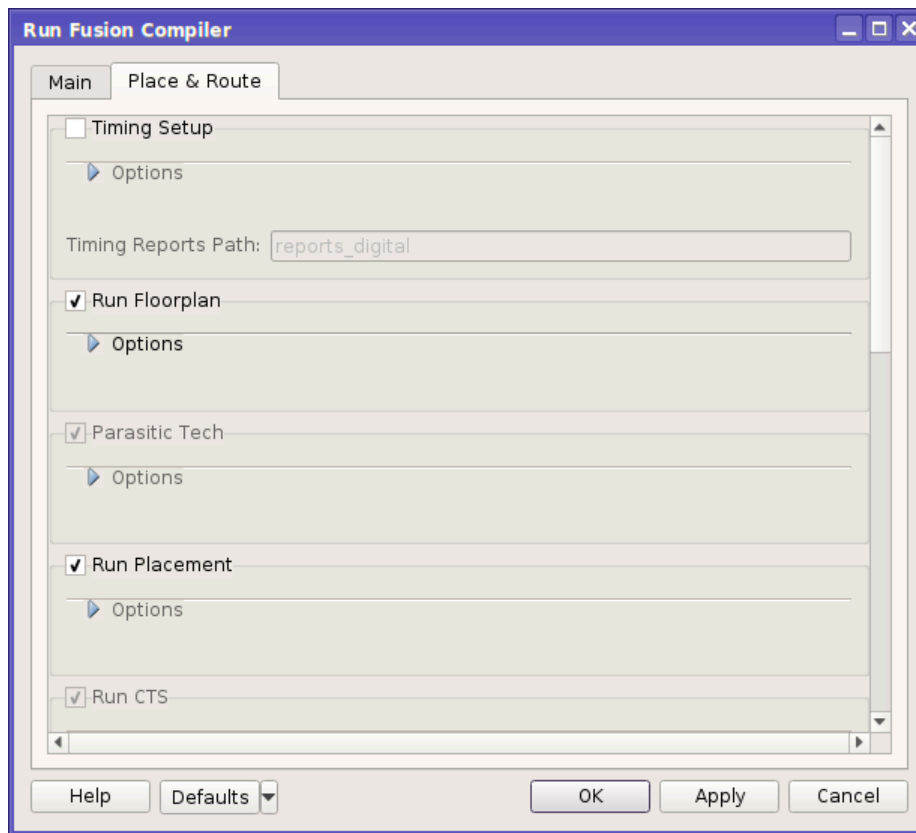


4. In the **Fusion Compiler Script** box, specify the location of `digital_place_route.tcl`. See [Place-and-Route Tcl Script](#).

If the file contains the default contents or if you customized the contents to include additional graphical user interface controls, the **Place & Route** tab appears.

If the file contains simple Fusion Compiler commands without any graphical user interface controls, no **Place & Route** tab will display.

If you previously specified a Tcl script, click **Reload the specified script**  to ensure that updates made to the file since you last ran the script are synchronized with your current session.
5. In the **Script Preview** box, review the script to make sure it is correct.
6. To create another copy of the currently opened cellview, select **Create New Cellview**, and select the **Cell**. This setting prevents the overwriting of the currently opened design and allows you to use the same Fusion Compiler design for multiple place-and-route iterations.
7. (Default and customized scripts only) Click the **Place & Route** tab and set the variables and options. For the descriptions of Fusion Compiler options, see the [Fusion Compiler user documentation](#).



8. Click **OK**.

The place-and-route process starts. You can monitor the progress using the Job Monitor.

When the process is finished, the completed block appears within the viewport in the Layout Editor. The output files are stored in the `synopsys_custom/digitalbridge` directory, or the directory specified by the `SYNOPSYS_CUSTOM_LOCAL` environment variable. You can also view the `ndm2oa.log` file that is generated and stored in `synopsys_custom/digitalbridge/<LibName>/logFiles`.

9. Close the design.

Integrating the Digital Block at the Top Level

After you place and route the block in the mixed-signal design, you need to integrate the block to the top-level design. This topic describes the two ways you can integrate the digital blocks:

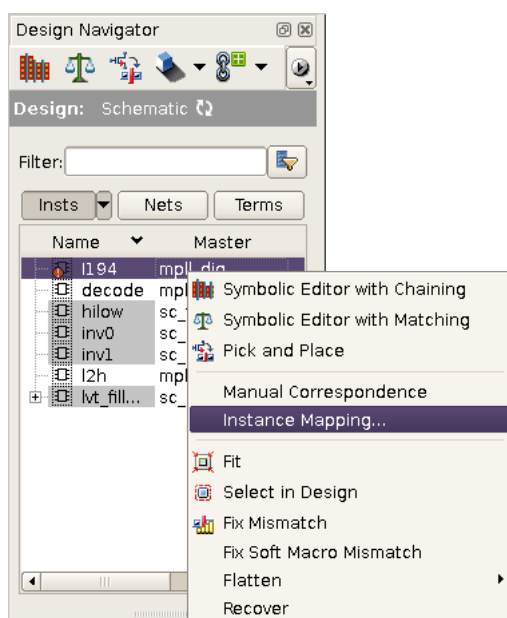
- [Placing Digital Block Instances](#)
- [Remastering the Instances](#)

Placing Digital Block Instances

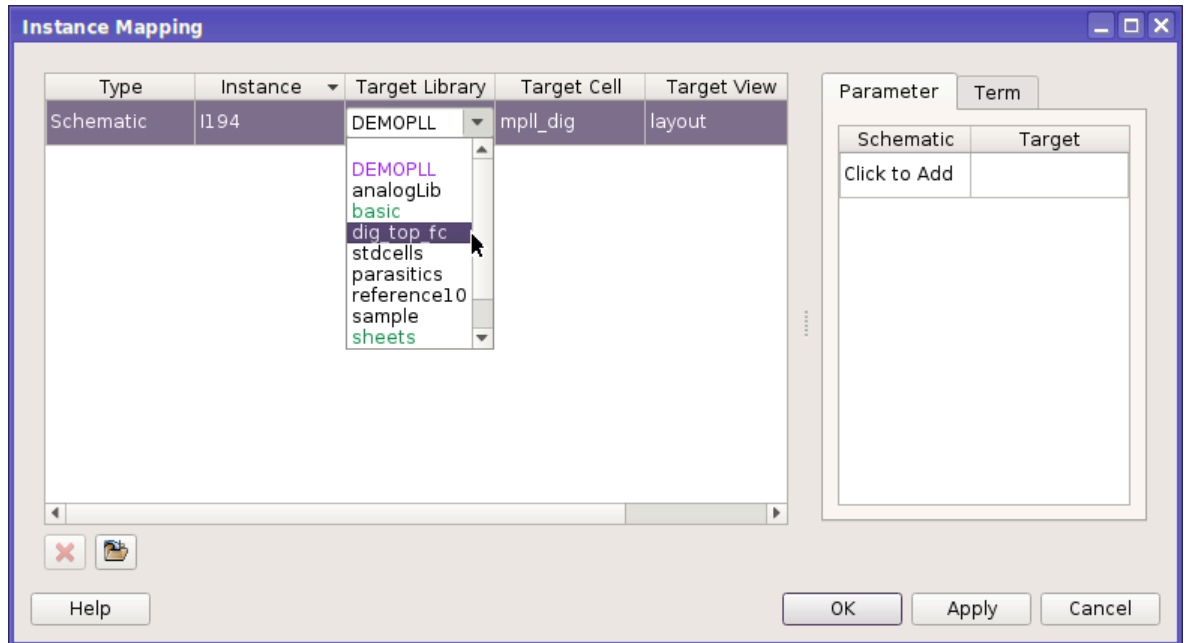
This section describes how to place a digital block instance in the mixed-signal design. You would need to do this after you created an Fusion Compiler block that contains floorplan components.

To place an instance of the digital block in the mixed-signal design:

1. From the Library Manager, open the top-level design layout in the SDL mode.
2. In the Design Navigator, change the instance mapping from the Custom Compiler library to the Fusion Compiler library as follows:
 - a. Click the **Insts** tab.
 - b. Right-click the digital block instance, and then choose **Instance Mapping**.



- c. In the **Instance Mapping** dialog box, double-click the **Target Library** name, and then choose the Fusion Compiler library.



3. Place the block instance in the top-level design as follows:
 - a. In the Design Navigator, make sure the block instance is still selected, and then click  **Pick and Place**.
 - b. On the canvas, place the instance at the desired location.
4. On the Design Navigator **Design** selector bar, click . The design changes to .

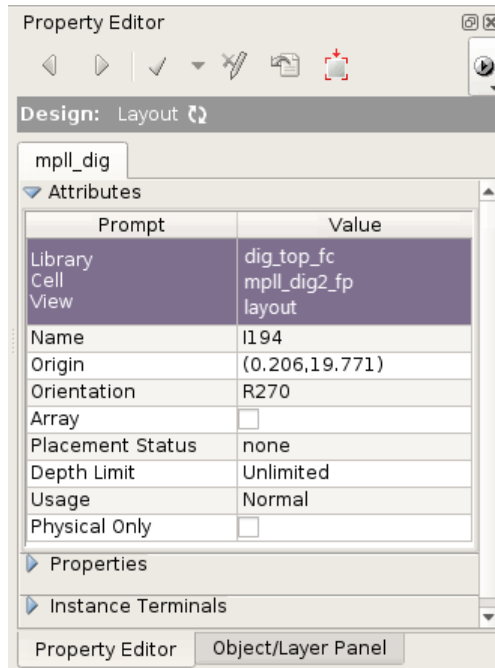
Remastering the Instances

You can remaster the digital block instance so that it references the Fusion Compiler library, cell, and view.

Before you start, make sure the instance is permitted to be remastered. The restrictions are set up using the `dbAllowedRemasterCells` CDF parameter. If the CDF parameter is not defined for the library, all instances in the library can be remastered. See Parameter Definition Editor.

To remaster the digital block instance:

1. Open the top-level design layout.
2. Select the instance of the block.
3. Select the digital block.
4. In the Property Editor, choose the Fusion Compiler **Library**, **Cell**, and **View**.



5. Click  **Apply** to save the changes.

10

Creating Frame Views

This chapter describes how to create frame (abstract) views in preparation for Fusion Compiler place-and-route.

This chapter describes the following:

- [About Frame Views and the Create Frame Tcl Script](#)
- [Before You Start](#)
- [Creating Frame Views from the Custom Compiler Environment](#)

About Frame Views and the Create Frame Tcl Script

To facilitate place and route operations and to optimize performance, you must create Fusion Compiler frame views for the Custom Compiler design blocks (instances, macros, and I/O devices). Frame views are physical abstractions of the blocks, containing only the components necessary for place and route, such as the block boundary, pins, via regions, and routing blockages. See the [Fusion Compiler user documentation](#) for information about frame views.

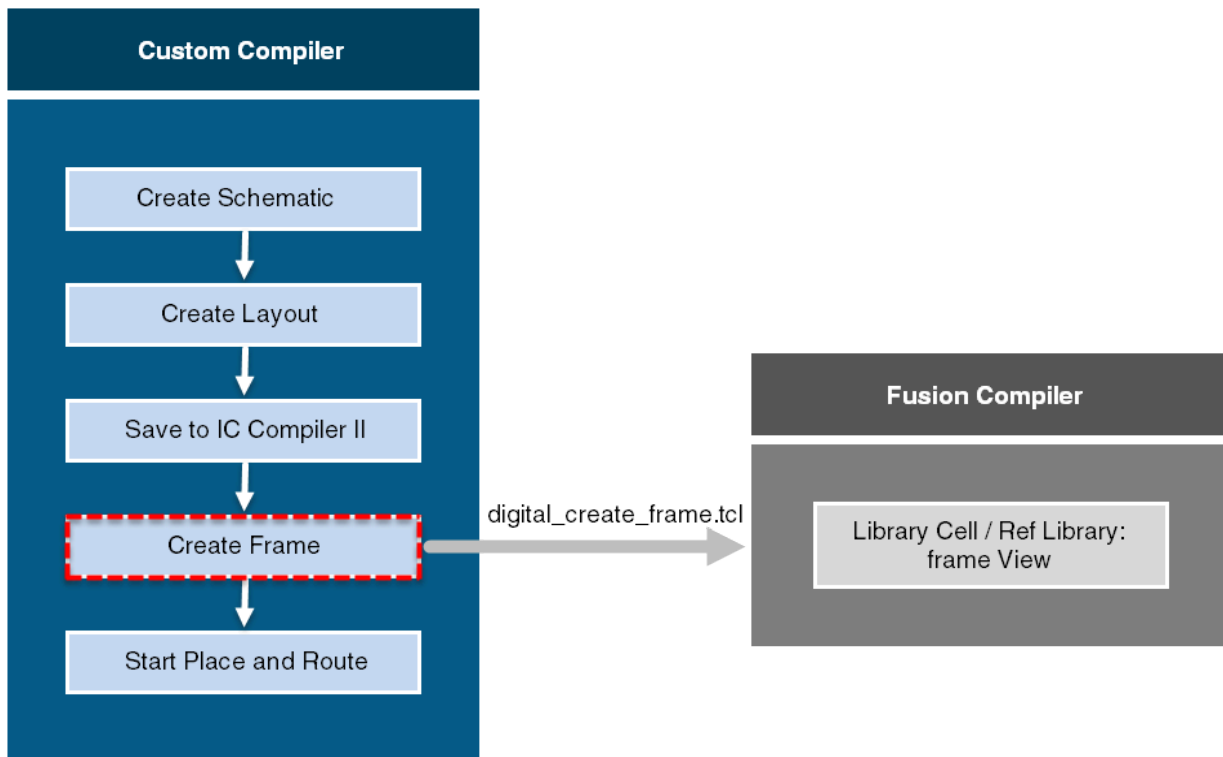
You can create the Fusion Compiler frame view from within the Custom Compiler environment. The Custom Compiler co-design assistant interface lets you:

- Set options that are used during the frame view creation process.
- Run the create-frame Tcl script, which contains Fusion Compiler frame view creation commands.

A `digital_create_frame_template.tcl` script file is supplied with the Custom Compiler environment. You can locate the script in the following directory:

```
$SYNOPSIS_CUSTOM_INSTALL/ndm_oa/digital_scripts
```

The script serves as a template and contains Fusion Compiler commands that creates frame views. You can modify the script to meet your design requirements. For the descriptions of Fusion Compiler commands, see the [Fusion Compiler user documentation](#).



Instead of using the graphical user interface, you can use the `xt::createFCFrame` command to create Fusion Compiler frame views. The command runs the frame creation flow in batch mode.

Before You Start

Make sure you:

- Review the `digital_create_frame.tcl` file and ensure the correct create-frame commands and settings are used. See [About Frame Views and the Create Frame Tcl Script](#).
- Check that the Fusion Compiler and IC Validator executable files are present in the path where Custom Compiler can launch them.

- Set the following Tcl preferences:
 - `dmICC2DataPrepMaskColors`: Specifies the mask colors to use (`{layer color}`). For example:

```
db::setPrefValue dmICC2DataPrepMaskColors -value " {M1 2} {M2 2}"
```
 - `dmICC2DataPrepTimingFiles`: Specifies the timing files to use. For example:

```
db::setPrefValue dmICC2DataPrepTimingFiles -value "/slowfs/liberty/db/*.db"
```
 - `dmICC2ScaleFactor`: Adjusts the design scale to ensure proper scaling during co-design operations. See [Adjusting the Design Scale](#).
- If you want to use existing Fusion Compiler reference libraries as target libraries for frame views, be sure to add the library to the Custom Compiler environment. See [Adding Fusion Compiler Libraries](#).
- If you want to use the Pin Access Checker, make sure you prepare the following files:

File	Description
<code>digital_pin_access_checker.tcl</code>	(Required for pin access checking) Script that helps you identify pin accessibility issues for specified standard cells after the Create Frame operation finishes. By default, the file is provided in the <code>ndm_oa/digital_script/</code> directory. The script calls the following customizable checkers that must be located in the same directory: ◦ <code>pin_check_variables_file.tcl</code> ◦ <code>pin_check_pg_script_file.tcl</code> ◦ <code>pin_check_post_route.tcl</code> ◦ <code>pin_check_preplace_options_file.tcl</code> ◦ <code>pin_check_route_options_file.tcl</code> The Pin Access Checker script contains the variables that are required for running the checker. The contents of the <code>pin_check_variables_file.tcl</code> override the variable specification in this Tcl script. The script is similar to running the Fusion Compiler Pin Access Checker Utility. See Using the Fusion Compiler Pin Access Checker Utility.
<code>pin_check_variables_file.tcl</code>	(Optional) Tcl file containing variables to be used by the Pin Access Checker script. The variables in this file override the variable specifications in the default Script File. By default, the file is provided in the installation directory for reference only. If you want to use the file, you must first copy it to your local directory and customize the contents to meet your requirements.

File	Description
pin_check_pg_script_file.tcl	(Optional) Tcl file containing power and ground (PG) routing settings. By default, the file is provided in the installation directory for reference only. If you want to use the file, you must first copy it to your local directory and customize the contents to meet your requirements. The settings in this file are equivalent to the Fusion Compiler pin_check.common.pg_script_file application option settings.
pin_check_post_route.tcl	(Optional) Tcl file containing settings to be applied after routing. By default, the file is provided in the installation directory for reference only. If you want to use the file, you must first copy it to your local directory and customize the contents to meet your requirements. The settings in this file are equivalent to the Fusion Compiler pin_check.route.post_route_tcl application option settings.
pin_check_preplace_options_file.tcl	(Optional) Tcl file containing floorplan, placement, and legalization settings that are sourced before floorplan creation. When preparing the preplace options Tcl file, make sure you include: ◦ Node-specific commands and options (for example, colored track definitions) for 10-nm and below libraries. ◦ Routing direction and track offset information for all metal layers if they are not already defined in the technology library. By default, the file is provided in the installation directory for reference only. If you want to use the file, you must first copy it to your local directory and customize the contents to meet your requirements. The settings in this file are equivalent to the Fusion Compiler pin_check.place.preplace_option_file application option settings.
pin_check_route_options_file.tcl	(Optional) Tcl file containing routing-related settings to be sourced before routing. The settings in this file has priority over other router specifications. For best results, add nonpersistent settings in the route options Tcl file. By default, the file is provided in the installation directory for reference only. If you want to use the file, you must first copy it to your local directory and customize the contents to meet your requirements. The settings in this file are equivalent to the Fusion Compiler pin_check.route.route_option_file application option settings.

Creating Frame Views from the Custom Compiler Environment

You can create frame views of a complete library from the Library Manager. If a layout is already open, you can create frame views from within that session of the Layout Editor.

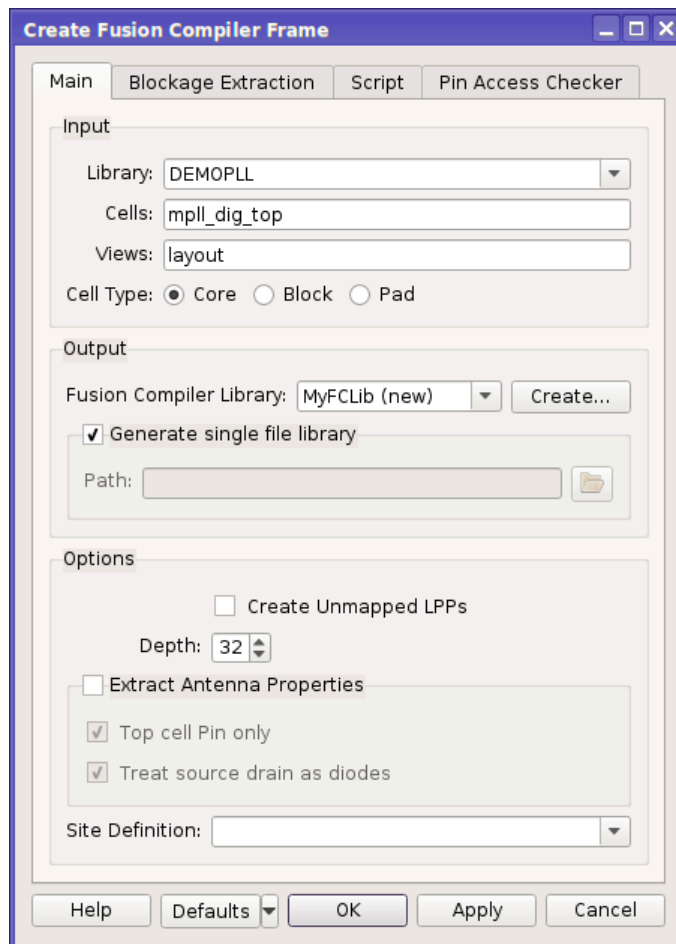
 [Show me](#) a basic example of creating frame views. Although the video shows working with an IC Compiler II design, the example is also applicable for Fusion Compiler designs.

 [Show me](#) an example of creating a frame view for an analog block in an IC Compiler II top-level design. The example is also applicable for co-designing with Fusion Compiler designs.

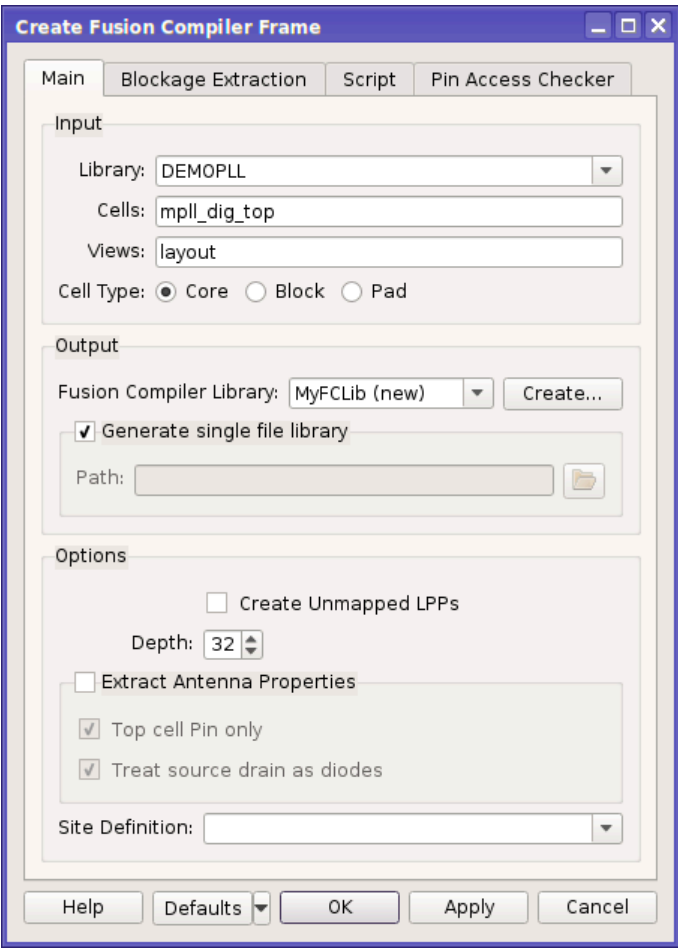
To create Fusion Compiler frame views from the Custom Compiler environment:

1. In the Custom Compiler environment:
 - (If none of the layouts is open) In the Library Manager, choose **File > Create Frame**.
 - (If one of the source layouts is already opened) In the Layout Editor, choose **Fusion Compiler > Create Frame**.

The **Create Fusion Compiler Frame** dialog box opens.



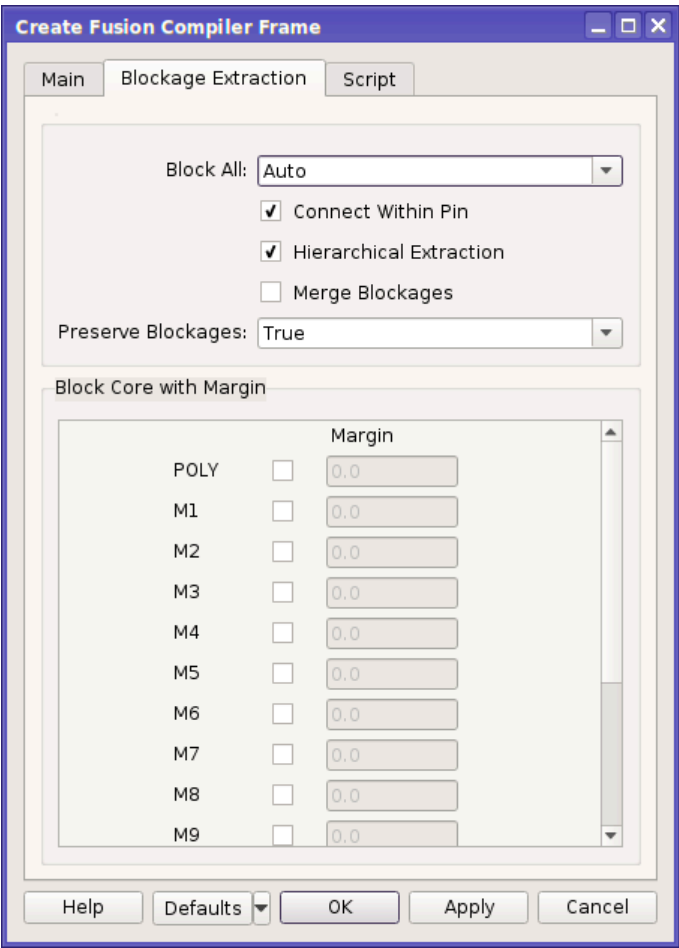
2. In the **Main** tab, set the following:



Area	Description
Input	The Library, Cell, and View of the design for which you want to create the frame view. By default, information about the currently opened design populates the Input area. Select one of the following to identify the design (Cell Type): ◦ Core: Standard cells ◦ Block: Macros ◦ Pad: I/O device If you select Core, make sure site definitions exist in the OA technology library. When you choose Create Frame in the Library Manager, you can generate frame for the complete library by specifying asterisk (*) in the Cells field.

Area	Description
Output	The target library where the frame view is stored. Choose an existing library from the Fusion Compiler Library list, or click Create to create a new library. When creating a new library, you can specify the Fusion Compiler mapping file, the layer mapping file, and the via mapping file. Select Generate single file library if you want to store the output in a single cell library file. The Path you specify for the single file must not exist. When you select this option, the Pin Access Checker tab appears. See Step 5.
Options	Additional options to consider during the frame view creation process: ◦ Create Unmapped LPPs: Creates LPPs in the target IC Compiler II technology for OA LPPs that do not have mappings to IC Compiler II LPPs. ◦ Depth: Accesses the hierarchy levels you specify when creating the frame view. ◦ Extract Antenna Properties: Extracts antenna rules. Select this option if you will use the Hercules or IC Validator tool. ◦ Top Cell Pin Only: Extracts hierarchical antenna properties only for nets that connect to pins at the top-level of the hard macro. ◦ Treat Source Drain as Diodes: Treats the MOS source and drain as protection diodes and considers any MOS source and drain regions that are connected to a net to be a part of the diode protection area available to that net. ◦ (Core only) Site Definition: Maps the Custom Compiler site definition to the Fusion Compiler site definition you choose.

3. In the **Blockage Extraction** tab, set the following:

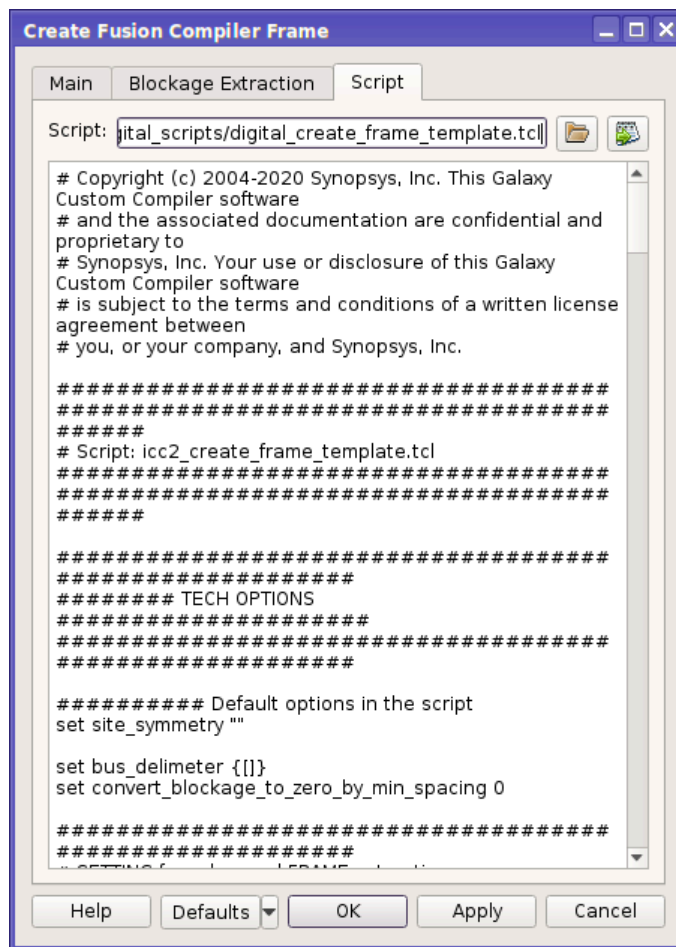


Setting	Description
Block All	Creates routing blockages in the frame view for the entire block, except for the area around the pins. For the description of the Block All selections, see the Fusion Compiler user documentation .
Connect Within Pin	Ensures the via enclosure is within the pin shape.
Hierarchical Extraction	Extracts objects in subblocks.
Merge Blockages	Merges metal shapes and routing blockages to reduce the number of shapes.

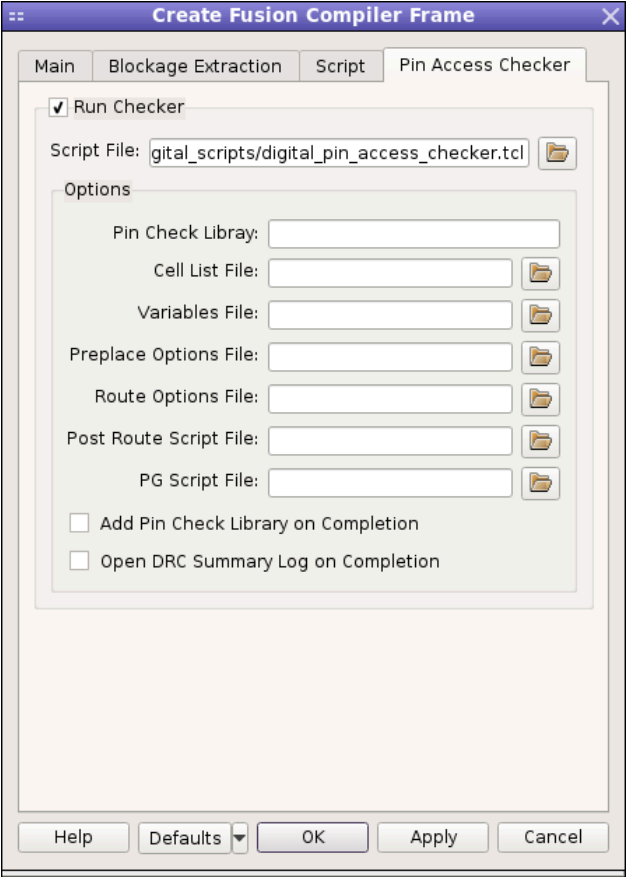
Setting	Description
Preserve Blockages	Preserves all metal shapes and routing blockages around pins as they exist in the design view. For the description of the Preserve Blockages selections, see the Fusion Compiler user documentation .

4. In the **Script** tab, locate and select the `digital_create_frame.tcl` file. The content of the file displays in the tab.

Review the script to make sure it contains the correct commands and settings.



5. (Optional) If you selected **Cell Type: Core** and **Output: Generate Single File Library** in [Step 2](#), set the following in the **Pin Access Checker** tab to verify pin accessibility in standard cells before place-and-route:



Settings	Description
Run Checker	Runs the Pin Access Checker for specified standard cells after the Create Frame operation finishes. See Before You Start .
Script File	Location of the default Pin Access Checker script, digital_pin_access_checker.tcl. See Before You Start .
Pin Check Library	Name of the library that contains all the library cells. During the checking operation, the Pin Access Checker places all the library cells in the top cell with all possible cell combinations and orientations. The pin check library is a new library created using the technology from the Fusion Compiler library, which you specified in the Output section in Step 2 . The Fusion Compiler library you specified in the Output section becomes attached as a reference library to the pin check library.

Settings	Description
Cell List File	Names of cells to be checked by the Pin Access Checker.
Variables File	Tcl file containing variables to be used by the Pin Access Checker script. The variables in this file override the variable specifications in the default Script File.
Preplace Options File	Tcl file containing floorplan, placement, and legalization settings that are sourced before floorplan creation. See Before You Start .
Route Options File	Tcl file containing routing-related settings to be sourced before routing. See Before You Start .
Post Route Script File	Tcl file containing settings to be applied after routing. See Before You Start .
PG Script File	Tcl file containing power and ground (PG) routing settings. See Before You Start .
Add Pin Check Library on Completion	Adds the pin access check library in the output so that you can review the cell combinations created.
Open DRC Summary Log on Completion	Automatically opens the DRC summary file after the pin access check is finished. The summary file displays cell-related DRC errors that are found.

6. Click **OK**. The create frame process starts.

When the process is finished, an Fusion Compiler design view and frame view are created. The following output files are created and stored in the `synopsys_custom/digitalbridge` directory:

```
synopsys_custom/digitalbridge/<digital_lib>/logFiles/  
<blockName>:layout:maskLayout_oa2ndm_createframe.log
```

Output	Description
<digital_lib>	The Fusion Compiler output library that you specified in the Fusion Compiler Create Frame dialog box.
FCDataPrepnnn	The directory containing the <code>create-frame.tcl</code> file, the Tcl script containing Fusion Compiler commands and settings that were used to create the frames. nnn is the sequential numbering of the directories if multiple are generated from rerunning the process (for example, FCDataPrep001, FCDataPrep002, and FCDataPrep003).

Output	Description
<code><digital_lib>/logFiles/</code>	The directory containing the <code>create-frame.log</code> file.

After you create the frame view, you can run place and route. See [About the Place-and-Route Process](#).

11

Creating Schematics from Verilog Files

This chapter describes how to create schematics from Fusion Compiler Verilog files.

This chapter describes the following:

- [About Schematic Creation From Verilog Files](#)
- [Before You Start](#)
- [Creating Schematics from Fusion Compiler Verilog Files](#)

About Schematic Creation From Verilog Files

To run layout-vs-schematic (LVS) comparisons in the Custom Compiler environment, you must have a design schematic in the Custom Compiler database. You can use the **Fusion Compiler > Create Schematic from Verilog** command to:

- Export the Fusion Compiler Verilog data.
- Import the Fusion Compiler Verilog files into the Custom Compiler environment.
- Create schematics in the Custom Compiler environment using the imported Verilog files as input.

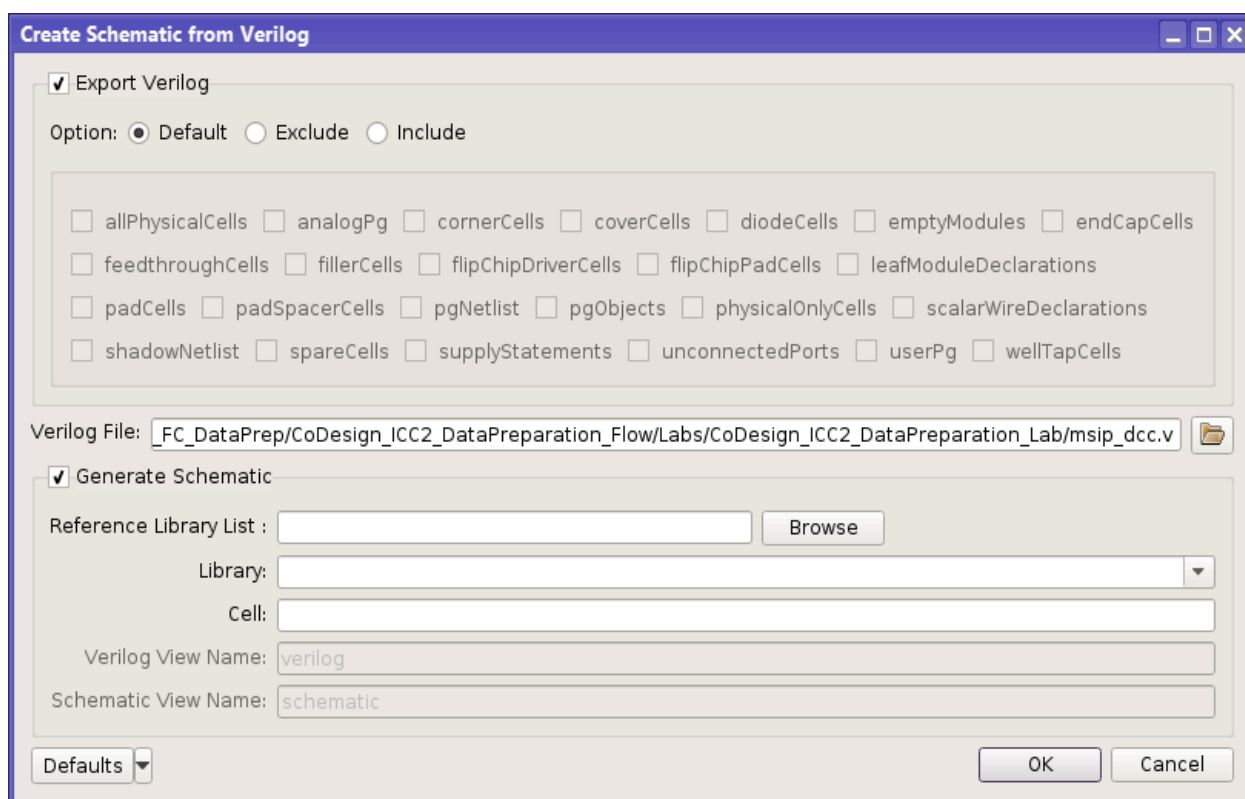
Before You Start

Make sure you start the Fusion Compiler tool. The **Fusion Compiler > Create Schematic from Verilog** command requires that the Fusion Compiler tool is open.

Creating Schematics from Fusion Compiler Verilog Files

To create a schematic from a Fusion Compiler Verilog File:

1. In the Custom Compiler environment, open the design layout for which you want to create the schematic.
2. In the Layout Editor, choose **Fusion Compiler > Create Schematic from Verilog**. The **Create Schematic from Verilog** dialog box opens.



3. If you need to export the Verilog data from the Fusion Compiler design, select **Export Verilog**. If you have the Verilog file, clear the check box.
4. Select the **Option** you want to use:
 - **Default:** Imports all data.
 - **Exclude:** Lets you specify the data you want to exclude during the import process.
 - **Include:** Lets you specify only the data you want to include during the import process.

To create a schematic when no symbol views exist for leaf cells, make sure you select **leafModuleDeclarations**.

5. Specify the location and name of the **Verilog File**.
6. Select **Generate Schematic**, and set the following:
 - **Reference Library List**: The reference library associated with the newly created schematic. Click **Browse** to select the applicable reference libraries.
 - **Library**: The design library of the newly created schematic.
 - **Cell**: The design cell name of the newly created schematic.

By default, if you are creating a schematic from an existing Verilog file, the **Cell** box is automatically populated with the first block name in the Verilog file. If you are exporting the Verilog file as part of the schematic creation process, the **Cell** box is empty.

If you specify a cell name, and the name is found in the Verilog file, the command only creates a schematic for the specified block, and uses the block name for the cell in the Custom Compiler environment.

If you leave the **Cell** box empty, the command creates schematic for every block in the Verilog file, and uses the block names for the newly created cells in the Custom Compiler environment.

- **Verilog View Name**: The view name of the Verilog file that is imported.
 - **Schematic View Name**: The view name of the newly created schematic.
7. Click **OK**.

When the export and import finishes, messages appears in the Console to let you know the number of cellviews generated, and any that failed or were skipped.

12

Exporting Custom Compiler Designs to the Fusion Compiler Tool

You can export Custom Compiler designs as input to the Fusion Compiler tool.

This chapter describes the following:

- [Validating the Custom Compiler Designs](#)
- [Exporting Custom Compiler Designs](#)

Validating the Custom Compiler Designs

Before exporting the Custom Compiler design, you can check for selected violations. You can fix the violations in the Custom Compiler environment, or retain the markers and fix the violations in the Fusion Compiler tool.

You use the **Validate for Fusion Compiler** command to check whether the design to be exported satisfies some minimum conditions, such as:

- All the pins should have a placement status.
- All the shapes should have connectivity.
- All the instance terminals have connectivity.
- All the instances within the boundary have a placement status.
- All the instances outside the boundary do not require a placement status.
- Power and ground nets have the Power/Ground signal type.

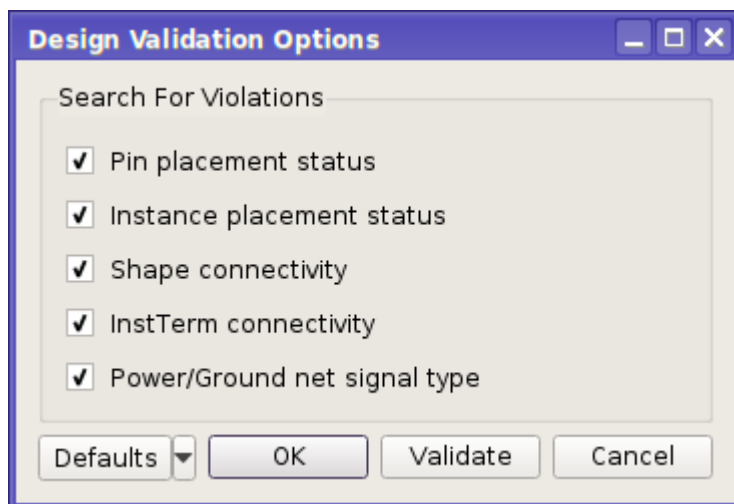
If there are any errors, they will be marked by error markers. Use the Error Viewer Assistant to browse and fix the errors.

You can run **Validate for Fusion Compiler** independently before you start the export process, or you can run the validation as part of the export process.

To validate the Custom Compiler design:

1. Open the design layout in the Custom Compiler environment. See [Opening Fusion Compiler Designs in the Custom Compiler Environment](#).
2. Enable the co-design assistant. See [Enabling the Co-design Assistant](#).
3. In the Layout Editor, choose **Fusion Compiler > Validate for Fusion Compiler**.

The **Design Validation Options** dialog box opens.



4. Select the rules you want to check.
5. Click one of the following:
 - **Validate**: Checks the design immediately.
 - **OK**: Saves the settings and runs the check when you export the Custom Compiler design.

If violations are found, markers are added in the design. You can use the markers to locate the violations in the design.

Warnings appear in the Console if the design does not contain any tracks or rows, or if tracks are found but track definitions do not exist.

Exporting Custom Compiler Designs

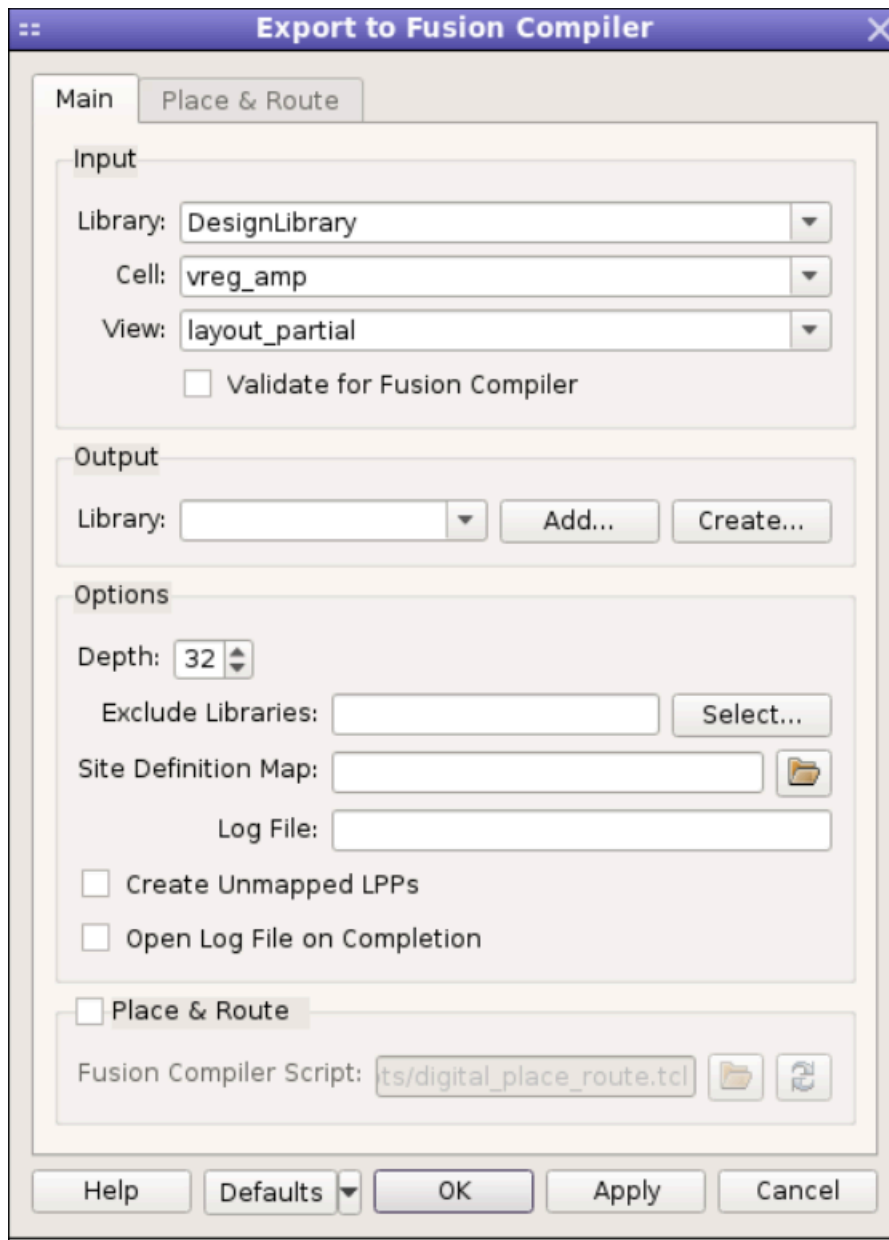
Exporting a Custom Compiler design creates the logical netlist representation in the target Fusion Compiler design. The logical netlist enables implementation flows for the Fusion Compiler design without requiring a separate Verilog Import process in the Fusion Compiler tool.

Before you start, be sure to set the design validation options. See [Validating the Custom Compiler Designs](#).

To export a Custom Compiler design:

1. Open the design layout in the Custom Compiler environment. See [Opening Fusion Compiler Designs in the Custom Compiler Environment](#).
2. Enable the co-design assistant. See [Enabling the Co-design Assistant](#).
3. In the Layout Editor, choose **Fusion Compiler > Export to Fusion Compiler**.

The **Export to Fusion Compiler** dialog box opens.



4. Specify the options in the following tabs:

- [Main tab](#)
- [Place & Route tab](#)

5. Click **OK**.

The design is checked based on the settings you specified in [Validating the Custom Compiler Designs](#).

After the check is finished, the design export process starts. Depending on the size and complexity of the design, the export process might take a long time. You can open the Job Monitor to check the progress.

When the export process finishes, two log files are generated:

- `export_to_digital.log`: Shows whether the export of the various design components were successful. You can locate the file in `synopsys_custom/digitalbridge/dig_top_icc2`.
- `fusion_compiler.log`: Shows the place-and-route commands and other information used during the export process.

You can view the log files in the Text Viewer.

10 /remote/amscae11/Maxwell/vasanth/Trainings/T-2022.06/ExportTOICC2/synopsys_custom/digitalbridge/dig_top_icc2/export_to_digital.log - T SYNOPSYS

Text Viewer

File Edit View Window Help

export_to_digital.log fusion_compiler.log

```
[2023-01-08 19:18:37] Translation statistics for design 'DEMOPLL/mpll_logic_placed/layout'
Created
Success Failure
PRBoundary      1      0
Shape
  Path          7      0
  Rect         37      0
Via
  StdVia       20      0
Net
  ScalarNet    54      0
Term
  ScalarTerm   35      0
Pin           36      0
Inst
  ScalarInst   31      0
InstTerm     137      0
Constraint
  LayerConstraint 12      0
TrackPattern  44      0

[2023-01-08 19:18:37] Application execution time (wall time in seconds)

Reflecting Nets                0.001      47487 PS
Reflecting Instances           0.024      1291 PS
Reflecting Instances Terminals 0.004      34037 PS
Reflecting InstHeader Boundries 0.000
Reflecting Layer Shape Patterns 0.000
Reflecting Shapes              0.001      80547 PS
Reflecting Vias                0.001      37070 PS
Reflecting Via Matrices        0.000
Reflecting Fig Groups          0.000
Reflecting Nets Constraints     0.000
Reflecting Groups              0.000
Reflecting Voltage Areas       0.000

Total time spent                0.053
```

Find: Next Previous Match Case Regexp

```
10 /remote/amscall/Maxwell/vasanth/Trainings/T-2022.06/ExportTOICC2/synopsys_custom/digitalbridge/place_route/icc2.log - Text Viewer - (SYNOPTYS)
File Edit View Window Help
export_to_digital.log fusion_compiler.log

Fusion Compiler (TM)

Version T-2022.03-SP4 for linux64 - Aug 31, 2022 -SLE

Copyright (c) 1988 - 2022 Synopsys, Inc.
This software and the associated documentation are proprietary to Synopsys, Inc. This software may only be used in accordance with the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, or distribution of this software is strictly prohibited. Licensed Products communicate with Synopsys servers for the purpose of providing software updates, detecting software piracy and verifying that customers are using Licensed Products in conformity with the applicable License Key for such Licensed Products. Synopsys will use information gathered in connection with this process to deliver software updates and pursue software pirates and infringers.

Inclusivity & Diversity - Visit SolvNetPlus to read the "Synopsys Statement on Inclusivity and Diversity" (Refer to article 000036315 at https://solvnetplus.synopsys.com)

Loading user preference file /u/vasanth/.synopsys_icc2_gui/preferences.tcl
=== CDICC2 set_app_options -name link.generate_dmm -value false
=== CDICC2 open_lib /remote/amscall/Maxwell/vasanth/Trainings/T-2022.06/ExportTOICC2/dig_top_icc2
Information: Loading library file '/remote/amscall/Maxwell/vasanth/Trainings/T-2022.06/ExportTOICC2/dig_top_icc2' (FILE-007)
Information: Loading library file '/remote/amscall/Maxwell/vasanth/Trainings/T-2022.06/ExportTOICC2/stdcells.ndm' (FILE-007)
=== CDICC2 open_block -edit mpll_logic_placed
Opening block 'dig_top_icc2:mpll_logic_placed.design' in edit mode
PR: DIGITAL_RUN_FLOORPLAN : 1
PR: DIGITAL_ADD_TAP_CELL : 0
PR: DIGITAL_RUN_PLACEMENT : 1
PR: DIGITAL_ADD_FILLER_CELL : 0
PR: DIGITAL_CREATE_PG_RINGS : 1
PR: DIGITAL_RUN_ROUTING : 1
PR: DIGITAL_RUN_VERIFICATION : 1
PR: DIGITAL_RUN_TIMING_FLOW : 0
PR: DIGITAL_RUN_CTS : 1
PR: DIGITAL_PARASITIC_TECH : 1
PR: DIGITAL_CONTROL_TYPE : core
PR: DIGITAL_FLOORPLAN_SHAPE_TYPE : R
PR: DIGITAL_SIDE_LENGTH : 0
PR: DIGITAL_CORE_LEFT : 1
PR: DIGITAL_CORE_RIGHT : 1
PR: DIGITAL_CORE_TOP : 1
PR: DIGITAL_CORE_BOTTOM : 1

Find: [ ] [Next] [Previous] [Match Case] [Regexp]
```

See Also

- [Job Monitor Tool](#)
- [Text Viewer Tool](#)

Specifying Main Tab Options

In the **Export to Fusion Compiler** dialog box, you can specify the design you want to export in the **Main** tab. This topic describes how to set the options in the **Main** tab.

The screenshot shows the 'Export to Fusion Compiler' dialog box with the 'Main' tab selected. The dialog has a title bar with a close button. Inside, there are two tabs: 'Main' and 'Place & Route'. The 'Main' tab is active and contains several sections: 'Input', 'Output', 'Options', and a 'Place & Route' section at the bottom. The 'Input' section has three dropdown menus for 'Library' (set to 'DesignLibrary'), 'Cell' (set to 'vreg_amp'), and 'View' (set to 'layout_partial'), followed by an unchecked checkbox 'Validate for Fusion Compiler'. The 'Output' section has an empty 'Library' dropdown, 'Add...' and 'Create...' buttons. The 'Options' section includes a 'Depth' spinner set to 32, an 'Exclude Libraries' text field with a 'Select...' button, a 'Site Definition Map' text field with a folder icon, a 'Log File' text field, and two unchecked checkboxes: 'Create Unmapped LPPs' and 'Open Log File on Completion'. The 'Place & Route' section has an unchecked checkbox and a 'Fusion Compiler Script' text field containing 'ts/digital_place_route.tcl' with folder and refresh icons. At the bottom are 'Help', 'Defaults' (with a dropdown arrow), 'OK', 'Apply', and 'Cancel' buttons.

Export to Fusion Compiler

Main | Place & Route

Input

Library: DesignLibrary

Cell: vreg_amp

View: layout_partial

☐ Validate for Fusion Compiler

Output

Library: Add... Create...

Options

Depth: 32

Exclude Libraries: Select...

Site Definition Map:

Log File:

☐ Create Unmapped LPPs

☐ Open Log File on Completion

☐ Place & Route

Fusion Compiler Script: ts/digital_place_route.tcl

Help Defaults OK Apply Cancel

In the **Main** tab of the **Export to Fusion Compiler** dialog box:

1. In the **Input** section:
 - a. Specify the **Library**, **Cell**, and **View** of the Custom Compiler design.
 - b. Select **Validate for Fusion Compiler** if you want to check for violations at the beginning of the export process. The rule selections in the **Design Validation Options** dialog box are used. See [Validating the Custom Compiler Designs](#).
2. In the **Output** section, choose the destination Fusion Compiler **Library**.

To populate the **Library** list, click one of the following:

 - **Add**: Opens the **Add Existing Fusion Compiler Library** dialog box. For instructions, see [Adding Fusion Compiler Libraries](#).
 - **Create**: Opens the **New Library** dialog box so that you can create a new Custom Compiler library for exporting. For instructions, see [Creating Fusion Compiler Libraries](#).
3. Set the **Depth** to export the specified number of hierarchy levels.
4. In the **Exclude Libraries** box, type the names of the libraries you want to exclude from the export process. Add a character space between the library names.

Alternatively, click **Select** to select the libraries to exclude from the export process.
5. Locate and select the **Site Definition Map** file.

A site definition defines how a standard cell must be placed in the layout. The mapping file maps the Custom Compiler site definitions to the Fusion Compiler site definitions. The information is stored in an ASCII file, and each row in the file provides a Custom Compiler to Fusion Compiler mapping. For example:

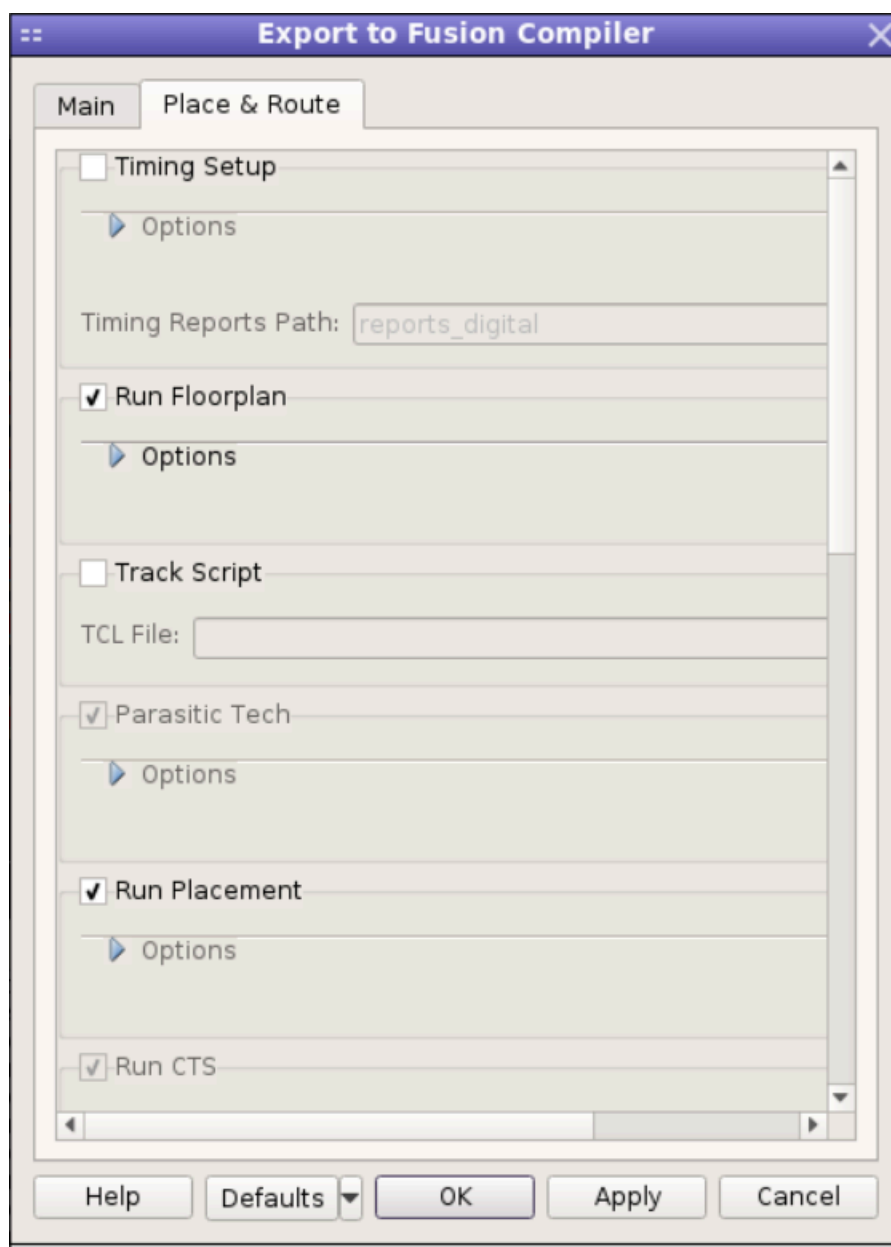
```
oa_unit_ySym ndm_unit_xSym
```
6. Type the name of the **Log File** that contains the record of events during the export process.
7. Select **Create Unmapped LPPs** if you want to create new LPPs if the Custom Compiler LPPs do not map to existing Fusion Compiler LPPs.
8. Select **Open Log File on Completion** if you want the log file to open automatically after the export process is finished.

9. If you want to run place-and-route as part of the export process:
 - a. Select **Place & Route**.
 - b. Specify the digital place-and-route Tcl script in the **Fusion Compiler Script** box.
 - c. Set the options in the [Place & Route](#) tab.
-

Specifying Place & Route Tab Options

In the **Export to Fusion Compiler** dialog box, you can specify the place-and-route settings in the **Place & Route** tab.

For the descriptions of Fusion Compiler options, see the [Fusion Compiler user documentation](#).



13

Implementing ECOs

This chapter describes how to implement engineering change orders (ECOs) in the Custom Compiler and Fusion Compiler co-design process.

This chapter describes the following:

- [About ECO Editing in Co-design Flows](#)
- [Creating and Deleting Nets](#)
- [Creating and Deleting Instances](#)
- [Connecting and Disconnecting Nets](#)

About ECO Editing in Co-design Flows

An engineering change order (ECO) is an incremental change made to a completed or nearly completed design. To facilitate the co-design process, you can make ECO edits in either the Custom Compiler environment or in the Fusion Compiler tool.

 [Show me](#) an example of top-level ECO editing in co-design flows. Although the video shows working with an IC Compiler II design, the example is also applicable for Fusion Compiler designs.

The Custom Compiler environment lets you make the following top-level and hierarchical ECO edits:

Editing Task in the Custom Compiler Environment	Equivalent Fusion Compiler Command
Creating and Deleting Nets	<code>create_net</code> <code>remove_net</code>
Creating and Deleting Instances	<code>create_cell</code> <code>remove_cell</code>
Connecting and Disconnecting Nets	<code>connect_net</code> <code>disconnect_net</code>

For example, you can:

- Connect top-level nets to instances.
- Create hierarchical instances and nets.
- Connect hierarchical instances to hierarchical nets within the same hierarchy level. For example, you can connect net I1|n1 to instance term I1|I1/a.
- Connect top-level nets to hierarchical instances. When you save the design, the tool creates module ports to establish the connection.

During ECO editing, you cannot:

- Connect different modules.
- Connect a hierarchical net to a top-level instance term.

 [Show me](#) an example of top-level ECO editing in the Custom Compiler environment.

Creating and Deleting Nets

This topic describes the following:

- [Creating Nets](#)
- [Deleting Nets](#)

 [Show me](#) an example of top-level ECO editing in the co-design process.

Creating Nets

In the Custom Compiler environment, you can create nets so that modules can be connected.

Use one of the following methods to create nets in the mixed-signal design:

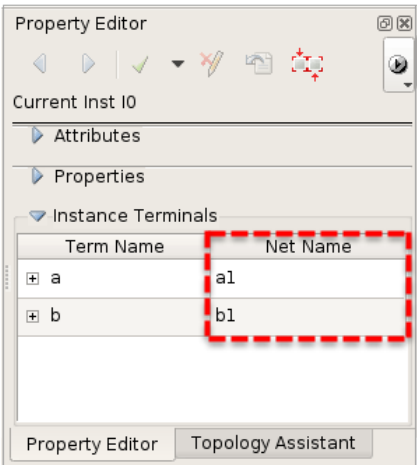
Method	Description
Create object	In the Layout Editor, when you create an object, you have the option of creating a new net. In the following example, when creating an interconnect, you can specify a new net in the Nets box.



See [Creating and Editing Basic Shapes and Text](#).

Method	Description
--------	-------------

Use the Property Editor
In the Layout Editor, you can select an existing instance object, and specify a new net in the Property Editor **Instance Terminals** section.



See Selecting and Editing Instance Pins.

Run the Fusion
Compiler
`create_net`
command

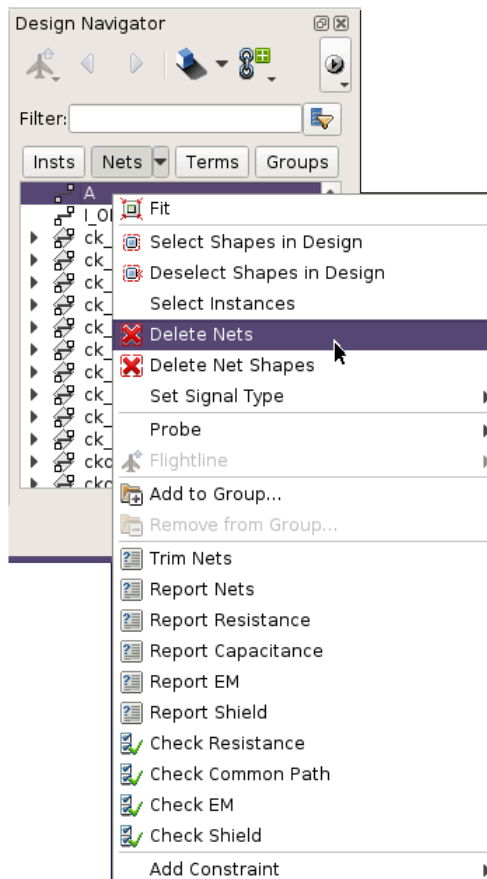
In the following example, net A is created in the I_ORCA_TOP module.



Deleting Nets

To delete net associations:

1. In the Layout Editor, display the Design Navigator.
2. On the **Nets** tab, right-click the net, and choose **Delete Nets**.



Creating and Deleting Instances

This topic describes the following:

- [Creating Instances](#)
- [Deleting Instances](#)
- [Creating and Deleting Physical-Only Cells](#)

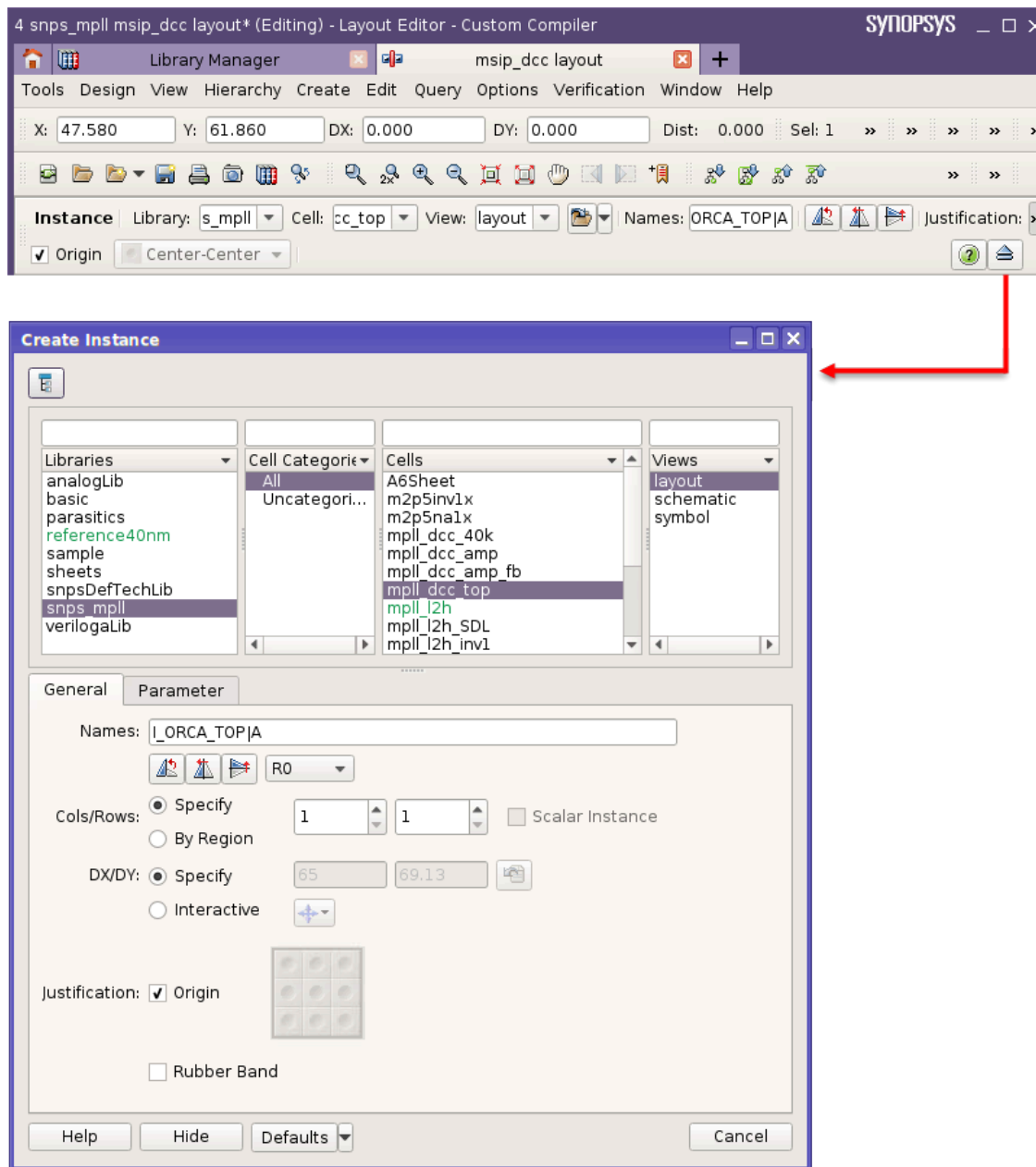
 [Show me](#) an example of top-level ECO editing in the co-design process.

Creating Instances

In the Custom Compiler environment, you can create instances (cells) in modules.

To create an instance:

1. In the Layout Editor, choose **Create > Instance**. The **Instance** toolbar or the **Create Instance** dialog box opens.



2. On the **Instance** toolbar or the **Create Instance** dialog box, set the options. See Instance Toolbar and Create Instance Dialog Box.

In the step1 example, the command is set up to create instance A in the I_ORCA_TOP module.

Deleting Instances

To delete an instance (cell):

1. On the Layout Editor canvas, select the instance.
2. Press Delete.

Creating and Deleting Physical-Only Cells

Before you create or delete physical-only cells, you must ensure that the master cells have the `physicalOnly` attribute. To check the existence of the `physicalOnly` attribute in the master cells, use the `db::getAttr design.cellSubtype` command.

To create instances of physical-only master cells in the IC Compiler II design using the Custom Compiler environment:

1. Set the `leNetlistMode` command to `read`.
2. Set the `leNetlistModeReadEnableCreateInst` preference to `true`.

Note:

If you create or delete physical instances that have the master cells in the Open Access standard library, and the cells do not have the `physicalOnly` attribute enabled, then the following error message appears:

```
The Create Instance command is in the invalid state because the
creation of non-physical only instances is not permitted when
the leNetlistMode preference is read-only. (ILE_CREATE-161)
```

Connecting and Disconnecting Nets

This topic describes the following:

- [Connecting Nets](#)
- [Disconnecting Nets](#)

 [Show me](#) an example of top-level ECO editing in the co-design process.

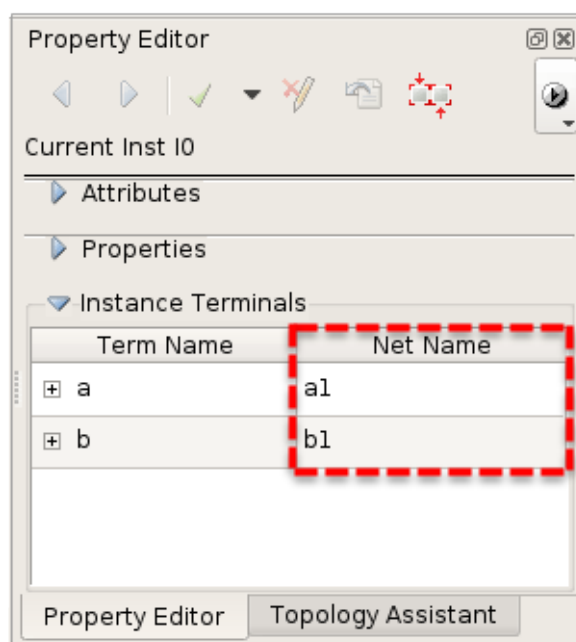
Connecting Nets

In the Custom Compiler environment, you use the Property Editor to connect nets to instances.

To connect nets to instances:

1. In the Layout Editor, select the instance and display the Property Editor. Information about the instance appears in the Property Editor.
2. In the Property Editor, locate the **Instance Terminals** section, and set the **Net** name for the instance terminal.

You can specify a new or an existing net name. If you specify a new net, the new net is created.

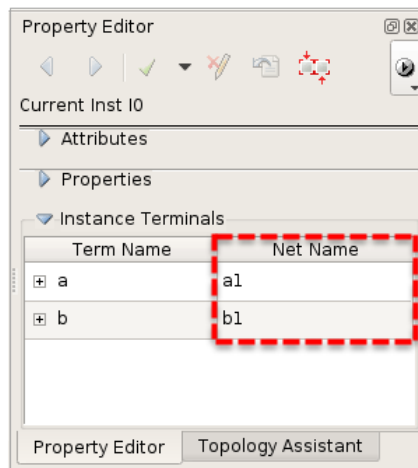


3. When you are finished, click **Apply Changes**  on the Property Editor toolbar.

Disconnecting Nets

To disconnect nets:

1. In the Layout Editor, select the instance and display the Property Editor. Information about the instance appears in the Property Editor.
2. In the Property Editor, locate the **Instance Terminals** section, and remove the **Net** name for the instance terminal.



3. When you are finished, click **Apply Changes**  on the Property Editor toolbar.

14

Preparing for Physical Verification (IC Validator)

To run the IC Validator physical verification tool in the Custom Compiler environment, you must have PDK-based GDS mapping files for Fusion Compiler designs. If the mapping files are not available, you can generate them.

To generate GDS mapping files for Fusion Compiler designs:

1. Use the following commands to attach the Fusion Compiler GDS layer mapping file to the Fusion Compiler library, save, and exit:

```
Fusion Compiler_shell
open_lib design_lib2
set_layer_map_file -map_file layermapfile_Fusion Compiler.txt -format
gds
save_lib
exit
```

2. In the Custom Compiler environment, add the Fusion Compiler `design_lib2` library.

The GDS layer mapping file you attached in step1 is parsed, and the Custom Compiler GDS mapping file is created in the following directory:

```
synopsys_custom/digitalbridge/<Fusion CompilerLibName>/tech/<Fusion
CompilerLibName>_tech.layermap
```

With the newly created GDS mapping file, you can proceed to run IC Validator in the Custom Compiler environment. For example, the following GDS mapping file can be used to run IC Validator DRC and LVS flows.

```
synopsys_custom/digitalbridge/design_lib2/tech/
design_lib2_tech.layermap
```

15

Running Formality Verification

Instead of running verification using simulations, you can run formal verifications to detect unexpected differences that might have been introduced into a design. This chapter describes the use of the Formality Verification tool to run formal verifications.

This chapter describes the following:

- [About the Formality Verification Flow](#)
- [Workflow for Formality Verification](#)
- [Enabling the Formality Verification Flow](#)
- [Setting Up Formality Verification Flow Options](#)
- [Starting the Formality Verification Flow Process](#)

About the Formality Verification Flow

The Formality Verification flow detects unexpected differences that might have been introduced into a design during development. You can run the verification as part of the Fusion Compiler co-design flows.

This topic describes the following:

- [Formal Verification and the Formality Verification Tool](#)
- [Using the Formality Verification Tool in the Co-design Process](#)

Formal Verification and the Formality Verification Tool

An alternative to verification through simulation, formal verification uses mathematical techniques to compare the logic to be verified against a logical specification or a reference design. Unlike verification through simulation, formal verification does not require input vectors and is independent of the physical properties of the design, such as layout and timing. These advantages allow formal verification to process large designs faster than simulation while providing 100 percent coverage.

The Formality Verification tool uses formal verification techniques to determine if two versions of a design are functionally equivalent. The tool provides both auto and manual setup modes to accommodate data availability and user preferences. For more information, see the [Formality Verification user documentation](#).

Using the Formality Verification Tool in the Co-design Process

While co-designing within the Custom Compiler environment, you can run the Formality Verification tool to report unexpected differences that might have been introduced into a design during development. Many key Formality Verification tool features are accessible in the Custom Compiler graphical user interface to facilitate the setup and run process. For example, you have the option to run auto setup with the specified SVF file, specify the two designs you want to compare, run the Verify mode, run the Match mode, and so on.

Workflow for Formality Verification

The following table presents the high-level steps for running the Formality Verification flow.

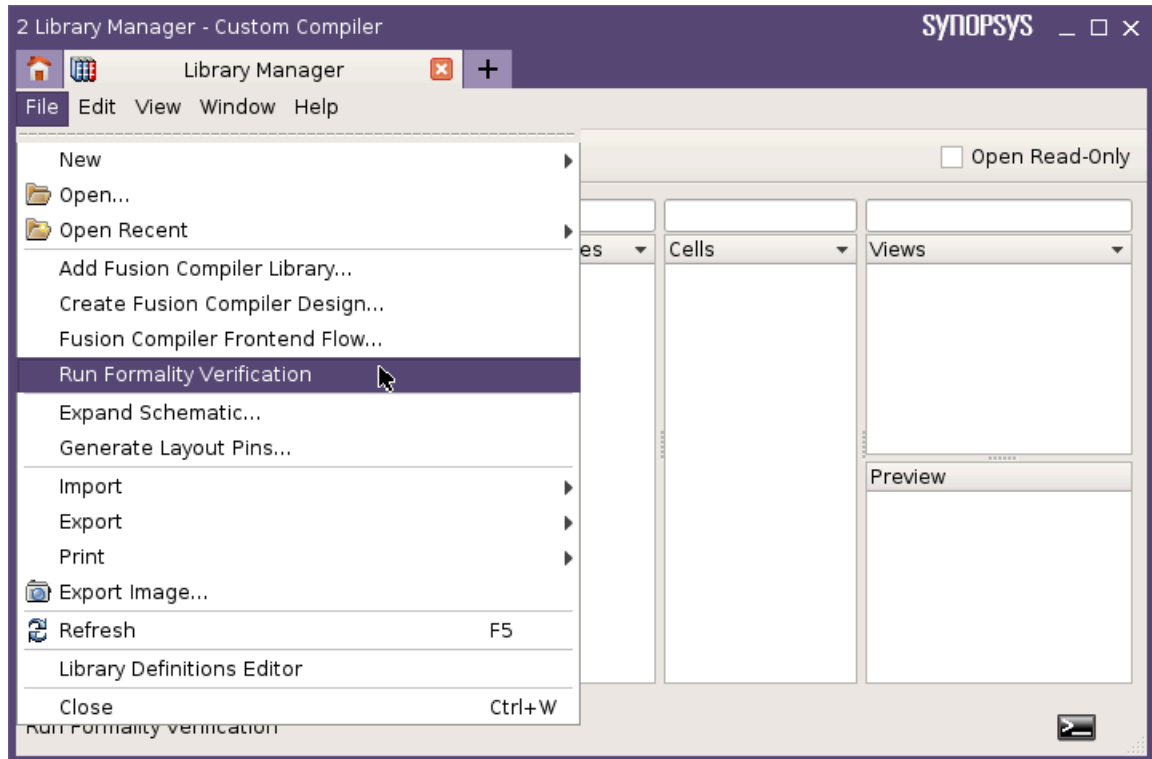
Task	See...
1 Turn on and display the Formality Verification flow options.	Enabling the Formality Verification Flow
2 Set the Formality Verification flow options.	Setting Up Formality Verification Flow Options
3 Start the Formality Verification flow operation.	Starting the Formality Verification Flow Process

Enabling the Formality Verification Flow

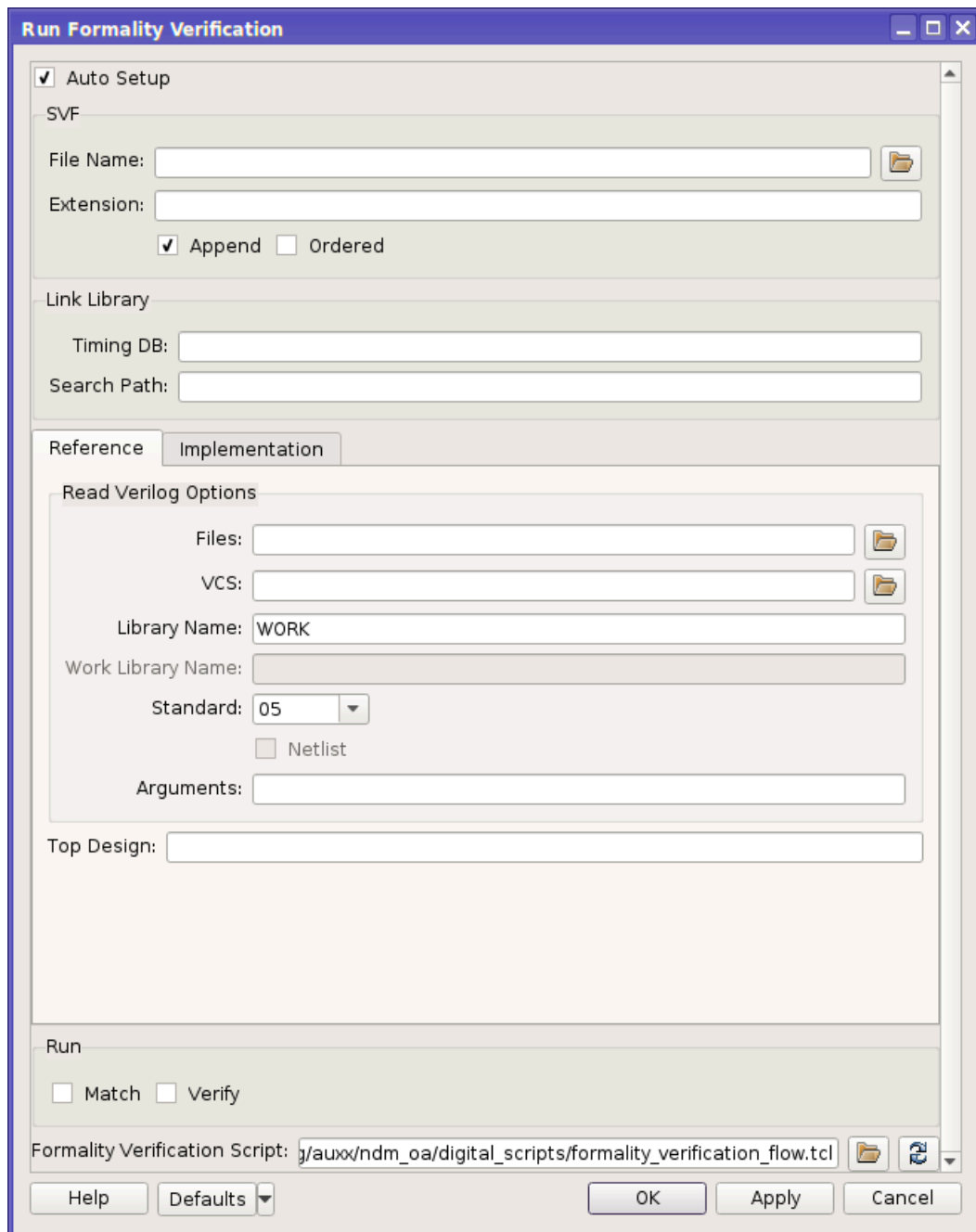
You must enable the Formality Verification flow to access the user interface.

To enable the Formality Verification flow:

- In the Library Manager, choose **File > Run Formality Verification**.



The **Run Formality Verification** dialog box opens.



The **Run Formality Verification** dialog box is shown. It has a title bar with standard window controls. The main area is divided into several sections:

- Auto Setup:** A checked checkbox.
- SVF:** A section containing:
 - File Name:** A text field with a folder icon to its right.
 - Extension:** A text field.
 - Append:** A checked checkbox.
 - Ordered:** An unchecked checkbox.
- Link Library:** A section containing:
 - Timing DB:** A text field.
 - Search Path:** A text field.
- Reference** and **Implementation** tabs. The **Implementation** tab is selected.
- Read Verilog Options:** A section containing:
 - Files:** A text field with a folder icon to its right.
 - VCS:** A text field with a folder icon to its right.
 - Library Name:** A text field containing the text "WORK".
 - Work Library Name:** A text field.
 - Standard:** A dropdown menu showing "05".
 - Netlist:** An unchecked checkbox.
 - Arguments:** A text field.
- Top Design:** A text field.
- Run:** A section containing:
 - Match:** An unchecked checkbox.
 - Verify:** An unchecked checkbox.
- Formality Verification Script:** A text field containing the path "j/auxx/ndm_oa/digital_scripts/formality_verification_flow.tcl" with a folder icon to its right.

At the bottom, there are buttons for **Help**, **Defaults** (with a dropdown arrow), **OK**, **Apply**, and **Cancel**.

After you enable the Formality Verification flow, you can set the options. See [Setting Up Formality Verification Flow Options](#).

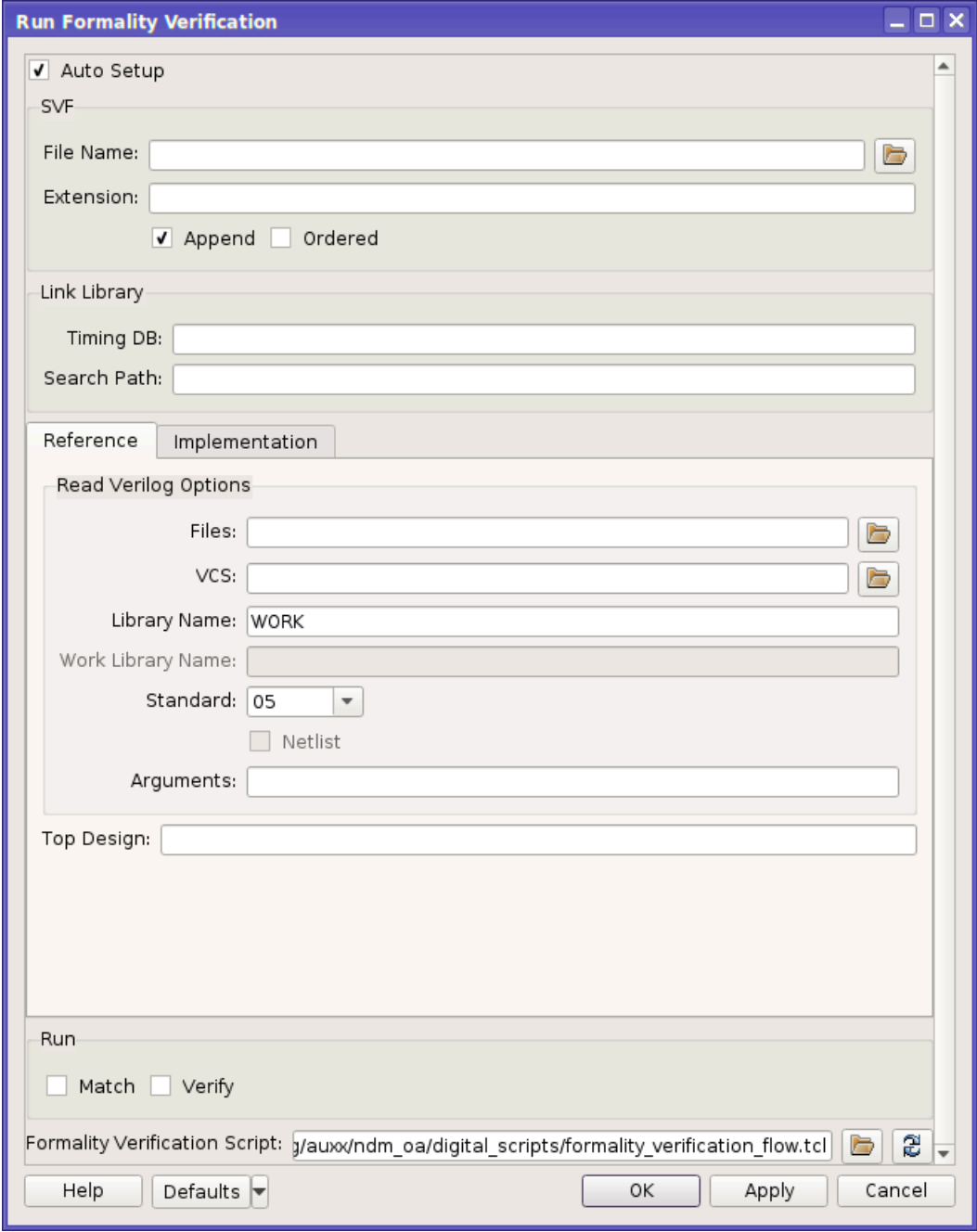
Setting Up Formality Verification Flow Options

You must set the Formality Verification flow options to specify how the tool should run the verification. This topic summarizes the flow options available.

All the flow options available in the **Run Formality Verification** dialog box match the options available in the Formality Verification tool. For detailed descriptions of the options, see the [Formality Verification user documentation](#).

The **Run Formality Verification** dialog box consists of the following sections:

- [Auto Setup](#)
- [Link Library](#)
- [Reference Tab](#)
- [Implementation Tab](#)
- [Run](#)
- [Formality Verification Script](#)



The **Run Formality Verification** dialog box is used to configure the formal verification process. It includes sections for SVF file setup, linking libraries, reading Verilog options, and running the verification.

Auto Setup (checked)

SVF

File Name: 

Extension:

☒ Append ☐ Ordered

Link Library

Timing DB:

Search Path:

Reference **Implementation**

Read Verilog Options

Files: 

VCS: 

Library Name:

Work Library Name:

Standard: 

☐ Netlist

Arguments:

Top Design:

Run

☐ Match ☐ Verify

Formality Verification Script:  

Help Defaults  OK Apply Cancel

Auto Setup

Option	Description
Auto Setup	Runs the verification using the SVF guidance file, which contains flow setup information, thus eliminating the need to enter the information manually. The SVF file is generated from Synopsys implementation tools, such as the Design Compiler tool. By default, the Auto Setup mode runs the appropriate Formality commands and variables to account for external constraints and RTL interpretation considered by Synopsys implementation tools.
File Name	Specifies the location and name of the SVF file.
Extension	Specifies the extension name to be used when recursively searching directories for SVF files.
Append	Appends setup information to an existing SVF file. Use Append to keep all the setup information in a single file rather than using separate SVF files.
Ordered	Preserves the order of the file names when reading.

Link Library

Option	Description
Timing DB	Specifies the name and location of the .db file that contains the technology or design data.
Search Path	Searches the specified directories for design and library files if you do not specify the directory in Timing DB .

Reference Tab

The **Reference** tab specifies details about the reference design. The flow compares the implementation design against the reference design.

Read Verilog Options	Description
Files	Specifies one or more Verilog files.

Read Verilog Options	Description
VCS	Specifies VCS options or the name and location of Verilog files. For the VCS options you can specify, see the Formality Verification user documentation .
Library Name	Specifies the name of the design library or technology library into which Verilog designs or cell descriptions are placed. You can specify any alphanumeric string for the library name.
Work Library Name	Specifies the default library into which designs are placed.
Standard	Specifies the IEEE standard to use for language interpretation. The default is -05, which specifies the IEEE 1364-2005. Use only the -95 and -01 options with the -netlist option.
Netlist	Identifies the file as a Verilog structural netlist. This option can reduce the time required to read the file.
Arguments	Lets you supply additional arguments to be applied during the verification operation.
Top Design	Resolves cell references and elaborates the RTL designs for reference.

Implementation Tab

The **Implementation** tab specifies details about the implementation design. The flow compares the implementation design against the reference design.

For the summary of the **Read Verilog Options** section, see [Reference Tab](#).

Read Database Options	Description
Read Database	Reads technology libraries or designs in the selected database format: • DB: The .db format. • NDM: The .ndm format.
Files	Specifies one or more .db or .ndm files to read.
Library Name	Specifies the name of the design library or technology library into which Verilog designs or cell descriptions are placed. You can specify any alphanumeric string for the library name.

Read Database Options	Description
Merge	Replaces black box cells and test cells in the specified library with those that are read from the .db or .ndm format library. This option also adds cells if there are no corresponding cells already in the specified existing library.
Replace Black box	Replaces black box cells and test cells in the specified library with those that are read in from the .db or .ndm format library.
Top Design	Resolves cell references and elaborates the RTL designs for reference.

Run

Option	Description
Match	Matches the implementation design against the reference design based on compare points.
Verify	Verifies the functional equivalence between the two specified designs or two comparable design objects.

Formality Verification Script

Specifies the Tcl script to run. To run the Formality Verification flow, specify the following script:

```
formality_verification_flow.tcl
```

By default, the script is located in the following Custom Compiler installation directory:

```
<product_installation_directory>/ndm_oa/digital_scripts/
```

Starting the Formality Verification Flow Process

After you finish setting up the Formality Verification flow options, you can start the flow process.

To start the Formality Verification flow process:

- In the **Run Formality Verification** dialog box, click **OK**.

The specified flow operations start. You can open the Job Monitor to check the progress.

A

Technology Mappings

Custom Compiler and Fusion Compiler co-designing supports various flows. This appendix describes how to set up the Custom Compiler and Fusion Compiler environments for different scenarios of technology information availability.

This chapter describes the following:

- [Viewing Mapped Constraints](#)
- [Mapping Fusion Compiler Constraints](#)
- [Mapping Custom Compiler Constraints](#)

For information about setting and editing constraints, see [Setting and Editing Constraints](#).

Viewing Mapped Constraints

To review the constraint mappings before you start to edit the design, see the following:

Fusion Compiler constraints > Custom Compiler constraints	Custom Compiler constraints > Fusion Compiler constraints
Viewing Constraints	Reviewing Newly Added or Modified Constraints

Mapping Fusion Compiler Constraints

This topic describes the following:

- [Constraints](#)
- [Design Constraints](#)

Constraints

The following table shows the mapping of Fusion Compiler constraints to Custom Compiler constraints. ✓ indicates that the characteristic is supported. x indicates that the characteristic will be supported in a future release. NA indicates that the characteristic is not applicable.

Fusion Compiler Constraint	Single Net	List of Nets	Net Bundles	Custom Compiler Constraint
Shield	✓	✓	✓	Net Shielding
Bus	NA	✓	✓	Bus
Diff	NA	✓	x	Diff Group
Wire Matching	NA	✓	✓	Wire Matching
Driver Taper Distance (Routing Rule)	✓	✓	NA	Driver Taper Halo on net group
Taper Distance (Routing Rule)	✓	✓	NA	Taper Halo on net group
Valid Routing Vias (Routing Rule)	✓	✓	NA	Vias on net group
Valid Routing Layer	✓	✓	NA	Layer
Nondefault width and spacing for a layer (Routing Rule)	✓	✓	NA	Layers: Width and Spacing parameters

Cut class definitions are mapped individually. Two cut class definitions that are transposed versions of one another will appear as two cut class definitions in the Custom Compiler environment.

Design Constraints

The following table shows the mapping of Fusion Compiler command options to Custom Compiler design constraints.

Fusion Compiler Command	Fusion Compiler Option	Custom Compiler Design Constraint	Description
create_via_def	excludedForSignalRoute	excludeRoutingVias	The vias of the selected signal net defined in the excludedForSignalRoute constraint in Fusion Compiler are mapped to the excludeRoutingVias constraint and added to the default constraint group in Custom Compiler. If the selected net is not a signal net, then all vias are available in the design.

Mapping Custom Compiler Constraints

The following table shows the mapping of Custom Compiler constraints to Fusion Compiler constraints. ✓ indicates that the characteristic is supported. x indicates that the characteristic will be supported in a future release. NA indicates that the characteristic is not applicable.

Custom Compiler Constraint	Single Net	List of Nets	Net Groups	Fusion Compiler Constraint
Net Shielding	✓	✓	✓	create_net_shielding
Bus	NA	✓	✓	create_bus_routing_style
Diff Group	NA	✓	x	create_differential_group
Wire Matching	NA	✓	✓	create_wire_matching
Driver Taper Halo	✓	x	✓	create_routing_rule [-driver_taper_distance distance]
Taper Halo	✓	x	✓	create_routing_rule [-taper_distance distance]
Vias	✓	x	✓	create_routing_rule [-vias {via_definition_name site_spec rotation_spec} ...] -cuts {cut_layer_name {cut_name num_of_cuts} ...} ...]
Layers	✓	✓	✓	set_routing_rule [-min_routing_layer layer] [-max_routing_layer layer]
Layers: Width and Spacing parameters	✓	x	✓	create_routing_rule -widths {layer_name width ...} [-spacings {layer_name {spacing ...} ...}]

B

Character and Object Mappings

This appendix shows the object mappings between the Fusion Compiler tool and the Custom Compiler environment.

When you open Fusion Compiler designs in the Custom Compiler environment, the Fusion Compiler objects automatically map to Custom Compiler environment naming conventions. This appendix lists the following mappings:

- [Character Mappings](#)
- [Object Mappings](#)

Character Mappings

The following characters are used in either net and instance names.

Fusion Compiler Tool	Custom Compiler Environment	Character is used for...
/		nets and instances
[	<	nets
]	>	nets

Object Mappings

To ensure designs can transition correctly between the Fusion Compiler and Custom Compiler environments, constraints and objects are mapped.

This topic lists the mappings of the following supported objects:

- [Object Properties](#)
- [Shapes](#)
- [Vias](#)
- [Instances](#)
- [Bounds](#)
- [Logical Components](#)
- [Boundary](#)
- [Blockage](#)
- [Routing](#)
- [Routing Corridor](#)
- [Anchor](#)
- [Route Guide](#)
- [Antenna Properties](#)
- [Voltage Area](#)
- [Custom Compiler Objects](#)

Object Properties

Fusion Compiler Tool	Custom Compiler Environment
User-defined attributes	Object properties (OA properties)

Supported data types: Boolean, string, integer, floating numbers

See [Viewing and Editing Object Properties](#) and [Retrieving New Object Attributes](#).

Shapes

Fusion Compiler Tool	Custom Compiler Environment
path	Path
pathseg	Path Segment
polygon	Polygon
rectangle	Rectangle
text	Text

Note:

`is_dummy_metal` attribute of the NDM shape is mapped to the `isDummyMetal` boolean property in the Custom Compiler environment.

Vias

Fusion Compiler Tool	Custom Compiler Environment
simple via	Standard Via
simple array via	Standard Via
custom via	Custom Via
through-silicon via (TSV)	Custom Via

Standard Via

Fusion Compiler Tool	Custom Compiler Environment
orientation	Orientation
origin	Origin
offset	Custom Via
topMask	Via MP Color
cutMask	Via MP Color

Fusion Compiler Tool	Custom Compiler Environment
botMask	Via MP Color
status	Route Status
rows	Rows
columns	Columns
cutPattern	Cut Pattern
xPitch	Cut Spacing X, or Cut Width
yPitch	Cut Spacing Y, or Cut Height
is_via_ladder	Via Ladder

Custom Via

Fusion Compiler Tool	Custom Compiler Environment
orientation	Orientation
status	Route Status
is_via_ladder	Via Ladder

Instances

Fusion Compiler Tool	Custom Compiler Environment
instance	Instance
instance array	Array Instance

Bounds

Fusion Compiler Tool	Custom Compiler Environment
Move bound	Cluster

Fusion Compiler Tool	Custom Compiler Environment
Bound types: • SOFT • HARD • EXCLUSIVE	Cluster type: • suggested • inclusive • exclusive

Logical Components

Fusion Compiler Tool	Custom Compiler Environment
nets	Net
ports	Bit Term
instance terminals	Instance Term
pin	Pin

Boundary

Fusion Compiler Tool	Custom Compiler Environment
block boundary	PR Boundary

Blockage

For information about Routing Blockages, see [Creating and Editing Blockages](#).

Fusion Compiler Tool	Custom Compiler Environment
placement blockage	Placement Blockage
pin blockage	Pin Blockage (or LayerRangeBlockage)
block and block instance halo attributes	Area Halo, Layer Halo, Layer Range Halo
Routing blockage	Routing Blockage

Placement Blockage

For the description of the subtypes, see the [Fusion Compiler user documentation](#).

Fusion Compiler Tool	Custom Compiler Environment
Subtypes: • hard • soft • hard_macro • partial • allow_buffer_only • allow_rp_only • register • category • rp_group	Subtypes: • Hard • Soft • Hard Macro • Partial • Allow Buffer Only • Allow RP Only • Register • Category • RP Group
boundary	Points List

Pin Blockage (or LayerRangeBlockage)

Fusion Compiler Tool	Custom Compiler Environment
boundary	Points List
layers	Layer

Routing Blockage

Fusion Compiler Tool	Custom Compiler Environment
boundary	Points LlSt
zero_spacing	Spacing If zero_spacing is true, then Spacing is 0.
mask: • mask_none • mask_one • mask_two • mask_three • mask_four • mask_same	Color: • No Color • Color1 • Color2 • Color3 • Color4 • same
netTypes: • signal • nwell • pwell • deep_nwell • deep_pwell • tie_high • tie_low • power • ground • clock • reset • analog_signal • analog_power • analog_ground • scan	Signal Types: • Signal • Signal • Signal • Signal • Signal • TieHi, Tieoff • TieLo • Power • Ground • Clock • Reset • Analog • Analog • Analog • Scan
layerNumber	Layer
• is_dummy_metal	• isDummyMetal

Routing

Fusion Compiler Tool	Custom Compiler Environment
row	Row See Creating and Editing Rows of Sites.
tracks	Track Pattern definition See Creating and Editing Track Pattern Definitions.

Routing Corridor

For information about routing corridors, see [Creating and Editing Routing Corridors](#).

Fusion Compiler Tool	Custom Compiler Environment
RoutingCorridor (path and polygon) • name • nets • region	leRouteCorridor (line and rectangle) • name • nets • corridorAreas

Anchor

The anchor object ensures that the Anchor and the Dependent components move together.

The Anchor is the cell type and the Dependent is the blockage type attributes when you convert instances from NDM library to OA library.

The NDM anchor is mapped to the **Attach** property of a routing blockage that is attached to an instance in the Custom Compiler environment.

Route Guide

For information about route guides, see [Editing Route Guides](#).

Fusion Compiler Tool	Custom Compiler Environment
boundary	Points List
layers	Layers
routing_guide_type	Type
preferred_direction_only	Preferred Direction
switch_preferred_direction	Switch Preferred Direction
horizontal_track_utilization	Horizontal Track Utilization
vertical_track_utilization	Vertical Track Utilization

Antenna Properties

The antenna properties are stored in the port object.

Fusion Compiler Tool	Custom Compiler Environment
gate_area - n_gate_area - p_gate_area	gate
diff_area	diff
antenna_area	partialMetal
antenna_side_area	partialMetalSide
cut_area	partialCut
max_area_car	maxCAR
max_side_area_car	maxSideCAR
max_cut_area_car	maxCutCAR

In addition to the antenna properties listed, layerNumber and oxideModel are also imported.

Voltage Area

For information voltage areas, see the [Fusion Compiler user documentation](#).

Fusion Compiler Tool	Custom Compiler Environment
power_domains	Power Domain
target_utilization	Target Utilization
regions	Area Boundary in figGroup
power supply_net	Power Net
power ground_net	Ground Net
guard_band (X) guard_band (Y)	Guard Band X Guard Band Y

Custom Compiler Objects

Objects that are specific to the Custom Compiler environment are mapped to Fusion Compiler as follows:

Fusion Compiler Tool	Custom Compiler Environment
EditGroup	Striped Wire See Striping Wires. Group with Sync Group Definition (sync clones) See Grouping Objects. Multipart path (MPP). See About Multipart Paths
